



slington college
(इस्लिङ्टन कलेज)

Module Code & Module Title

CS6P05NI Project

Final Year

40% Individual Coursework

Submission: Final Submission

AY 2025 2026

Project Title: Intelligence Recon System

Credit: 30 Credit Year Long Module

Student Name: Prabesh Sundar Taksari

London Met ID: 23047464

College ID: NP01NT4A230138

Assignment Due Date: Wednesday, April 20, 2026

Assignment Submission Date: Monday, April 22, 2026

Internal Supervisor: Prashant Pudasaini

External Supervisor: Suraj Neupane

Word Count: 8105

OneDrive Link: [23047464 - Prabesh Sundar Taksari - Intelligence Recon System](#)

I confirm that I understand my coursework needs to be submitted online via MST Classroom under the relevant module before the deadline for my assignment to be accepted and marked. I am fully aware that late submissions will be treated as non-submission and a mark of zero will be awarded.

ACKNOWLEDGEMENT

I would like to appreciate my outstanding supervisors, Mr. Prashant Pudasaini and Mr. Suraj Neupane, who have supported me in every way possible at any stage of this project and helped me out.

The supervision of Mr. Prashant Pudasaini as the Internal Supervisor has played a crucial role in offering the academic guidance of the project and ensuring that the project meets all the required university and academic standards. In the same way, the supervision and expertise of the External Supervisor, Mr. Suraj Neupane, with the valuable experience in the industry and broader perspective he gave me, have assisted me considerably in the development of the project concept and through the implementation of the project concept.

I would like to express my deep gratitude to both supervisors who support and encourage me to discover new directions of research and provide regular and informative feedback.

Lastly, I would like to say that I appreciate Islington College a lot, which gave me a chance to explore, learn and work in the sphere that I am interested in.

ABSTRACT

The Intelligence Recon System (IRS) is an AI-powered, web-based cybersecurity reconnaissance platform built to eliminate the inefficiency and fragmentation of manual command-line security testing. A combination of fifteen open-source CLI tools and routed execution by an AI Decision Node powered by Gemini API is used to select and sequence the most contextually relevant tools by IRS based on live reconnaissance clues and a user-selected OWASP Top 10 category. Tools are asynchronous, independent, such that a single failure does not cause a stop in the larger scan. Raw outputs are normalized into a common Finding model, deduplicated with OWASP scope rule, and persistently stored to be reviewed and audited in the long term. The scheduling engine provides one time, daily, weekly, and monthly automated scans and the reporting layer produces professional, downloadable HTML and PDF reports, to be used in communications with the stakeholders. There were fifteen-unit tests and one end-to-end system test and all of them were successful. An after-deployment survey of thirteen subjects revealed that three-quarters of respondents (76.9 percent) believed that the system was as effective or more so than expected, confirming IRS as a viable, accessible alternative to disjointed manual reconnaissance to students and small security teams.

TABLE OF CONTENTS

CHAPTER 1: INTRODUCTION	1
1.1. MOTIVATION.....	2
1.2. PROJECT DESCRIPTION.....	2
1.2.1. PROJECT DELIVERABLES	3
1.3. CURRENT SCENARIO.....	4
1.4. PROBLEM DOMAIN AND SOLUTION	5
1.4.1. Problem Domain	5
1.4.2. PROJECT AS SOLUTION	5
1.5. AIM AND OBJECTIVES.....	6
1.5.1. Aim.....	6
1.5.2. Objectives	6
1.6. STRUCTURE OF REPORT	7
1.6.1. BACKGROUND	7
1.6.2. DEVELOPMENT	7
1.6.3. TESTING AND ANALYSIS	7
1.6.4. CONCLUSION	7
CHAPTER 2: BACKGROUND	8
2.1. TARGET CLIENT.....	9
2.2. UNDERSTANDING THE SOLUTION	9
2.3. SIMILAR PROJECTS	10
2.3.1. Project 1: reNgine	10
2.3.2. Project 2: Pentest-Tools.....	11
2.3.3. Project 3: SOCRadar	12
2.4. COMPARISON TABLE	13
2.5. ANALYSIS AND CONCLUSION OF COMPARISON.....	14
CHAPTER 3: DEVELOPMENT	15

3.1.	CONSIDERED METHODOLOGY	16
3.1.1.	WATERFALL METHODOLOGY	16
3.1.2.	V-MODEL METHODOLOGY	16
3.2.	SELECTED METHODOLOGY	16
3.2.1.	Agile Framework	16
3.3.	SURVEY ANALYSIS	18
3.3.1.	Pre-Survey Analysis	18
3.3.2.	Post-Survey Analysis	18
3.4.	DESIGN	19
3.4.1.	Gantt Chart	19
3.4.2.	Work Breakdown Structure	23
3.4.3.	Milestone	24
3.4.4.	Logo	24
3.4.5.	Wireframe	25
3.4.6.	Flowchart	26
3.4.7.	Use Case Diagram	28
3.4.8.	System Architecture Diagram	31
3.5.	IMPLEMENTATION	32
3.5.1.	Sprint 1: Initial Setup and Tools installation	32
3.5.2.	Sprint 2: Backend API and Architecture Development	37
3.5.3.	Sprint 3: Intelligent Workflow Development	42
3.5.4.	Sprint 4: Frontend Development	48
3.5.5.	Sprint 5: Reporting Module Development	51
3.5.6.	Sprint 6: Integration and Improve	55
CHAPTER 4:	TESTING AND ANALYSIS	63
4.1.	TESTING	64
4.1.1.	Unit Testing, Test Plan	64

4.1.2.	System Testing, Test Plan	65
4.2.	UNIT TESTING	66
4.2.1.	Test Case 1	66
4.2.2.	Test Case 2	67
4.2.3.	Test Case 3	70
4.2.4.	Test Case 4	75
4.2.5.	Test Case 5	77
4.2.6.	Test Case 6	79
4.2.7.	Test Case 7	81
4.2.8.	Test Case 8	83
4.2.9.	Test Case 9	86
4.2.10.	Test Case 10	89
4.2.11.	Test Case 11	91
4.2.12.	Test Case 12	93
4.2.13.	Test Case 13	94
4.2.14.	Test Case 14	95
4.2.15.	Test Case 15	97
4.3.	SYSTEM TESTING	99
	Test Case 1	99
	Test Case 2	111
4.4.	CRITICAL ANALYSIS	119
CHAPTER 5: CONCLUSION		120
5.1.	LEGAL, SOCIAL AND ETHICAL ISSUES	121
5.1.1.	Legal Issues	121
5.1.2.	Social Issues	121
5.1.3.	Ethical Issues	122
5.2.	ADVANTAGES	122

5.3.	LIMITATIONS	123
5.4.	FUTURE WORK	123
CHAPTER 6:	REFERECES	125
CHAPTER 7:	APPENDIX	129
7.1.	APPENDIX 1: PRE-SURVEY.....	130
7.1.1.	Pre-Survey Questions.....	130
7.1.2.	Findings of the pre-survey	134
7.2.	APPENDIX: 2: POST SURVEY	143
7.2.1.	Post-Survey Questions	143
7.2.2.	Post-Survey Responses	148
7.3.	APPENDIX 3: CODES	155
7.3.1.	Sprint 1: Initial Setup and Tools Installation	155
7.3.2.	Sprint 2 (Backend API and Architecture Development) and Sprint 3 (Intelligent Workflow Development).....	169
7.3.3.	Sprint 4: Frontend Development	177
7.3.4.	Sprint 5: Reporting Module Development	180
7.3.5.	Sprint 6: Integration and Improve.....	183
7.4.	APPENDIX 4: DESIGN	185
7.4.1.	Gantt Chart	185
7.4.2.	WORK BREAKDOWN STRUCTURE	191
7.4.3.	MILESTONE	194
7.4.4.	FLOWCHART	197
7.4.5.	SYSTEM ARCHITECTURE DIAGRAM	198
7.5.	APPENDIX 5: SCREENSHOTS OF THE SYSTEM.....	199
7.6.	APPENDIX 6: WATERFALL METHODOLOGY	203
7.7.	APPENDIX 7: V-MODEL METHODOLOGY	204
7.8.	APPENDIX 7: AGILE METHODOLOGY	206

7.9.	APPENDIX 9: SCRUM METHODOLOGY.....	207
7.9.1.	Role of scrum.....	207
7.9.2.	Justification for choosing Methodology.	209
7.10.	APPENDIX 10: FUTURE PLAN	210
7.11.	APPENDIX 11: SOFTWARE REQUIREMENTS SPECIFICATIONS ...	211
7.11.1.	Software Requirements Specifications.....	211
7.12.	APPENDIX 12: PRODUCT BACKLOG	216
8.13.	APPENDIX 13: PROJECT RISKS, THREATS & CONTINGENCY PLAN	217
8.14.	APPENDIX 14: RESOURCES REQUIREMENTS.....	218
8.14.1.	Hardware Requirement	218
8.14.2.	Software Requirement.....	218

TABLE OF FIGURES

Figure 1 Vulnerability Landscape (Alexander Kolesnikov, 2024).	2
Figure 2 Manual Reconnaissance vs IRS	4
Figure 3 Workflow of reNgin.....	10
Figure 4 System Architecture of Pentest Tools	11
Figure 5 System Architecture of SOCRadar.....	12
Figure 6 Comparison between similar projects and IRS.....	13
Figure 7 Scrum Methodology Process (Donetonic, 2025).	16
Figure 8 Gantt Chart.....	20
Figure 9 Gantt Chart.....	21
Figure 10 Detailed Gantt Chart and Progress Table	22
Figure 11 Detailed Gantt Chart and Progress Table	23
Figure 12 WBS.....	23
Figure 13 Milestone.....	24
Figure 14 IRS Logo	24
Figure 15 Dashboard Wireframe	25
Figure 16 System Flow Chart.....	26
Figure 17 System Scheduled Scan Flowchart.....	27
Figure 18 System User Use Case Diagram	28
Figure 19 System User Use Case Diagram	29
Figure 20 System Developer Use Case Diagram.....	30
Figure 21 System Architecture Diagram.....	31
Figure 22 Activation of Virtual Environment and Python3 version check	32
Figure 23 Project Requirements File.....	32
Figure 24 Successful Startup of the Application Server	33
Figure 25 Dependency Installation Script.....	33
Figure 26 Project Settings code and Environment Configuration	34
Figure 27 Configuration for AI and Tools.....	34
Figure 28 OWASP category configuration.....	35
Figure 29 Basic Base Class in Reconnaissance Tools	35
Figure 30 FastAPI Application and API Route Registration.....	36
Figure 31 Asynchronous Processing of Results and Tools	36
Figure 32 Execution and Monitoring of Commands When Running Tools	37

Figure 33 Handling of Timeout and Error Recovery of Tools	37
Figure 34 Normalized Finding Model for Recon Results	38
Figure 35 Setting up the database and managing sessions	39
Figure 36 Initialize Database and Setup Tables Routine.....	39
Figure 37 Scan Model of Reconnaissance Workflow and Result Storage.....	40
Figure 38 Live Status Updates and Error handling Asynchronous Tool Execution ..	40
Figure 39 Tool Output parser, Error Handling and Finalization of Results Workflow	41
Figure 40 Scan Creation and validation Workflow to Tool execution	41
Figure 41 Scan Request Validation and Target Normalization Schema.....	42
Figure 42 Nuclei Timeout Adjustment Based on OWASP Category and Scan Complexity	43
Figure 43 Tool execution and Coordination of Scan	43
Figure 44 Intelligent Nuclei Timeout Calculation and Tool Execution Process	44
Figure 45 AI Decision Module Gemini-Based Tool Selection	44
Figure 46 AI Decision Request and Response parsing workflow	45
Figure 47 Filtering and Deduplication Logic of AI Tool Selection	46
Figure 48 Description of Tools and OWASP Category mapping	46
Figure 49 Gemini Decision Prompt Construction for Recon Tool Selection	47
Figure 50 Domain Checks and Reachability Checks on Scan Targets	47
Figure 51 Scan Target Validation and Host Verification	48
Figure 52 Setting up Templates and Themes	49
Figure 53 Launch Recon Scan Form.....	49
Figure 54 API Routes and API Config	50
Figure 55 Live Updates and Websocket Setup	50
Figure 56 Live Status Tracker and Polling workflow.....	51
Figure 57 Tool List and OWASP Mapping.....	51
Figure 58 Report Generation and Cache Set-up.....	52
Figure 59 Report Generation based on Gemini with Duplicate Filtering and Retry Logic.....	53
Figure 60 Gemini API Retry Logic to generate Reports	53
Figure 61 Gemini backend Fallback and Retry	54
Figure 62 HTML and PDF Generation scan report Endpoints.....	54
Figure 63 Mark Scan as Report Persistence Endpoints and Saved.....	55
Figure 64 Scan Imports and Orchestration Service.....	56

Figure 65 Finding Classification to avoid misclassification	56
Figure 66 Finding Normalization and OWASP Scope Check	57
Figure 67 OWASP Category Lookup and Local Target Check.....	58
Figure 68 AI plan validation and implementation overview.....	59
Figure 69 AI Implementation Quality and Findings Logic	60
Figure 70 Planned Scan Runner and Missed Scan Processing	61
Figure 71 Plan Scan Trigger and Scan Exec Flow	62
Figure 72 Backend Initialization and API Requests Logs	66
Figure 73 IRS Dashboard.....	67
Figure 74 Test Evidence: Select Start date.	68
Figure 75 Test Evidence: Select End Date.....	68
Figure 76 Test Evidence: Click the Apply Button.	69
Figure 77 Test Evidence: Hover mouse to chart to view Findings and Vulnerabilities.	69
Figure 78 Enter Domain and message before Run full Scan was not clicked.	71
Figure 79 Choose OWASP Category, leave tools and clicked Run Full Scan.....	71
Figure 80 Enter Invalid Target Domain.....	72
Figure 81 Validation domain message appeared.	72
Figure 82 Enter a Valid Domain	73
Figure 83 Choose OWASP Category	73
Figure 84 Choose tools for scanning.....	74
Figure 85 Scan Run Successfully and Scan Created message appeared.	74
Figure 86 Navigate to Saved Reports.	75
Figure 87 Filter target with zero.webappsecurity.com	76
Figure 88 Choose OWASP A03:2021	76
Figure 89 Select Status as Running.....	76
Figure 90 Open Scanned Targets page and click Details.	77
Figure 91 Opened Overview of Scans and choose option Delete	78
Figure 92 Click Delete button to delete scan.....	78
Figure 93 Confirm Delete Validation.....	78
Figure 94 Check list of scans	79
Figure 95 Check Update after Scan #57 Delete.....	79
Figure 96 Input Valid domain, OWASP Category and tools, set one time for running scan.....	80

Figure 97 Enable Schedule Toggle and Set Date and Time.	80
Figure 98 Add Schedule Button turns blue after all conditions are met.	81
Figure 99 Opened Settings.	81
Figure 100 Switch to the Light Theme.....	82
Figure 101 Dashboard Light Theme View.....	82
Figure 102 Hover and Clicked Unique Targets.	84
Figure 103 Details of Unique Targets.....	84
Figure 104 Hover and Clicked Targets with Critical.....	84
Figure 105 Detail view for Targets with Critical.	85
Figure 106 Hover to Clean Assets and Clicked Clean Assets.....	85
Figure 107 Detail view for Clean Assets.....	85
Figure 108 Hover to Scheduled Scans and Clicked.	86
Figure 109 Scheduled Scans open directly by clicking Scheduled Scans Card.	86
Figure 110 Open Settings and Click Enable.....	87
Figure 111 Add Email Which Receive Alert.....	87
Figure 112 Alert Email Enabled.....	88
Figure 113 Terminal log after scanning a complete email alert.	88
Figure 114 Gmail Account that received Alert.	88
Figure 115 Scroll to Scanned Target Summary and Click “Select a scan” dropdown.	89
Figure 116 Dropdown displays list of saved scans.....	90
Figure 117 Choose scan loaded with all correct fields.	90
Figure 118 Scan Status Update from Scan Creation to Clue gathering phase.....	91
Figure 119 Scan Status updates in Tools execution phase.....	92
Figure 120 Scan status updates from tools execution to completion.	92
Figure 121 Open Saved Reports and Clicked View Report.....	93
Figure 122 Opened new tab for Report.....	93
Figure 123 Click Quick target search.	94
Figure 124 Type target to view details.....	94
Figure 125 Opened Target Analysis for pentest-ground.com.....	95
Figure 126 Navigate to Tool Library.	96
Figure 127 CLI Command DNSx.....	96
Figure 128 Opened Details and select different scans with same attack type.....	97
Figure 129 Clicked Naabu Slice on chart.	98

Figure 130 Findings details by Naabu.....	98
Figure 131 Start System and Filled Scan Requirement.....	99
Figure 132 Scan Created Successfully	100
Figure 133 Scan created and run in backend.....	100
Figure 134 Initial Clue Gathering Phase by naabu.....	100
Figure 135 Initial Clue gathering phase by httpx	101
Figure 136 AI decision for Tools.....	101
Figure 137 Starting tool execution phase	102
Figure 138 Live Status for Tool run	102
Figure 139 Nuclei run as per AI decision Phase-1	103
Figure 140 Nuclei run Phase 2 & 3	103
Figure 141 Output by Nuclei from all 3 phases	104
Figure 142 Scanned Target Summary, Nuclei Output.....	104
Figure 143 Active tester/Response Analyzer Engine run phase and completed full scan.....	105
Figure 144 Active Testing Report.....	105
Figure 145 Response Analysis Report.....	106
Figure 146 Relevance to Chosen Attack Type	106
Figure 147 Full Scan Summary	107
Figure 148 Completed Green Badge after scanning complete.....	107
Figure 149 Executive Summary	108
Figure 150 Generating PDF	108
Figure 151 Successfully Downloaded Scan pdf	109
Figure 152 Scan Saved.....	109
Figure 153 Saved scan added on Saved Reports	110
Figure 154 Findings list shown on Security Trend Analytics	110
Figure 155 Filled requirement and Schedule Time for scanning	111
Figure 156 Schedule scan added on "Your Schedule"	112
Figure 157 Scan had been triggered	112
Figure 158 Scan appears in "Scheduled scan result".....	112
Figure 159 Tool Execution	113
Figure 160 Tool Execution	113
Figure 161 Tool Execution	114
Figure 162 Tool Execution	114

Figure 163 Tool Execution	115
Figure 164 Final phase of scan	115
Figure 165 Scanning Completed	115
Figure 166 Scan Completed within 4 minutes	116
Figure 167 Saved Report	116
Figure 168 Scan Saved on "Saved Reports"	117
Figure 169 Turn off scan	117
Figure 170 Turned off Successfully	118
Figure 171 Findings shown on "Security Trend Analysis"	118
Figure 172 Scan Summary.....	118
Figure 173 Pre-Survey Form: Personal Detail.....	130
Figure 174 Pre-Survey Form: Question 1.....	130
Figure 175 Pre-Survey Form: Question 2.....	130
Figure 176 Pre-Survey Form: Question 3.....	131
Figure 177 Pre-Survey Form: Question 4.....	131
Figure 178 Pre-Survey Form: Question 5.....	131
Figure 179 Pre-Survey Form: Question 6.....	132
Figure 180 Pre-Survey Form: Question 7.....	132
Figure 181 Pre-Survey Form: Question 8.....	132
Figure 182 Pre-Survey Form: Question 9.....	133
Figure 183 Pre-Survey Form: Question 10.....	133
Figure 184 Pre-Survey Form: Question 11.....	133
Figure 185 Pre-Survey Form: Question 12.....	133
Figure 186 Pre-Survey Form: Response Email ID	134
Figure 187 Pre-Survey Form: Response Emails ID	135
Figure 188 Pre-Survey Form: Response Identity	135
Figure 189 Pre-Survey Form: Response Identity	136
Figure 190 Pre-Survey Form Response: Question 1	136
Figure 191 Pre-Survey Form Response: Question 2	137
Figure 192 Pre-Survey Form Response: Question 3	137
Figure 193 Pre-Survey Form Response: Question 4	138
Figure 194 Pre-Survey Form Response: Question 5	138
Figure 195 Pre-Survey Form Response: Question 6	139
Figure 196 Pre-Survey Form Response: Question 7	139

Figure 197 Pre-Survey Form Response: Question 8	140
Figure 198 Pre-Survey Form Response: Question 9	140
Figure 199 Pre-Survey Form Response: Question 10.....	141
Figure 200 Pre-Survey Form Response: Question 11.....	141
Figure 201 Pre-Survey Form Response: Question 12.....	142
Figure 202 Post-Survey Form: Personal Detail	143
Figure 203 Post-Survey Form: Question 1	143
Figure 204 Post-Survey Form: Question 2	144
Figure 205 Post-Survey Form: Question 3	144
Figure 206 Post-Survey Form: Question 4	144
Figure 207 Post-Survey Form: Question 5	145
Figure 208 Post-Survey Form: Question 6	145
Figure 209 Post-Survey Form: Question 7	145
Figure 210 Post-Survey Form: Question 8	146
Figure 211 Post-Survey Form: Question 9	146
Figure 212 Post-Survey Form: Question 10	147
Figure 213 Post-Survey Form: Question 11	147
Figure 214 Post-Survey Form: Question 12	148
Figure 215 Post-Survey Form: Question 13	148
Figure 216 Post-Survey Form Response: Identity.....	148
Figure 217 Post-Survey Form Responses: Identity	149
Figure 218 Post-Survey Form Responses: Question 1	149
Figure 219 Post-Survey Form Responses: Question 2	150
Figure 220 Post-Survey Form Responses: Question 3	150
Figure 221 Post-Survey Form Responses: Question 4	151
Figure 222 Post-Survey Form Responses: Question 5	151
Figure 223 Post-Survey Form Responses: Question 6	152
Figure 224 Post-Survey Form Responses: Question 7	152
Figure 225 Post-Survey Form Responses: Question 8	153
Figure 226 Post-Survey Form Responses: Question 9	153
Figure 227 Post-Survey Form Responses: Question 10	154
Figure 228 Post-Survey Form Responses: Question 11	154
Figure 229 Post-Survey Form Responses: Question 12	155
Figure 230 Post-Survey Form Responses: Question 13	155

Figure 231 Project Path Configuration and Database URL Setup.....	156
Figure 232 OWASP Top 10 2021 Categories Configuration	156
Figure 233 Installing Python dependencies.....	156
Figure 234 BaseTool Abstract Class Definition and Interface	157
Figure 235 Fast API Swagger page showing the root endpoint	157
Figure 236 Installing Nabbu tool with its go modules	158
Figure 237 Verify Successful Installation.....	158
Figure 238 Installation of nuclei scanner and go	159
Figure 239 Verified installation of nuclei.....	159
Figure 240 Install httpx and verified	160
Figure 241 Install Subfinder	160
Figure 242 Verified installation of subfinder	161
Figure 243 Install Assetfinder with go and fetching dependencies	161
Figure 244 Verified installation of assetfinder.....	162
Figure 245 Install Amass.....	162
Figure 246 Cloning Sublist3r, installing requirements and verified installation	162
Figure 247 Install GAU	163
Figure 248 Verified installation of GAU	163
Figure 249 Install Katana	164
Figure 250 Verified installation of katana	164
Figure 251 Install Gospider	165
Figure 252 Verified installation of gospider	165
Figure 253 Install and Verified ffuf tool.....	166
Figure 254 Install and verified cewl	166
Figure 255 Install and Verified dnsx	167
Figure 256 Installed and Verified Shuffledns.....	167
Figure 257 FastAPI Application Lifespan, Middleware and Router Registration	168
Figure 258 IRS environment configuration (.env) for Gemini and tool settings	168
Figure 259 Fast API and CORS setup	169
Figure 260 Tool Descriptions and OWASP Nuclei Tag Mapping Configuration	170
Figure 261 Scan delete and retry endpoint for managing scans in IRS API.....	170
Figure 262 Scan status, detail, and list endpoints querying IRS scan results	171
Figure 263 Scan Request Schema with Target and OWASP Category Validation	172
Figure 264 Tool run status enum and Tool Run model	172

Figure 265 Asynchronous Database Engine and Session Configuration	173
Figure 266 Scan Service create_scan.....	173
Figure 267 Gather initial clues with Naabu and Httpx.....	174
Figure 268 Execute recon tools and save findings	175
Figure 269 AI decision class and Gemini API call	176
Figure 270 Execute a single tool with timeout and output file.....	176
Figure 271 API Routes Configuration.....	177
Figure 272 API Config Initialization and Same-Origin Handling	177
Figure 273 Live Status Tracker State and Polling Variable	178
Figure 274 Scan Progress Widget and Tracking	178
Figure 275 API Request Handler and URL Builder	179
Figure 276 SSE Connection and Disconnect Handler.....	179
Figure 277 Scan Launch Form HTML Structure.....	180
Figure 278 HTML Report Generation and Cache Check.....	181
Figure 279 Report Validation, Tool Runs and Findings Deduplication	181
Figure 280 Severity Counts and Top Findings Builder	182
Figure 281 AI Executive Summary Generation with Gemini.....	182
Figure 282 AI Conclusion and Remediation Summary Generation	183
Figure 283 Scheduled Scan Imports and Frequency Calculator	183
Figure 284 Due Schedule Runner and Missed Scan Handler	184
Figure 285 Scan Trigger, Schedule Update and Error Handling	184
Figure 286 Project Planning	185
Figure 287 Requirement Analysis	185
Figure 288 Sprint 1.....	185
Figure 289 Sprint 2.....	185
Figure 290 Sprint 3.....	186
Figure 291 Sprint 4.....	186
Figure 292 Sprint 5.....	186
Figure 293 Sprint 6.....	186
Figure 294 Control.....	186
Figure 295 Closure!.....	187
Figure 296 Interim Report Gantt Chart.....	187
Figure 297 Interim Report Gantt Chart.....	188
Figure 298 Interim Report Gantt Chart.....	188

Figure 299 Interim Report Gantt Chart in Detail	189
Figure 300 Updated Gantt Chart in detail.....	190
Figure 301 Previous WBS	191
Figure 302 Updated WBS	192
Figure 303 Updated Work Breakdown Structure	193
Figure 304 Milestone.....	194
Figure 305 Clear System Flow Chart	197
Figure 306 Clear System Architecture Diagram	198
Figure 307 Dashboard of IRS.....	199
Figure 308 Dashboard of IRS.....	199
Figure 309 Dashboard Output Section of IRS	200
Figure 310 Saved Reports of IRS	200
Figure 311 Scanned Targets of IRS	200
Figure 312 Scheduled Scans of IRS	201
Figure 313 Scheduled Scans of IRS	201
Figure 314 Tools & OWASP Mapping	202
Figure 315 Settings of IRS	202
Figure 316 Waterfall Methodology (Welch, 2025).	203
Figure 317 V-Model Methodology (Simran, 2025).....	204
Figure 318 Agile Methodology (Kamis, 2019).	206

TABLE OF TABLES

Table 1 Unit Testing, Test Plan	65
Table 2 System Testing, Test Plan	65
Table 3 Unit Test Case 1.....	66
Table 4 Unit Test Case 2.....	67
Table 5 Unit Test Case 3.....	70
Table 6 Unit Test Case 4.....	75
Table 7 Unit Test Case 5.....	77
Table 8 Unit Test Case 6.....	79
Table 9 Unit Test Case 7.....	81
Table 10 Unit Test Case 8.....	83
Table 11 Unit Test Case 9.....	87
Table 12 Unit Test Case 10.....	89
Table 13 Unit Test Case 11.....	91
Table 14 Unit Test Case 12.....	93
Table 15 Unit Test Case 13.....	94
Table 16 Unit Test Case 14.....	95
Table 17 Unit Test Case 15.....	97
Table 18 System Test Case 1	99
Table 19 System Test Case 2	111
Table 20 Product Backlog	217
Table 21 Table of Software Requirements	219

TABLE OF ABBREVIATIONS

Abbreviations	Full Form
IRS	Intelligence Recon System
CLI	Command Line Interface
SRS	Software Requirements Specification
OWASP	Open Web Application Security Project
WBS	Work Breakdown Structure
AI	Artificial Intelligence
API	Application Programming Interface
URL	Uniform Resource Locator
IP	Internet Protocol
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
HTML	Hypertext Markup Language
CSS	Cascading Style Sheets
JS	JavaScript
PDF	Portable Document Format
DNS	Domain Name System
UI	User Interface
SDLC	Software Development Life Cycle
GDPR	General Data Protection Regulation
SSE	Server-Sent Events
JWT	JSON Web Token

CHAPTER 1: INTRODUCTION

1.1. MOTIVATION

The motivation to develop the system is to integrate various reconnaissance tools into one web-based application, allowing one to scan and report targets through the same interface. It provides easy to understand, standardized vulnerability data to cybersecurity professionals and learners, eliminating the need to rely on complex command-line usage.

1.2. PROJECT DESCRIPTION

Web applications and online services have become part of modern organizations, increasing their attack surface exponentially, and revealing repeated vulnerabilities to misconfigurations, missing access controls, and injections, so automated reconnaissance based on OWASP delineations is required in addition to manual checks (Sumanth, 2023).

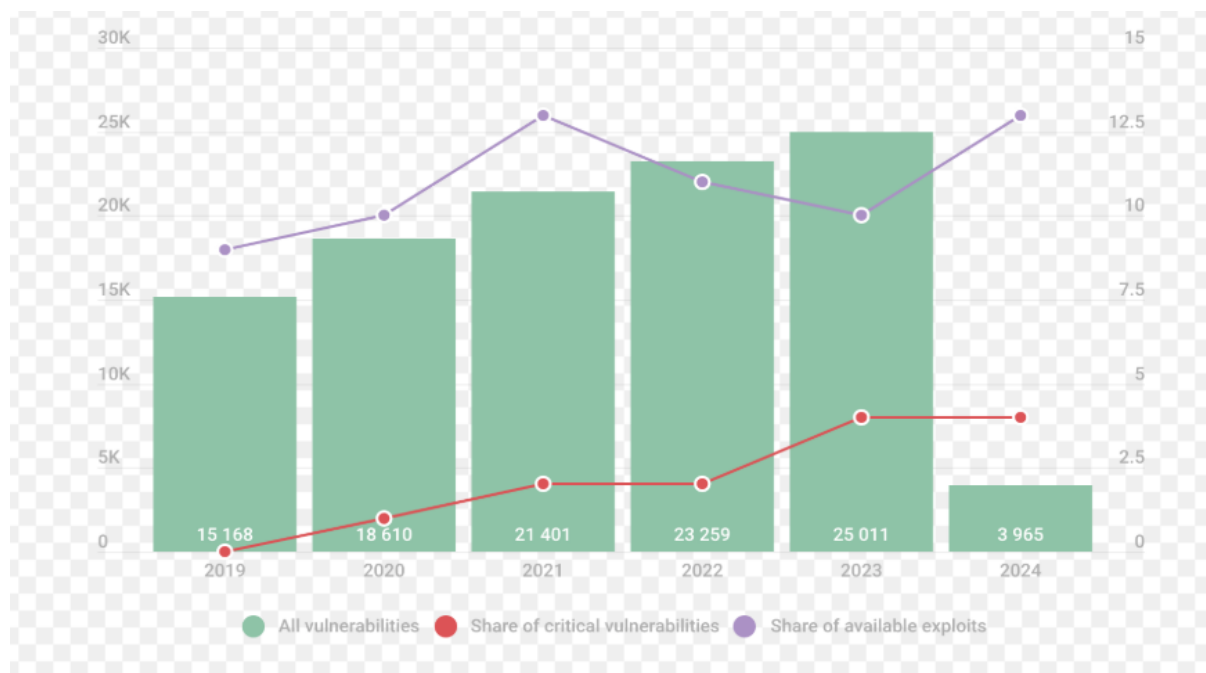


Figure 1 Vulnerability Landscape (Alexander Kolesnikov, 2024).

The IRS is a web-based reconnaissance cybersecurity tool that substitutes manual command-line operations with one AI-supported dashboard. It combines 15 open-source tools including Nuclei, Naabu, Httpx, Subfinder, Amass, FFuf, GAU, DNSx, Katana, Gospider, CeWL, ShuffleDNS, Assetfinder, Sublist3r, and Wfuzz that automate subdomain discovery, port scanning, HTTP probing, endpoint enumeration

and OWASP Top 10 centric vulnerability checks. [Hardware and Software Requirements](#).

IRS is a three-tier application that has an HTML/CSS/JavaScript frontend, Fast API backend, and an execution layer that invokes all CLI tools asynchronously using Python subprocess. Users specify a target domain and make an OWASP Top 10 focus, and then an AI Decision Node driven by Gemini narrows down the scanned tools to only context-relevant scans to minimize redundancy and human error.

Additionally, the system includes an engine layer for active testing and endpoint classification, as well as scheduled scan functionality that allows One-time, daily, weekly, or monthly scheduled scan with chosen tools. The results of scans are normalized and stored and displayed in a single dashboard with running progress and downloadable HTML/PDF reports. Using automated multi-tool reconstruction, AI-based tool selection, and central reporting, IRS allows small security teams and students to conduct an effective and professionally-quality reconnaissance, rather than lumbering through CLI-based reconnaissance. To view a screenshots picture of the system interface and dashboard, see [Appendix 5](#).

1.2.1. PROJECT DELIVERABLES

- **IRS:** The complete system can execute hyper-focused, AI-optimized security assessments via one web screen.
- **Unified Interface and Monitoring Dashboard:** This allows analysts to manage all security scans, track real-time progress, and review aggregated findings from a single central hub.
- **Structured Output and Persistent Results:** It transforms raw data so that the user can easily analyze the standardized results and securely store all the historical data for long-term review.
- **Security Tool Execution Layer:** This layer runs the 15 in-built tools, enabling the system to execute a complete and powerful suite of security checks instructed by the AI.
- **Downloadable Standardized Report:** Users can generate professional, ready-to-share HTML reports for audits, documentation, or incident review.

1.3. CURRENT SCENARIO

As of today, security professionals performing web reconnaissance operations use a combination of CLI based tools chained together in succession, which is rather tedious and largely relies on personal experience (Fouz Barman, 2023). The time required to collect and analyze data by manual reconnaissance may require days or weeks and is not suitable in a fast-paced security environment. According to the results of Armis Research, the average number of tools that organizations are utilizing to manage security assessment is 11, though 44% continue to use manual means, exposing large gaps in visibility across their attack surface (Simpson, 2023). A 2023 study by the Armis Security organization demonstrated that decentralization of security practices compromises uniformity and effectiveness, and organizations cannot define and prioritize the most important assets (Simpson, 2023). Security Boulevard Point security solutions introduce blind spots and are uncontrollable, and a consolidated platform because of a single platform allows sharing data more effectively and remediate automatically across all the security architecture of an organization (Aguar, 2024).

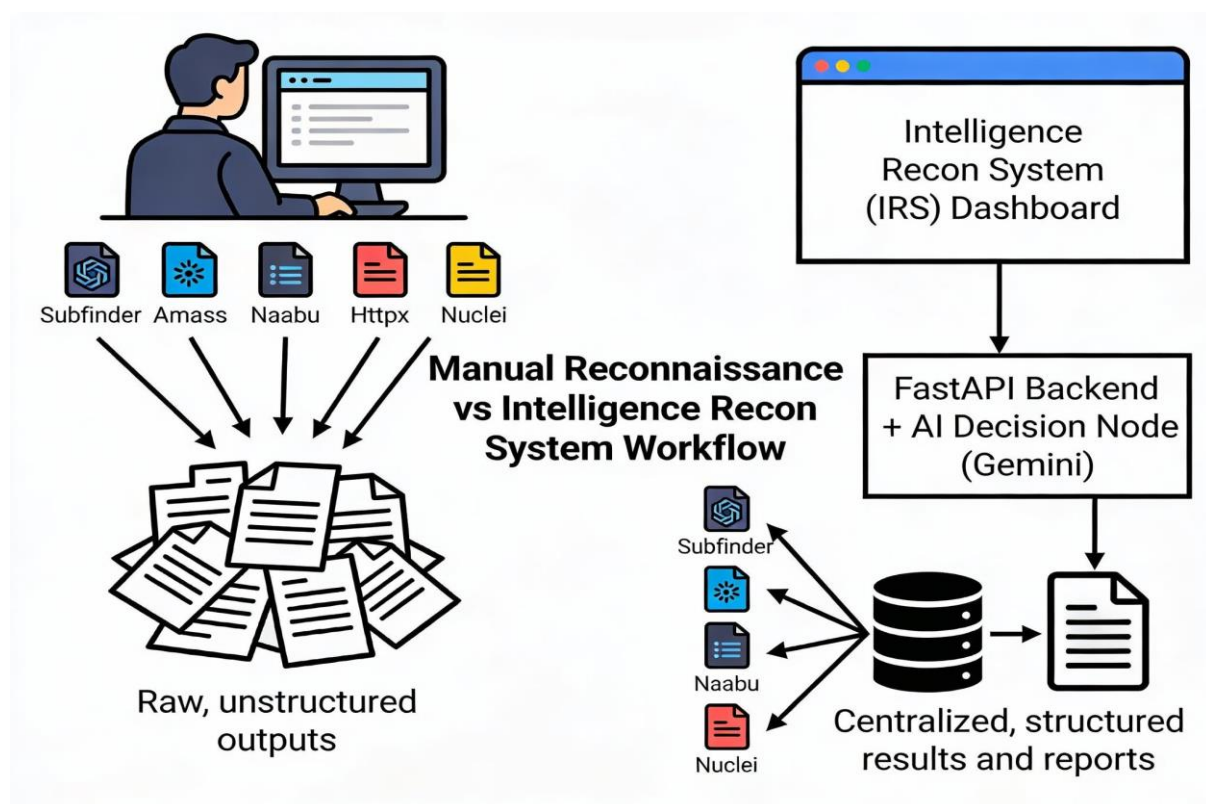


Figure 2 Manual Reconnaissance vs IRS

1.4. PROBLEM DOMAIN AND SOLUTION

1.4.1. Problem Domain

1. **Manual and Time Consuming:** Manual recon is time-consuming, failing to keep up with changing cyber threats and demand of timely security opinions. (Sumanth, 2023).
2. **Human Error and Bias:** Even professional ethical hackers tend to make errors, such as failing to interpret information or not detecting vulnerability. False alarms or missing threats can be caused by unconscious bias.
3. **Manual and Fragmented Execution:** Recon involves chaining numerous manual CLI tools. This fragmentation wastes time in complicated syntax and leads to inefficiency and inconsistency in the outcomes (Ariganello, 2022).
4. **Absence of Centralized Management and Professional Reporting:** Scan results are distributed locally, thus making it difficult to retrieve the data and track its history. Report generation is slow and manual, and prone to errors.

1.4.2. PROJECT AS SOLUTION

- **Removes Manual Time Waste:** The system automates the whole working system allowing the security analysis to take place in a fast and timely manner, even in extensive settings.
- **Reduces Minimizes Error and Bias:** The inclusion of AI Decision Node makes the tool selection objective and optimized, which reduces human misinterpretation or bias when making triage decisions.
- **Automated and Parallel execution:** This system incorporates the specialty of CLI tools into one scalable framework, permitting automated and parallel chaining to conquer the disjointed and manual execution.
- **Centralized Management and Reporting:** Data is gathered, normalized, and stored continuously, to give consistent and structured outputs to real-time analysis and the creation of professional reports to the stakeholders with a single click.

1.5. AIM AND OBJECTIVES

1.5.1. Aim

The Aim of the IRS is to create, design, and thoroughly test an integrated, unified, and smartly coordinated security platform that automates the multi-step, multi-stage reconnaissance process of professional CLI programs, and thus replaces manual and fragmented vulnerability testing with automated, centralized, and report-enabled security testing based on the OWASP Top 10 threats.

1.5.2. Objectives

To achieve the project goal, the following objectives that need to be achieved:

- Created a strong, 3-tier (Frontend, Fast Api Backend, and Tool Execution) web and Python-based platform.
- Develop and implement one easy-to-use web dashboard to minimize inputs, provide real time response, and enable OWASP-oriented target and tool selection.
- Have asynchronous execution with dynamic chaining of different CLI tools with a reliable data flow and a responsive non-blocking user interface.
- Add an AI Decision Node that is used to optimize the tool execution plan contextually by smartly utilizing the possibility of excluding irrelevant or redundant scanning checks.
- Create an effective data parsing component to process all raw and inconsistent tool output and provide this in a centralized, consistent, and structured format.
- Apply persistent storage to provide all historical scan results enabling users to monitor the history of asset discovery and vulnerability.
- Offer a real-time web dashboard with scans and the capacity to create and download professional, standardized PDF/HTML reports.
- Study and contrast the performance of the IRS to conventional manual CLI reconnaissance processes regarding scan time, checks redundancy, and coverage of OWASP consistent findings.
- Prepare and assess the system as a structured technical reporting with background research, reviewing of similar projects, architectural design, development progress and testing results to show the feasibility and learning effectiveness of the IRS platform.

1.6. STRUCTURE OF REPORT

1.6.1. BACKGROUND

This subsection introduces the IRS and explains why traditional, manual CLI based reconnaissance is inefficient and fragmented. It motivates integrating multiple tools into a single OWASP aligned platform. The target users are students, small security teams, and bug bounty hunters are briefly characterized..

1.6.2. DEVELOPMENT

This subsection explains the three-tiered architecture system frontend, Fast API backend, and tool execution layer. It describes the way Python subprocesses and asynchronous calls coordinate approximately 15 built in CLI tools. The features of the AI Decision Node workflow, unified dashboard, organized outputs, and report generation are also described.

1.6.3. TESTING AND ANALYSIS

This part discusses unit and system testing performed on the IRS. Unit test cases were performed using the actual domain permitted as the test domain, confirming fundamental features such as scan execution, input validation, live status monitoring, report management, and email notifications. Every finalized test case yielded positive outcomes

1.6.4. CONCLUSION

In this subsection, the IRS is compared to established legal, social, and ethical frameworks such as GDPR, the Electronic Transactions Act 2063 of Nepal, Privacy Act 2075, and ISO/IEC 27001. It fairly evaluates the strengths and weaknesses of the system in fulfilling its responsibilities and where loopholes still exist, specifically in the areas of authorization, transport security, and AI transparency. Suggestions on how to solve such gaps are provided as a roadmap towards the subsequent development process.

CHAPTER 2: BACKGROUND

2.1. TARGET CLIENT

The primary target clients for IRS are designed for three main types of users:

- The students and learners of cybersecurity who need to train on the actual reconnaissance and OWASP-based testing but have access to the help of a simple web dashboard and AI rather than complicated terminal routines.
- Small security teams in small companies, colleges or labs that do not have big commercial platforms though they require automated and consistent recon with centralized results and easy reports.
- Single security analysts/bug-bounty hunters that need to accelerate multi-tool recon, lessen manual and human error, and store history of their scans in a single location.

2.2. UNDERSTANDING THE SOLUTION

Security reconnaissance commonly involves several tools and the analysis of multiple outputs by hand and tools. This may be a lengthy and complicated process. IRS is a solution to this issue because it offers a one-stop web-based platform that makes the whole process of reconnaissance and vulnerability assessment faster and more organized with the help of AI.

The system has a definite scan procedure. The user types a target, which could be a (domain, IP or URL), select an OWASP Top 10 category and tools. IRS performs initial scanning of basic tools such as Naabu and Httpx to gather clues, such as ports open, running services, and technologies. This is analyzed by an artificial intelligence system that runs on Google Gemini, which will pick the most appropriate tools to scan further without irrelevant steps and enhances efficiency.

The chosen tools are executed simultaneously and their results translated into organized findings with information such as type, severity, location, and evidence. The scan is shown as live and the results saved to be used later. Once finished, IRS creates an understandable and clear report in HTML or PDF format, displaying findings by severity and OWASP category and recommended solutions, which simplifies the entire process, results in efficiency and makes it easy to comprehend.

2.3. SIMILAR PROJECTS

2.3.1. Project 1: reNgin

Author: Yogesh Ojha

URL: <https://github.com/yogeshojha/rengine>

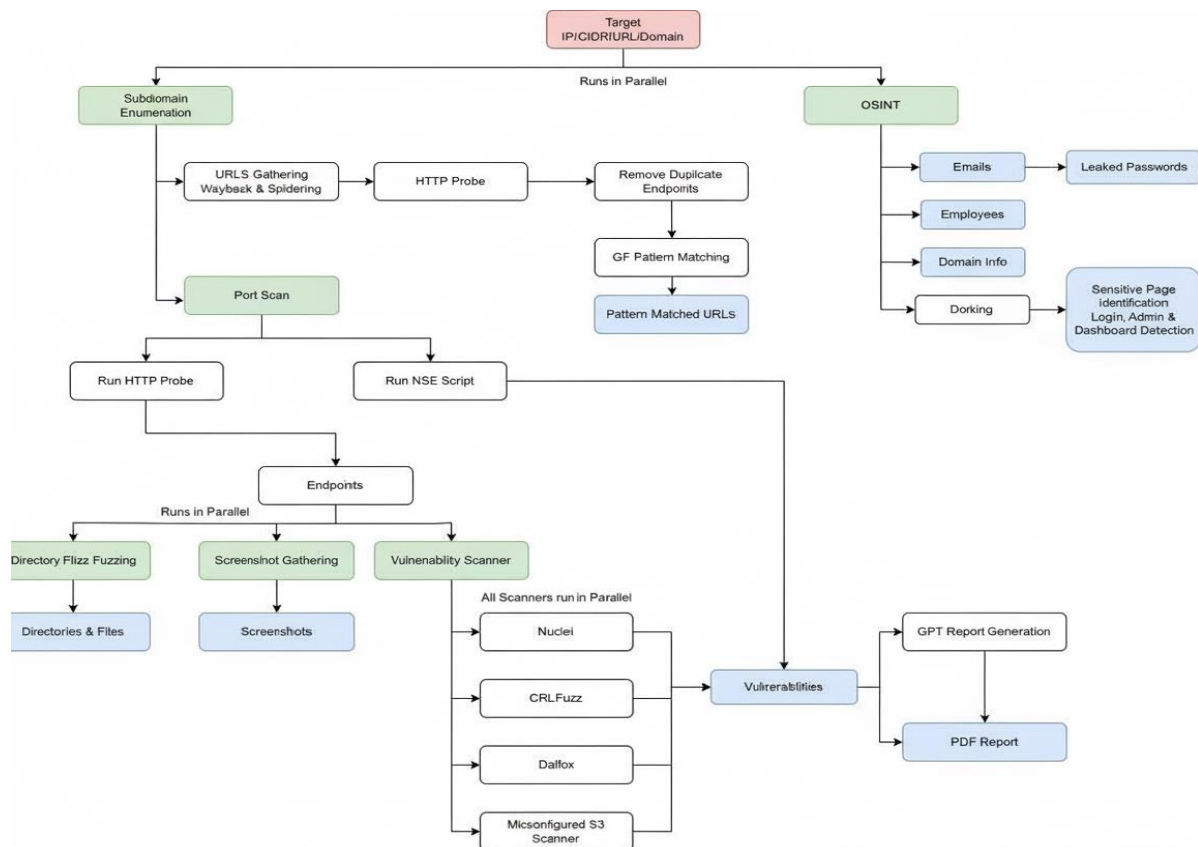


Figure 3 Workflow of reNgin

Description: reNgin is an open-source web-based framework that will automate reconnaissance of bug hunters and penetration testers attending bug bounties. It allows them to specify recon profiles and targets via a web UI and executes a large set of popular tools in the background. The platform will carry out subdomain discovery, DNS resolution, port scanning, HTTP probing, screen shots and vulnerability scanning within one workflow. The data obtained is all mined, standardized and stored in the central database that is easily analyzed. Each target has dashboards displaying subdomains, open ports, technologies, and screenshots and identified vulnerabilities. This assists security professionals in saving on time and ensuring uniform recons in numerous engagements (Ojha, 2025).

2.3.2. Project 2: Pentest-Tools

Author: Pentest-Tools Team

URL: <https://pentest-tools.com/>

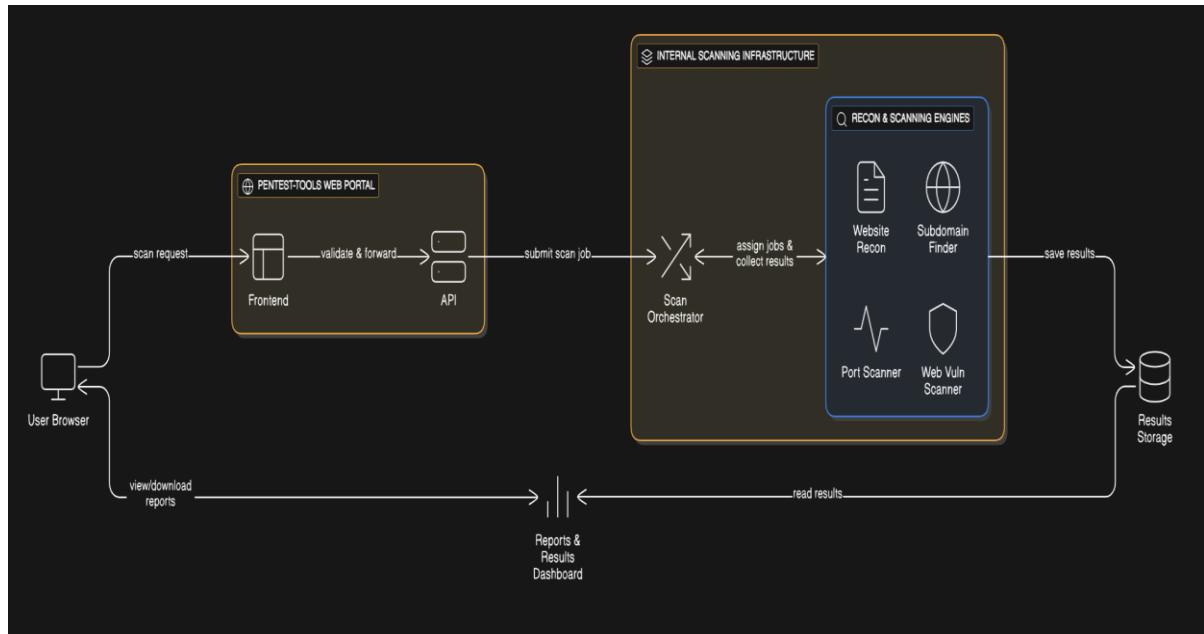


Figure 4 System Architecture of Pentest Tools

Description: Pentest -Tools.com is a commercial web-based service that provides a set of reconnaissance and vulnerability-scanning tools that are available via browser. Its modules are web reconnaissance, subdomain finder, port scanner, technology fingerprinting, and web vulnerability scanners. One just needs to enter a target URL or domain and choose one or a few tools; the scans are performed on their own infrastructure, and the results are displayed in a single report. The tool of recon of the site, e.g., determines technologies, HTTP headers, files of interest, and misconfigurations. This will reflect your IRS idea of having many checks running on one web dashboard rather than per tool of the CLI (Team, 2025).

2.3.3. Project 3: SOCRadar

Author: SOCRadar Cyber Intelligence Inc.

URL: <https://socradar.io/labs>

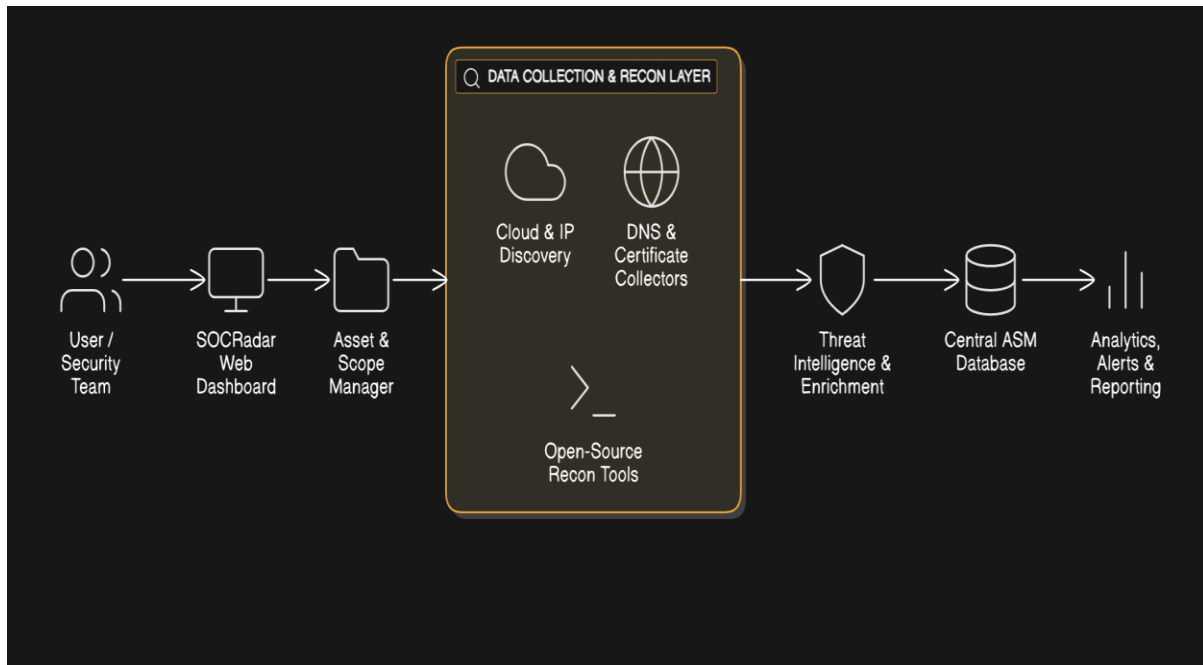


Figure 5 System Architecture of SOCRadar

Description: SOCRadar is a business tool, which integrates External Attack Surface Management (EASM), digital risk protection, and threat intelligence. It is constantly identifying and scanning internet-facing resources of the organization's domains, IPs, cloud computing, exposed services and matching them with threat-intelligence feeds. The system applies automated reconnaissance, open-source tools as well as its own data sources and identifies newly exposed services, misconfigurations, and vulnerabilities. There is a central dashboard with assets, findings, risk scores, and alerts to assist security teams in prioritizing remediation (SOCRadar, 2025).

2.4. COMPARISON TABLE

Features	P1	P2	P3	IRS
Web-based dashboard	Yes	Yes	Yes	Yes
Automated multi-tool scanning	Yes	Partly (per tool)	Yes	Yes
Recon focused on web targets and domains	Yes	Yes	No	Yes
Visibility of all external organization assets	No	No	Yes	No
AI-generated summary and findings report	No	No	No	Yes
Ability to keep and review past scan results	Yes	Yes	Yes	Yes
Assessment guided by OWASP Top-10 risks	No	No	No	Yes
Requires installation on your own server	Yes	No	No	Yes
Easy to learn and use	Medium (setup is harder)	Yes	Yes	Yes
Continuous / repeat scans	Yes	Yes	Yes	Yes

Figure 6 Comparison between similar projects and IRS

2.5. ANALYSIS AND CONCLUSION OF COMPARISON

IRS features are also affected by the three reference tools. Whereas reNgin is about the ability to highly configure reconnaissance pipelines to technical users, IRS is about making automated multi-tool recon simpler by a guided web interface and a tool selection system with AI to help keep small teams undertaking professional-grade reconnaissance with less manual effort. Similarly to the case when Pentest-tools.com has a cloud dashboard to run a scan of a web-site and view clear vulnerability-reports, IRS takes the concept of a single web-based dashboard with multi-target scans, live progress, and standardized HTML/PDF report, which is implemented using open-source elements that may be used in a lab and student setup.

The principle of SOCRadar to centralize the discovered assets and exposures over time also influences IRS, which maintains the results of recon and assets in persistent storage to allow users to revisit targets and review the past results, albeit at a smaller scale than the enterprise attack-surface-management systems. Unlike these tools, IRS has an AI decision node to minimize human bias when selecting a tool and a parsing layer which transforms various command-line outputs into a single standard format to enhance analysis and reporting.

These systems have several features common to IRS but also encompass several sophisticated features like internet-wide asset discovery, rich external threat-intelligence feeds, and highly automated remediation workflows not yet available in IRS but potentially available in future editions. Therefore, IRS is a hybrid between reNgin, Pentest-Tools.com and SOCRadar powered by AI-driven organization, formatted outputs, and a minimal web structure, thus being a convenient reconnaissance tool to use among students and small security teams.

CHAPTER 3: DEVELOPMENT

3.1. CONSIDERED METHODOLOGY

In IRS, the [risk mitigation](#) required the proper choice of SDLC which has led to the option of both Waterfall and Agile approaches.

3.1.1. [WATERFALL METHODOLOGY](#)

3.1.2. [V-MODEL METHODOLOGY](#)

3.2. SELECTED METHODOLOGY

3.2.1. Agile Framework

Agile project management is chosen to be used in the IRS as it allows iterative development, constant feedback and allowing flexibility in case of change in requirements or technical details. [Learn more](#)

3.2.1.1. Scrum Methodology

Scrum Methodology is officially chosen to be the project management tool used in this IRS project. This option has been informed by the fact that it requires a very flexible strategy in dealing with challenging regulatory requirements and a fast-changing specification. [Learn more](#)

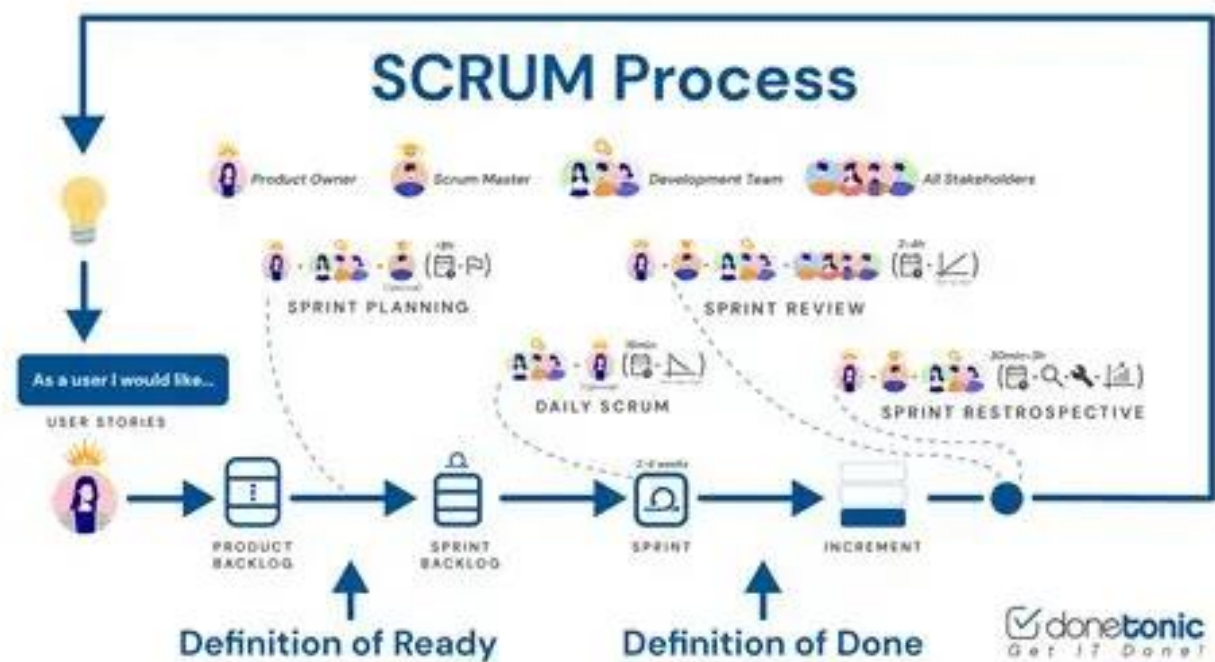


Figure 7 Scrum Methodology Process (Donetonic, 2025).

In the case of IRS, Scrum divides the development process into the following phases:

- **Initiation phase:** Establish project objectives, scope, key features, and resources needed (backend, tools, AI, and storage).
- **Planning and estimate phase:** Divide the project into manageable sprints, develop a [product backlog](#), and create a story point assignment on tasks to determine effort and priority.
- **Implementation phase:** Within every sprint, finish tasks that have been selected and provide a working bit of IRS. The progress of the project is ongoing.
- **Review and retrospective phase:** During a review with the supervisor, after every sprint, assess the progress made, collect feedback, and discuss problems to make improvements in future sprints.
- **Release:** Each set of sprints should be followed by a stable increment (e.g., a complete recon workflow with report) and ensure that it is to expectations.

3.2.1.2. Justification for Choosing Methodology

- Scrum is suitable for increasing changes and involves adapting to changes with repeated sprints, as reflected in the [Appendix 11](#)
- The development is carried out with Sprint so that the delivery of working features is made continuously.
- Constant feedback enhances quality, and it is facilitated by Scrum practices.

[Details.](#)

3.3. SURVEY ANALYSIS

3.3.1. Pre-Survey Analysis

The IRS pre-survey was carried out on a varied sample of 13 individuals who included final year cybersecurity students in Islington College. The major task was to find out the current pain points in the reconnaissance stage of Vulnerability Assessment. Its results showed the following:

- **Existing Workflow Gaps:** Although the intermediate level of experience in using web reconnaissance tools was the most common result (53.8% of respondents), they all (100%) agreed that a system that would automate the process of scanning and reporting would be very advantageous.
- **Manual Effort Bottleneck:** Most respondents observed that the current tools were disjointed and required manual data correlation. The respondents affirmed that a built-in, web-based system would address the issue of data silo that is prevalent in manual recon.
- **Challenges of Infrastructure:** According to the poll, most of the respondents are currently using various separate CLI tools, which makes their operation inefficient when handling massive assessments.
- **Industry Expectation:** The respondents were keen on having a tool that would not only collect information but also present it in a manner that will aid in giving priority to remediation.

Based on the pre-survey responses, it was realized that unquestionably, there is a necessity for an automated system such as IRS that would help fill the gap between tool orchestration and professional reporting.

Details of survey are placed in [Appendix 1](#)

3.3.2. Post-Survey Analysis

The IRS was demonstrated thoroughly after which, a post-survey was carried out on the same 13 participants to assess the effectiveness and professional usefulness of the system. The reception was very positive:

- **Operation Support:** More than 61% of the respondents rated the system as Well or Very Well in terms of supporting real world security workflows.

- **Integration and Efficiency:** The key aspect of the system, which is the combination of various scanners into one and a single report, was very successful as 69.3 percent of participants agreed that the system enhances efficiency when it comes to evaluating security.
- **Reporting Quality:** According to the output, 69.3 percent of respondents rated the reports created (Executive Summary and Detailed Reconnaissance) as either excellent or as good with few gaps, which is an indicator that the tool is approaching the professional level.
- **Adoption and Use Cases:** 61.6% of the participants indicated that they were likely or very likely to use the system in a professional situation, e.g. to provide internal assessments or to engage with clients.
- **Impact on Stakeholders:** A substantial 38.5% of respondents felt that the greatest value was going to be to Security Teams and SOC's, the rest of the respondents felt the value of training, pre-deployment checks.
- **Expectation Alignment:** 76.9% of those interviewed said that the system either was or surpassed their expectations regarding a contemporary reconnaissance and reporting system.

At the end of the post-survey, the IRS was found to be effective in dealing with fragmentation of current tools. The basic functionality is strong, but the users have proposed to improve scalability and expand the range of tools in the future, which will make it better fit large-scale manufacturing processes.

Details of survey are placed in [Appendix 2](#)

3.4. DESIGN

Designing the development workflow is crucial before building the IRS application. Knowing characteristics in advance makes it easy to incorporate planned scans and tool execution. Wireframe, Flowchart, Use Case Diagram and System Architecture have been developed which offer finer blueprints of the cybersecurity platform.

3.4.1. Gantt Chart

A Gantt chart is a horizontal bar chart that is utilized to represent a project on a timeline. Every task is presented as a bar where the beginning of work is displayed, the duration of the work is indicated, the due date is shown, and the percentage of

fulfillment is indicated (LaPrad, 2025). For Previous Gantt Chart you can view [Appendix 4](#) . Gantt Chart was switched due to some [reasons](#). The following Gantt Chart shows the activities of IRS project with their respective timeline.

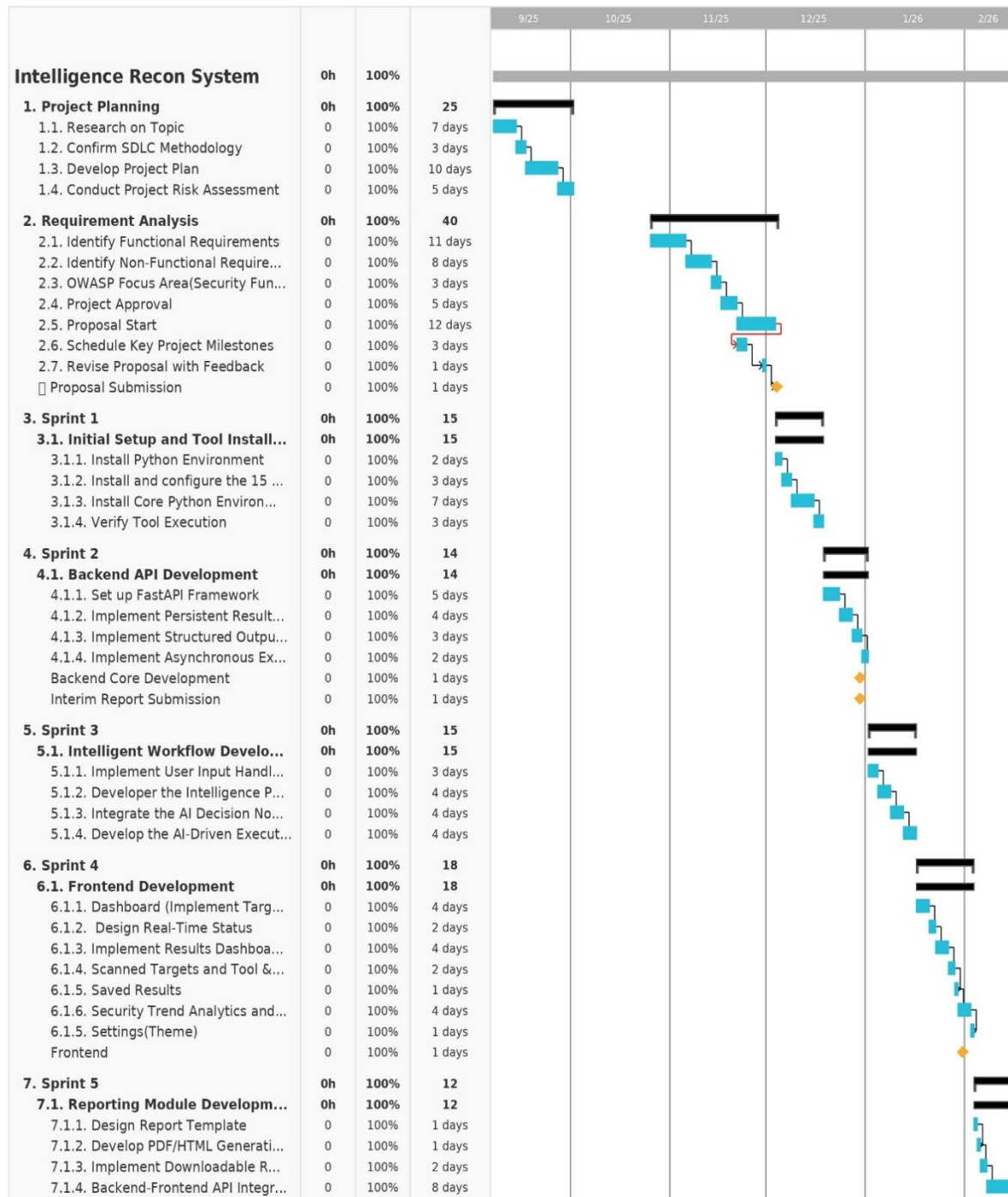


Figure 8 Gantt Chart

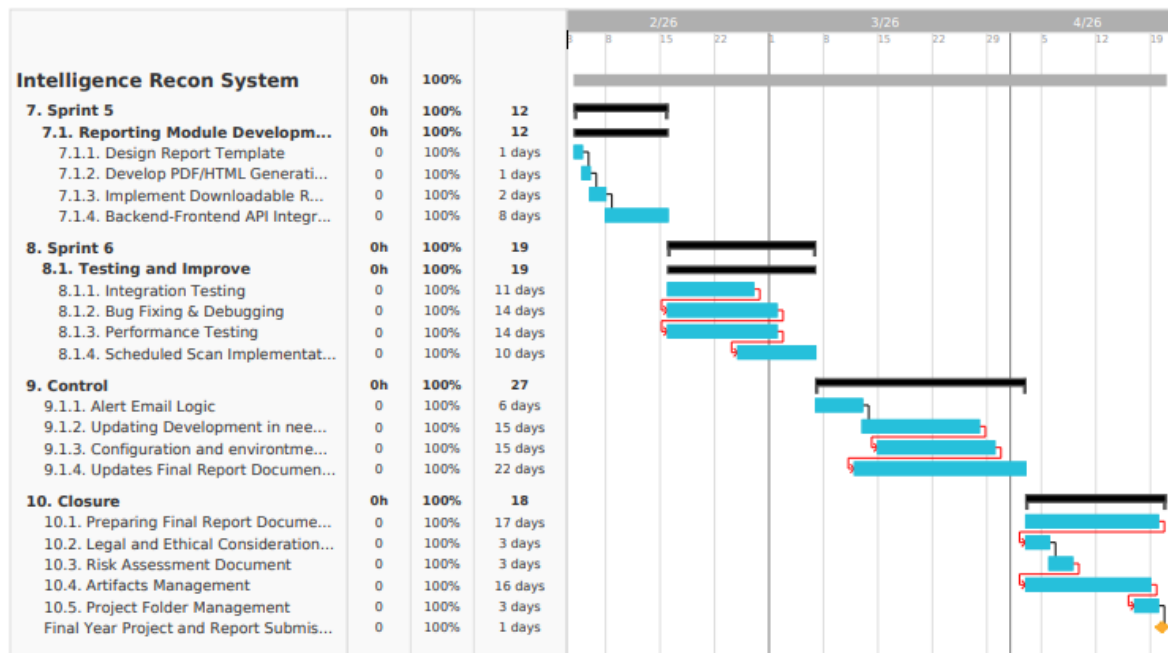


Figure 9 Gantt Chart

Tasks	Start Date	End Date	Status
Project Planning	9/7/2025	10/2/2025	Completed
Research on Topic	9/7/2025	9/13/2025	Completed
Confirm SDLC Methodology	9/14/2025	9/26/2025	Completed
Develop Project Plan	9/17/2025	9/27/2025	Completed
Conduct Project Risk Assessment	9/28/2025	10/2/2025	Completed
Requirement Analysis	10/26/2025	12/3/2025	Completed
Identify Functional Requirements	10/26/2025	11/5/2025	Completed
Identify Non-Functional Requirements	11/6/2025	11/13/2025	Completed
OWASP Focus Area (Security Fundamentals)	11/14/2025	11/16/2025	Completed
Project Approval	11/17/2025	11/21/2025	Completed
Proposal Start	11/22/2025	12/3/2025	Completed
Schedule Key Project Milestone	11/22/2025	11/24/2025	Completed
Revised Proposal with feedback	11/30/2025	11/30/2025	Completed
Proposal Submission	12/3/2025	12/3/2025	Completed
Sprint 1	12/4/2025	12/18/2025	Completed
Initial Setup and Tool Installation	12/4/2025	12/4/2025	Completed
Install Python environment	12/4/2025	12/5/2025	Completed
Install and configure the 15 Open-Source Security Tools	12/6/2025	12/8/2025	Completed
Install Core Python Environment (Fast API, Subprocess, Gemini API)	12/9/2025	12/15/2025	Completed
Verify Tool Execution via Python Scripts	12/16/2025	12/18/2025	Completed
Sprint 2	12/19/2025	1/1/2026	Completed
Backend API and Architecture Development	12/19/2025	1/1/2026	Completed
Set up Fast API Framework & Core Logic	12/19/2025	12/23/2025	Completed
Implementing Persistent Results Data Storage	12/24/2025	12/27/2025	Completed
Implement Structured Output Logic & Parsers	12/28/2025	12/30/2025	Completed
Implement Asynchronous Execution Logic	12/31/2025	1/1/2026	Completed
Backend Core Development	12/30/2025	12/30/2025	Completed
Interim Report Submission	12/30/2025	12/30/2025	Completed
Sprint 3	1/2/2026	1/16/2026	Completed
Intelligent Workflow Development	1/2/2026	1/16/2026	Completed
Implement User Input Handler (Domain, OWASP Focus)	1/2/2026	1/5/2026	Completed
Develop the Intelligent Pre-Check (Naabu, Httpx)	1/6/2026	1/10/2026	Completed
Integrate the AI Decision Node (Gemini API)	1/10/2026	1/14/2026	Completed
Develop the AI-Driven Execution Validation Logic	1/14/2026	1/16/2026	Completed
Sprint 4	1/17/2026	2/3/2026	Completed
Frontend Development	1/17/2026	2/3/2026	Completed
Dashboard (Implement Target/OWASP Attack/Tool Selection Forms)	1/17/2026	1/20/2026	Completed
Design Real-time Status	1/21/2026	1/22/2026	Completed
Implement Results Dashboard Display	1/23/2026	1/26/2026	Completed
Scanned Targets and Tools & OWASP	1/27/2026	1/28/2026	Completed
Saved Reports	1/29/2026	1/29/2026	Completed
Security Trend Analytics and Severity Distribution	1/30/2026	2/2/2026	Completed
Settings (Theme)	2/3/2026	2/3/2026	Completed
Frontend	1/31/2026	1/31/2026	Completed

Figure 10 Detailed Gantt Chart and Progress Table

Sprint 5	2/4/2026	2/15/2026	Completed
Reporting Module Development	2/4/2026	2/15/2026	Completed
Design Report Template	2/4/2026	2/4/2026	Completed
Develop PDF/HTML Generation Logic	2/5/2026	2/5/2026	Completed
Implement Downloadable Report Feature	2/6/2026	2/7/2026	Completed
Backend-Frontend API Integration	2/8/2026	2/15/2026	Completed
Sprint 6	2/16/2026	3/6/2026	Completed
Integration and improve	2/16/2026	3/6/2026	Completed
Integration Testing	2/16/2026	2/26/2026	Completed
Bug Fixing & Debugging	2/16/2026	3/1/2026	Completed
Performance Testing	2/16/2026	3/1/2026	Completed
Scheduled Scan Implementation	2/25/2026	3/6/2026	Completed
Control	3/7/2026	4/2/2026	Completed
Alert Email Logic	3/7/2026	3/12/2026	Completed
Updating Development in needed parts	3/13/2026	3/27/2026	Completed
Configuration and environment management	3/15/2026	3/29/2026	Completed
Updates Final Report Documentation	3/12/2026	4/2/2026	Completed
Closure	4/3/2026	4/22/2026	Completed
Preparing Final Report Documentation	4/3/2026	4/18/2026	Completed
Legal and Ethical Considerations Report	4/3/2026	4/5/2026	Completed
Risk Assessment Document	4/6/2026	4/8/2026	Completed
Artifacts Management	4/3/2026	4/18/2026	Completed
Project Folder Management	4/13/2026	4/18/2026	Completed
Final Year Project and Report Submission			Completed

Figure 11 Detailed Gantt Chart and Progress Table

3.4.2. Work Breakdown Structure

The WBS illustrated below separates the project into distinct phases and tasks, facilitating the organization, scheduling, and management of all work required for successful completion (Manager, 2025). [Clear View](#)

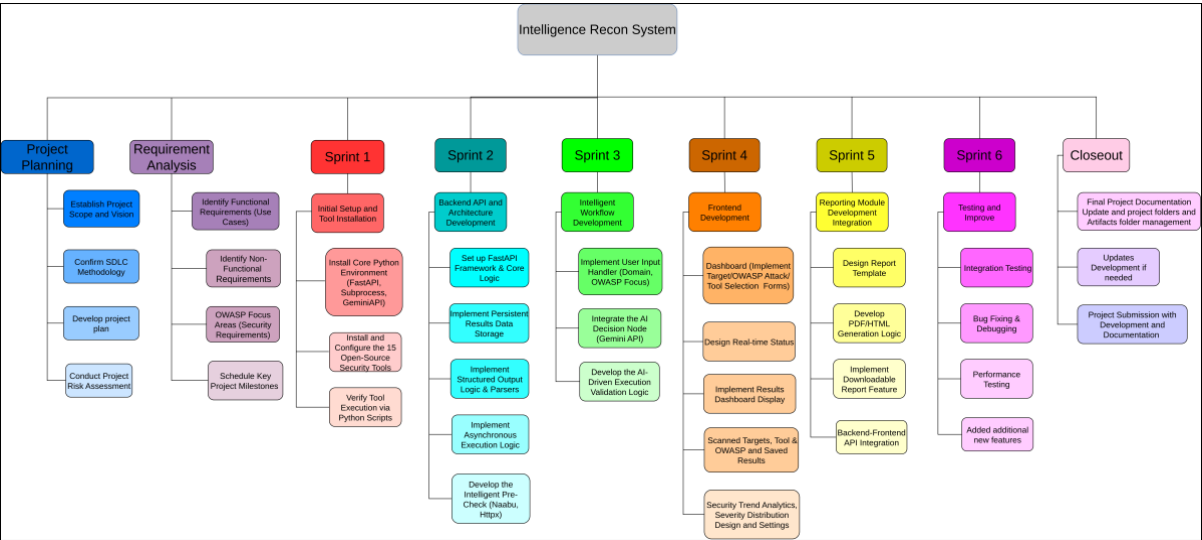


Figure 12 WBS

3.4.3. Milestone

The project milestone is a reference or benchmark to showcase a significant goal, event, or task in a project lifecycle (Staff, 2025). The following figure is the milestone of the IRS project which gives clear directions of the project in the long run.

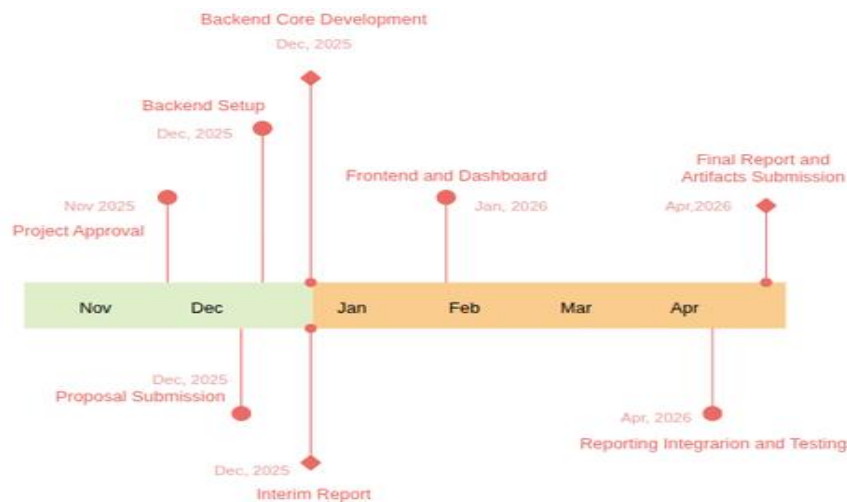


Figure 13 Milestone

[Clear view](#)

3.4.4. Logo

The logo was designed to represent the identity and purpose of the system. The logo reflects the main idea of cybersecurity and system protection, and it is in the shape of circle and a circle and a shield, which can be interpreted as a symbol of security and stability.

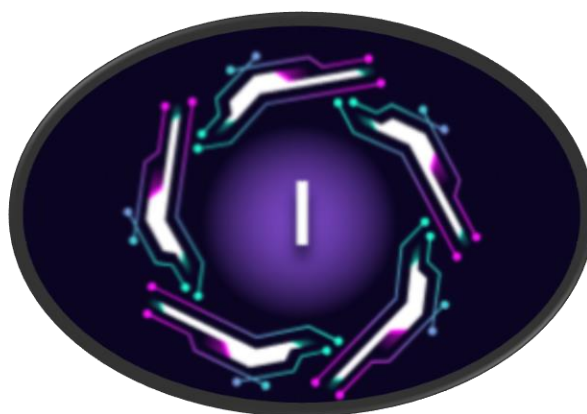


Figure 14 IRS Logo

3.4.5. Wireframe

A wireframe is the frame of your online project. Consider it as the basis of your site, application, or dashboard. It concerns layout, and content positioning, but not colors, fonts, or visual refinement (Guilizzoni, 2025). The wireframe was designed as visual blueprint to define the dashboard, it outlines the placement of scan input fields, output section, saved reports and charts.

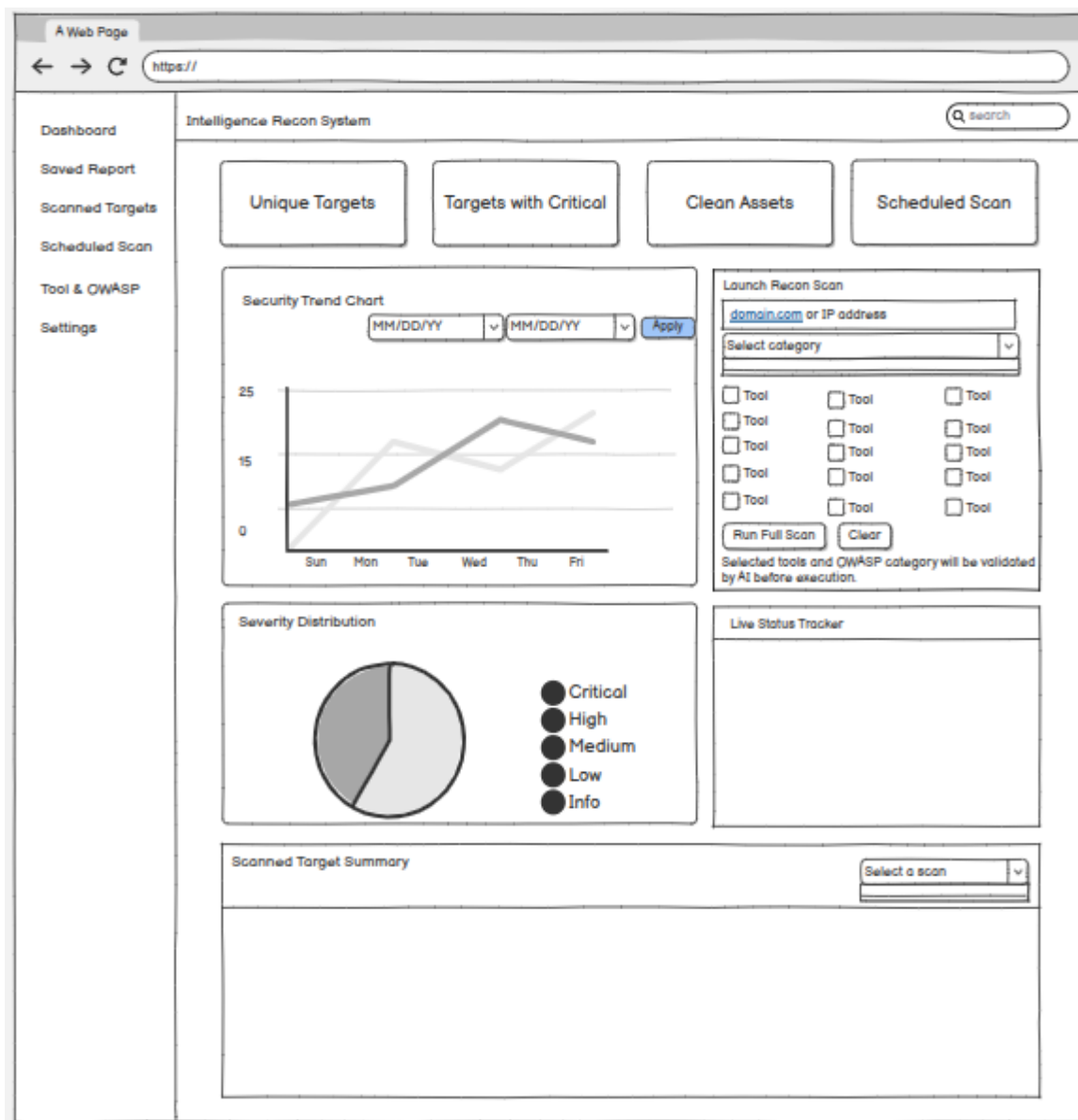


Figure 15 Dashboard Wireframe

3.4.6. Flowchart

A flow chart of this project is a step-by-step diagram, which displays the sequence of tasks or processes the system undertakes in a recon scan, such as common symbols as a rectangle (process), a diamond (decision), and a control flow arrow. [Clear View](#).

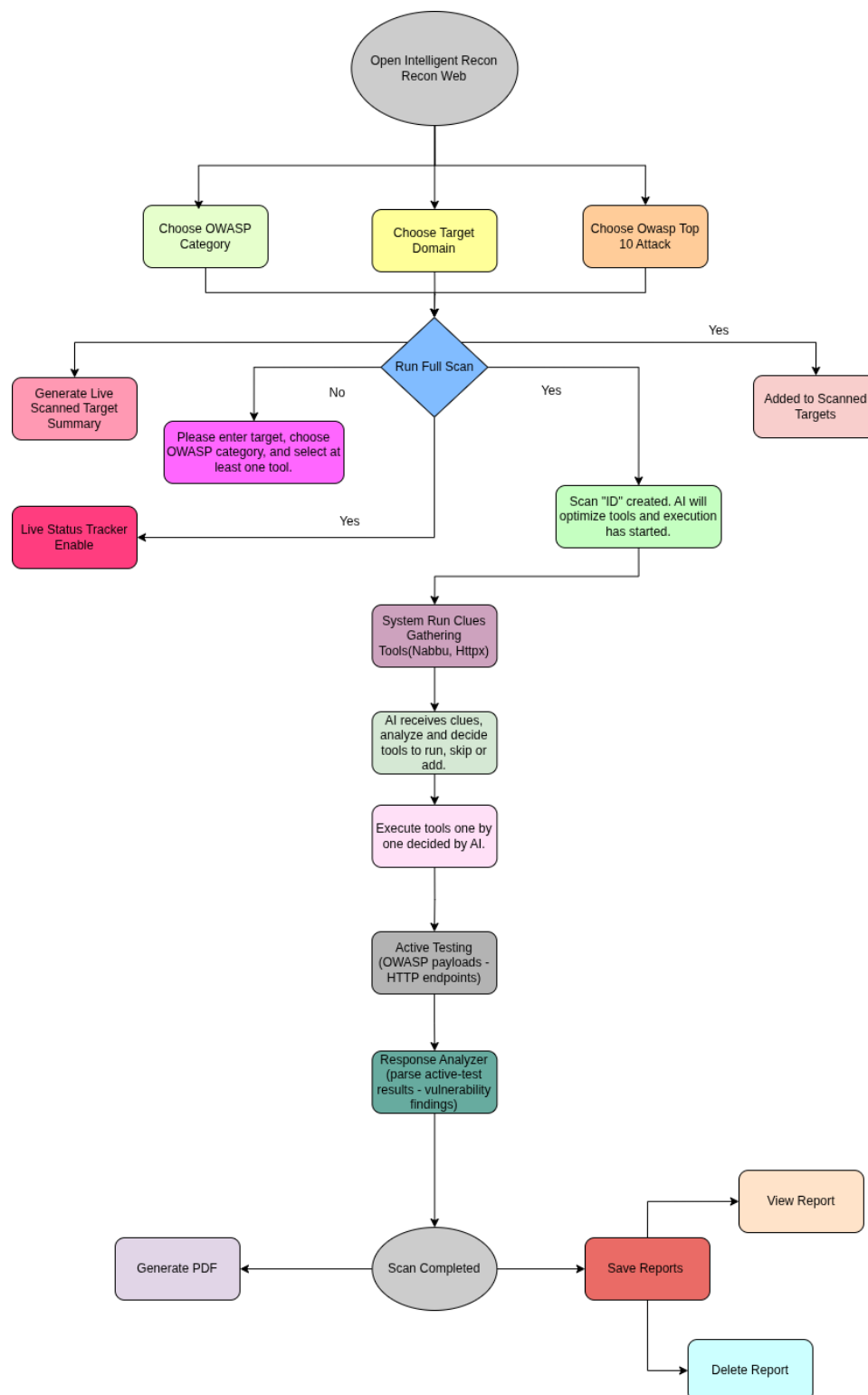


Figure 16 System Flow Chart

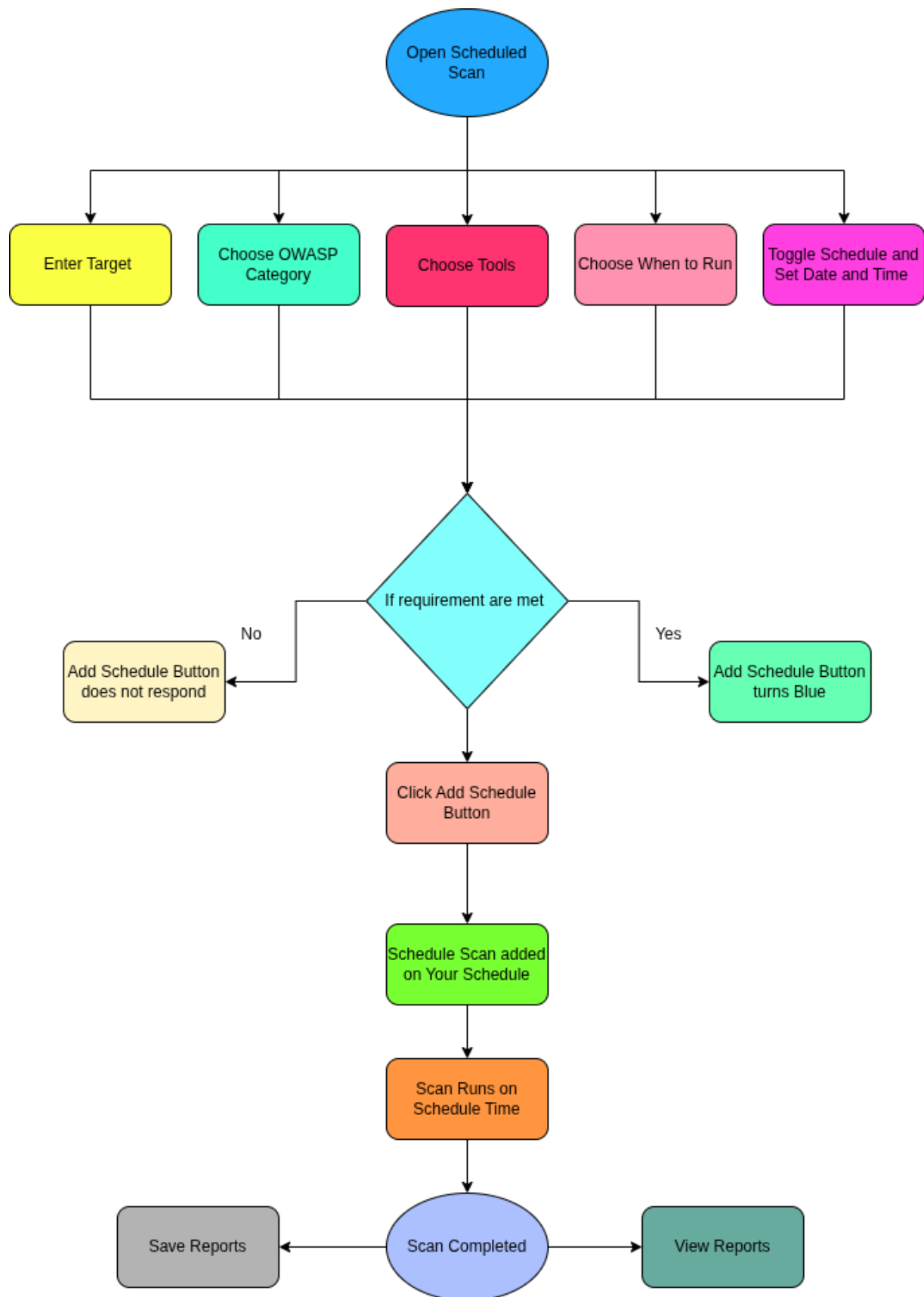


Figure 17 System Scheduled Scan Flowchart

3.4.7. Use Case Diagram

The Unified Modeling Language is a visual representation known as a Use Case Diagram which shows the interactions between users (actors) and a system (geeksforgeeks.org, 2025). The diagram below illustrates how the system will be used by the security analyst and system developer.

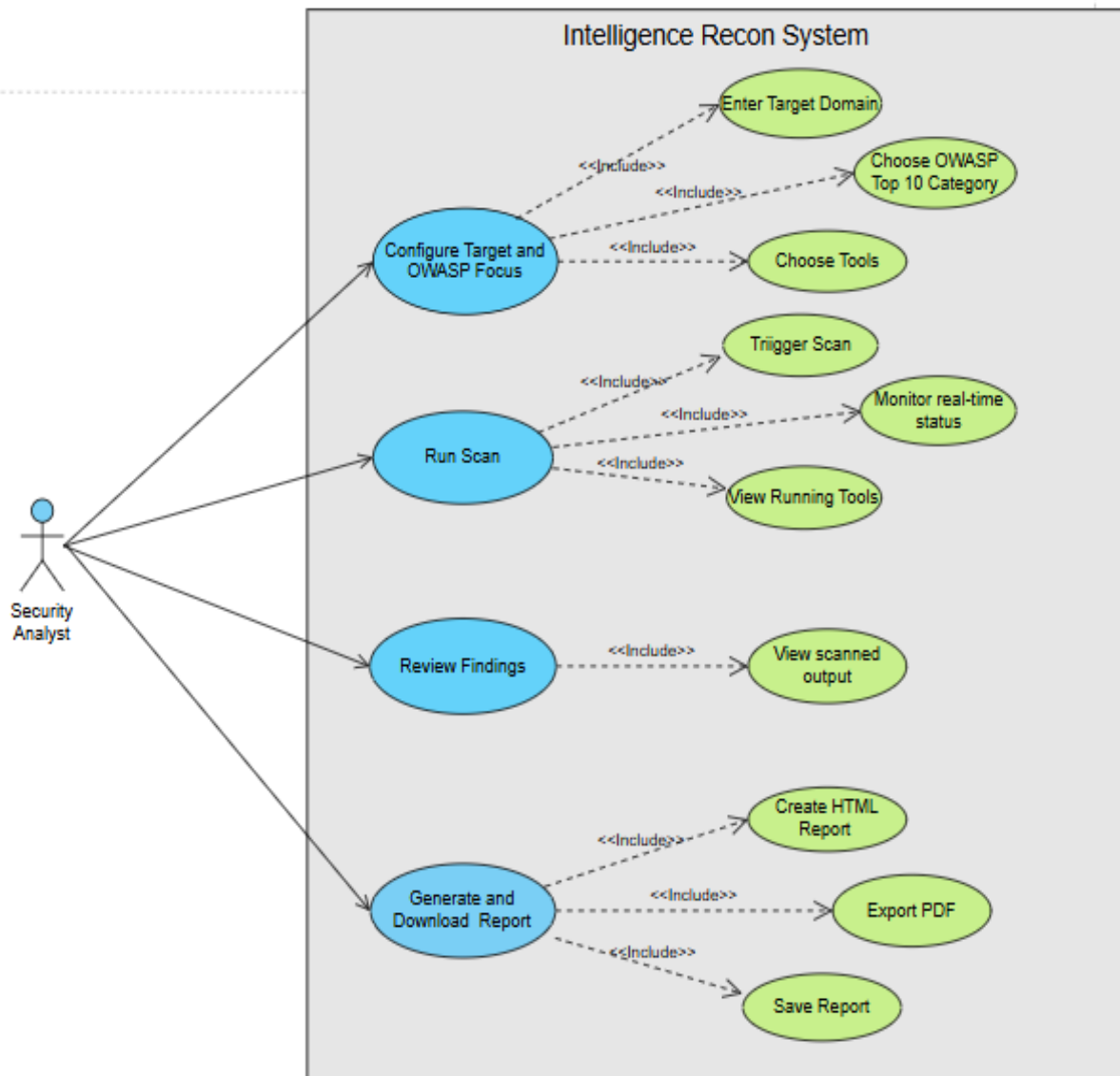


Figure 18 System User Use Case Diagram

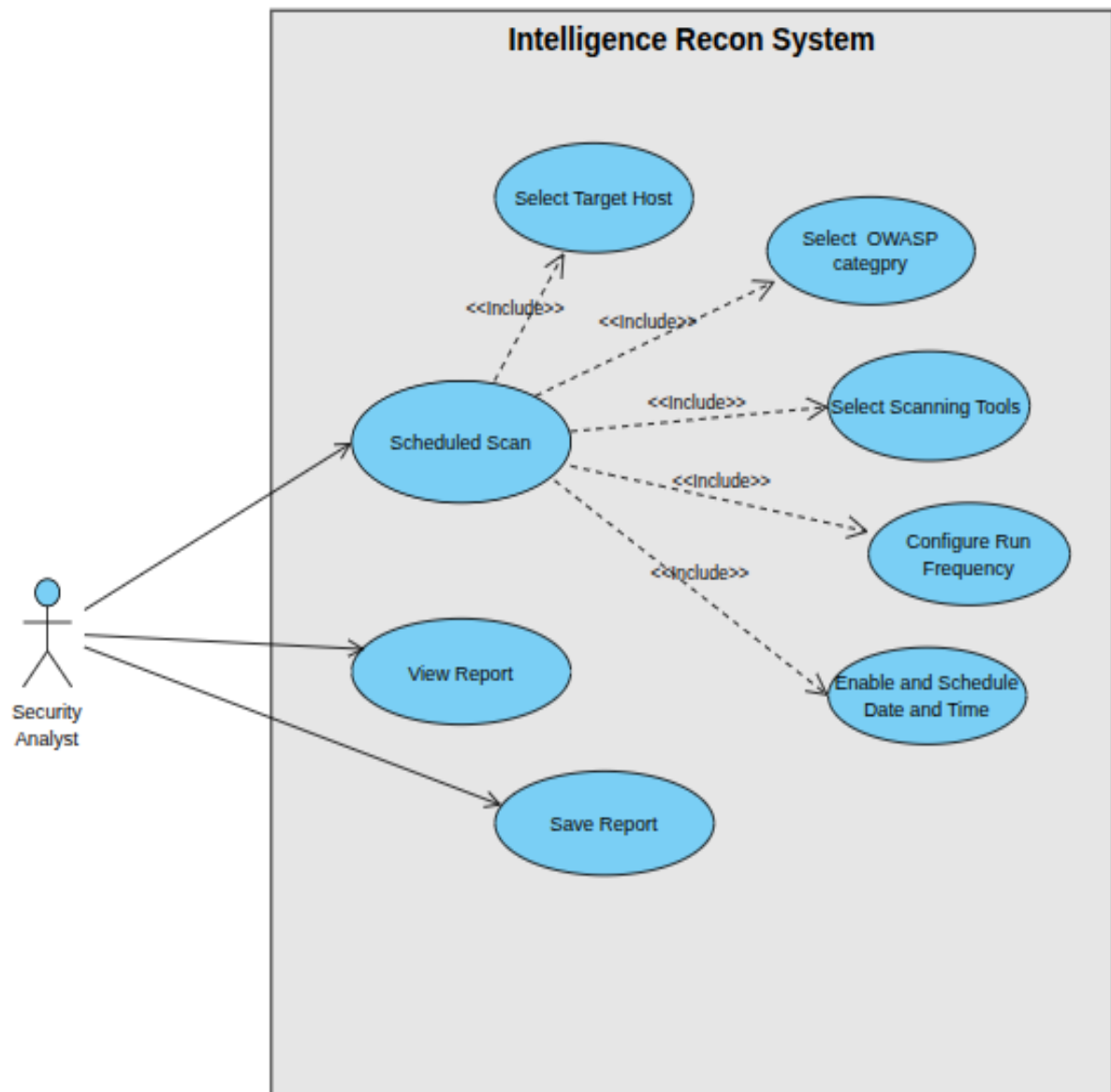


Figure 19 System User Use Case Diagram

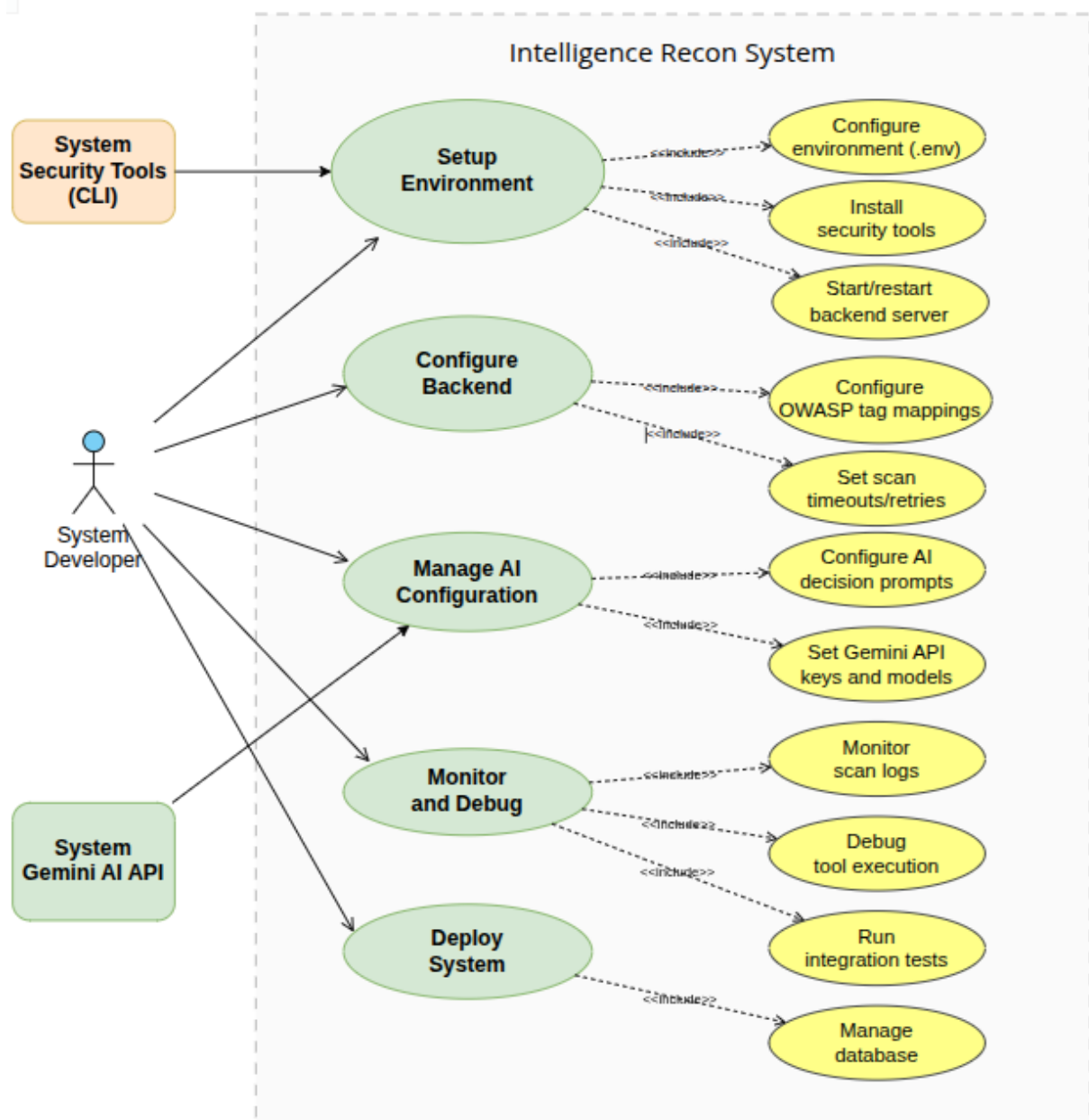


Figure 20 System Developer Use Case Diagram

3.4.8. System Architecture Diagram

The system architecture diagram represents an abstract diagram of the system component architecture. It gives a brief definition of the component architecture of the system so that it may help to establish component-relationships as well as the working of the system (InterviewBit, 2023). System architecture diagram displays HTML/CSS/JS, Fast API, Python, and Gemini API as the frontend, API, and the 15 security tools, as well as the flow of the data between them.

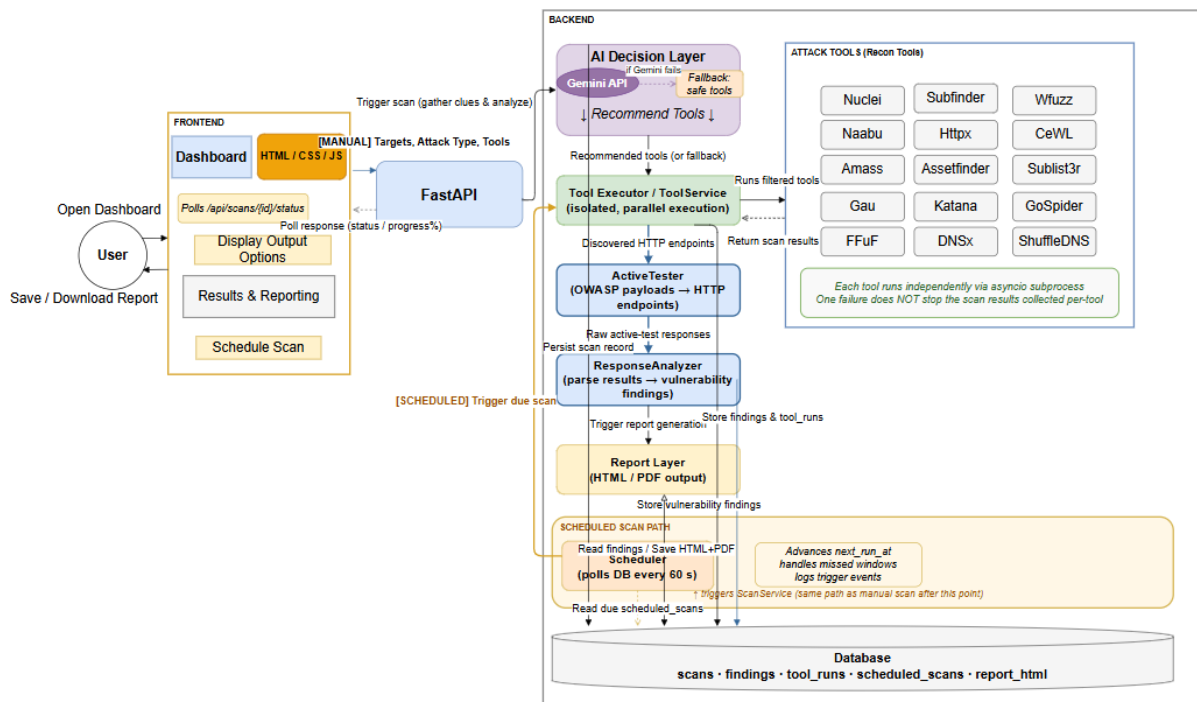


Figure 21 System Architecture Diagram

[Clear View](#)

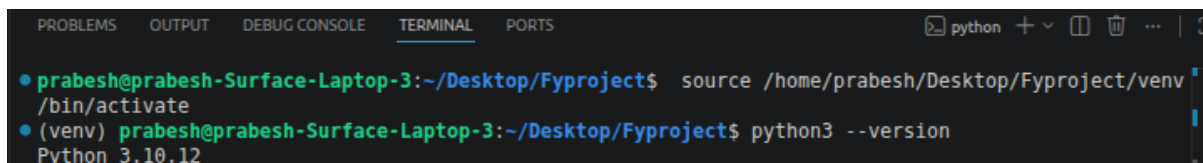
3.5. IMPLEMENTATION

3.5.1. Sprint 1: Initial Setup and Tools installation

Sprint 1 provided the basic architecture of the IRS with the development environment configuration and the core backend architecture implementation. The 15 open-source security tools were installed and checked, and the tool execution layer was developed to enable asynchronous parallel execution.

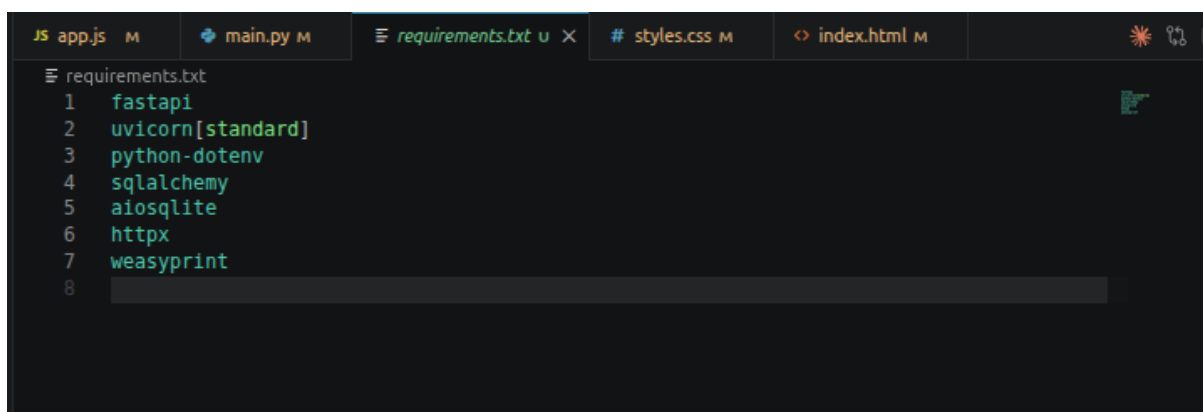
Key implementations:

1. Virtual environment setup with Python, FastAPI and Uvicorn.
2. Centralized config.py that sets the categories of OWASP, list of tools, and AI credentials.
3. Installation of all 15 security tools via github repository.
4. Base.py Abstract BaseTool class whose subprocesses can be run asynchronously.
5. FastAPI application with all API routers and middleware set up.



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
python + v [icon] [icon] [icon] [icon]
● prabesh@prabesh-Surface-Laptop-3:~/Desktop/Fyproject$ source /home/prabesh/Desktop/Fyproject/venv
/bin/activate
● (venv) prabesh@prabesh-Surface-Laptop-3:~/Desktop/Fyproject$ python3 --version
Python 3.10.12
```

Figure 22 Activation of Virtual Environment and Python3 version check



```
JS app.js M main.py M requirements.txt u x # styles.css M <> index.html M
requirements.txt
1 fastapi
2 uvicorn[standard]
3 python-dotenv
4 sqlalchemy
5 aiosqlite
6 httpx
7 weasyprint
8
```

Figure 23 Project Requirements File


```
Python 3.10.12
(venv) prabesh@prabesh-Surface-Laptop-3: ~/Desktop/Fyproject$ uvicorn app.main:app --host 0.0.0.0 -
-port 8080
INFO:      Started server process [14145]
INFO:      Waiting for application startup.
INFO:      Application startup complete.
INFO:      Uvicorn running on http://0.0.0.0:8080 (Press CTRL+C to quit)
```

Figure 24 Successful Startup of the Application Server

```
$ install-tools.sh
1 set -euo pipefail
2 cd "$(dirname "$0")"
3 GO_BIN=""
4 if command -v go >/dev/null 2>&1; then
5     GO_BIN=$(go env GOPATH)
6     if [ -z "$GO_BIN" ] || [ "$GO_BIN" = "/" ]; then
7         GO_BIN=$(go env GOPATH)/bin
8     fi
9     export PATH="$GO_BIN:$PATH"
10 fi
11
12 install_go_tool() {
13     local pkg="$1"
14     echo "Installing $pkg..."
15     if ! go install "$pkg@latest"; then
16         echo "Warning: failed to install $pkg"
17     fi
18 }
19
20 if command -v go >/dev/null 2>&1; then
21     install_go_tool github.com/projectdiscovery/httpx/cmd/httpx
22     install_go_tool github.com/projectdiscovery/nuclei/v2/cmd/nuclei
23     install_go_tool github.com/projectdiscovery/naabu/v2/cmd/naabu
24     install_go_tool github.com/projectdiscovery/dnsx/cmd/dnsx
25     install_go_tool github.com/projectdiscovery/shuffledns/cmd/shuffledns
26     install_go_tool github.com/projectdiscovery/katana/cmd/katana
27     install_go_tool github.com/projectdiscovery/gau/v2/cmd/gau
28     install_go_tool github.com/jaeles-project/gospider
29     install_go_tool github.com/projectdiscovery/subfinder/v2/cmd/subfinder
30     install_go_tool github.com/tomnomnom/assetfinder
31     install_go_tool github.com/ffuf/ffuf
32     if ! install_go_tool github.com/OWASP/Amass/v3/...; then
33         echo "Warning: Amass installation failed. Please install Amass manually if needed."
34     fi
35 else
36     echo "Warning: Go is not installed. Skipping Go-based tool installation."
37     echo "Install Go and re-run ./install-tools.sh to install httpx, nuclei, naabu, dnsx, shuffledns, katana, gau, gospider, subfinder, assetfinder, ffuf, and amass."
38 fi
39
40 if [ -f "venv/bin/activate" ]; then
41     printf "Activating Python virtual environment...\n"
42     # shellcheck disable=SC1091
43     . "venv/bin/activate"
44     pip install --upgrade pip
45     pip install wfuzz sublist3r
46 else
47     echo "Warning: Python virtual environment not found. Run ./install.sh first."
48 fi
49
50 if command -v gem >/dev/null 2>&1; then
51     echo "Installing CeWL via gem..."
52     gem install cewl
53 else
54     echo "Warning: gem is not installed. Skipping CeWL installation."
55 fi
56
57 echo "External tool installation complete."
58 if [ -n "$GO_BIN" ]; then
59     echo "Go binaries installed to: $GO_BIN"
60     echo "Ensure $GO_BIN is in your PATH."
61 fi
62
```

Figure 25 Dependency Installation Script

```

app > core > config.py > Settings
1  """Application configuration and settings."""
2  import os
3  from pathlib import Path
4  from typing import List, Tuple
5
6  class Settings:
7      """Application settings loaded from environment variables."""
8
9      # Project paths
10     BASE_DIR: Path = Path(__file__).resolve().parent.parent.parent
11     DATA_DIR: Path = BASE_DIR / "data"
12     SCANS_DIR: Path = DATA_DIR / "scans"
13
14     # Database
15     DATABASE_URL: str = os.getenv(
16         "DATABASE_URL",
17         f"sqlite+aiosqlite:///{BASE_DIR}/data/irs.db"
18     )
19

```

Figure 26 Project Settings code and Environment Configuration

```

# AI Configuration
GEMINI_API_KEY: str = os.getenv("GEMINI_API_KEY", "")
GEMINI_MODEL: str = os.getenv("GEMINI_MODEL", "gemini-2.5-flash")
# Optional: multiple backends "model:key" or "key" (uses GEMINI_MODEL). Example: "gemini-2.5-flash:key1,gemini-2.0-flash:key2" or "key1,key2"
GEMINI_BACKENDS: str = os.getenv("GEMINI_BACKENDS", "").strip()
GEMINI_429_BACKOFF_BASE: int = int(os.getenv("GEMINI_429_BACKOFF_BASE", "3"))
INTELLIGENCE_AI_ENABLED: bool = os.getenv("INTELLIGENCE_AI_ENABLED", "true").lower() == "true"

# Tool execution settings
TOOL_TIMEOUT: int = int(os.getenv("TOOL_TIMEOUT", "300")) # 5 minutes default
NUCLEI_TIMEOUT: int = int(os.getenv("NUCLEI_TIMEOUT", "1800")) # 30 minutes base for OWASP-focused scans
NUCLEI_REQUEST_TIMEOUT: int = int(os.getenv("NUCLEI_REQUEST_TIMEOUT", "5"))
NUCLEI_RETRIES: int = int(os.getenv("NUCLEI_RETRIES", "0"))
NUCLEI_MAX_URLS: int = int(os.getenv("NUCLEI_MAX_URLS", "60"))
NUCLEI_STOP_ON_FINDINGS: int = int(os.getenv("NUCLEI_STOP_ON_FINDINGS", "1"))
MAX_CONCURRENT_TOOLS: int = int(os.getenv("MAX_CONCURRENT_TOOLS", "3"))

# Recon tools - canonical names
AVAILABLE_TOOLS: List[str] = [
    "Nuclei",
    "Naabu",
    "Httpx",
    "Subfinder",
    "Amass",
    "Assetfinder",
    "Sublist3r",
    "GAU",
    "Katana",
    "GoSpider",
    "FFuf",
    "Wfuzz",
    "CeWL",
    "DNSx",
    "ShuffleDNS"
]

```

Figure 27 Configuration for AI and Tools

```

@property
def API_BASE_URL(self) -> str:
    """Get the API base URL - server-side only, frontend gets it via /api/config"""
    return f"{self.API_SCHEME}://{self.API_HOST}:{self.API_PORT}"

# OWASP Top 10 2021 categories
OWASP_CATEGORIES: List[dict] = [
    {"id": "A01:2021", "name": "Broken Access Control"},
    {"id": "A02:2021", "name": "Cryptographic Failures"},
    {"id": "A03:2021", "name": "Injection"},
    {"id": "A04:2021", "name": "Insecure Design"},
    {"id": "A05:2021", "name": "Security Misconfiguration"},
    {"id": "A06:2021", "name": "Vulnerable and Outdated Components"},
    {"id": "A07:2021", "name": "Identification and Authentication Failures"},
    {"id": "A08:2021", "name": "Software and Data Integrity Failures"},
    {"id": "A09:2021", "name": "Security Logging and Monitoring Failures"},
    {"id": "A10:2021", "name": "Server-Side Request Forgery (SSRF)"}
]

```

Figure 28 OWASP category configuration

```

@dataclass
class ToolResult:
    """Holds the result of a single tool execution."""
    tool_name: str
    success: bool
    summary: str
    findings: List[Dict[str, Any]] = field(default_factory=list)
    raw_output: str = ""
    error_message: Optional[str] = None
    started_at: Optional[datetime] = None
    finished_at: Optional[datetime] = None

Windsurf: Refactor | Explain
class BaseTool(ABC):
    """
    Abstract base class for all recon tools.

    Handles:
    - Command execution
    - Timeout handling
    - Output parsing
    - WebSocket notifications
    """
    Windsurf: Refactor | Explain | Generate Docstring
    def __init__(self, tool_name: str):
        self.tool_name = tool_name
        self.logger = get_logger(f"tool.{tool_name.lower()}")

    Windsurf: Refactor | Explain
    @abstractmethod
    def build_command(self, target: str, output_file: Path, **kwargs) -> List[str]:
        """
        Build the command line arguments for this tool.

        Returns:
        --- List of strings representing the command.
        """
        pass

    Windsurf: Refactor | Explain
    @abstractmethod
    def parse_output(self, output: str, target: str) -> List[Dict[str, Any]]:
        """
        Parse raw tool output into a structured list of findings.
        Each finding is a dictionary with keys:
        - type
        - severity
        - location
        - description
        - evidence
        """
        pass

```

Figure 29 Basic Base Class in Reconnaissance Tools

```

        yield

        task.cancel()
    try:
        await task
    except asyncio.CancelledError:
        pass

app = FastAPI(
    title="Intelligence Recon System (IRS)",
    description="AI-assisted OWASP Top 10 reconnaissance platform",
    version="1.0.0",
    lifespan=lifespan,
)

app.add_middleware(NoCacheStaticMiddleware)
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

# Mount static files
app.mount("/static", StaticFiles(directory="Frontend"), name="static")

# API prefix like "/api"
app.include_router(health.router, prefix=settings.API_PREFIX)
app.include_router(scans.router, prefix=settings.API_PREFIX)
app.include_router(reports.router, prefix=settings.API_PREFIX)
app.include_router(tools.router, prefix=settings.API_PREFIX)
app.include_router(scheduled_scans.router, prefix=settings.API_PREFIX)
app.include_router(settings_router.router, prefix=settings.API_PREFIX)

```

Figure 30 FastAPI Application and API Route Registration

```

Windsurf: Refactor | Explain
async def execute(
    self,
    target: str,
    output_file: Path,
    timeout: int = 300,
    scan_id: Optional[int] = None,
    **kwargs
) -> ToolResult:
    """
    Execute the tool asynchronously and parse results.

    Args:
        target: target domain or host
        output_file: path to save raw output
        timeout: max time in seconds
        scan_id: optional scan identifier for WS updates
        **kwargs: tool-specific options
    """
    started_at = datetime.utcnow()

    # Notify WS: tool start
    if scan_id:
        await send_tool_start_update(scan_id, self.tool_name)
        await send_tool_status_update(scan_id, self.tool_name, "running", {"command": " ".join(self.build_command(target, output_file, **kwargs))})

    # Capture OWASP category for post-processing in parse_output
    self.owasp_category = kwargs.get("owasp_category")

    # Check installation
    if not self.check_installed():
        error_msg = f"{self.tool_name} is not installed or not in PATH"
        self.logger.error(error_msg)
        if scan_id:
            await send_tool_complete_update(scan_id, self.tool_name, False)
        return ToolResult(
            tool_name=self.tool_name,
            success=False,
            summary="Tool not installed",
            error_message=error_msg,
            started_at=started_at,
            finished_at=datetime.utcnow()
        )

```

Figure 31 Asynchronous Processing of Results and Tools

```

try:
    # Build command
    command = self.build_command(target, output_file, **kwargs)
    self.logger.info(f"Executing command: {' '.join(command)}")
    if scan_id:
        await send_log_message(scan_id, self.tool_name, f"Running command: {' '.join(command)}")

    # Run command asynchronously (limit=1MB so long lines e.g. Nuclei JSONL don't raise LimitOverrunError)
    process = await asyncio.create_subprocess_exec(
        *command,
        stdout=asyncio.subprocess.PIPE,
        stderr=asyncio.subprocess.PIPE,
        limit=1024 * 1024,
    )

```

Figure 32 Execution and Monitoring of Commands When Running Tools

```

try:
    await asyncio.wait_for(run_with_timeout(), timeout=timeout)
    raw_output = "\n".join(raw_output_lines)
    stderr_output = "\n".join(stderr_lines)
except asyncio.TimeoutError:
    process.kill()
    await process.wait()
    error_msg = f"Execution timed out after {timeout} seconds"
    self.logger.error(error_msg)
    if scan_id:
        await send_tool_complete_update(scan_id, self.tool_name, False)
    return ToolResult(
        tool_name=self.tool_name,
        success=False,
        summary="Execution timeout",
        error_message=error_msg,
        started_at=started_at,
        finished_at=datetime.utcnow()
    )

```

Figure 33 Handling of Timeout and Error Recovery of Tools

For more installation and setup code, see **Appendix 3**, [Section 7.3.1](#)

3.5.2. Sprint 2: Backend API and Architecture Development

The second sprint was dedicated to building the main backend such as database schemas, API endpoints, and the tool execution pipeline. SQLAlchemy with asynchronous SQLite was used as a persistent data layer, to store and retrieve efficiently all scan findings and results.

Key implementations:

1. finding.py and scan.py database models that characterize the types of findings, the levels of severity and scan lifecycle.
2. Asynchronous database engine and session management set up in database.py. ScanService class used to control the entire scan orchestration life cycle.
3. RESTful API endpoints created in scans.py to create a scan, poll on its status and retrieve the findings.
4. Designed output parsing logic to normalize and categorize results of all tools.

```

1  """Finding model - normalized security findings from tools."""
2  from sqlalchemy import Column, Integer, String, Text, ForeignKey, Enum as SQLAlchemyEnum
3  from sqlalchemy.orm import relationship
4  import enum
5
6  from app.models.base import BaseModel
7
8  class FindingType(str, enum.Enum):
9      """Finding type enumeration."""
10     ASSET = "asset"
11     ENDPOINT = "endpoint"
12     PORT = "port"
13     VULNERABILITY = "vulnerability"
14     MISCONFIGURATION = "misconfiguration"
15     INFORMATION = "information"
16
17  class FindingSeverity(str, enum.Enum):
18      """Finding severity enumeration."""
19     INFO = "info"
20     LOW = "low"
21     MEDIUM = "medium"
22     HIGH = "high"
23     CRITICAL = "critical"
24
25
26  class Finding(BaseModel):
27      """Finding model representing a normalized piece of reconnaissance data."""
28
29      __tablename__ = "findings"
30
31      # Scan relationship
32      scan_id = Column(Integer, ForeignKey("scans.id", ondelete="CASCADE"), nullable=False, index=True)
33      scan = relationship("Scan", back_populates="findings")
34
35      # Source information
36      tool_name = Column(String(100), nullable=False, index=True)
37
38      # Classification
39      type = Column(SQLAlchemyEnum(FindingType), nullable=False, index=True)
40      severity = Column(SQLAlchemyEnum(FindingSeverity), default=findingSeverity.INFO, nullable=False, index=True)
41      owasp_category = Column(String(20), nullable=True, index=True)
42
43      # Finding details
44      location = Column(String(500), nullable=False) # URL, host:port, or resource identifier
45      description = Column(Text, nullable=False)
46      evidence = Column(Text, nullable=True) # Supporting data (headers, payloads, etc.)
47
48      def __repr__(self):
49          return f"<Finding(id={self.id}, type={self.type}, severity={self.severity}, location={self.location})>"

```

Figure 34 Normalized Finding Model for Recon Results

```

1  """Database configuration and session management."""
2  from typing import AsyncGenerator
3  import json
4  import time
5  from sqlalchemy import text
6  from sqlalchemy.ext.asyncio import (
7      AsyncEngine,
8      AsyncSession,
9      create_async_engine,
10     async_sessionmaker
11 )
12 from sqlalchemy.orm import declarative_base
13
14 from app.core.config import settings
15 # Create async engine
16 engine: AsyncEngine = create_async_engine(
17     settings.DATABASE_URL,
18     echo=False,
19     future=True,
20     pool_pre_ping=True
21 )
22 # Create async session factory
23 AsyncSessionLocal = async_sessionmaker(
24     engine,
25     class_=AsyncSession,
26     expire_on_commit=False,
27     autocommit=False,
28     autoflush=False
29 )
30 # Declarative base for models
31 Base = declarative_base()
32
33 async def get_db() -> AsyncGenerator[AsyncSession, None]:
34     """Dependency for getting async database sessions.
35
36     Yields:
37         AsyncSession instance
38     """
39     async with AsyncSessionLocal() as session:
40         try:
41             yield session
42         finally:
43             await session.close()

```

Figure 35 Setting up the database and managing sessions

```

Windsurf: Refactor | Explain
async def init_db() -> None:
    """Initialize database tables."""
    from app.models import scan, tool_run, finding, scheduled_scan # Import models to register them

    async with engine.begin() as conn:
        await conn.run_sync(Base.metadata.create_all)
        await conn.run_sync(_ensure_scheduled_scan_id_column)
        await conn.run_sync(_ensure_scheduled_scans_next_run_at)
        await conn.run_sync(_ensure_report_html_column)
        await conn.run_sync(_ensure_endpoint_classification_column)

```

Figure 36 Initialize Database and Setup Tables Routine

```

19
20 Windsurf: Refactor | Explain
21 class Scan(BaseModel):
22     """Scan model representing a complete recon run for a target."""
23
24     __tablename__ = "scans"
25
26     # Target information
27     target = Column(String(255), nullable=False, index=True)
28     owasp_category = Column(String(20), nullable=False, index=True)
29
30     # Status tracking
31     status = Column(SQLEnum(ScanStatus), default=ScanStatus.CREATING, nullable=False, index=True)
32     completed_at = Column(DateTime, nullable=True)
33
34     # Tool selection
35     user_selected_tools = Column(Text, nullable=True) # JSON list of tool names
36     ai_tools_to_run = Column(Text, nullable=True) # JSON list of tool names AI recommended
37     ai_tools_skipped = Column(Text, nullable=True) # JSON list of {tool, reason}
38     ai_raw_response = Column(Text, nullable=True) # Raw AI response for debugging
39
40     # Error tracking
41     error_summary = Column(Text, nullable=True)
42
43     # Set when scan was triggered by a scheduled scan (scheduled_scans.id)
44     scheduled_scan_id = Column(Integer, nullable=True, index=True)
45
46     # Persisted AI-generated report HTML; once set, View Report never calls AI again for this scan
47     report_html = Column(Text, nullable=True)
48
49     # Intelligence layer data - endpoint classification results
50     endpoint_classification = Column(Text, nullable=True) # JSON classification data
51
52     # Relationships
53     tool_runs = relationship("ToolRun", back_populates="scan", cascade="all, delete-orphan")
54     findings = relationship("Finding", back_populates="scan", cascade="all, delete-orphan")

```

Figure 37 Scan Model of Reconnaissance Workflow and Result Storage

```

async def execute(
    self,
    target: str,
    output_file: Path,
    timeout: int = 300,
    scan_id: Optional[int] = None,
    **kwargs
) -> ToolResult:
    """
    Execute the tool asynchronously and parse results.

    Args:
        target: target domain or host
        output_file: path to save raw output
        timeout: max time in seconds
        scan_id: optional scan identifier for WS updates
        **kwargs: tool-specific options
    """
    started_at = datetime.utcnow()

    # Notify WS: tool start
    if scan_id:
        await send_tool_start_update(scan_id, self.tool_name)
        await send_tool_status_update(scan_id, self.tool_name, "running", {"command": " ".join(self.build_command(target, output_file, **kwargs))})

    # Capture OWASP category for post-processing in parse_output
    self.owasp_category = kwargs.get("owasp_category")

    # Check installation
    if not self.check_installed():
        error_msg = f"{self.tool_name} is not installed or not in PATH"
        self.logger.error(error_msg)
        if scan_id:
            await send_tool_complete_update(scan_id, self.tool_name, False)
            return ToolResult(
                tool_name=self.tool_name,
                success=False,
                summary="Tool not installed",
                error_message=error_msg,
                started_at=started_at,
                finished_at=datetime.utcnow()
            )

    try:
        # Build command
        command = self.build_command(target, output_file, **kwargs)
        self.logger.info(f"Executing command: {' '.join(command)}")
        if scan_id:
            await send_log_message(scan_id, self.tool_name, f"Running command: {' '.join(command)}")

        # Run command asynchronously (limit=1MB so long lines e.g. Nuclei JSONL don't raise LimitOverrunError)
        process = await asyncio.create_subprocess_exec(
            *command,
            stdout=asyncio.subprocess.PIPE,
            stderr=asyncio.subprocess.PIPE,
            limit=1024 * 1024,
        )

```

Figure 38 Live Status Updates and Error handling Asynchronous Tool Execution


```

# Many tools write to the output file and use stdout (stdout empty). Read file first.
output_file.parent.mkdir(parents=True, exist_ok=True)
file_content = ""
if output_file.exists():
    try:
        file_content = output_file.read_text(encoding="utf-8", errors="replace")
    except Exception as e:
        self.logger.warning(f"Could not read tool output file: {e}")
# Prefer file content for parsing (where tools write results); fallback to stdout
file = file_content.strip()
if "=== STDOUT ===" in file:
    file = file.split("=== STDOUT ===")[0].strip()
content_for_parsing = (file or raw_output) or ""
# Persist full output: file + stdout + stderr so nothing is lost
combined = "\n\n=== STDOUT ===\n" + raw_output + "\n\n=== STDERR ===\n" + stderr_output
output_file.write_text(combined)

# Parse findings from the content the tool actually produced (file or stdout)
findings = self.parse_output(content_for_parsing, target)

# Generate summary
summary = self.generate_summary(findings, content_for_parsing)

finished_at = datetime.utcnow()

# If execution error occurred, notify failure and return early
if has_execution_error:
    error_msg = f"{self.tool_name} exited with error code {tool_return_code}"
    self.logger.error(error_msg)
    if scan_id:
        await send_tool_complete_update(scan_id, self.tool_name, False)
    return ToolResult(
        tool_name=self.tool_name,
        success=False,
        summary=f"Execution error: {summary}",
        error_message=error_msg,
        findings=findings, # Include any partial findings
        started_at=started_at,
        finished_at=finished_at
    )

# Notify WS: tool complete
if scan_id:
    await send_tool_complete_update(scan_id, self.tool_name, True)

return ToolResult(
    tool_name=self.tool_name,
    success=True,
    summary=summary,
    findings=findings,
    raw_output=raw_output,
    started_at=started_at,
    finished_at=finished_at
)

```

Figure 39 Tool Output parser, Error Handling and Finalization of Results Workflow

```

@router.post("", response_model=ScanResponse)
async def create_scan(
    scan_data: ScanCreate,
    db: AsyncSession = Depends(get_db),
):
    """Create and start a new scan (dashboard flow: clues → AI decision → execute). Never uses scheduled_scan_id."""
    # Validate selected tools
    invalid_tools = [
        tool for tool in scan_data.selected_tools
        if tool not in settings.AVAILABLE_TOOLS
    ]
    if invalid_tools:
        msg = f"Invalid tools: {', '.join(invalid_tools)}"
        logger.warning("POST /api/scans 400: %s", msg)
        raise HTTPException(status_code=400, detail=msg)

    if not scan_data.selected_tools:
        msg = "You must select at least one tool before starting a scan. AI will refine this list based on clues, but cannot choose tools from an empty selection."
        logger.warning("POST /api/scans 400: %s", msg)
        raise HTTPException(status_code=400, detail=msg)

    try:
        scan = await ScanService.create_scan(
            db=db,
            target=scan_data.target,
            owasp_category=scan_data.owasp_category,
            selected_tools=scan_data.selected_tools,
        )
    except ValueError as e:
        logger.warning("POST /api/scans 400: %s", str(e))
        raise HTTPException(status_code=400, detail=str(e))

    return ScanResponse(
        scan_id=scan.id,
        status=scan.status.value,
        message="Scan created successfully. AI decision and tool execution in progress.",
    )

```

Figure 40 Scan Creation and validation Workflow to Tool execution

More backend and architecture code can be found in **Appendix 3**, [Section 7.3.2](#).

3.5.3. Sprint 3: Intelligent Workflow Development

Sprint 3 used the intelligent decision-making layer of the system, which incorporated the Gemini AI API to choose and optimize the execution of tools dynamically based on target reconnaissance clues. The Naabu and Httpx pre-check mechanism was created to collect the first intelligence prior to the AI decision making.

Key implementations:

1. schemas/scan.py that defines the user input handler and has target and OWASP category validation.
2. executor.py to perform clever Nuclei timeouts calculation according to the complexity of OWASP categories.
3. decision_node.py this code incorporates Gemini AI to make decisions by analyzing clues and deciding whether to run a certain tool or not.
4. prompts.py that sets the description of tools, mapping of tag mappings with OWASP nuclei, and the AI decision prompt.
5. core/validation.py providing domain format, DNS resolution and reachability validation.

```

Scan-related schemas:
from typing import List, Optional, Dict, Any
from datetime import datetime
from urllib.parse import urlparse
from pydantic import BaseModel, Field, field_validator

Windsurf: Refactor | Explain
class ScanCreate(BaseModel):
    """Request schema for creating a new scan."""
    target: str = Field(..., min_length=1, max_length=255, description="Target domain or host")
    owasp_category: str = Field(..., description="OWASP Top 10 category ID (e.g., A01:2021)")
    selected_tools: List[str] = Field(default_factory=list, description="List of tool names to run")

Windsurf: Refactor | Explain
@field_validator("target")
@classmethod
def validate_target(cls, v: str) -> str:
    """Validate and sanitize target."""
    if not isinstance(v, str):
        raise ValueError("Target must be a string")

    v = v.strip()
    if not v:
        raise ValueError("Target cannot be empty")

    if v.startswith("//"):
        v = "http:" + v

    if v.startswith(("http://", "https://")):
        parsed = urlparse(v)
        host = parsed.hostname
        port = parsed.port
        if host:
            v = f"{host}:{port}" if port else host
        else:
            v = parsed.netloc or parsed.path

    # Remove any remaining path/query/fragment after URL normalization.
    v = v.split("/", 1)[0].split("?", 1)[0].split("#", 1)[0].strip()
    v = v.lower().rstrip("/")

    if not v:
        raise ValueError("Target cannot be empty")
    return v

Windsurf: Refactor | Explain
@field_validator("owasp_category")
@classmethod
def validate_owasp(cls, v: str) -> str:
    """Validate OWASP category."""
    valid_categories = [
        "A01:2021", "A02:2021", "A03:2021", "A04:2021", "A05:2021",
        "A06:2021", "A07:2021", "A08:2021", "A09:2021", "A10:2021"
    ]
    if v not in valid_categories:
        raise ValueError(f"Invalid OWASP category. Must be one of: {', '.join(valid_categories)}")
    return v

```

Figure 41 Scan Request Validation and Target Normalization Schema

```

"""Tool execution coordinator."""
from pathlib import Path
from typing import Optional, List, Dict, Any

from app.tools.base import ToolResult, BaseTool
from app.tools.tools_impl import TOOL_REGISTRY
from app.core.logging import get_logger
from app.core.config import settings

logger = get_logger(__name__)

Windsurf: Refactor | Explain
def _estimate_nuclei_timeout(
    base_timeout: int,
    discovered_urls: Optional[List[str]],
    clues: Optional[Dict[str, Any]],
    owasp_category: Optional[str] = None,
) -> int:
    """Scale Nuclei timeout based on URL count and OWASP category complexity.

    Different OWASP categories use different template types with varying execution times:
    - Fast categories (A05): Header checks, misconfigurations (~milliseconds per request)
    - Medium categories (A01, A07, A08): Access control, auth tests (~1-3s per request)
    - Slow categories (A03, A06): Injection, CVE scanning (~3-10s+ per request)
    """

    # Category-specific base timeout multipliers
    CATEGORY_TIMEOUT_MULTIPLIERS = {
        # Fast: Security Misconfiguration
        "A05:2021": 1.0,
        # Medium: Access Control, Authentication, Integrity Failures, SSRF
        "A01:2021": 1.5,
        "A07:2021": 1.5,
        "A08:2021": 1.5,
        "A10:2021": 1.5,
        # Medium-Slow: Cryptographic, Design, Logging
        "A02:2021": 1.75,
        "A04:2021": 1.75,
        "A09:2021": 1.75,
        # Slow: Injection, Vulnerable Components
        "A03:2021": 2.0,
        "A06:2021": 2.0,
    }

    # Get category multiplier (default to 1.5 if unknown or not specified)
    category_multiplier = CATEGORY_TIMEOUT_MULTIPLIERS.get(owasp_category, 1.5)

    # Apply category multiplier to base timeout
    adjusted_timeout = int(base_timeout * category_multiplier)

```

Figure 42 Nuclei Timeout Adjustment Based on OWASP Category and Scan Complexity

```

Windsurf: Refactor | Explain
async def execute_tool(
    tool_name: str,
    target: str,
    scan_id: int,
    timeout: Optional[int] = None,
    discovered_urls: Optional[List[str]] = None,
    clues: Optional[Dict[str, Any]] = None,
    owasp_category: Optional[str] = None,
) -> ToolResult:
    """Execute a single recon tool.

    Args:
        tool_name: Name of the tool to execute
        target: Target domain/host
        scan_id: ID of the scan this tool run belongs to
        timeout: Optional timeout override
        discovered_urls: URLs from discovery tools (for Nuclei, Ffuf, Wfuzz)
        clues: Initial clues from Naabu/Httpx (http_services, etc.)

    Returns:
        ToolResult object
    """
    # Get tool class
    tool_class = TOOL_REGISTRY.get(tool_name)

    if not tool_class:
        logger.error(f"Unknown tool: {tool_name}")
        return ToolResult(
            tool_name=tool_name,
            success=False,
            summary="Unknown tool",
            error_message=f"Tool '{tool_name}' not found in registry"
        )

    # Create output file path
    output_dir = settings.SCANS_DIR / str(scan_id)
    output_dir.mkdir(parents=True, exist_ok=True)
    output_file = output_dir / f"{tool_name.lower()}.out"

    # Build kwargs for tools that use discovered URLs
    extra_kwargs = {}
    if discovered_urls is not None:
        extra_kwargs["discovered_urls"] = discovered_urls
    if clues is not None:
        extra_kwargs["clues"] = clues
    if owasp_category is not None:
        extra_kwargs["owasp_category"] = owasp_category

```

Figure 43 Tool execution and Coordination of Scan

```

# Nuclei needs longer timeout (runs many templates)
# Timeout now scales based on both URL count AND OWASP category complexity
if tool_name == "Nuclei" and timeout is None:
    timeout = _estimate_nuclei_timeout(
        base_timeout=settings.NUCLEI_TIMEOUT,
        discovered_urls=discovered_urls,
        clues=clues,
        owasp_category=owasp_category, # Pass category for smart timeout calculation
    )
    logger.info(f"SMART TIMEOUT: Calculated Nuclei timeout={timeout}s (base={settings.NUCLEI_TIMEOUT}, urls={len(discovered_urls) if discovered_urls else 0}, owasp={owasp_category})")
elif tool_name == "Nuclei":
    logger.info(f"SMART TIMEOUT: Using provided timeout={timeout}s (not calculating)")

# Execute tool with exception handling
try:
    tool: BaseTool = tool_class()
    timeout_value = timeout or settings.TOOL_TIMEOUT

    if tool_name == "Nuclei":
        logger.info(f"NUCLEI FINAL TIMEOUT: timeout_value={timeout_value}s, timeout_param={timeout}, settings.TOOL_TIMEOUT={settings.TOOL_TIMEOUT}")

    logger.info(f"Executing {tool_name} for scan {scan_id}, target={target}, timeout={timeout_value}s")

    result = await tool.execute(
        target=target,
        output_file=output_file,
        timeout=timeout_value,
        scan_id=scan_id,
        **extra_kwargs
    )

    logger.info(f"{tool_name} completed: success={result.success}, findings={len(result.findings)}")
    return result
except Exception as e:
    logger.exception(f"Execution failed for {tool_name} on scan {scan_id}: {e}")
    return ToolResult(
        tool_name=tool_name,
        success=False,
        summary="Execution exception",
        error_message=str(e),
        findings=[]
    )

```

Figure 44 Intelligent Nuclei Timeout Calculation and Tool Execution Process

```

"""AI Decision Node using Gemini for tool selection."""
import json
from typing import List, Dict, Any, Optional
from dataclasses import dataclass
import httpx

from app.core.config import settings
from app.core.logging import get_logger
from app.ai.prompts import build_decision_prompt
from app.ai.report_generator import _gemini_post_with_retry

# Fixed safe fallback tools used when AI decision is unavailable or invalid.
SAFE_FALLBACK_TOOLS = ["Naabu", "Httpx", "Nuclei", "Subfinder", "DNSx"]

logger = get_logger(__name__)

Windsurf: Refactor | Explain
@dataclass
class AIDecision:
    """AI decision result."""
    tools_to_run: List[str]
    tools_skipped: List[Dict[str, str]]
    execution_batches: Optional[List[List[str]]] = None
    reasoning: Optional[str] = None
    raw_response: Optional[str] = None
    success: bool = True
    error: Optional[str] = None

Windsurf: Refactor | Explain
async def decide_tools(
    target: str,
    owasp_category: str,
    owasp_name: str,
    selected_tools: List[str],
    clues: Dict[str, Any]
) -> AIDecision:
    """Call Gemini AI to decide which tools to run."""

```

Figure 45 AI Decision Module Gemini-Based Tool Selection

```

logger.info(f"Requesting AI decision for target={target}, owasp={owasp_category}")

if not settings.get_gemini_backends():
    logger.error("No Gemini backend configured (GEMINI_API_KEY or GEMINI_BACKENDS)")
    return _fallback_decision(
        selected_tools,
        "Gemini API key not configured",
        raw_response=None,
    )

try:
    prompt = build_decision_prompt(
        target, owasp_category, owasp_name, selected_tools, clues
    )
    payload = {
        "contents": [{"parts": [{"text": prompt}]}],
        "generationConfig": {
            "temperature": 0.2,
            "maxOutputTokens": 8192,
        },
    }
    # Uses multi-backend + retries (429/connection → try next model/key)
    response = await _gemini_post_with_retry(payload, timeout=30.0)
    result = response.json()

    # Extract response text
    try:
        response_text = result["candidates"][0]["content"]["parts"][0]["text"]
    except (KeyError, IndexError) as e:
        logger.error(f"Unexpected Gemini response structure: {e}")
        return _fallback_decision(
            selected_tools,
            "Invalid API response structure",
            raw_response=json.dumps(result),
        )

    logger.info(f"Raw AI response: {response_text}")

    # Parse JSON from response (tolerant)
    decision_data = _parse_json_response(response_text)

    if not decision_data:
        logger.error(f"Failed to parse AI response as JSON. Response length: {len(response_text)}")
        logger.debug(f"Full response that failed to parse: {response_text[:500]}...") # Log first 500 chars
        return _fallback_decision(
            selected_tools,
            "Failed to parse AI response",
            raw_response=response_text,
        )

```

Figure 46 AI Decision Request and Response parsing workflow

```

# Merge tools to run and tools added to AI-recommended additions (e.g. Atcp, Gospider) are actually run
tools_from_run = decision_data.get("tools_to_run", []) or []
tools_from_added = [
    item.get("tool")
    for item in decision_data.get("tools_added", []) or []
    if isinstance(item, dict) and item.get("tool")
]
all_tools = list(dict.fromkeys(tools_from_run + tools_from_added))
valid_tools = [
    tool for tool in all_tools
    if tool in settings.AVAILABLE_TOOLS
]

if not valid_tools:
    logger.warning("AI returned no valid tools, using fallback")
    return fallback_decision(
        selected_tools,
        "No valid tools in AI response",
        raw_response=response_text,
    )

# IMPORTANT: Only include skipped tools if user selected them
# Do not show tools as "skipped" if user never selected them
skipped_tools = decision_data.get("tools_skipped", [])
filtered_skipped = [
    skip for skip in skipped_tools
    if skip.get("tool") in selected_tools # Only if user selected this tool
]

# Guardrail: avoid contradictory AI output (same tool in both run + skipped).
# If it happens, prefer the "skip" instruction for execution consistency.
skipped_tool_names = {
    str(skip.get("tool")).strip()
    for skip in filtered_skipped
    if isinstance(skip, dict) and skip.get("tool")
}
if skipped_tool_names:
    valid_tools = [t for t in valid_tools if str(t).strip() not in skipped_tool_names]

# Log tools that AI wanted to skip but user didn't select (these are ignored)
all_skipped_from_ai = [skip.get("tool") for skip in skipped_tools]
non_selected_skipped = [
    tool for tool in all_skipped_from_ai
    if tool not in selected_tools
]
if non_selected_skipped:
    logger.info(f"AI suggested skipping tools user didn't select (ignored): {non_selected_skipped}")

return AIDecision(
    tools_to_run=valid_tools,
    tools_skipped=filtered_skipped, # Only user-selected tools that should be skipped
    execution_batches=decision_data.get("execution_batches"),
    reasoning=decision_data.get("reasoning"),
    raw_response=response_text,
    success=True,
    error=None,
)

```

Figure 47 Filtering and Deduplication Logic of AI Tool Selection

```

1  """Prompt templates for AI decision node."""
2
3
4  TOOL_DESCRIPTIONS = {
5      "Nuclei": "Fast vulnerability scanner with extensive template library for detecting security issues across web applications, APIs, and network services. Excellent for all OWASP categories.",
6      "Nmap": "High-speed port scanner to discover open ports and services. Essential for initial reconnaissance and A05 (Security Misconfiguration).",
7      "Httpx": "HTTP probe to analyze web servers, status codes, headers, titles, and technologies. Critical for initial clues and all web-based OWASP categories.",
8      "Subfinder": "Passive subdomain discovery tool using multiple sources. Essential for asset discovery (A01, A05, A10).",
9      "Amass": "Comprehensive subdomain enumeration and network mapping. Deep reconnaissance for all categories.",
10     "Assetfinder": "Fast subdomain discovery tool. Good for quick asset enumeration (A01, A05).",
11     "Sublist3r": "Subdomain enumeration using search engines. Complements other subdomain tools.",
12
13     "GAU": "GetAllURLs - fetches known URLs from AlienVault's OTX, Wayback Machine, and Common Crawl. Excellent for A01, A03, A05, A10.",
14     "Katana": "Next-generation web crawler for gathering URLs, endpoints, and parameters. Essential for A01, A03, A05.",
15     "GoSpider": "Fast web spider for crawling and extracting endpoints. Good for A01, A03, A07.",
16     "Ffuf": "Fast web fuzzer for directory/file brute-forcing and parameter discovery. Critical for A01, A03, A05.",
17     "Wfuzz": "Web application fuzzer for security testing and OWASP attack vector discovery. Excellent for A01, A03, A04, A07 - specializes in access control and injection testing.",
18     "CmWf": "Custom wordlist generator from web content for password attacks. Useful for A07 (Authentication Failures).",
19     "DNSx": "Fast DNS toolkit for DNS enumeration and validation. Important for A05, A10.",
20     "ShuffleDNS": "Subdomain bruteforcer with wildcard detection. Useful when passive methods are insufficient."
21 }
22
23 OWASP_NUCLEI_TAGS = {
24     "A01:2021": "unauth, traversal, lfi, idor, redirect",
25     "A02:2021": "ssltls, expired-ssl, certificate",
26     "A03:2021": "sqli, xss, command-injection, ssti, xxe",
27     "A04:2021": "logic-bypass, auth-bypass, access-control, misconfig",
28     "A05:2021": "misconfig, cors, headers, exposure",
29     "A06:2021": "cve, known-vuln, version-detect, outdated",
30     "A07:2021": "default-login, auth-bypass, jwt, login",
31     "A08:2021": "deserialization, file-upload, xxe, path-traversal",
32     "A09:2021": "log4j, logging, exposure, debug-page",
33     "A10:2021": "ssrf, graphql",
34 }

```

Figure 48 Description of Tools and OWASP Category mapping


```

def build_decision_prompt(
    target: str,
    owasp_category: str,
    owasp_name: str,
    selected_tools: list,
    clues: dict
) -> str:
    """Build the prompt for Gemini AI decision node.

    Args:
        target: Target domain
        owasp_category: OWASP category ID (e.g., A01:2021)
        owasp_name: Full OWASP category name
        selected_tools: List of tools user selected
        clues: Dictionary containing initial reconnaissance data

    Returns:
        Formatted prompt string
    """
    tools_list = "\n".join(
        [f"- {tool}: {desc}" for tool, desc in TOOL_DESCRIPTIONS.items()]
    )

    nuclei_tags = OWASP_NUCLEI_TAGS.get(owasp_category, "misconfig,security-headers")

    clues_summary = """
Initial Reconnaissance Clues:
- Open Ports: {clues.get('open_ports', 'None detected')}
- HTTP Services: {clues.get('http_services', 'None detected')}
- Server Headers: {clues.get('server_headers', 'None detected')}
- Page Titles: {clues.get('page_titles', 'None detected')}
- Status Codes: {clues.get('status_codes', 'None detected')}
- Technologies Detected: {clues.get('technologies', 'None detected')}
- Discovered URLs: {clues.get('discovered_urls', 'None yet')}
- Subdomains Found: {clues.get('subdomains', 'None yet')}
    """

    selected_tools_str = ", ".join(selected_tools) if selected_tools else "All tools available"

    prompt = f"""You are a senior application security recon analyst optimizing tool selection for OWASP {owasp_category} ({owasp_name}) assessments.

## Mission
Produce a professional, evidence-driven execution plan that:
1) expands target intelligence with high-signal reconnaissance,
2) minimizes redundant/no-output tools,
3) improves vulnerability detection probability for Nuclei.

## Target Information
- Domain: {target}
- OWASP Focus: {owasp_category} - {owasp_name}
- User Selected Tools: {selected_tools_str}

{clues_summary}
    
```

Figure 49 Gemini Decision Prompt Construction for Recon Tool Selection

```

1 import socket
2 import re
3 from urllib.parse import urlparse
4 from typing import Tuple
5 import dns.resolver
6 import requests
7 from app.core.logging import get_logger
8
9 logger = get_logger(__name__)
10
11 Windowurf: Refactor | Explain
12 def is_valid_domain_format(domain: str) -> bool:
13     """Check if the domain format is valid."""
14     # Regular expression for validating domain name
15     pattern = r'^[a-zA-Z0-9]{1,61}[a-zA-Z0-9]([a-zA-Z0-9]{1,61}[a-zA-Z0-9])*\.[a-zA-Z]{2,6}$'
16     return bool(re.match(pattern, domain))
17
18 Windowurf: Refactor | Explain
19 def is_valid_ip_format(ip: str) -> bool:
20     """Check if the IP address format is valid."""
21     try:
22         socket.inet_aton(ip)
23         return True
24     except socket.error:
25         return False
26
27 Windowurf: Refactor | Explain
28 def is_resolvable_domain(domain: str) -> bool:
29     """Check if the domain is resolvable via DNS."""
30     try:
31         # Try to resolve the domain
32         socket.gethostbyname(domain)
33         return True
34     except socket.gaierror:
35         try:
36             # Alternative DNS resolution using dns.resolver
37             import dns.resolver
38             answers = dns.resolver.resolve(domain, 'A')
39             return len(answers) > 0
40         except:
41             return False
42
43 Windowurf: Refactor | Explain
44 def is_reachable_domain(domain: str) -> bool:
45     """Check if the domain is reachable via HTTP/HTTPS."""
46     try:
47         # Try HTTP
48         response = requests.head(f'http://{domain}', timeout=5, allow_redirects=True)
49         return response.status_code < 400
50     except:
51         try:
52             # Try HTTPS
53             response = requests.head(f'https://{domain}', timeout=5, allow_redirects=True)
54             return response.status_code < 400
55         except:
56             return False
    
```

Figure 50 Domain Checks and Reachability Checks on Scan Targets

```

    return False

def validate_domain(target: str) -> Tuple[bool, str]:
    """Validate if the target is a real, hosted domain or IP.

    Args:
        target: The target domain or IP address

    Returns:
        Tuple of (is_valid, message)
    """
    # Remove protocol if present, then normalize to host (strip :port for validation).
    # This allows targets like 'localhost:3000' or '127.0.0.1:8080'.
    original = target
    if target.startswith(("http://", "https://")):
        parsed = urlparse(target)
        target = parsed.netloc or parsed.path

    # Strip port from host:port
    host = (target or "").strip()
    # IPv6 in brackets: [::1]:3000
    if host.startswith("["):
        # Try bracketed host:port first
        m = re.match(r"^\[([^\]]+)\]:(\d+)$", host)
        if m:
            host = m.group(1)
        else:
            # No port, just strip brackets
            host = host.strip("[]")
    else:
        # IPv4/hostname:port
        if ":" in host:
            h, p = host.split(":", 1)
            if p.isdigit():
                host = h
            else:
                host = h

    # Check if it's an IP address
    if is_valid_ip_format(host):
        return True, f"Valid IP address: {host}"

    # Check if it's a domain
    if not is_valid_domain_format(host):
        return False, f"Invalid domain format: {original}"

    # Check if domain is resolvable
    if not is_resolvable_domain(host):
        return False, f"Domain is not resolvable via DNS: {host}"

    # Optionally, check if domain is reachable (can be skipped for privacy reasons)
    # if not is_reachable_domain(target):
    #     return False, f"Domain is not reachable via HTTP/HTTPS: {target}"

    return True, f"Valid hosted domain: {host}"

```

Figure 51 Scan Target Validation and Host Verification

To see more intelligent workflow code, see **Appendix 3**, [Section 7.3.2](#).

3.5.4. Sprint 4: Frontend Development

Sprint 4 worked on creating the entire user interface of the IRS, which included an easy-to-use dashboard to manage the scans, monitor their status, and display the results. It was designed on a single-page application architecture, which is responsive, based on HTML, CSS, and JavaScript.

Key implementations:

1. Sidebar navigation that allows you to access Dashboard, Saved Reports, Scanned Targets and Settings.
2. Scan launch form, with target input and OWASP category selection and tool selection.
3. OWASP_MAP and TOOL_LIST specified in app.js that mapped the 15 tools to OWASP categories.
4. Status reporting of scans in real-time through polling with WebSocket broadcast capability in ws_updates.py.
5. Security trend analytics and severity distribution charts with Chart.js. Dark and light mode with theme support using local Storage.


```

1 <!doctype html>
2 <html lang="en">
3 <head>
4   <meta charset="utf-8" />
5   <title>Intelligence Recon System (IRS)</title>
6   <link rel="icon" href="/static/favicon.svg?v=3" type="image/svg+xml" />
7   <link rel="shortcut icon" href="/static/favicon.svg?v=3" type="image/svg+xml" />
8   <meta name="viewport" content="width=device-width, initial-scale=1" />
9   <!-- Prevent caching -->
10  <meta http-equiv="Cache-Control" content="no-cache, no-store, must-revalidate">
11  <meta http-equiv="Pragma" content="no-cache">
12  <meta http-equiv="Expires" content="0">
13  <link rel="stylesheet" href="/static/styles.css?v=1.2.6" />
14  <script src="https://cdn.jsdelivr.net/npm/chart.js"></script>
15  <script>
16    (function() {
17      var allowed = ["dark-soc", "light-minimal"];
18      var t = localStorage.getItem("irs_theme");
19      var theme = (t && allowed.indexOf(t) !== -1) ? t : "dark-soc";
20      document.documentElement.setAttribute("data-theme", theme);
21    })();
22  </script>
23 </head>
24 <body class="theme-dark-soc app-loading">
25  <script>
26    (function() {
27      var allowed = ["dark-soc", "light-minimal"];
28      var t = localStorage.getItem("irs_theme");
29      if (t && allowed.indexOf(t) !== -1) {
30        document.body.classList.remove("theme-dark-soc", "theme-light-minimal");
31        document.body.classList.add("theme-" + t);
32      }
33    })();
34  </script>

```

Figure 52 Setting up Templates and Themes

```

<!-- Row 1: Launch Recon Scan -->
<div class="card">
  <div class="card-header">
    <span class="card-title"><svg class="card-icon" aria-hidden="true"><use href="#icon-launch"/></svg> Launch Recon Scan</span>
  </div>
  <div class="card-body">
    <form id="launchForm" class="launch-form">
      <label class="field">
        <span class="field-label">Target Host</span>
        <input type="text" id="targetInput" placeholder="domain.com or IP address" required>
      </label>

      <label class="field">
        <span class="field-label">OWASP Category</span>
        <select id="owaspSelect" required>
          <option value="">Select category...</option>
          <option value="A01:2021">Broken Access Control</option>
          <option value="A02:2021">Cryptographic Failures</option>
          <option value="A03:2021">Injection</option>
          <option value="A04:2021">Insecure Design</option>
          <option value="A05:2021">Security Misconfiguration</option>
          <option value="A06:2021">Vulnerable and Outdated Components</option>
          <option value="A07:2021">Identification and Authentication Failures</option>
          <option value="A08:2021">Software and Data Integrity Failures</option>
          <option value="A09:2021">Security Logging and Monitoring Failures</option>
          <option value="A10:2021">Server-Side Request Forgery</option>
        </select>
      </label>

      <div class="field">
        <span class="field-label">Recon Toolchain Selection</span>
        <div id="toolCheckboxGrid" class="tool-grid"></div>
      </div>

      <div class="launch-actions">
        <button type="submit" id="runScanBtn" class="btn-primary"><svg class="btn-icon" aria-hidden="true"><use href="#icon-scan"/></svg> Run Full Scan</button>
        <button type="button" id="clearLaunchBtn" class="btn-ghost">Clear</button>
      </div>

      <p class="field-help" id="launchHelpText">
        Selected tools and OWASP category will be validated by AI before execution.
      </p>
    </form>
  </div>
</div>

```

Figure 53 Launch Recon Scan Form

```
// ===== CONFIG =====
// API Base URL: use same-origin first so API works when app is served from the same server
let defaultApiBase = window.location.origin || `http://${window.location.hostname}:8080`;

const API_ROUTES = {
  config: () => "/api/config", // Fetch API config from backend
  createScan: () => "/api/scans",
  listScans: () => "/api/scans",
  scanDetail: (id) => `/api/scans/${id}`,
  scanStatus: (id) => `/api/scans/${id}/status`,
  scanIntelligence: (id) => `/api/scans/${id}/intelligence-summary`,
  health: () => "/api/health",
  reportHtml: (id) => `/api/scans/${id}/report`,
  reportPdf: (id) => `/api/scans/${id}/report/pdf`,
  markScanSaved: (id) => `/api/scans/${id}/mark-saved`,
  severityStatsCompleted: () => `/api/scans/stats/severity-completed`,
  purgeScans: () => "/api/scans/purge",
  toolCommand: (toolName, target) => `/api/tools/${encodeURIComponent(toolName)}/command${target ? `?target=${encodeURIComponent(target)}` : ""}`,
};

// Initialize API config from backend on page load; also update apiBase so requests use correct URL
Windsurf: Refactor | Explain
async function initializeApiConfig() {
  try {
    // Use relative fetch so it always stays same-origin (works in internal/external browsers).
    const response = await fetch("/api/config");
    if (response.ok) {
      const config = await response.json();
      // Only update apiBase if backend returns something compatible with the current origin.
      // Otherwise, keep the safe same-origin default (prevents 127.0.0.1/localhost from breaking external browsers).
      try {
        const curOrigin = (window.location.origin || "").replace(/\/$/, "");
        const cfgOrigin = (config.api_base_url || "").replace(/\/$/, "");
        if (curOrigin && cfgOrigin && curOrigin === cfgOrigin) {
          defaultApiBase = config.api_base_url;
          apiBase = config.api_base_url;
          console.log("API Config loaded from backend:", apiBase);
        } else {
          apiBase = defaultApiBase;
        }
      } catch (_) {
        apiBase = defaultApiBase;
      }
    }
  } catch (error) {
    console.warn("⚠ Could not fetch API config from backend, using same-origin:", defaultApiBase);
    apiBase = defaultApiBase;
  }
}

```

Figure 54 API Routes and API Config

```
import logging
import time
from typing import Any, Dict

logger = logging.getLogger(__name__)

ws_manager = None
WEBSOCKET_MANAGER_AVAILABLE = False

Windsurf: Refactor | Explain
def init_websocket_manager():
    """No-op: live push disabled."""
    pass

Windsurf: Refactor | Explain
async def register_connection(websocket):
    """No-op."""
    pass

Windsurf: Refactor | Explain
async def unregister_connection(websocket):
    """No-op."""
    pass

Windsurf: Refactor | Explain
async def broadcast(message: Dict[str, Any]):
    """No-op: no SSE/WebSocket; frontend polls status."""
    pass

Windsurf: Refactor | Explain | Generate Docstring
async def send_tool_start_update(scan_id: int, tool_name: str):
    logger.info(f"[Scan {scan_id}] Tool started: {tool_name}")
    await broadcast({"type": "tool_start", "scan_id": scan_id, "tool": tool_name})

Windsurf: Refactor | Explain | Generate Docstring
async def send_tool_status_update(scan_id: int, tool_name: str, status: str, details: Dict[str, Any] = None):
    logger.debug(f"[Scan {scan_id}] {tool_name} status: {status}")
    await broadcast({
        "type": "tool_status",
        "scan_id": scan_id,
        "data": {"tool_name": tool_name, "status": status, "details": details or {}, "timestamp": time.time()}
    })

```

Figure 55 Live Updates and Websocket Setup

```

let apiBase = defaultApiBase;
let trendChart;
let severityChart;
/* ===== LIVE STATUS TRACKER – State (per Implementation Overview) =====
 * Data flow: Scan Launch → addLiveStatusItem() → startStatusPolling() + connectSSE()
 *           → Polling (every 5s) + SSE/WebSocket events
 *           → addLiveStatusItem() / handleWebSocketMessage()
 *           → renderLiveStatus() → Updates #liveStatusList in the DOM
 * Polling: primary source (GET /api/scans/{id}/status every 5s)
 * SSE: optional real-time (EventSource /api/sse/scan/{id})
 */
let liveStatus = []; // Rolling snapshot list (max 6 updates)
let statusPollingIntervals = {}; // Per-scan polling intervals
let commandTracking = {}; // Per-tool command/output tracking
let fallbackNotificationsShown = {}; // Track fallback alerts by scanId
const LIVE_STATUS_UPDATE_INTERVAL_MS = 5000; // Throttle: only show updates every 5 seconds
const lastLiveStatusUpdateByScan = {}; // scanId → timestamp
// Initialize liveStatus with OWASP category field

```

Figure 56 Live Status Tracker and Polling workflow

```

// Track whether the user is currently manually scrolling in summary panel
let isUserScrolling = false;

// Tools & OWASP
const TOOL_LIST = [
  "Nuclei",
  "Naabu",
  "Httpx",
  "Subfinder",
  "Amass",
  "Assetfinder",
  "Sublist3r",
  "GAU",
  "Katana",
  "GoSpider",
  "FFuf",
  "Wfuzz",
  "CeWL",
  "DNSx",
  "ShuffleDNS",
];

// All 15 tools mapped to OWASP Top 10 2021 categories (each tool appears in at least one category)
const OWASP_MAP = [
  { id: "A01:2021", name: "Broken Access Control", tools: ["Nuclei", "FFuf", "Wfuzz", "Katana", "GoSpider", "GAU", "Subfinder", "Sublist3r"] },
  { id: "A02:2021", name: "Cryptographic Failures", tools: ["Nuclei", "Httpx"] },
  { id: "A03:2021", name: "Injection", tools: ["Nuclei", "FFuf", "Wfuzz", "Katana", "GoSpider", "GAU"] },
  { id: "A04:2021", name: "Insecure Design", tools: ["Nuclei", "Wfuzz", "Katana", "FFuf"] },
  { id: "A05:2021", name: "Security Misconfiguration", tools: ["Naabu", "Httpx", "Nuclei", "Subfinder", "DNSx", "Sublist3r"] },
  { id: "A06:2021", name: "Vulnerable and Outdated Components", tools: ["Nuclei", "Subfinder", "Amass", "Assetfinder", "Sublist3r", "DNSx", "ShuffleDNS"] },
  { id: "A07:2021", name: "Identification & Authentication Failures", tools: ["Nuclei", "FFuf", "Wfuzz", "CeWL"] },
  { id: "A08:2021", name: "Software & Data Integrity Failures", tools: ["Nuclei", "Httpx"] },
  { id: "A09:2021", name: "Security Logging & Monitoring Failures", tools: ["Nuclei", "Httpx"] },
  { id: "A10:2021", name: "Server-Side Request Forgery", tools: ["Httpx", "Nuclei", "GAU", "Subfinder", "Amass", "DNSx"] },
];

```

Figure 57 Tool List and OWASP Mapping

More code of the frontend is in **Appendix 3**, [Section 7.3.3](#).

3.5.5. Sprint 5: Reporting Module Development

Sprint 5 added the full report generation feature to allow the system to generate AI-driven HTML and downloadable PDF reports of completed scans. The reporting layer is combined with the Gemini AI API to produce professional executive summaries, remediation advice, and analysis of relevance of attacks.

Key implementations:

1. report_generator.py that applies Gemini AI-based report content generation and multi-backend failure logic.
2. report-service.py writing the HTML report builder where the in-memory caching will allow the builder to not call the AI multiple times.

3. Endpoint to generate PDF reports with scan results and severity distribution and download. `api/reports.py` revealing HTML and PDF report-endpoints with scan persistence through mark-saved feature.
4. Deduplication logic is applied to prevent counting the same findings more than once in various tools.

```

"""Report generation service."""
import asyncio
import base64
import html
import json
import re
from datetime import datetime
from pathlib import Path
from typing import Optional, Dict, Any, List

from sqlalchemy import select
from sqlalchemy.ext.asyncio import AsyncSession

from app.models.scan import Scan
from app.models.tool_run import ToolRun
from app.models.finding import Finding
from app.core.logging import get_logger
from app.core.config import settings
from app.services.intelligence_service import IntelligenceService
from app.ai.report_generator import (
    generate_report_executive_summary,
    generate_report_conclusion,
    generate_remediation_playbook_table_html,
    _fallback_ai_analysis,
    _fallback_attack_relevance,
)

logger = get_logger(__name__)

# In-memory cache for generated HTML reports: scan_id -> html. Once generated, same scan serves cached.
_REPORT_HTML_CACHE: Dict[int, str] = {}
_REPORT_CACHE_MAX = 100

Windsurf: Refactor | Explain
class ReportService:
    """Service for generating scan reports."""

    Windsurf: Refactor | Explain | Generate Docstring
    @staticmethod
    def _severity_rank(severity: str) -> int:
        rank = {
            "info": 0,
            "low": 1,
            "medium": 2,
            "high": 3,
            "critical": 4,
        }
        return rank.get(str(severity).lower(), 0)

```

Figure 58 Report Generation and Cache Set-up

```

"""AI-powered report content generation using Gemini. Reports are strictly AI-only; no fallbacks."""
import asyncio
import html
import json
import httpx
from typing import Dict, Any, List, Optional

from app.core.config import settings
from app.core.logging import get_logger

logger = get_logger(__name__)

GEMINI_MAX_RETRIES = 3

Windsurf: Refactor | Explain
def deduplicate_findings(findings: List[Dict[str, Any]]) -> List[Dict[str, Any]]:
    """
    Deduplicate findings across tools to avoid over-counting the same issue.
    Keyed on vulnerability/template + location.
    """
    unique = {}
    for f in findings:
        key = (
            f.get("description") or f.get("template_id") or "",
            f.get("location") or ""
        )
        if key not in unique:
            unique[key] = f
    return list(unique.values())

Windsurf: Refactor | Explain
def _429_backoff_seconds(attempt: int) -> int:
    """Seconds to wait before retry on 429. Uses GEMINI_429_BACKOFF_BASE (default 3 ~ 3, 6, 12)."""
    base = getattr(settings, "GEMINI_429_BACKOFF_BASE", 3) or 3
    return base * (2 ** attempt)

```

Figure 59 Report Generation based on Gemini with Duplicate Filtering and Retry Logic

```

Windsurf: Refactor | Explain
async def _post_one_backend_with_retry(
    url: str, payload: Dict[str, Any], timeout: float
) -> httpx.Response:
    """POST to one Gemini URL with retries on 429 and connection/DNS errors. Raises when exhausted."""
    last_error: Optional[Exception] = None
    last_response: Optional[httpx.Response] = None
    for attempt in range(GEMINI_MAX_RETRIES):
        try:
            async with httpx.AsyncClient(timeout=timeout) as client:
                resp = await client.post(url, json=payload)
                last_response = resp
                if resp.status_code == 200:
                    return resp
                if resp.status_code == 429 and attempt < GEMINI_MAX_RETRIES - 1:
                    wait = _429_backoff_seconds(attempt)
                    logger.warning(
                        "Gemini 429 rate limit, retry %s/%s in %ss (quota resets midnight Pacific)",
                        attempt + 1,
                        GEMINI_MAX_RETRIES,
                        wait,
                    )
                    await asyncio.sleep(wait)
                    continue
                resp.raise_for_status()
        except (httpx.ConnectError, OSError) as e:
            last_error = e
            if attempt < GEMINI_MAX_RETRIES - 1:
                wait = _429_backoff_seconds(attempt)
                logger.warning(
                    "Gemini connection/DNS error (retry %s/%s in %ss): %s",
                    attempt + 1,
                    GEMINI_MAX_RETRIES,
                    wait,
                    e,
                )
                await asyncio.sleep(wait)
            else:
                raise
    if last_error:
        raise last_error
    if last_response is not None:
        last_response.raise_for_status()
    raise RuntimeError("Gemini request failed after retries")

```

Figure 60 Gemini API Retry Logic to generate Reports

```

Windsurf: Refactor | Explain
async def _gemini_post_with_retry(
    payload: Dict[str, Any], timeout: float = 20.0
) -> httpx.Response:
    """POST to Gemini: try each configured backend (model+key). On 429 or connection failure, try next."""
    backends = settings.get_gemini_backends()
    if not backends:
        raise ValueError(
            "Report generation requires AI (Gemini). Set GEMINI_API_KEY or GEMINI_BACKENDS in environment."
        )
    last_error: Optional[Exception] = None
    base_url = "https://generativelanguage.googleapis.com/v1beta/models/"
    for model, key in backends:
        url = f"{base_url}/{model}:generateContent?key={key}"
        try:
            return await _post_one_backend_with_retry(url, payload, timeout)
        except (httpx.HTTPStatusError, httpx.ConnectError, OSError, RuntimeError) as e:
            last_error = e
            status = getattr(getattr(e, "response", None), "status_code", None)
            if status == 429 or isinstance(e, (httpx.ConnectError, OSError)):
                logger.warning(
                    "Backend %s failed (%s), trying next backend if any.",
                    model,
                    e,
                )
                continue
            raise
    if last_error:
        status = getattr(getattr(last_error, "response", None), "status_code", None)
        if status == 429:
            logger.info(
                "All Gemini backends returned 429. Quota resets at midnight Pacific Time. "
                "Use an API key from a different Google Cloud project for a separate quota."
            )
        raise last_error
    raise RuntimeError("All Gemini backends failed")

```

Figure 61 Gemini backend Fallback and Retry

```

"""Report generation endpoints."""
from fastapi import APIRouter, Depends, HTTPException
from fastapi.responses import HTMLResponse, Response
from sqlalchemy.ext.asyncio import AsyncSession
from sqlalchemy import select
from app.models.scan import Scan

from app.core.database import get_db
from app.services.report_service import ReportService

router = APIRouter(prefix="/scans", tags=["reports"])

Windsurf: Refactor | Explain
@router.get("/{scan_id}/report")
async def get_scan_report(
    scan_id: int,
    db: AsyncSession = Depends(get_db)
):
    """Generate and return scan report as HTML."""
    html = await ReportService.generate_html_report(db, scan_id)

    if not html:
        raise HTTPException(status_code=404, detail="Scan not found")

    return HTMLResponse(content=html)

Windsurf: Refactor | Explain
@router.get("/{scan_id}/report/pdf")
async def get_scan_report_pdf(
    scan_id: int,
    db: AsyncSession = Depends(get_db)
):
    """Generate and return scan report as PDF."""
    pdf_bytes = await ReportService.generate_pdf_report(db, scan_id)

    if not pdf_bytes:
        raise HTTPException(
            status_code=500,
            detail="Failed to generate PDF. Ensure WeasyPrint is installed and the scan exists."
        )

    return Response(
        content=pdf_bytes,
        media_type="application/pdf",
        headers={"Content-Disposition": f"attachment; filename=IRS_Scan_{scan_id}_Report.pdf"}
    )

```

Figure 62 HTML and PDF Generation scan report Endpoints

```

Windsurf: Refactor | Explain
@router.post("/{scan_id}/mark-saved")
async def mark_scan_as_saved(
    scan_id: int,
    db: AsyncSession = Depends(get_db)
):
    """Mark a scan as saved for permanent storage.

    Also ensures the View Report content (AI-generated HTML) is persisted:
    if the report was never generated, generates it once and saves it, so that
    Saved Reports always have the same report content as View Report, and
    re-viewing never calls AI again.
    """
    result = await db.execute(select(Scan).where(Scan.id == scan_id))
    scan = result.scalar_one_or_none()

    if not scan:
        raise HTTPException(status_code=404, detail="Scan not found")

    # Ensure View Report content is persisted (generate once if not yet generated)
    if not (scan.report_html and scan.report_html.strip()):
        await ReportService.generate_html_report(db, scan_id)

    # Re-fetch scan after possible report generation (it commits)
    result = await db.execute(select(Scan).where(Scan.id == scan_id))
    scan = result.scalar_one()

    # Mark the scan as saved
    scan.is_saved = True
    if scan.error_summary:
        if "SAVED_SCAN" not in scan.error_summary:
            scan.error_summary = scan.error_summary + " | SAVED_SCAN"
    else:
        scan.error_summary = "SAVED_SCAN"

    await db.commit()

    return {"status": "success", "message": "Scan marked as saved", "scan_id": scan_id}

```

Figure 63 Mark Scan as Report Persistence Endpoints and Saved

More reporting module code, refer to **Appendix 3**, [Section 7.3.4](#)

3.5.6. Sprint 6: Integration and Improve

Sprint 6 was dedicated to integration, debugging, performance improvement and adding the scheduled scan functionality to finish the IRS. The system was tested end-to-end to make sure that all the parts were functioning properly, and the AI triage accuracy was checked against several test subjects.

Key implementations:

1. `validate_ai_plan_and_execution()` applied to check the accuracy of the selection of AI tools and the results of their execution.
2. `finding_classification` was developed as normal to avoid finding misclassification among OWASP categories.
3. `apply_owasp_scope_to_normalized()` to make sure that findings are properly scoped to the chosen OWASP category.
4. Scheduled scan service installed which enabled users to schedule recurring scans at set intervals.
5. Performance enhancements and bug fixes implemented in tool execution, AI decision, and report generation.


```

"""Scan orchestration service."""
import asyncio
import json
import shutil
from datetime import datetime, timedelta
from pathlib import Path
from typing import List, Dict, Any, Optional
from urllib.parse import urlparse
from sqlalchemy import select, func, desc, case
from sqlalchemy.ext.asyncio import AsyncSession
from app.models.scan import Scan, ScanStatus
from app.models.tool_run import ToolRun, ToolRunStatus
from app.models.finding import Finding, FindingSeverity, FindingType
from app.core.config import settings
from app.core.logging import get_logger
from app.tools.executor import execute_tool
from app.ai.decision_node import SAFE_FALLBACK_TOOLS, decide_tools
from app.core.ws_updates import init_websocket_manager
from app.ai.validation import validate_ai_plan_and_execution
from app.core.validation import validate_domain

AI_FALLBACK_MARKER = "AI_FALLBACK_MODE_ACTIVE"

logger = get_logger(__name__)

```

Figure 64 Scan Imports and Orchestration Service

```

Windsurf: Refactor | Explain
@staticmethod
def _normalize_finding_classification(
    raw_type: Optional[str],
    raw_severity: Optional[str],
    raw_owasp_category: Optional[str],
) -> Dict[str, Optional[str]]:
    """Normalize finding type/severity/owasp to prevent recon misclassification.

    Rules:
    - Recon/discovery finding types (asset/endpoint/port/information) are always INFO
      and must not carry OWASP vulnerability category.
    - Only vulnerability/misconfiguration findings may keep low/medium/high/critical and OWASP category.
    """
    type_map = {
        "asset": "asset",
        "endpoint": "endpoint",
        "port": "port",
        "vulnerability": "vulnerability",
        "misconfiguration": "misconfiguration",
        "information": "information",
        "info": "information",
    }
    sev_map = {
        "critical": "critical",
        "high": "high",
        "medium": "medium",
        "low": "low",
        "info": "info",
    }

    ftype = type_map.get((raw_type or "information").lower(), "information")
    fsev = sev_map.get((raw_severity or "info").lower(), "info")
    owasp_category = raw_owasp_category

    # Strict guard: non-vulnerability findings should never be elevated severity.
    # EXCEPTION: endpoint findings can retain their severity if they represent sensitive resource exposure
    # (e.g., Katana/GoSpider discovering aws_secrets.docx, /admin panels, etc.)
    if ftype not in {"vulnerability", "misconfiguration"}:
        # Allow endpoint findings to keep their classified severity (from intelligent URL analysis)
        # Only force to info for pure reconnaissance types (assets, ports, information)
        if ftype in {"asset", "port", "information"}:
            fsev = "info"
            owasp_category = None

    return {"type": ftype, "severity": fsev, "owasp_category": owasp_category}

```

Figure 65 Finding Classification to avoid misclassification


```
Windsurf: Refactor | Explain
@staticmethod
def _scan_has_owasp_focus(owasp_category: Optional[str]) -> bool:
    """True when the user picked an OWASP Top 10 category (A01-A10 style)."""
    if not owasp_category:
        return False
    o = str(owasp_category).strip().upper()
    return o.startswith("A") and ":2021" in o

Windsurf: Refactor | Explain
@staticmethod
def _apply_owasp_scope_to_normalized(
    scan_owasp_category: Optional[str],
    normalized: Dict[str, Any],
) -> Optional[Dict[str, Any]]:
    """When a scan is OWASP-focused, drop recon-only rows and align vuln/misconfig OWASP labels.

    Returns normalized dict to persist, or None to skip this finding entirely.
    """
    if not ScanService._scan_has_owasp_focus(scan_owasp_category):
        return normalized
    scan_o = str(scan_owasp_category).strip().upper()
    ftype = normalized.get("type")
    if hasattr(ftype, "value"):
        ftype = ftype.value
    ftype = (ftype or "information").lower()
```

Figure 66 Finding Normalization and OWASP Scope Check

```

Windsurf: Refactor | Explain
def _get_owasp_category_name(category_id: str) -> str:
    """Get the full OWASP category name from category ID.

    Args:
        category_id: OWASP category ID (e.g., A01:2021)

    Returns:
        Full category name
    """
    for category in settings.OWASP_CATEGORIES:
        if category["id"] == category_id:
            return category["name"]
    return f"Unknown ({category_id})"

Windsurf: Refactor | Explain
class ScanService:
    """Service for managing scan lifecycle and orchestration."""

    # Scans in CREATING/PREPARING/RUNNING older than this are treated as stuck and do not block new scans.
    STALE_RUNNING_THRESHOLD_MINUTES = 30

    # Passive/DNS-oriented tools can return irrelevant wildcard-like results
    # when the target is localhost / loopback. Skip them for accuracy.
    _LOCAL_DNS_TOOLS = {
        "Subfinder",
        "Amass",
        "Assetfinder",
        "Sublist3r",
        "DNSx",
        "ShuffleDNS",
    }

Windsurf: Refactor | Explain
    @staticmethod
    def _is_local_target(target: str) -> bool:
        """Return True if target is localhost/loopback (including host:port)."""
        t = (target or "").strip().lower()
        if not t:
            return False
        # Strip protocol if present
        if t.startswith(("http://", "https://")):
            parsed = urlparse(t)
            t = (parsed.hostname or "").strip().lower()
        else:
            # If host:port, strip port (ignore IPv6 for now)
            if ":" in t and not t.startswith("[") and not t.count(":") > 1:
                t = t.split(":", 1)[0].strip().lower()
            # If bracketed IPv6: [::1]
            if t.startswith("[") and t.endswith("]"):
                t = t.strip("[]")
        return t in {"localhost", "127.0.0.1", "::1"} or t.endswith(".localhost")

```

Figure 67 OWASP Category Lookup and Local Target Check

```

# app/ai/validation.py

from typing import List, Dict, Any, Tuple
from app.models.scan import Scan
from app.models.tool_run import ToolRun, ToolRunStatus

Windsurf: Refactor | Explain
def validate_ai_plan_and_execution(
    scan: Scan,
    tool_runs: List[ToolRun],
    ai_tools_to_run: List[str],
    ai_tools_skipped: List[str],
) -> Tuple[str, Dict[str, Any]]:
    """
    Validate AI-selected tools and their execution results.

    Returns:
        summary: Human-readable summary string.
        details: Structured dict with flags and counts.
    """
    summary_parts: List[str] = []
    details: Dict[str, Any] = {
        "ai_tools_to_run": ai_tools_to_run,
        "ai_tools_skipped": ai_tools_skipped,
        "total_tools_planned": len(ai_tools_to_run),
        "total_tools_executed": 0,
        "total_success": 0,
        "total_failed": 0,
        "total_timeout": 0,
        "missing_tool_runs": [],
    }

    # Index tool runs by name
    runs_by_name = {tr.tool_name: tr for tr in tool_runs}
    executed_names = set(runs_by_name.keys())

    # Count by status
    for tr in tool_runs:
        details["total_tools_executed"] += 1
        if tr.status == ToolRunStatus.COMPLETED:
            details["total_success"] += 1
        elif tr.status == ToolRunStatus.FAILED:
            details["total_failed"] += 1
        elif tr.status == ToolRunStatus.TIMEOUT:
            details["total_timeout"] += 1

    # Detect planned but missing executions
    for name in ai_tools_to_run:
        if name not in executed_names:
            details["missing_tool_runs"].append(name)

    # Build summary parts

```

Figure 68 AI plan validation and implementation overview

```

# Build summary text
if not ai_tools_to_run:
    summary_parts.append(
        "AI did not select any tools to run. This may indicate an error in the decision step."
    )
else:
    summary_parts.append(
        f"AI selected {len(ai_tools_to_run)} tools and {len(ai_tools_skipped)} tools were skipped."
    )

if details["missing_tool_runs"]:
    summary_parts.append(
        f"The following AI-selected tools have no execution record: {', '.join(details['missing_tool_runs'])}."
    )

if details["total_tools_executed"] == 0:
    summary_parts.append(
        "No tools were successfully started; scan results may be incomplete."
    )
else:
    summary_parts.append(
        f"{details['total_success']} tools completed, "
        f"{details['total_failed']} failed, "
        f"{details['total_timeout']} timed out."
    )

# Simple quality flag
if details["total_success"] == 0 and details["total_tools_planned"] > 0:
    details["overall_status"] = "poor"
    summary_parts.append(
        "Overall AI execution quality is poor because none of the planned tools completed successfully."
    )
elif details["missing_tool_runs"]:
    details["overall_status"] = "mixed"
else:
    details["overall_status"] = "good"

summary = " ".join(summary_parts)
return summary, details

```

Figure 69 AI Implementation Quality and Findings Logic

```

async def run_due_schedules(db: AsyncSession) -> int:
    """
    Find all enabled scheduled scans where next_run_at <= now and start a scan for each.

    Behaviour:
    - If the server was online at the scheduled time (within a small grace window),
      the scan is created as normal.
    - If the scheduled time is in the past beyond the grace window (server was down),
      the run is considered "missed": no scan is created, and next_run_at is advanced
      to the next occurrence (or the schedule is disabled for 'once').

    Returns the number of scans started.
    """
    now = datetime.utcnow()
    # How far past the scheduled time we still consider a run "on time".
    # Anything older than this is treated as a missed run and skipped.
    grace_window = timedelta(minutes=2)
    result = await db.execute(
        select(ScheduledScan).where(
            ScheduledScan.enabled.is_(True),
            ScheduledScan.next_run_at.isnot(None),
            ScheduledScan.next_run_at <= now,
        )
    )
    schedules = result.scalars().all()
    started = 0
    for s in schedules:
        # If this schedule's next_run_at is far in the past, treat it as missed.
        if s.next_run_at and s.next_run_at < now - grace_window:
            next_run = _next_run_after(now, s.frequency)
            if next_run is None:
                # 'once' schedule missed while server was down: disable without running
                s.enabled = False
                s.next_run_at = None
            else:
                s.next_run_at = next_run
            s.last_run_at = None
            await db.commit()
            logger.info(
                f"Scheduled scan {s.id} missed its window at {s.next_run_at} and was skipped; "
                f"next run is scheduled at {s.next_run_at or 'disabled'}"
            )
            continue
        tools_str = s.selected_tools
        if isinstance(tools_str, str):
            try:
                tools_list = json.loads(tools_str) if tools_str else []
            except Exception:
                tools_list = []

```

Figure 70 Planned Scan Runner and Missed Scan Processing

```

else:
    tools_list = list(tools_str) if tools_str else []
if not tools_list:
    continue
try:
    # scheduled_scan_id set + scan runs as scheduled flow (no clues, no AI; run chosen tools only)
    scan = await ScanService.create_scan(
        db=db,
        target=s.target,
        owasp_category=s.owasp_category,
        selected_tools=tools_list,
        scheduled_scan_id=s.id,
    )
    s.last_run_at = now
    next_run = _next_run_after(now, s.frequency)
    if next_run is None:
        # once: disable after run
        s.enabled = False
        s.next_run_at = None
    else:
        s.next_run_at = next_run
    await db.commit()
    started += 1
    logger.info(f"Scheduled scan {s.id} triggered scan {scan.id} for {s.target}")
except ValueError as e:
    logger.warning(f"Scheduled scan {s.id} skipped: {e}")
    continue
except Exception as e:
    logger.exception(f"Scheduled scan {s.id} failed: {e}")
    continue
return started

```

Figure 71 Plan Scan Trigger and Scan Exec Flow

Additional code of integration, see **Appendix 3**, [Section 7.3.5](#).

CHAPTER 4: TESTING AND ANALYSIS

4.1. TESTING

Testing is a necessary component of the development of this system, which is used to ensure that nothing goes wrong, and the existing problems that should be corrected by the time of the final deployment. During the development process, various tests were conducted to ensure every feature acts as per the plan, and the entire system is functional. Unit testing and system testing were also done in this project to test these individual units and the entire integrated system to make sure that all the units work well together to give the correct results.

4.1.1. Unit Testing, Test Plan

Test Cases	Objectives	Results
Case 1	To verify that system, backend runs with no errors.	<u>Successful</u>
Case 2	To Confirm Security Trend Analytics chart shows results of Findings and Vulnerabilities properly within a selected date range.	<u>Successful</u>
Case 3	To verify the capability of Launch Recon Scan in terms of input validation and successful reconnaissance execution.	<u>Successful</u>
Case 4	To validate the features of saved reports Filter by target, OWASP Category, and Status.	<u>Successful</u>
Case 5	To validate the delete feature for an individual Scanned Target.	<u>Successful</u>
Case 6	To validate the Schedule Scans form input and configuration logic.	<u>Successful</u>
Case 7	To verify the theme of switching logic of the system.	<u>Successful</u>
Case 8	To ensure that the Scanned Target Summary is showing the data of the Executive Summary properly after choosing a scan that has been saved.	<u>Successful</u>
Case 9	To validate Email Alert Activation and Receive Alert Email.	<u>Successful</u>
Case 10	To ensure that the Scanned Target Summary is showing the data of the Executive Summary properly after choosing a scan that has been saved.	<u>Successful</u>

Case 11	To ensure that the Live Status Tracker Runs and shows scan status updates in each 5 seconds.	<u>Successful</u>
Case 12	To validate, the report should open in a new tab when View Report is clicked from Saved Reports Security Reports Registry.	<u>Successful</u>
Case 13	To Validate “Quick target search” successfully open and shows details of searched target.	<u>Successful</u>
Case 14	To verify that when a tool Run button is clicked, the tool library correctly presents CLI Command which is configured and running in the system.	<u>Successful</u>
Case 15	To test that a particular tool slice on the donut chart or graph will show the tool details of its finding below the graph.	<u>Successful</u>

Table 1 Unit Testing, Test Plan

4.1.2. System Testing, Test Plan

Test Cases	Objectives	Results
Case 1	To verify the end-to-end system process runs correctly.	<u>Successful</u>
Case 2	To verify the end-to-end scheduled scan process.	<u>Successful</u>

Table 2 System Testing, Test Plan

4.2. UNIT TESTING

A unit test is a small section of code that verifies whether a particular function or method in an application is functioning properly. It will act as the input of the function and check the function output (GeeksforGeeks, 2025).

4.2.1. Test Case 1

Unit Testing	Objectives
Objective	To verify that system, backend runs with no errors.
Action	1. Start the Python virtual environment. 2. Run the uvicorn. 3. Browse to the dashboard on a web browser at <code>http:0.0.0.0:8080</code>
Expected Result	Server should run and health, scans and trend APIs should successfully 200 ok logs on terminal. The dashboard should run without errors.
Actual Result	Server started. The health, scans, and trend APIs have successfully 200 ok logs on terminal. The data was loaded to the dashboard.
Conclusion	Test is successful and UI and Backend are running and coordinated.

Table 3 Unit Test Case 1

Evidence:

```

prabesh@prabesh-Surface-Laptop-3:~/Desktop/Fyproject$ source /home/prabesh/Desktop/Fyproject/venv/bin/activate
(venv) prabesh@prabesh-Surface-Laptop-3:~/Desktop/Fyproject$ uvicorn app.main:app --host 0.0.0.0 --port 8080
INFO: Started server process [61060]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: Uvicorn running on http://0.0.0.0:8080 (Press CTRL+C to quit)
INFO: 127.0.0.1:37186 - "GET / HTTP/1.1" 200 OK
INFO: 127.0.0.1:37186 - "GET /styles.css?v=1.2.6 HTTP/1.1" 200 OK
INFO: 127.0.0.1:37194 - "GET /app.js?v=1.2.4 HTTP/1.1" 200 OK
INFO: 127.0.0.1:37194 - "GET /logo.png HTTP/1.1" 200 OK
INFO: 127.0.0.1:37194 - "GET /favicon.svg HTTP/1.1" 200 OK
INFO: 127.0.0.1:37186 - "GET /api/config HTTP/1.1" 200 OK
INFO: 127.0.0.1:37194 - "GET /api/scans?include_unsaved=false HTTP/1.1" 200 OK
INFO: 127.0.0.1:37214 - "GET /api/health HTTP/1.1" 200 OK
INFO: 127.0.0.1:37194 - "GET /api/scans/stats/severity HTTP/1.1" 200 OK
INFO: 127.0.0.1:37198 - "GET /api/scheduled-scans HTTP/1.1" 200 OK
INFO: 127.0.0.1:37214 - "GET /api/scans/stats/severity HTTP/1.1" 200 OK
INFO: 127.0.0.1:37186 - "GET /api/scans/stats/trend?start_date=2026-03-19&end_date=2026-03-25 HTTP/1.1" 200 OK
INFO: 127.0.0.1:37214 - "GET /api/settings/alert-email HTTP/1.1" 200 OK
INFO: 127.0.0.1:37198 - "GET /api/scheduled-scans/scan-results HTTP/1.1" 200 OK
INFO: 127.0.0.1:37214 - "GET /api/scans/stats/trend?start_date=2026-03-19&end_date=2026-03-25 HTTP/1.1" 200 OK
INFO: 127.0.0.1:37198 - "GET /api/scans?include_unsaved=true HTTP/1.1" 200 OK
INFO: 127.0.0.1:37198 - "GET /api/scheduled-scans HTTP/1.1" 200 OK
INFO: 127.0.0.1:37198 - "GET /api/scans?include_unsaved=false HTTP/1.1" 200 OK
INFO: 127.0.0.1:37198 - "GET /api/scans?include_unsaved=false HTTP/1.1" 200 OK

```

Figure 72 Backend Initialization and API Requests Logs

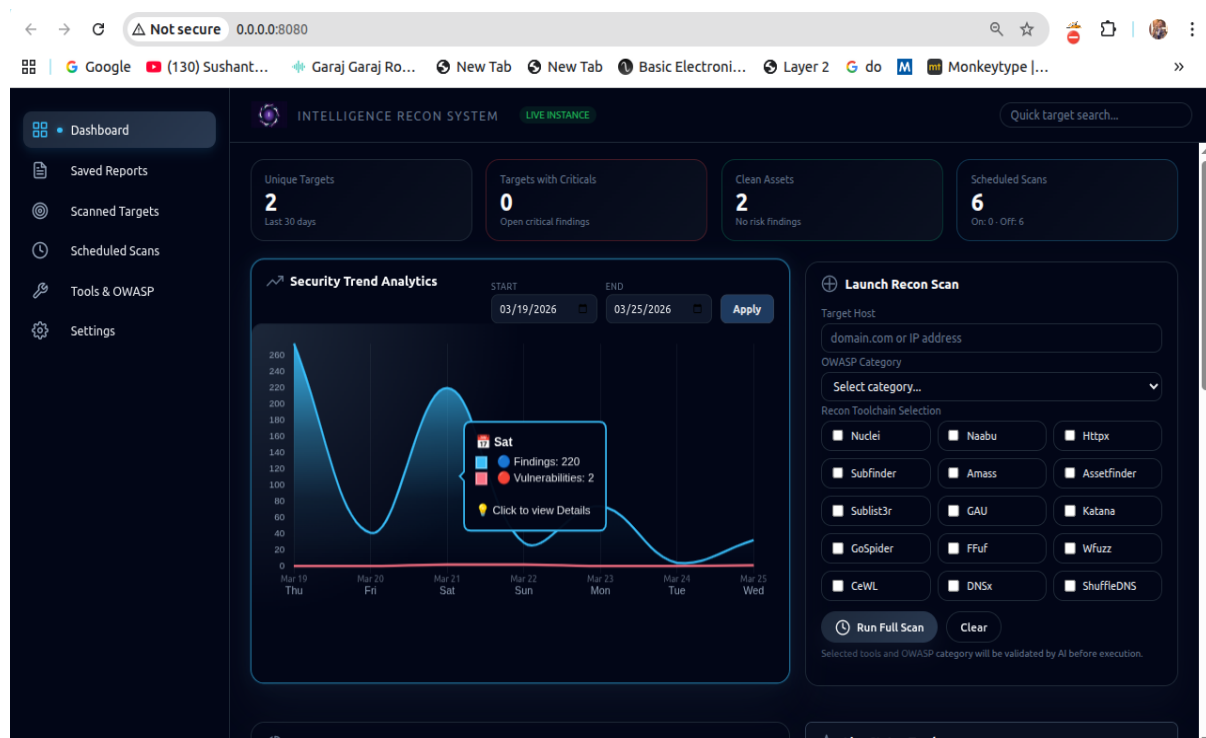


Figure 73 IRS Dashboard

4.2.2. Test Case 2

Unit Testing	Objectives
Objective	To Confirm Security Trend Analytics chart shows results of Findings and Vulnerabilities properly within a selected date range.
Action	1. Select a Start Date and End Date. 2. Click Apply Button 3. Hover mouse on chart to see data.
Expected Result	Chart data should show the Findings and Vulnerabilities of applied duration date.
Actual Result	Security Trend Analytics shows Findings and Vulnerabilities of selected Dates.
Conclusion	The test was successful.

Table 4 Unit Test Case 2

Evidence

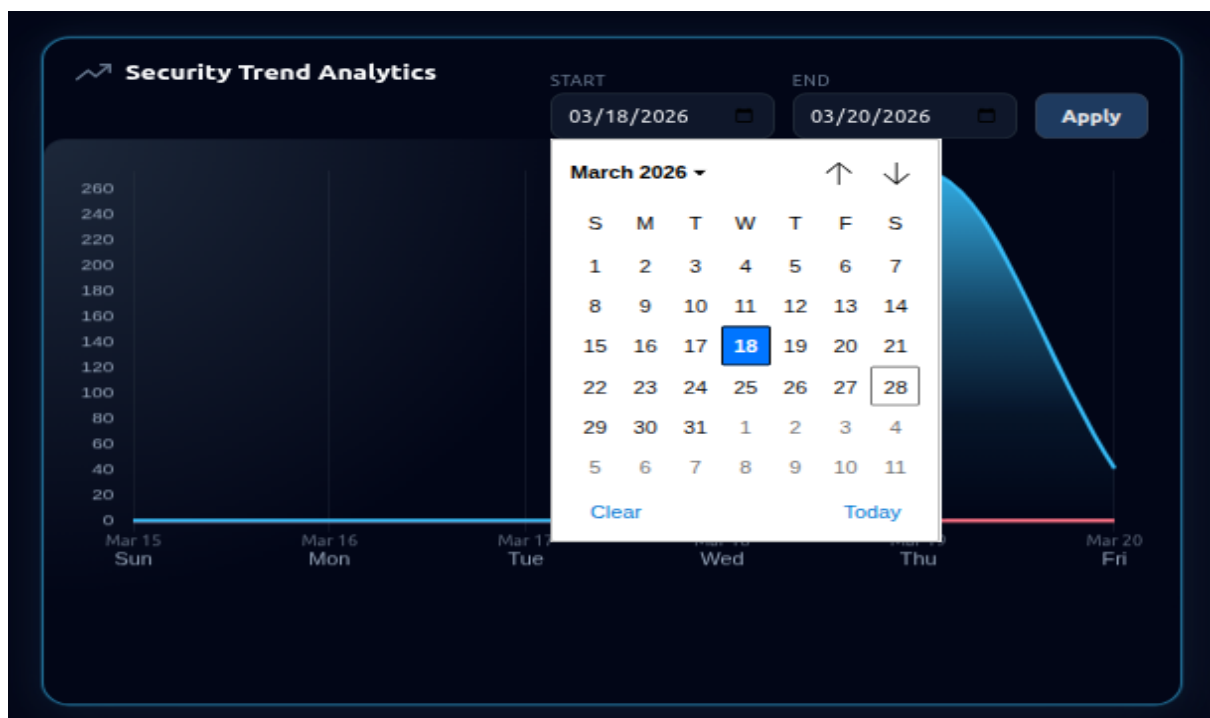


Figure 74 Test Evidence: Select Start date.

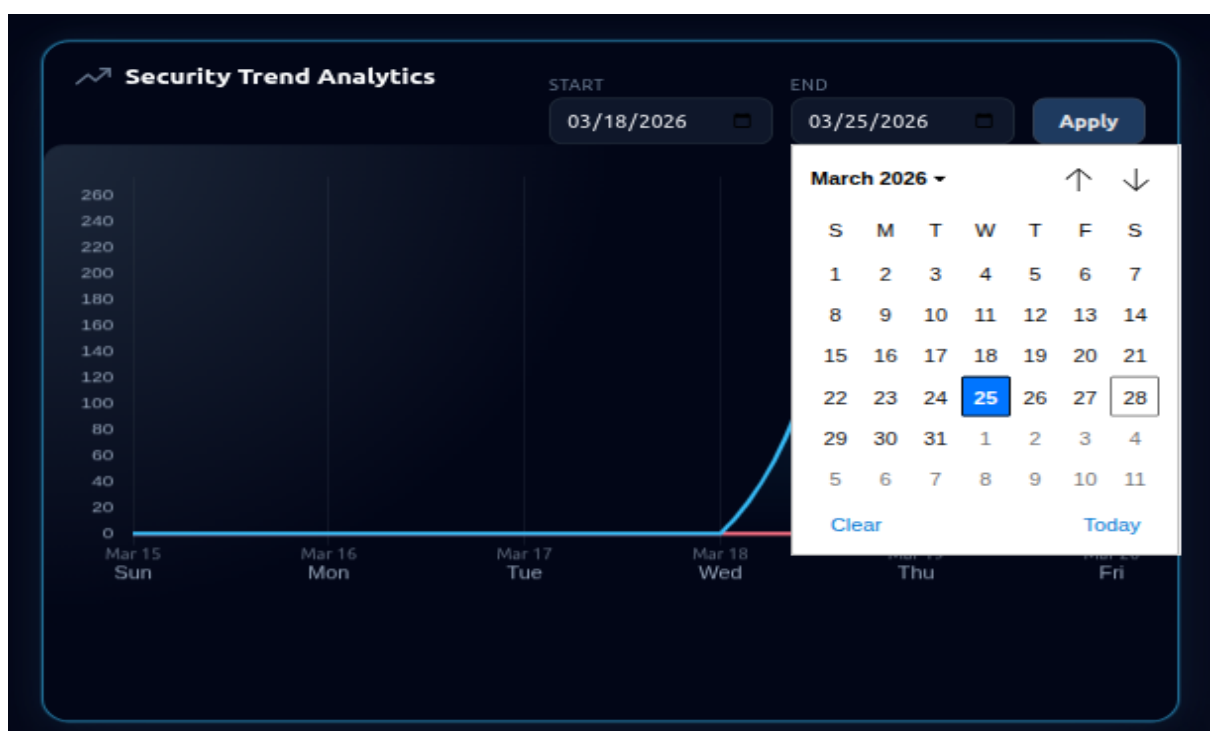


Figure 75 Test Evidence: Select End Date.



Figure 76 Test Evidence: Click the Apply Button.

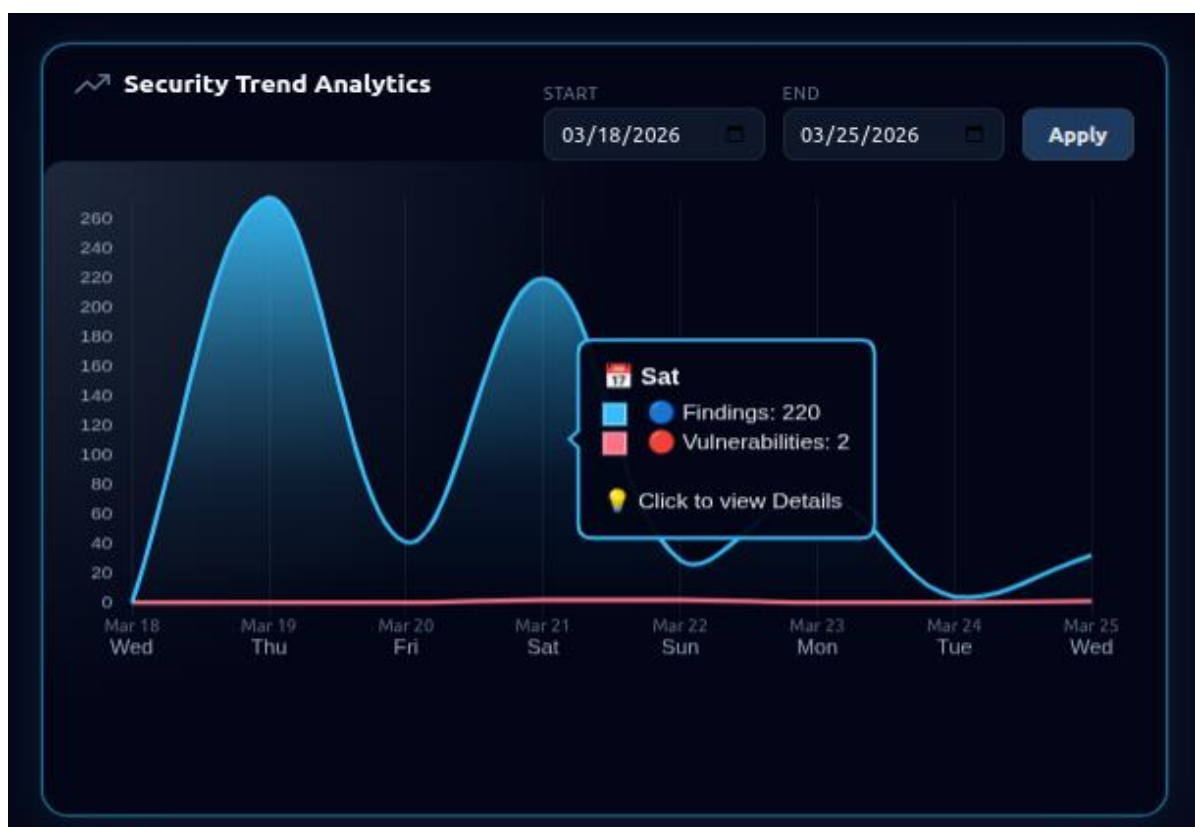


Figure 77 Test Evidence: Hover mouse to chart to view Findings and Vulnerabilities.

4.2.3. Test Case 3

Unit Testing	Objectives
Objective	To verify the capability of Launch Recon Scan in terms of input validation and successful reconnaissance execution.
Action	Case 1: • Fill target domain. • Choose OWASP category. • Leave Tools and Click Run Full Scan. Case 2: • Enter an invalid target domain. • Choose OWASP category, tools and try to attempt to run the scan. Case 3: • Enter a valid target domain. • Choose OWASP Category and tools and try to attempt to run the scan
Expected Results	Case 1: System will display the validation warning message” Please enter target, choose OWASP category, and select at least one tool”. Case 2: System will display warning message “Invalid Target” Case 3: System will execute the scan without any error with message.
Actual Result	Case 1: As expected, a validation warning appeared when the Run button was clicked without choosing tools. Case 2: As expected, warning message appeared when Invalid target was tried to execute. Case 3: As expected, new scan runs successfully without an error and “scan create” message appeared.
Conclusion	All the tests were successful.

*Table 5 Unit Test Case 3***Evidence Case 1**

Launch Recon Scan

Target Host
zero.webappsecurity.com

OWASP Category
Select category...

Recon Toolchain Selection

☐ Nuclei	☐ Naabu	☐ Httpx
☐ Subfinder	☐ Amass	☐ Assetfinder
☐ Sublist3r	☐ GAU	☐ Katana
☐ GoSpider	☐ FFuf	☐ Wfuzz
☐ CeWL	☐ DNSx	☐ ShuffleDNS

Run Full Scan **Clear**

Selected tools and OWASP category will be validated by AI before execution.

Figure 78 Enter Domain and message before Run full Scan was not clicked.

Launch Recon Scan

Target Host
zero.webappsecurity.com

OWASP Category
A01:2021 – Broken Access Control

Recon Toolchain Selection

☐ Nuclei	☐ Naabu	☐ Httpx
☐ Subfinder	☐ Amass	☐ Assetfinder
☐ Sublist3r	☐ GAU	☐ Katana
☐ GoSpider	☐ FFuf	☐ Wfuzz
☐ CeWL	☐ DNSx	☐ ShuffleDNS

Run Full Scan **Clear**

Please enter target, choose OWASP category, and select at least one tool.

Figure 79 Choose OWASP Category, leave tools and clicked Run Full Scan

Evidence Case 2

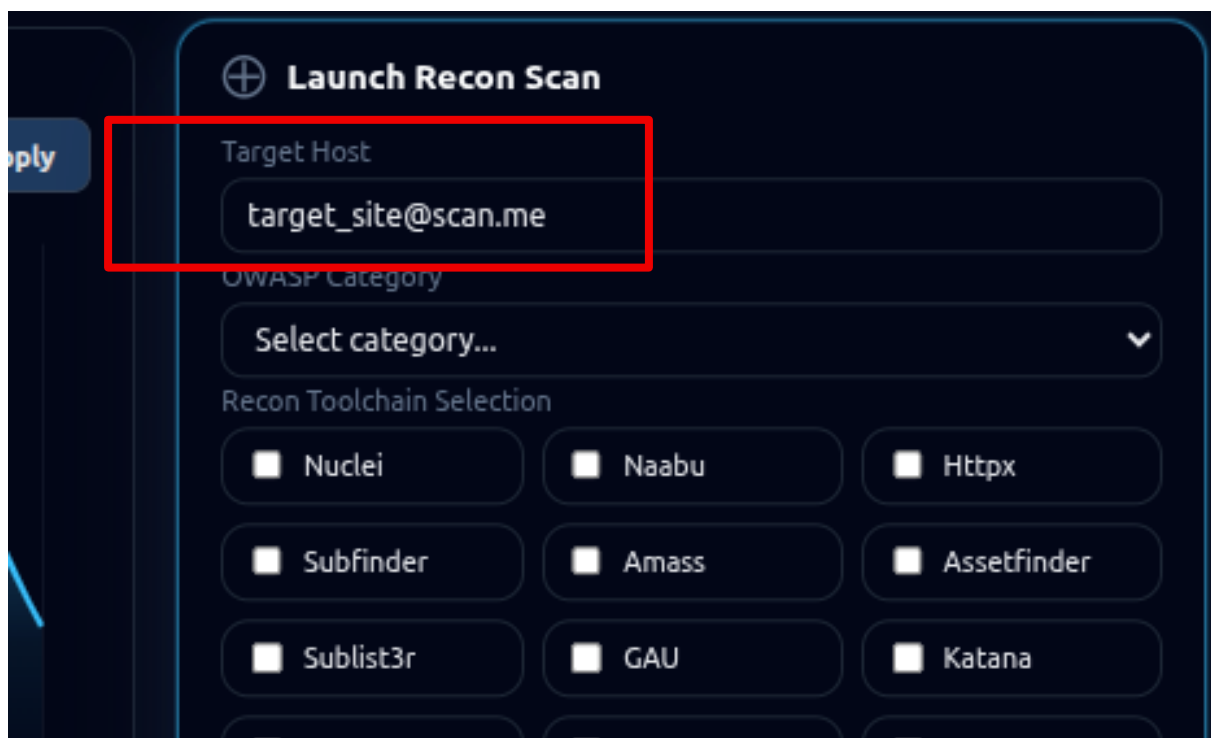


Figure 80 Enter Invalid Target Domain

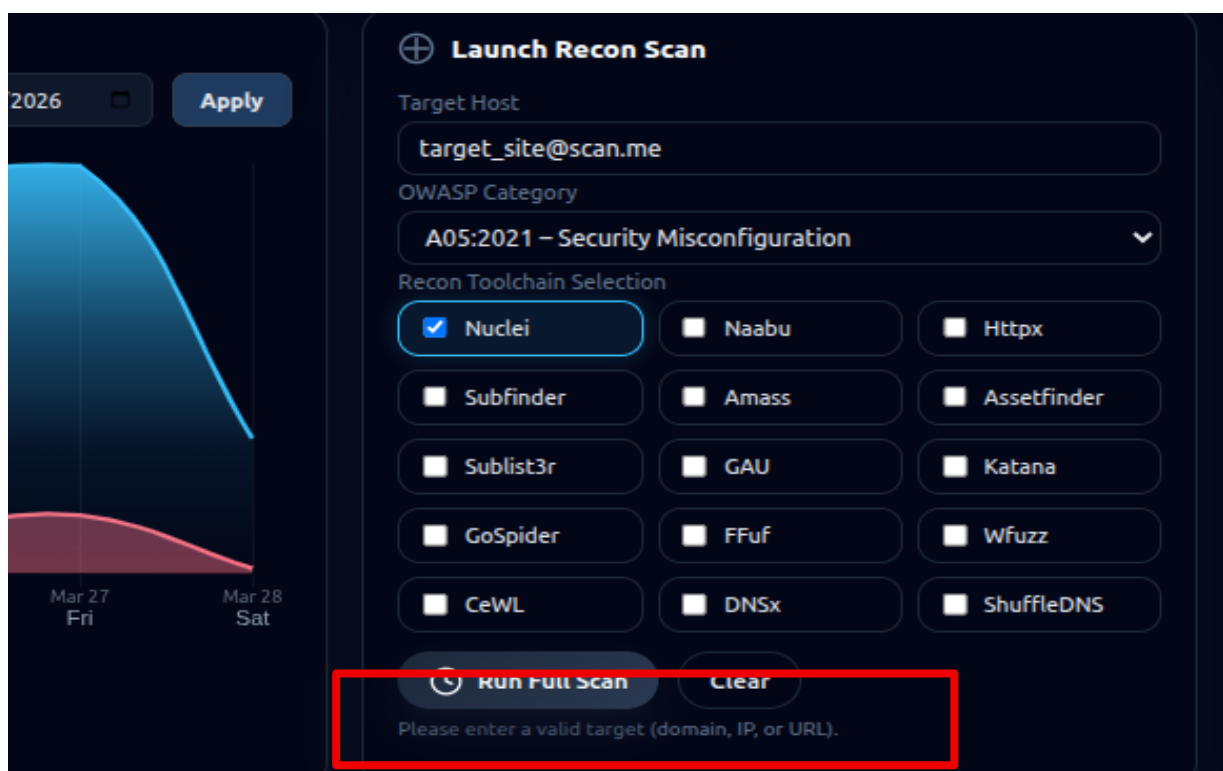
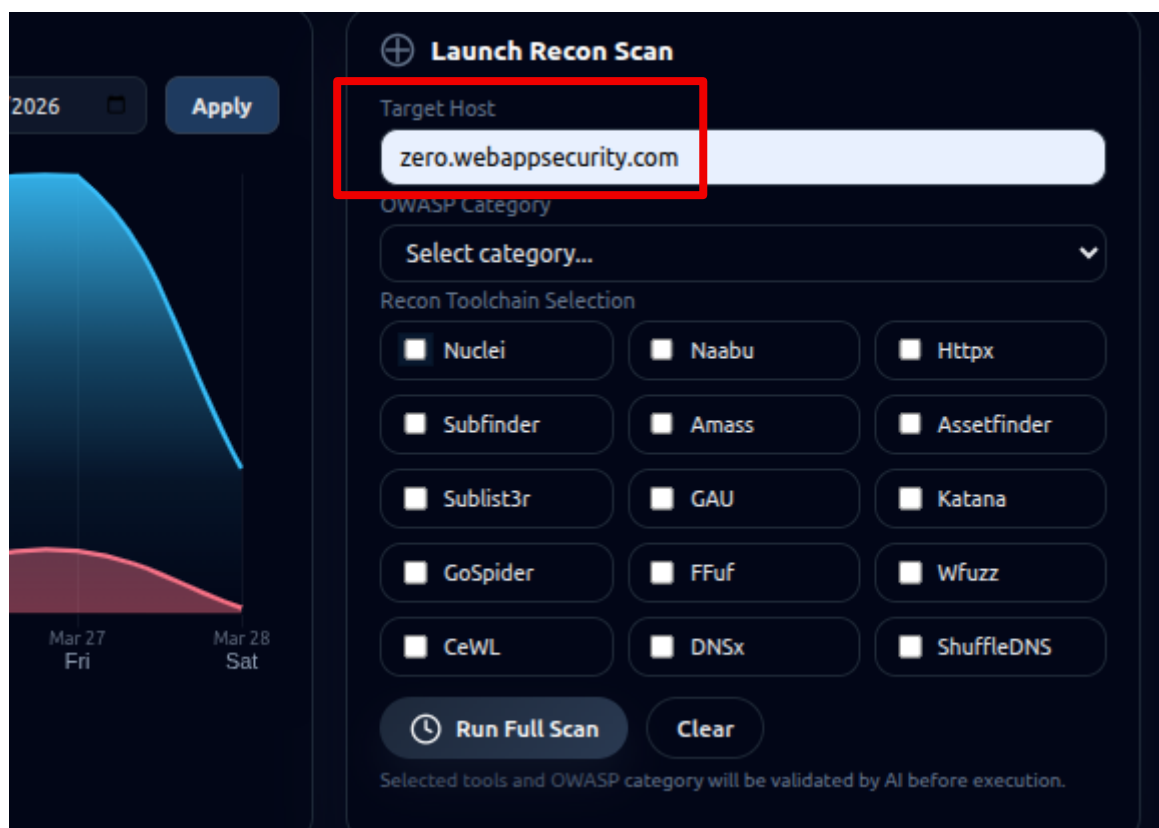


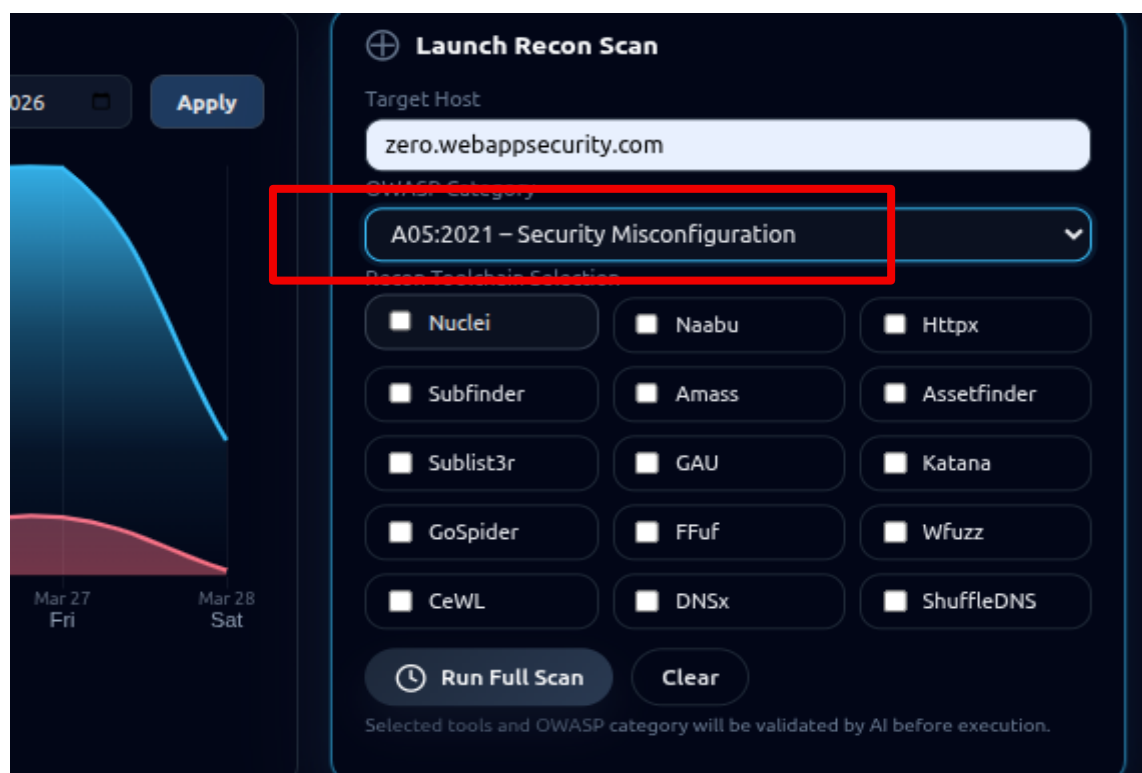
Figure 81 Validation domain message appeared.

Evidence Case 3



The screenshot shows the 'Launch Recon Scan' interface. On the left, there is a graph with a blue line and a red line, with labels for 'Mar 27 Fri' and 'Mar 28 Sat'. The main panel has a dark blue background. At the top, there is a plus icon and the text 'Launch Recon Scan'. Below this, the 'Target Host' field is highlighted with a red box and contains the text 'zero.webappsecurity.com'. Below the 'Target Host' field is the 'OWASP Category' dropdown menu, which is currently set to 'Select category...'. Below the 'OWASP Category' dropdown is the 'Recon Toolchain Selection' section, which contains a grid of buttons for various tools: Nuclei, Naabu, Httpx, Subfinder, Amass, Assetfinder, Sublist3r, GAU, Katana, GoSpider, FFuf, Wfuzz, CeWL, DNSx, and ShuffleDNS. At the bottom of the toolchain selection section are two buttons: 'Run Full Scan' and 'Clear'. Below these buttons is a small text note: 'Selected tools and OWASP category will be validated by AI before execution.'

Figure 82 Enter a Valid Domain



The screenshot shows the 'Launch Recon Scan' interface. On the left, there is a graph with a blue line and a red line, with labels for 'Mar 27 Fri' and 'Mar 28 Sat'. The main panel has a dark blue background. At the top, there is a plus icon and the text 'Launch Recon Scan'. Below this, the 'Target Host' field contains the text 'zero.webappsecurity.com'. Below the 'Target Host' field is the 'OWASP Category' dropdown menu, which is highlighted with a red box and contains the text 'A05:2021 – Security Misconfiguration'. Below the 'OWASP Category' dropdown is the 'Recon Toolchain Selection' section, which contains a grid of buttons for various tools: Nuclei, Naabu, Httpx, Subfinder, Amass, Assetfinder, Sublist3r, GAU, Katana, GoSpider, FFuf, Wfuzz, CeWL, DNSx, and ShuffleDNS. At the bottom of the toolchain selection section are two buttons: 'Run Full Scan' and 'Clear'. Below these buttons is a small text note: 'Selected tools and OWASP category will be validated by AI before execution.'

Figure 83 Choose OWASP Category

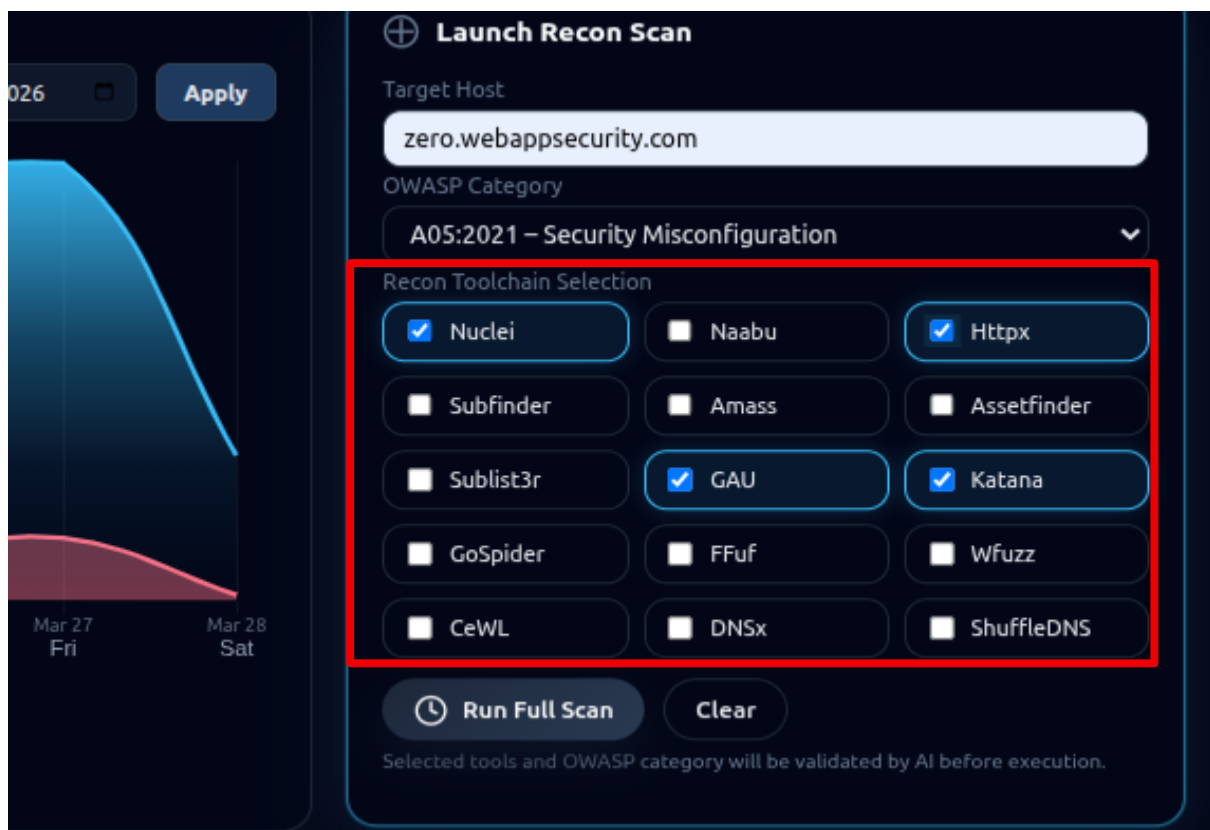


Figure 84 Choose tools for scanning.

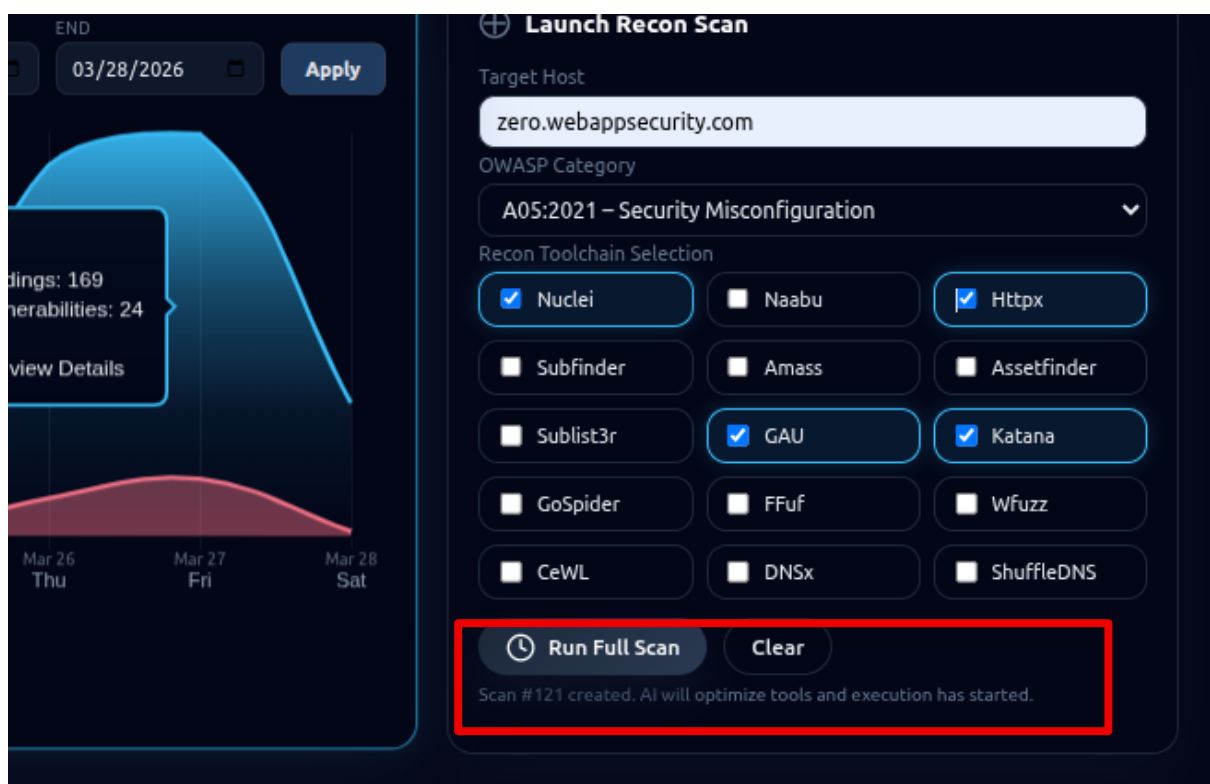


Figure 85 Scan Run Successfully and Scan Created message appeared.

4.2.4. Test Case 4

Unit Testing	Objectives
Objective	To validate the features of saved reports Filter by target, OWASP Category, and Status.
Action	1. Open Saved Reports page 2. In Filter by target box type zero.webappsecurity.com. 3. Choose OWASP Category (A03:2021). 4. Select Status as Running.
Expected Results	As there are no scans which are running and saved. So, Table remains blank with no records.
Actual Result	As expected, there was no running and saved scan for zero.webappsecurity.com. So, the table displays blank table.
Conclusion	The test was successful.

Table 6 Unit Test Case 4

Evidence

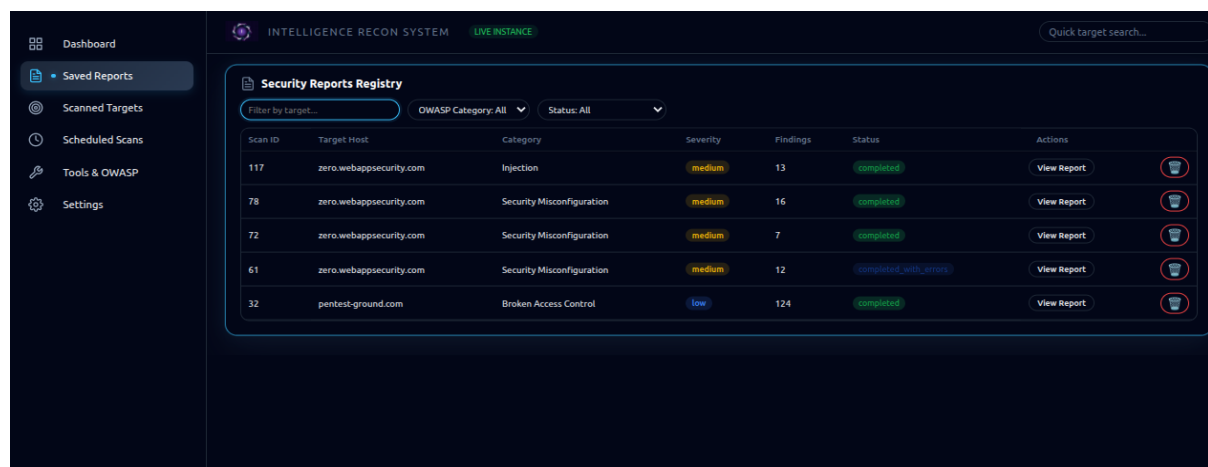


Figure 86 Navigate to Saved Reports.

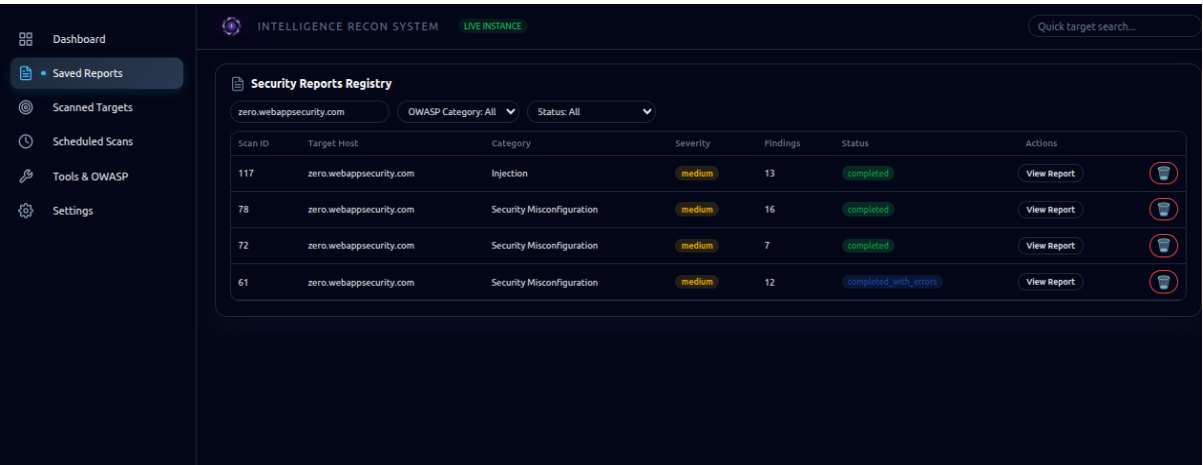


Figure 87 Filter target with zero.webappsecurity.com

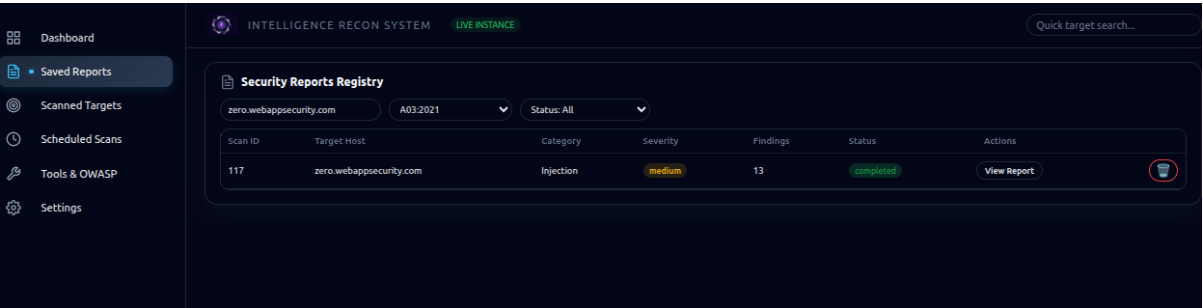


Figure 88 Choose OWASP A03:2021

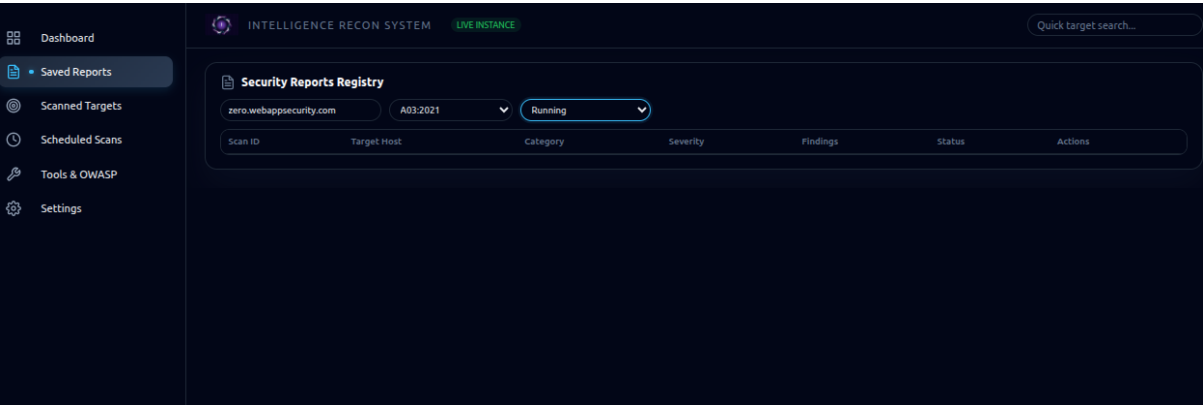


Figure 89 Select Status as Running.

4.2.5. Test Case 5

Unit Testing	Objectives
Objective	To validate the delete feature for an individual Scanned Target.
Action	1. Open the Scanned Targets page. 2. Click Details of Scan zerowebappsecurity.com. 3. Select the Delete option. 4. Delete Scan #57 and confirm Delete. 5. Check List of scans to confirm scan #57 was deleted and review the Total Scans and Total Findings updates.
Expected Results	Scan #57 is removed. Total Scans decrements from 46 to 45. Total Findings decreased from 315 to 304.
Actual Result	Scan #57 was deleted successfully. Metrics updated to 45 Total Scans and 304 Total Findings.
Conclusion	The test was Successful.

Table 7 Unit Test Case 5

Evidence

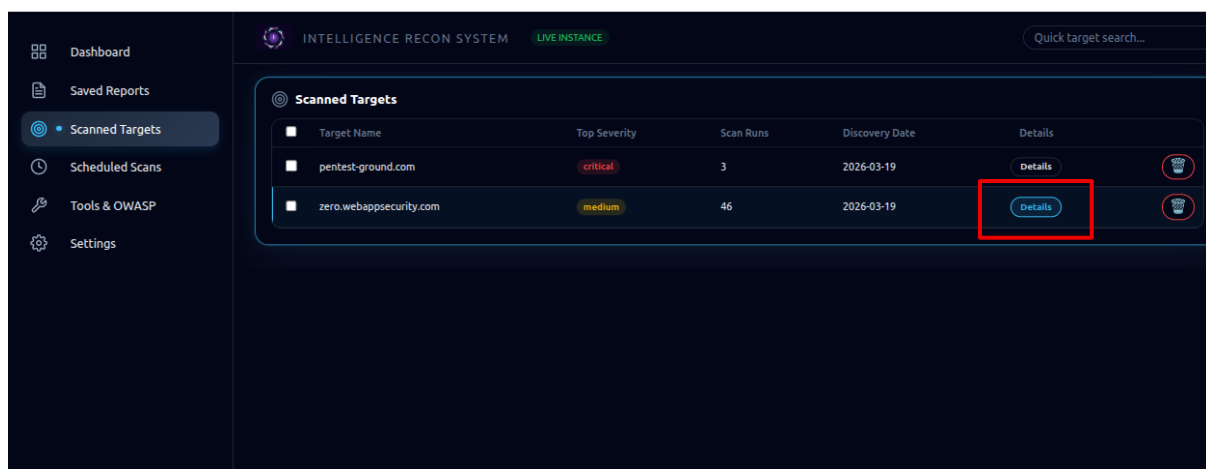


Figure 90 Open Scanned Targets page and click Details.

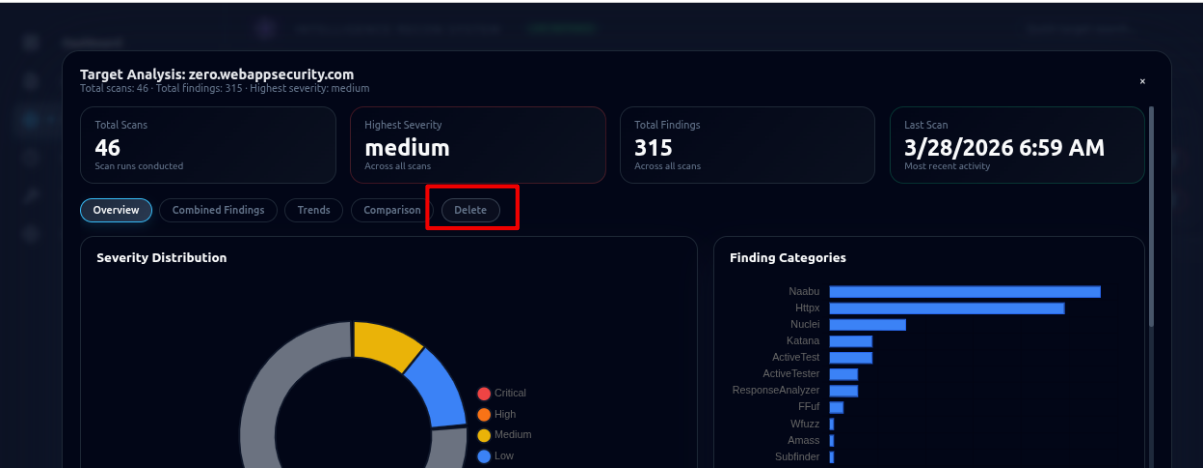


Figure 91 Opened Overview of Scans and choose option Delete



Figure 92 Click Delete button to delete scan

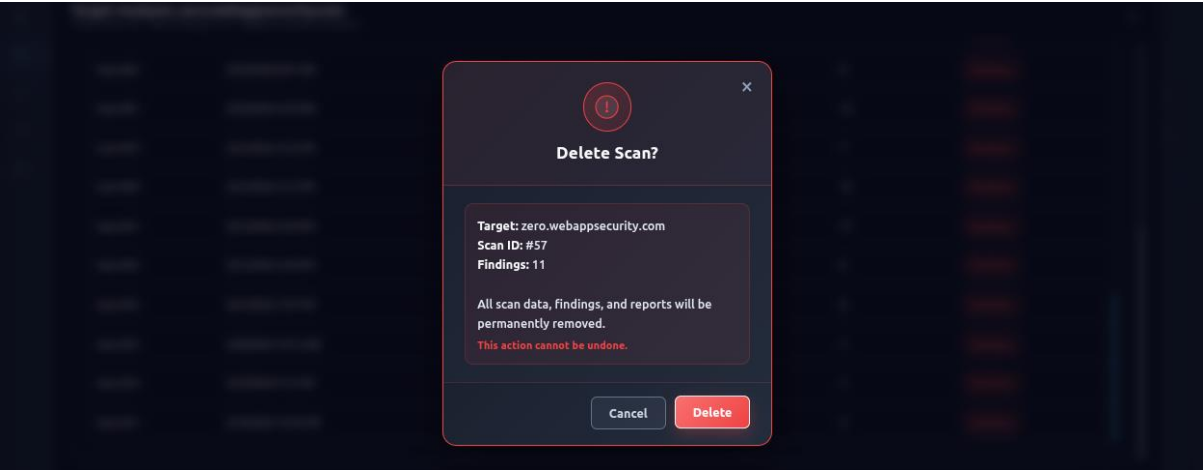
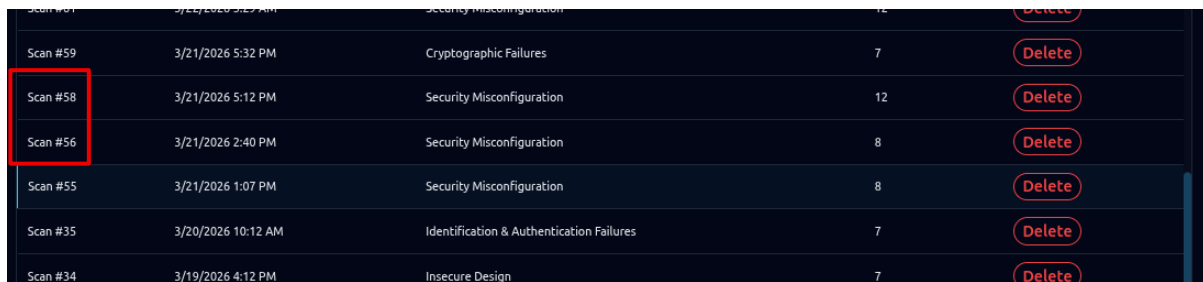


Figure 93 Confirm Delete Validation



Scan #59	3/21/2026 5:32 PM	Cryptographic Failures	7	Delete
Scan #58	3/21/2026 5:12 PM	Security Misconfiguration	12	Delete
Scan #56	3/21/2026 2:40 PM	Security Misconfiguration	8	Delete
Scan #55	3/21/2026 1:07 PM	Security Misconfiguration	8	Delete
Scan #35	3/20/2026 10:12 AM	Identification & Authentication Failures	7	Delete
Scan #34	3/19/2026 4:12 PM	Insecure Design	7	Delete

Figure 94 Check list of scans



Target Analysis: zero.webappsecurity.com
Total scans: 45 · Total findings: 304 · Highest severity: medium

Total Scans 45 Scan runs conducted	Highest Severity medium Across all scans	Total Findings 304 Across all scans	Last Scan 3/28/2026 6:59 AM Most recent activity
---	---	--	---

Overview Combined Findings Trends Comparison **Delete**

Delete a scan
Select a scan below to permanently delete it and all its data (findings, reports).

Figure 95 Check Update after Scan #57 Delete.

4.2.6. Test Case 6

Unit Testing	Objectives
Objective	To validate the Schedule Scans form input and configuration logic.
Action	1. Open Schedule Scans. 2. Enter a Target into the Target Host field. 3. Select OWASP Category (A03:2021-Injection). 4. Choose tools for scanning. 5. Toggle the Schedule switch to Enable.
Expected Results	The Add Schedule button should turn from gray to blue once all need conditions are met for Schedule Scan.
Actual Result	The button Add Schedule Successfully turns blue after all needed information for schedule scans are met.
Conclusion	The Test was Successful.

Table 8 Unit Test Case 6

Evidence

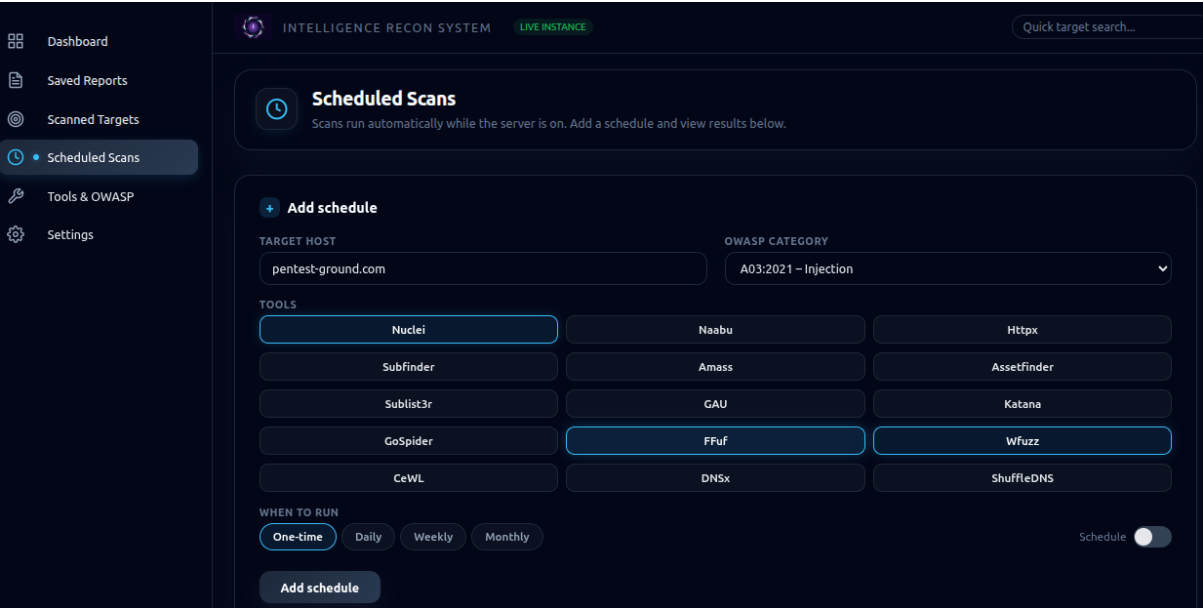


Figure 96 Input Valid domain, OWASP Category and tools, set one time for running scan.

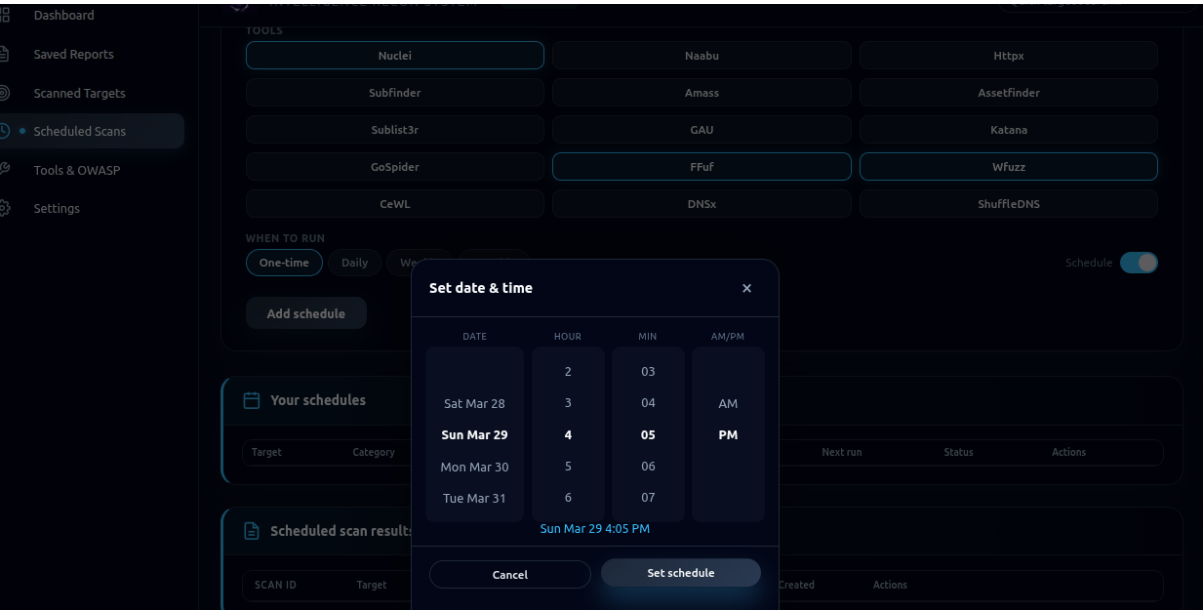


Figure 97 Enable Schedule Toggle and Set Date and Time.

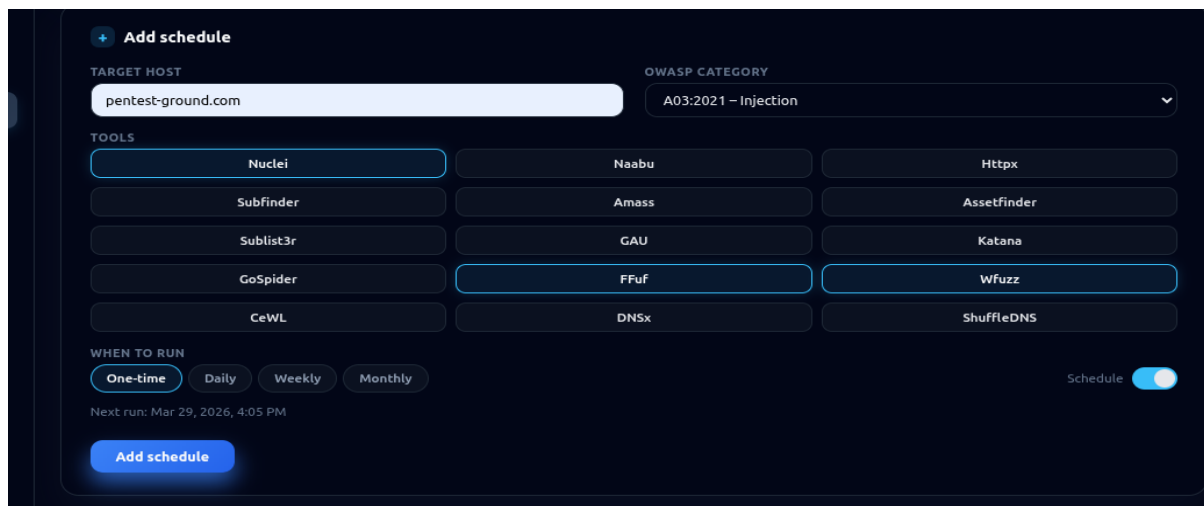


Figure 98 Add Schedule Button turns blue after all conditions are met.

4.2.7. Test Case 7

Unit Testing	Objectives
Objective	To verify the theme of switching logic of the system.
Action	1. Navigate to Settings. 2. Switch the system theme by selecting the Light theme.
Expected Results	The System should update the light theme immediately after selecting Light theme.
Actual Result	As expected, the system turns immediately from Dark mode to Light theme.
Conclusion	The test was Successful.

Table 9 Unit Test Case 7

Evidence

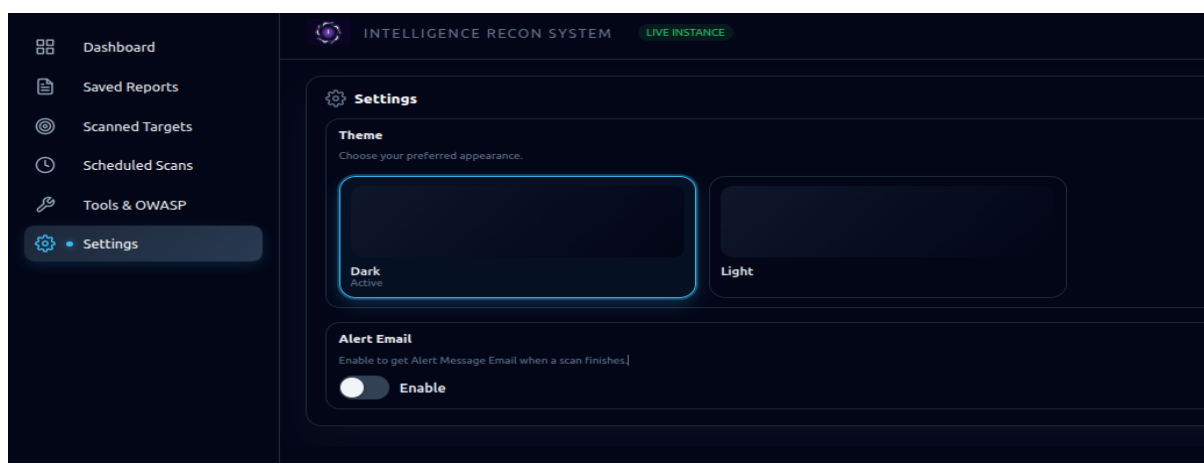


Figure 99 Opened Settings.

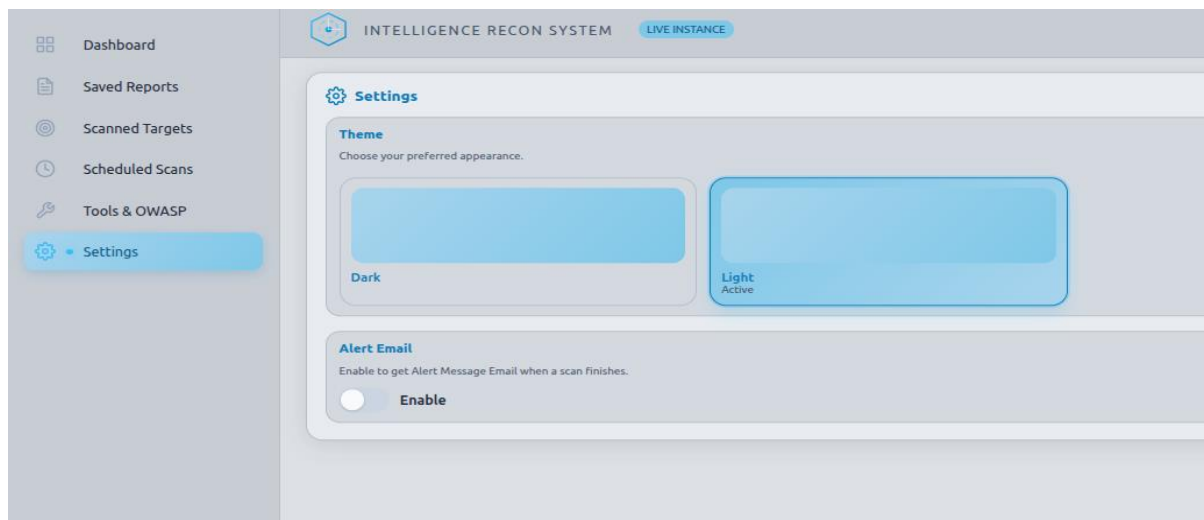


Figure 100 Switch to the Light Theme



Figure 101 Dashboard Light Theme View.

4.2.8. Test Case 8

Unit Testing	Objectives
Objective	To check the functionality of the dashboard Unique Targets, Targets with Critical, Clean Assets, and the Scheduled scans settings to view details.
Action	Case 1: 1. Start the system. 2. Click the Unique Targets card. 3. Details will be shown. Case 2: 1. Click the Targets with Critical. 2. View Details. Case 3: 1. Click on the Clean Assets. 2. View Details. Case 4: 1. Click on Scheduled Scans.
Expected Results	Case 1: Unique Targets should show the popup showing the Targets lists after Clicking Unique Targets Card. Case 2: Targets with Critical should open the popup with Critical Target details list. Case 3: Clean Assets should show the popup with a list of details for Clean Assets. Case 4: Scheduled Scans page should open directly to view details after clicking Scheduled Scans card.
Actual Result	Case 1: As Expected, the list of targets was shown with popup. Case 2: The list of details for Targets with Critical was shown with popup as expected. Case 3: The target with Clean Assets was opened with popup to view details. Case 4: The Scheduled Scans section was directly opened by clicking the Scheduled Scans card was clicked.
Conclusion	All the tests were successful.

Table 10 Unit Test Case 8

Evidence Case 1



Figure 102 Hover and Clicked Unique Targets.

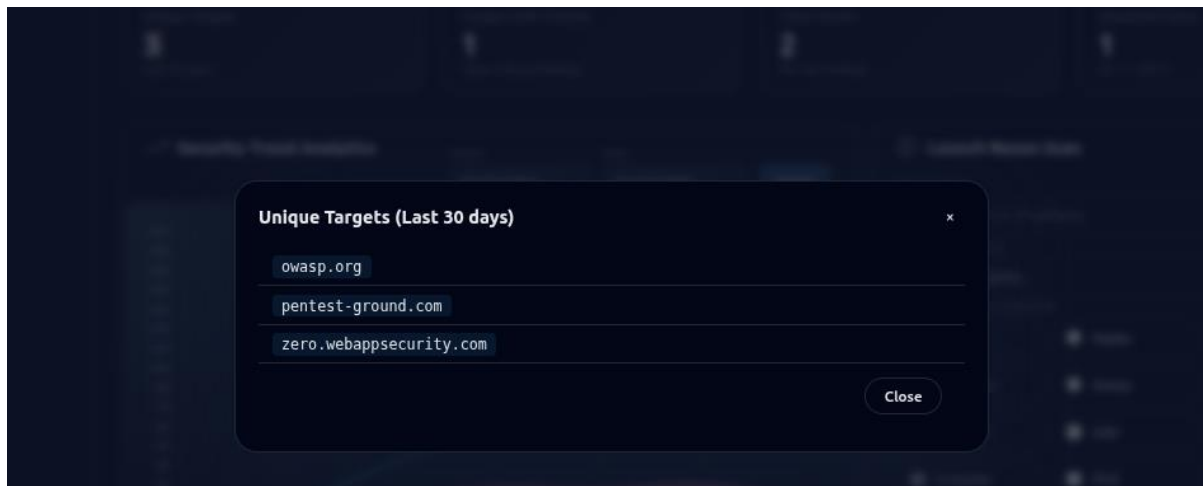


Figure 103 Details of Unique Targets.

Evidence Case 2

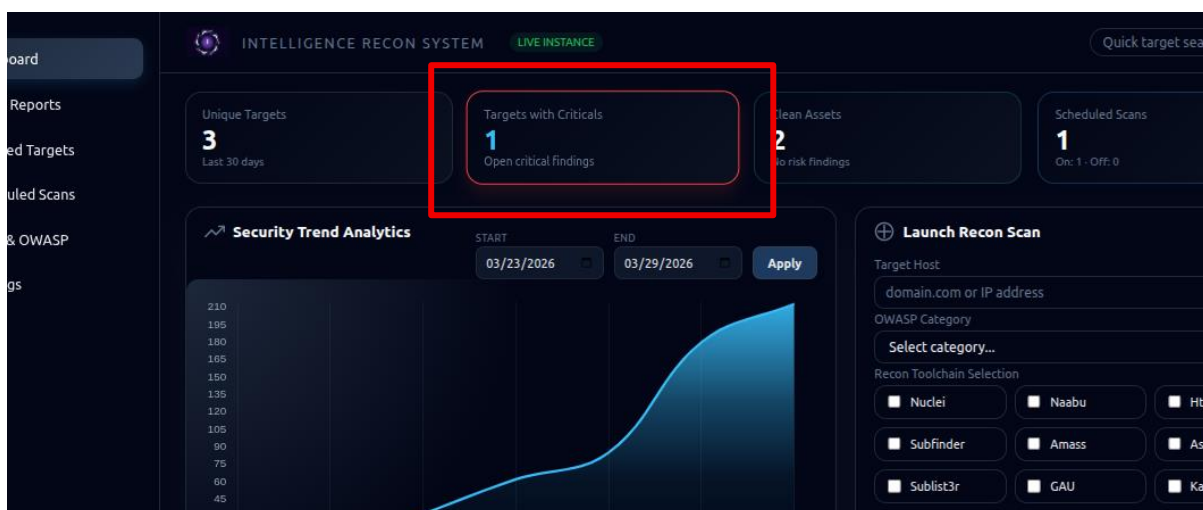


Figure 104 Hover and Clicked Targets with Critical.

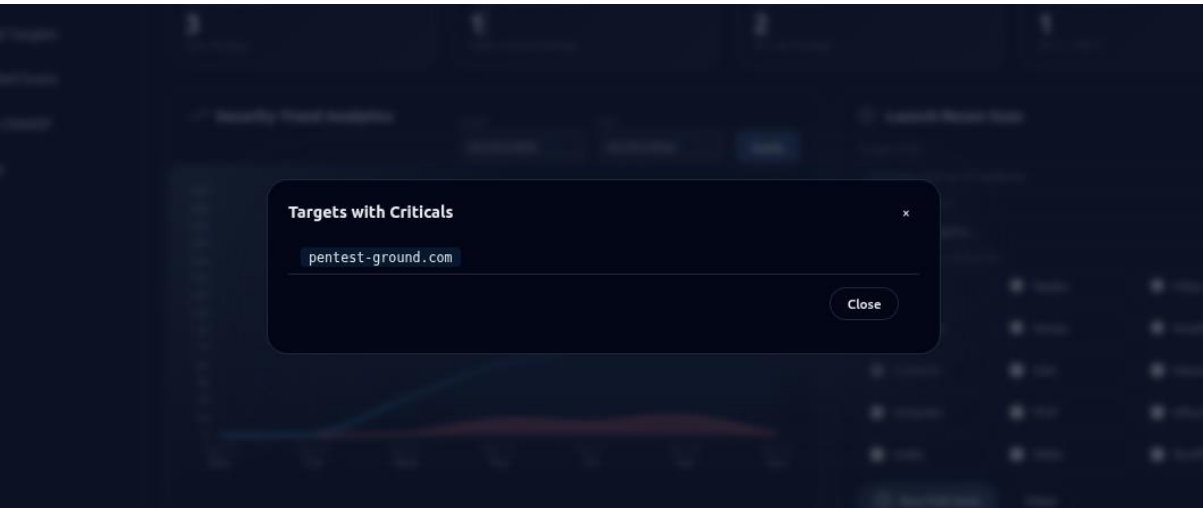


Figure 105 Detail view for Targets with Critical.

Evidence Case 3

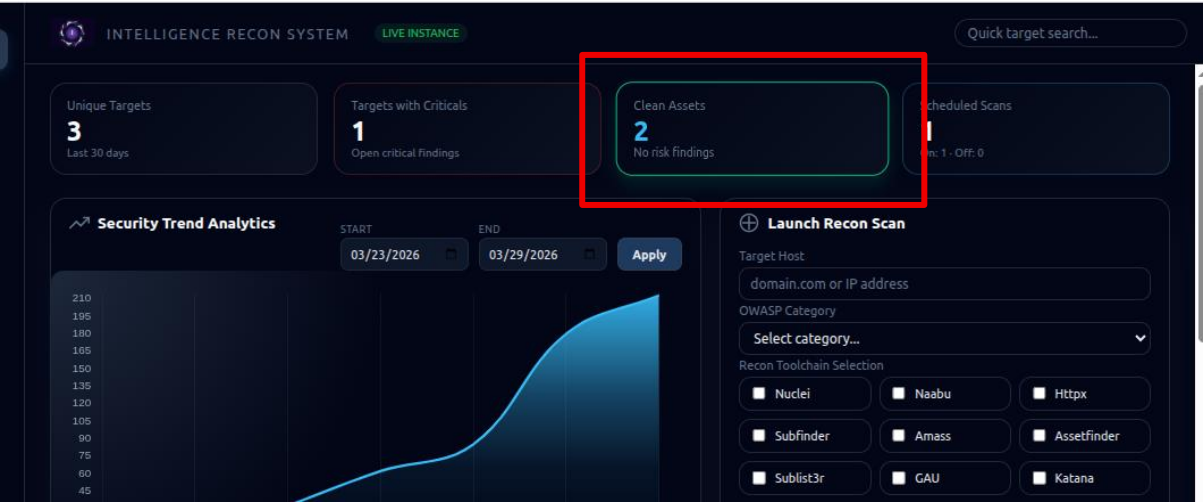


Figure 106 Hover to Clean Assets and Clicked Clean Assets.

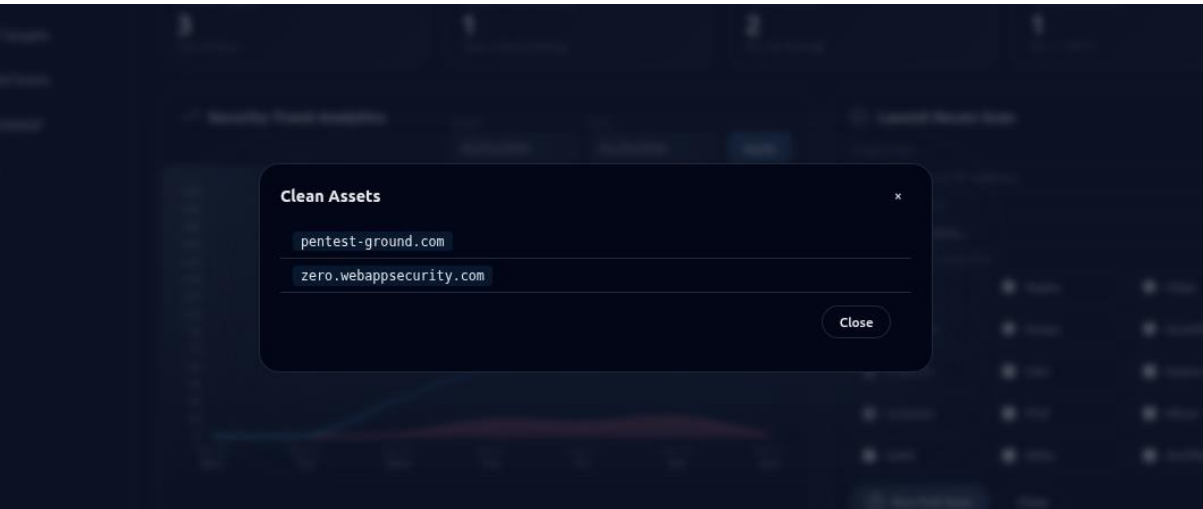


Figure 107 Detail view for Clean Assets.

Evidence Case 4

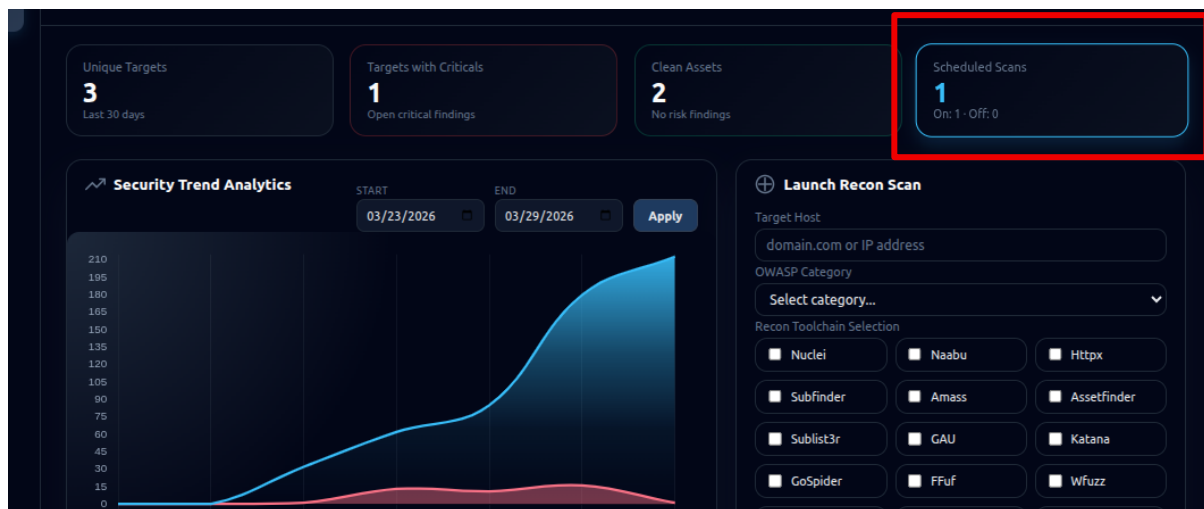


Figure 108 Hover to Scheduled Scans and Clicked.

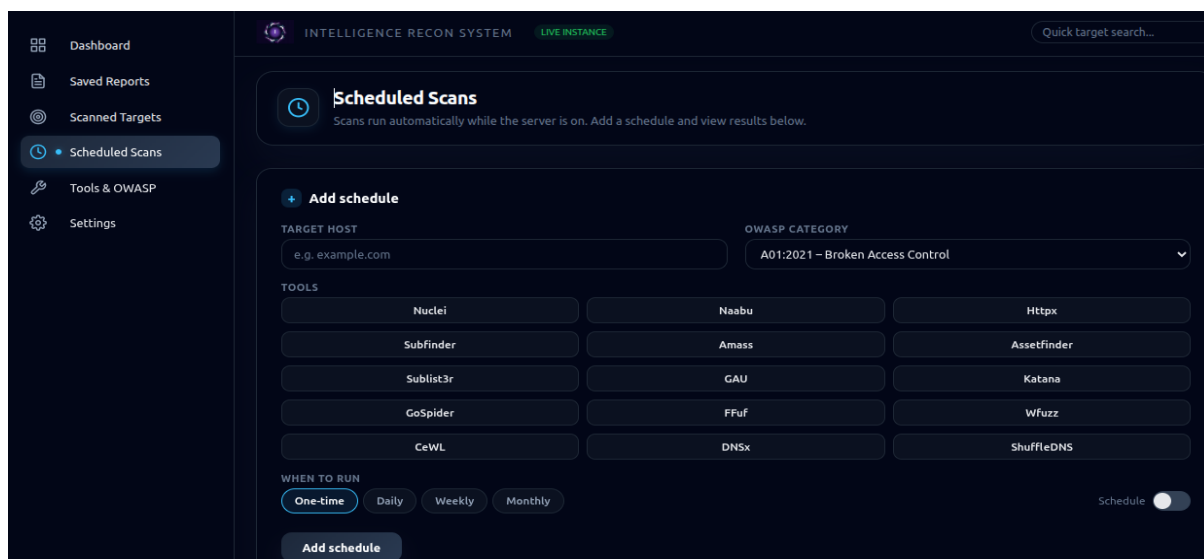


Figure 109 Scheduled Scans open directly by clicking Scheduled Scans Card.

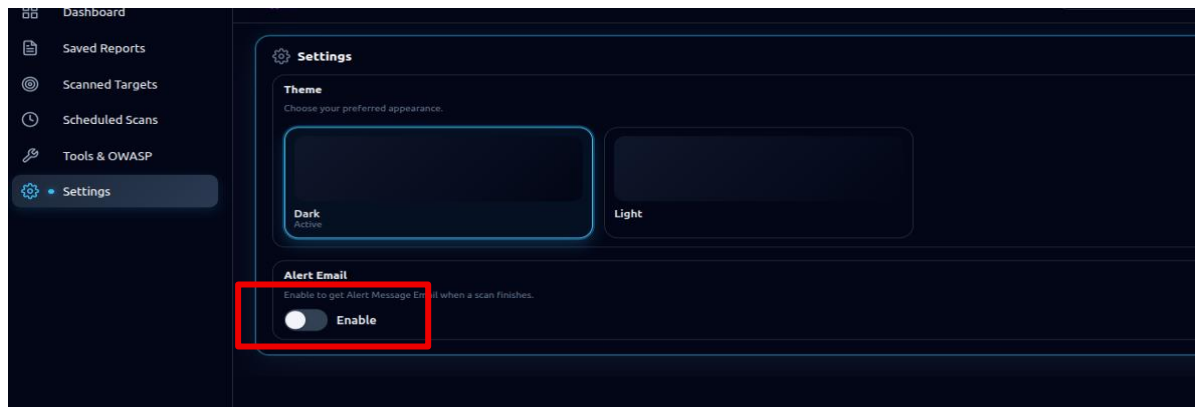
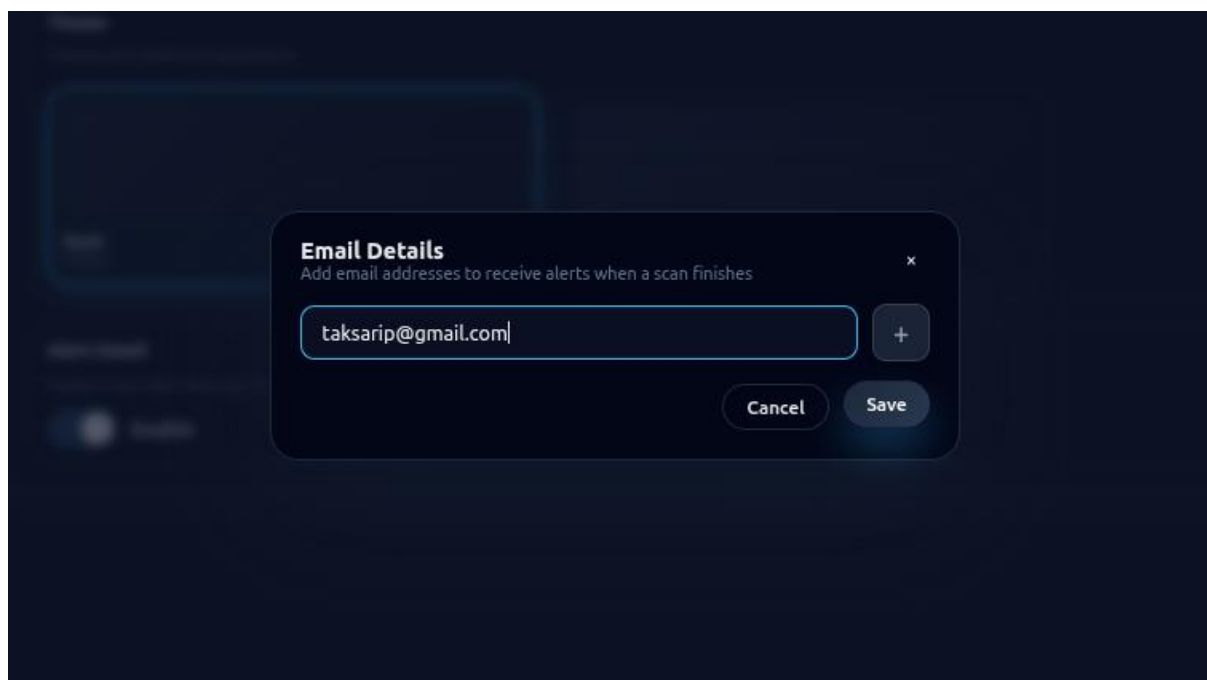
4.2.9. Test Case 9

Unit Testing	Objectives
Objective	To validate Email Alert Activation and Receive Alert Email.
Action	1. Open Settings. 2. Toggle the Enable to active Alert Email. 3. Add Email which receives alert email after scanning complete. 4. Run scan and verify email alert sent to added email.
Expected Results	Added Email should receive Alert Email after scanning completion by Enabling Alert Email.

Actual Result	As expected, added email received the Alert Email after scanning complete.
Conclusion	The test was successful.

Table 11 Unit Test Case 9

Evidence

*Figure 110 Open Settings and Click Enable.**Figure 111 Add Email Which Receive Alert.*

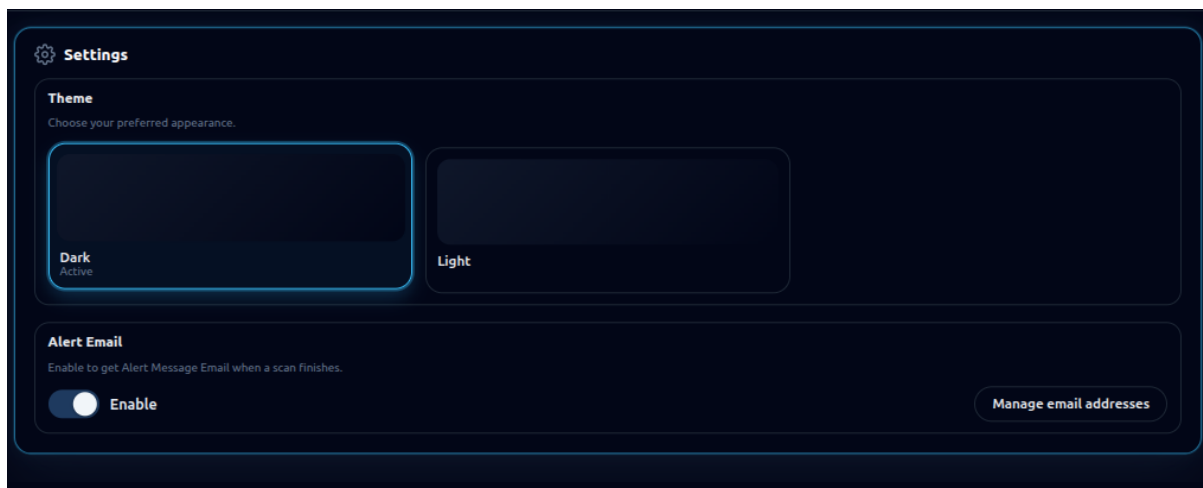


Figure 112 Alert Email Enabled.

```

2026-03-29 16:58:35,925 - app.services.scan_service - INFO - Risk Scoring: https://pentest-ground.com:7001 severity low -> medium (tool: ActiveTester)
2026-03-29 16:58:35,929 - app.services.scan_service - INFO - Smart Risk Scoring: completed severity adjustments for scan 136
2026-03-29 16:58:35,929 - app.services.scan_service - INFO - Scan 136: Finalizing
2026-03-29 16:58:35,936 - app.services.scan_service - INFO - Scan 136 finalized with status: completed
2026-03-29 16:58:40,712 - app.services.email_alert_service - INFO - Scan-finished alert email sent to ['taksarip@gmail.com']
INFO: 127.0.0.1:40490 - "GET /api/scans/136/status HTTP/1.1" 200 OK
INFO: 127.0.0.1:40480 - "GET /api/scans/136/intelligence-summary HTTP/1.1" 200 OK
INFO: 127.0.0.1:40490 - "GET /api/scans?page_size=100&include_unsaved=true HTTP/1.1" 200 OK
INFO: 127.0.0.1:40490 - "GET /api/scheduled-scans HTTP/1.1" 200 OK
INFO: 127.0.0.1:40490 - "GET /api/scans?page_size=100&include_unsaved=false HTTP/1.1" 200 OK
INFO: 127.0.0.1:40490 - "GET /api/scans?include_unsaved=false HTTP/1.1" 200 OK

```

Figure 113 Terminal log after scanning a complete email alert.

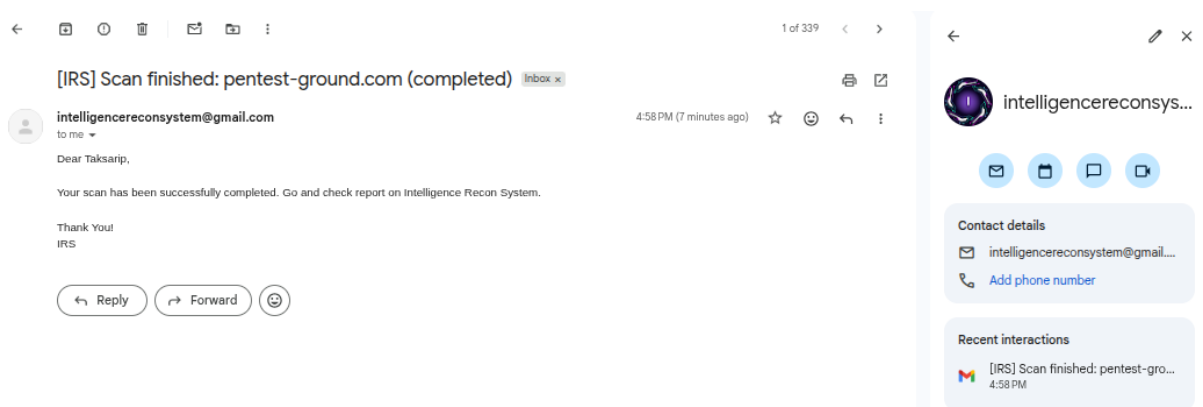
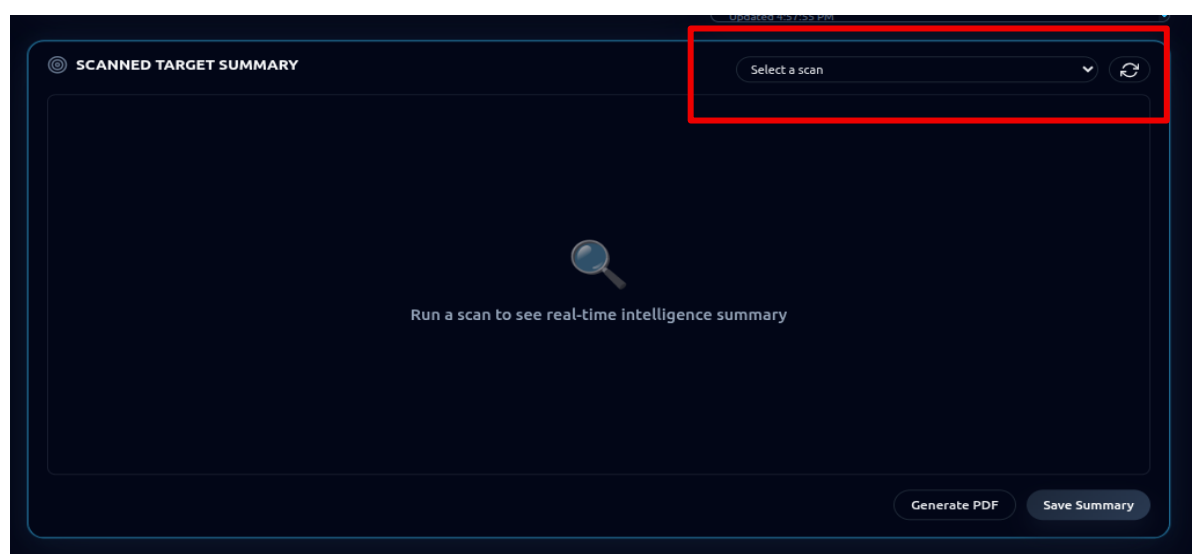


Figure 114 Gmail Account that received Alert.

4.2.10. Test Case 10

Unit Testing	Objectives
Objective	To ensure that the Scanned Target Summary is showing the data of the Executive Summary properly after choosing a scan that has been saved.
Action	Scroll down to Scanned Target Summary. Click on the dropdown Select a scan. Choose a completed scan 117 - zero.webappsecurity.com (A03:2021) - completed. Check and look at the loaded Executive Summary.
Expected Results	Finished scans will be shown in the dropdown. By choosing one, it displays the right Executive Summary information (Target, Status, Findings, Risk, Score, Tools, Severity, Key Findings).
Actual Result	The dropdown displayed the correct scans that have been completed. The scan, which was selected as scan number 117, loaded the Executive Summary with all fields correctly displayed.
Conclusion	The test was Successful.

*Table 12 Unit Test Case 10***Evidence***Figure 115 Scroll to Scanned Target Summary and Click "Select a scan" dropdown.*

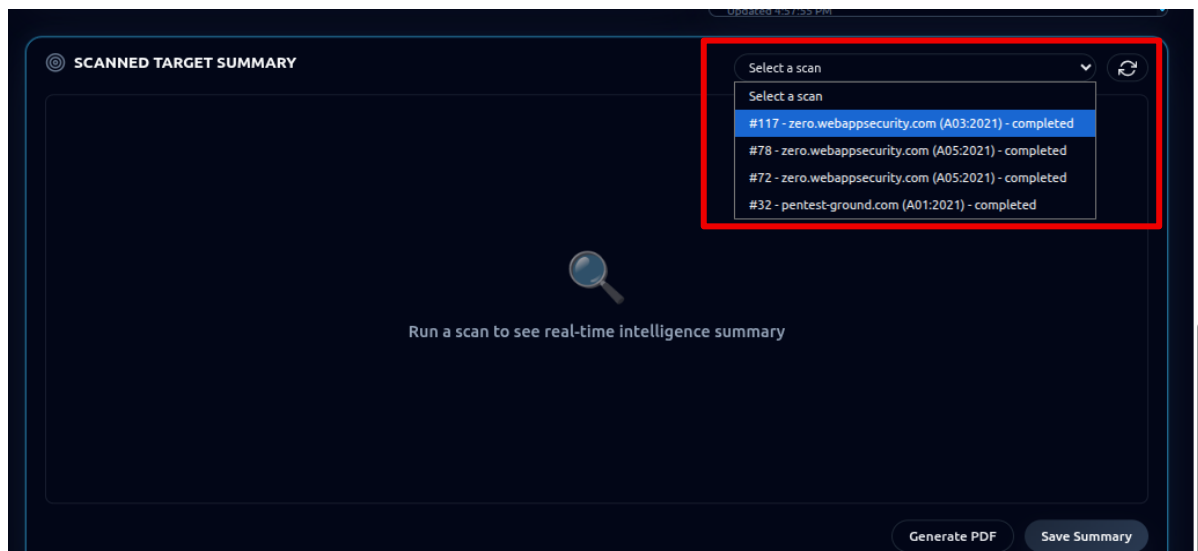


Figure 116 Dropdown displays list of saved scans.

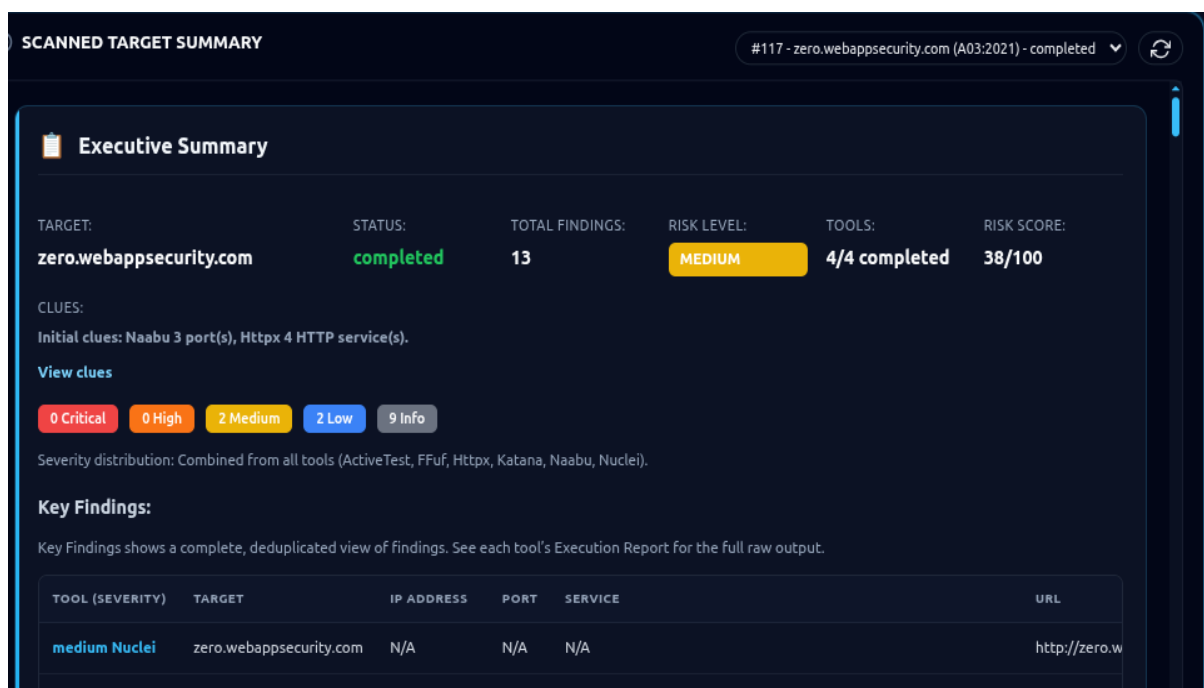


Figure 117 Choose scan loaded with all correct fields.

4.2.11. Test Case 11

Unit Testing	Objectives
Objective	To ensure that the Live Status Tracker Runs and shows scan status updates in each 5 seconds.
Action	1. Run a new Scan. 2. Scroll down to Live Status Tracker. 3. Check Scan Status Updates.
Expected Results	Live Status Tracker must show real time scan progress and status in each 5 seconds from Scan Creation to its completion.
Actual Result	As expected, Live Status Tracker showed real-time scan updates in each 5 seconds from its creation to completion.
Conclusion	The Test was Successful.

Table 13 Unit Test Case 11

Evidence

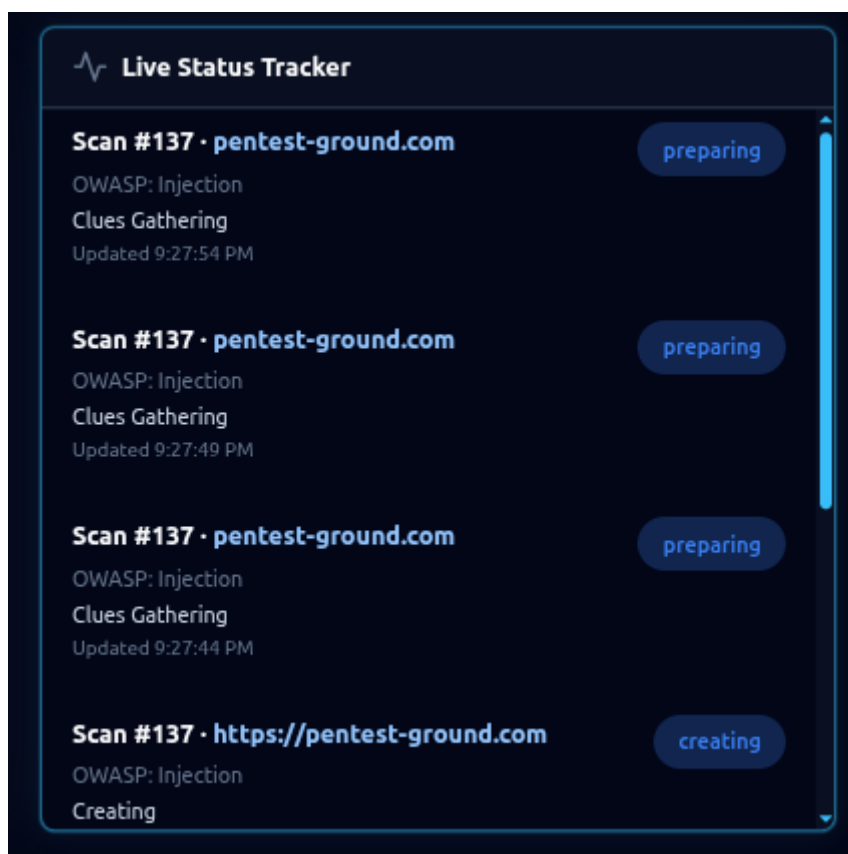


Figure 118 Scan Status Update from Scan Creation to Clue gathering phase.

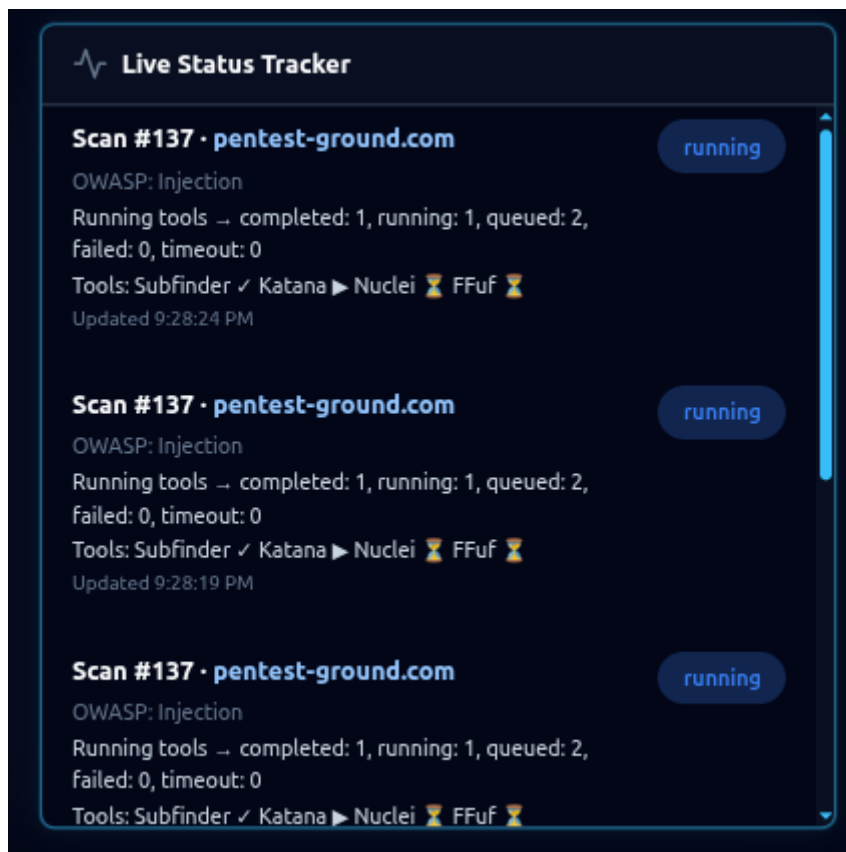


Figure 119 Scan Status updates in Tools execution phase.

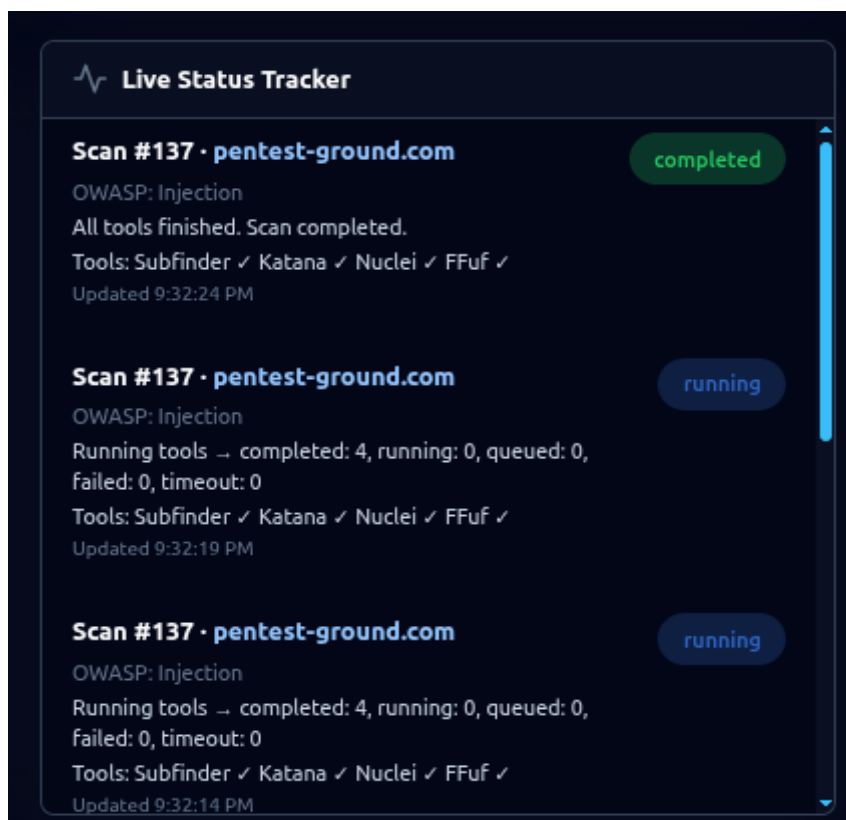


Figure 120 Scan status updates from tools execution to completion.

4.2.12. Test Case 12

Unit Testing	Objectives
Objective	To validate, the report should open in a new tab when View Report is clicked from Saved Reports Security Reports Registry.
Action	1. Open Saved Reports. 2. Click View Report for Scan ID 117.
Expected Results	New tabs should be opened on the same browser for report.
Actual Result	As expected, the Report was opened on a new tab when View Report was clicked.
Conclusion	The conducted test was Successful.

Table 14 Unit Test Case 12

Evidence

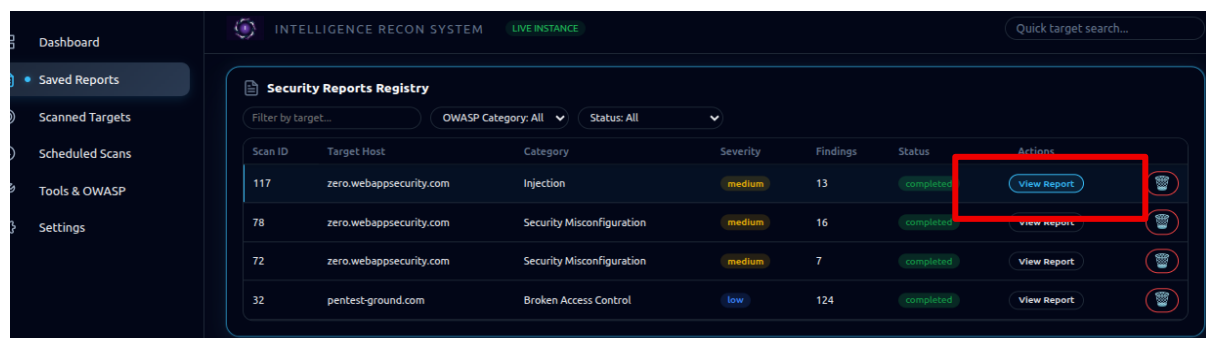


Figure 121 Open Saved Reports and Clicked View Report

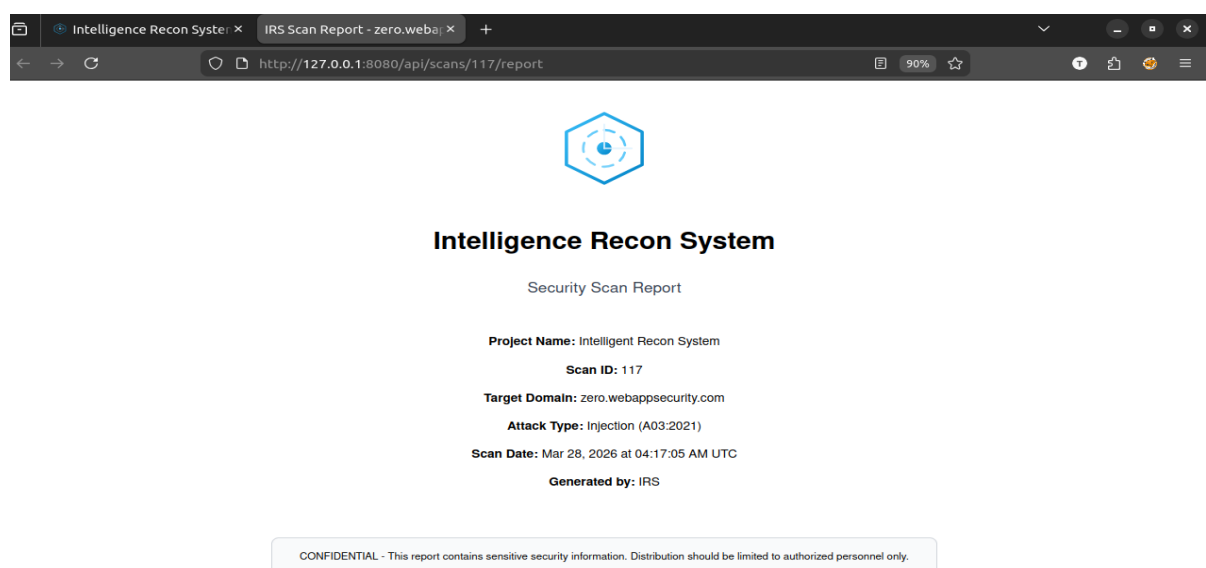


Figure 122 Opened new tab for Report

4.2.13. Test Case 13

Unit Testing	Objectives
Objective	To Validate “Quick target search” successfully open and shows details of searched target.
Action	1. Click the “Quick target search”. 2. Type “pentest-ground.com” as target view details. 3. Enter.
Expected Results	Target Analysis Details of “pentest-ground.com” should open and show when user searches on “Quick target search”.
Actual Result	As expected, Target Analysis Details were opened when targets were searched and entered.
Conclusion	The test was successful.

Table 15 Unit Test Case 13

Evidence

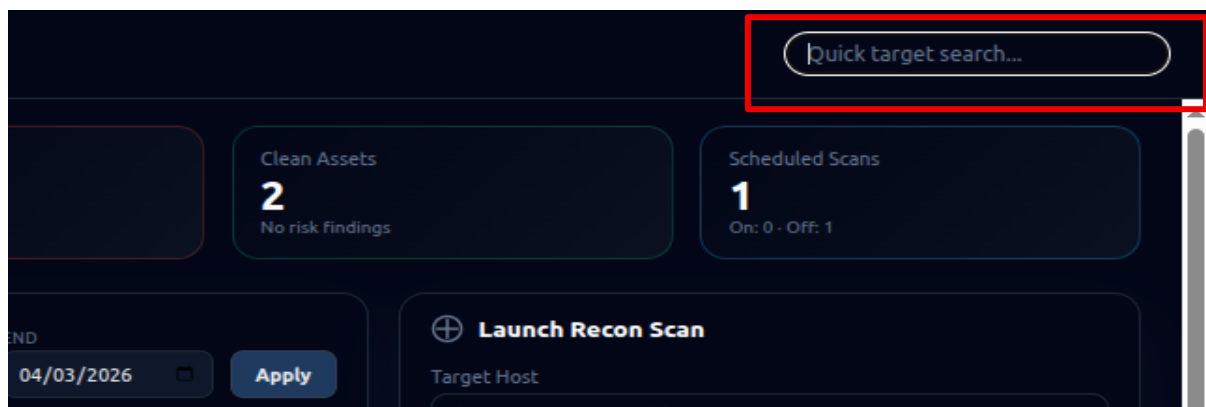


Figure 123 Click Quick target search.

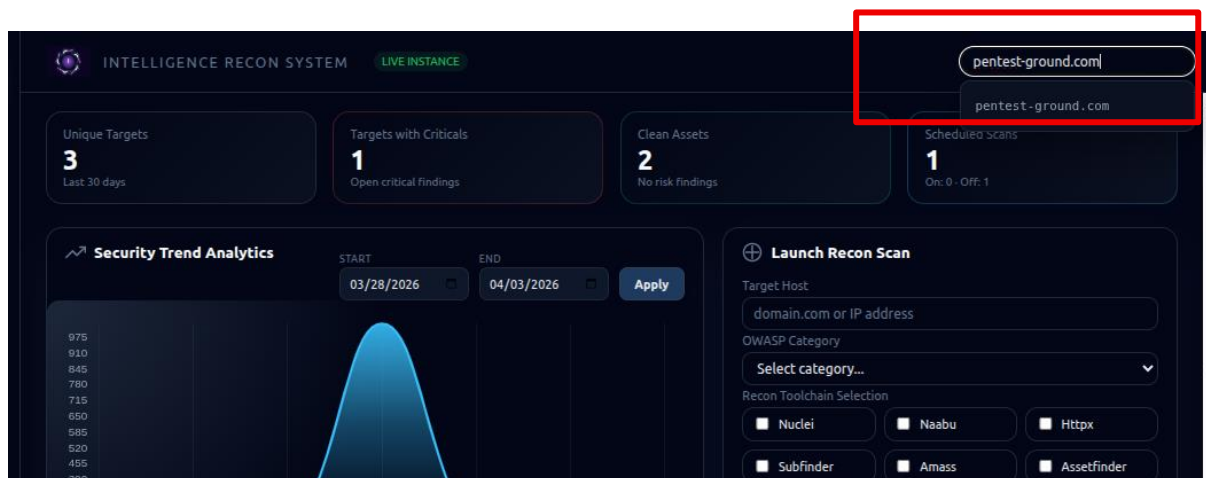


Figure 124 Type target to view details.

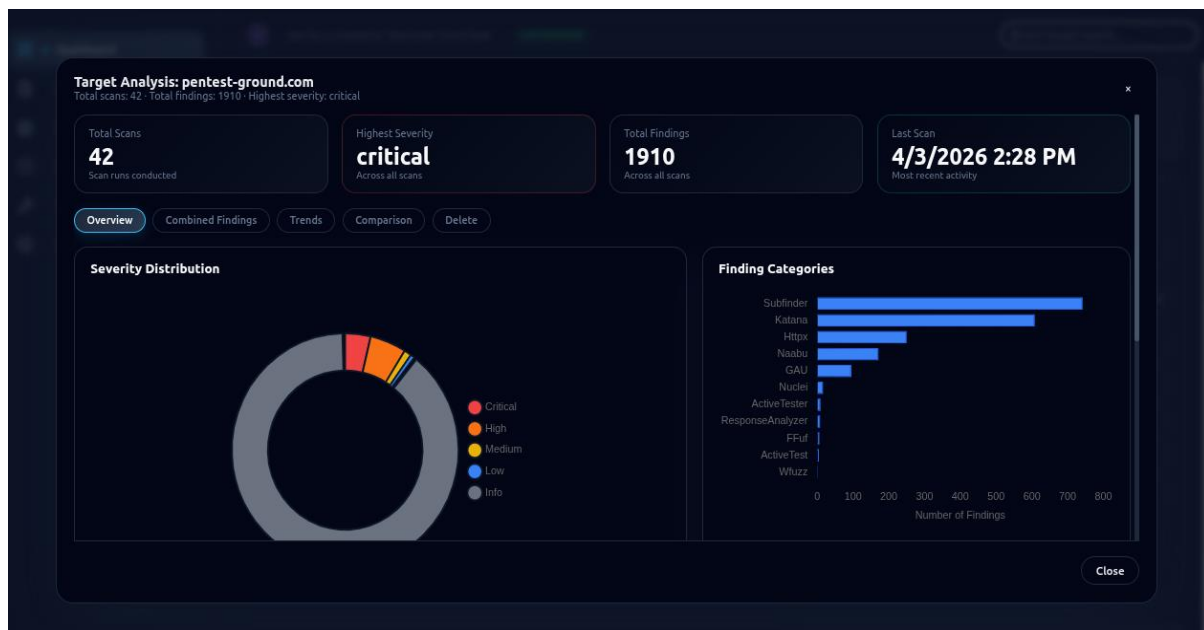


Figure 125 Opened Target Analysis for pentest-ground.com.

4.2.14. Test Case 14

Unit Testing	Objectives
Objective	To verify that when a tool Run button is clicked, the tool library correctly presents CLI Command which is configured and running in the system.
Action	1. Proceed to the Tool Library. 2. Choose any tool, for example "DNSx". 3. Click "Run DNSx".
Expected Results	System should correctly present the selected tool CLI command when Run DNSx is clicked.
Actual Result	The CLI command for DNSx was displayed as expected, when Run DNSx was clicked.
Conclusion	The test was successful.

Table 16 Unit Test Case 14

Evidence

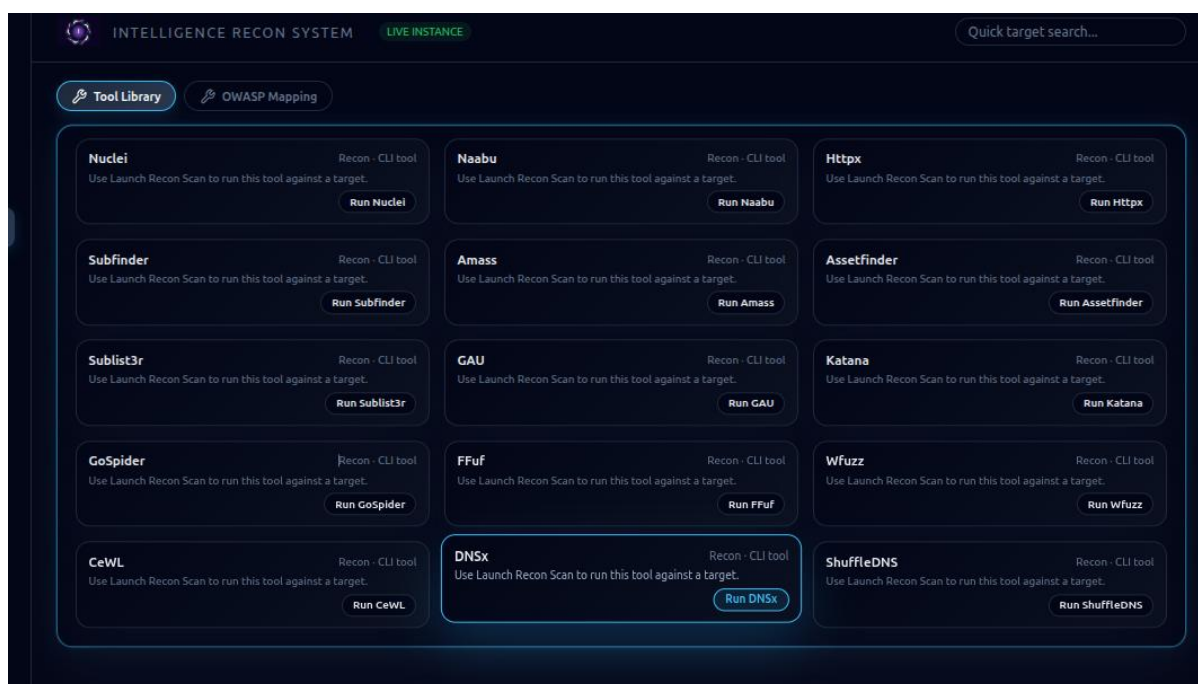


Figure 126 Navigate to Tool Library.

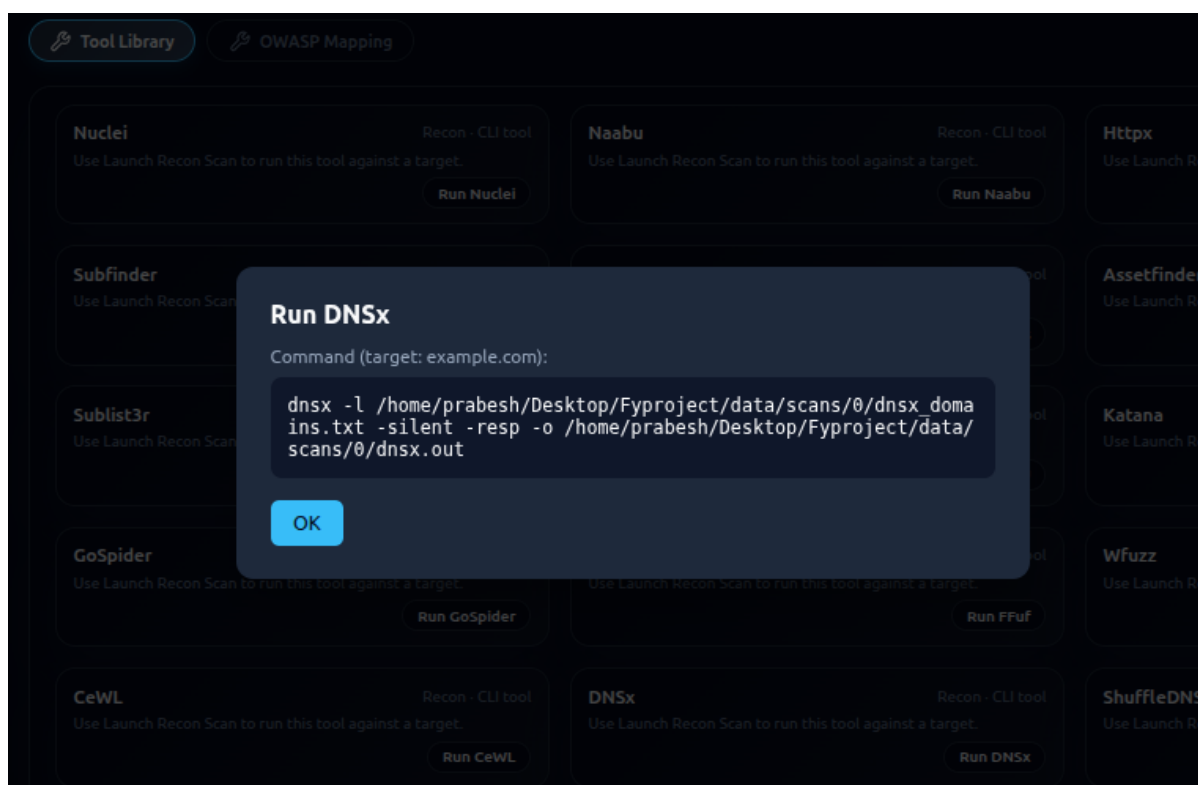


Figure 127 CLI Command DNSx.

4.2.15. Test Case 15

Unit Testing	Objectives
Objective	To test that a particular tool slice on the donut chart or graph will show the tool details of its finding below the graph.
Action	Open Details. Compare any two Scan with same Attack type. Click on Naabu slice of the Scan 1 Chart.
Expected Results	Below the chart, there should be new section which shows the findings by Naabu when the slice of naabu is clicked.
Actual Result	New section was opened as expected, when Naabu slice was clicked and new section showed the detailed findings by Naabu.
Conclusion	The test was successful.

Table 17 Unit Test Case 15

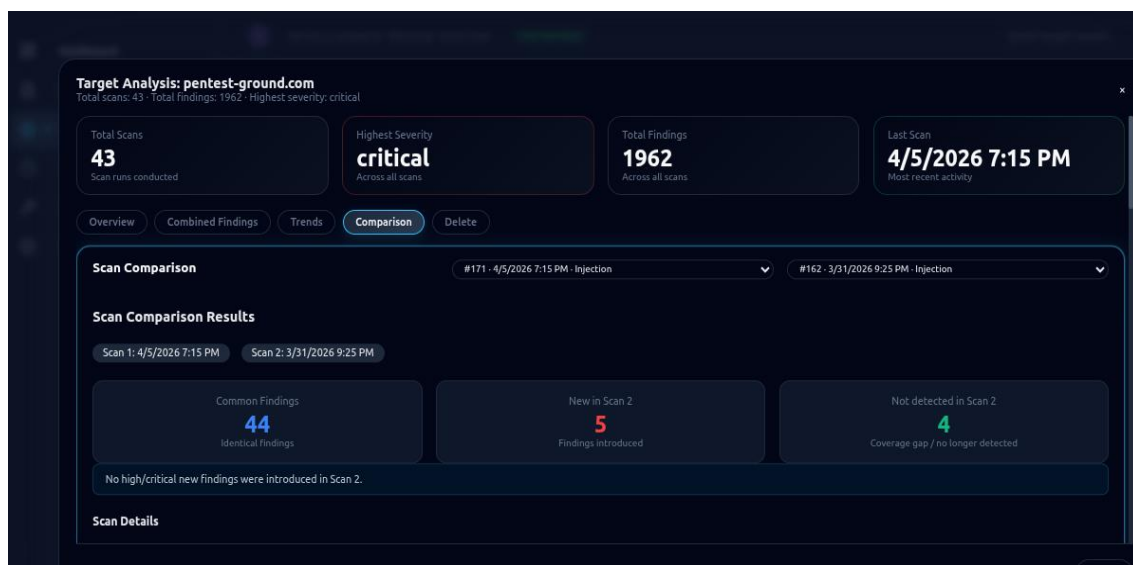
Evidence

Figure 128 Opened Details and select different scans with same attack type.

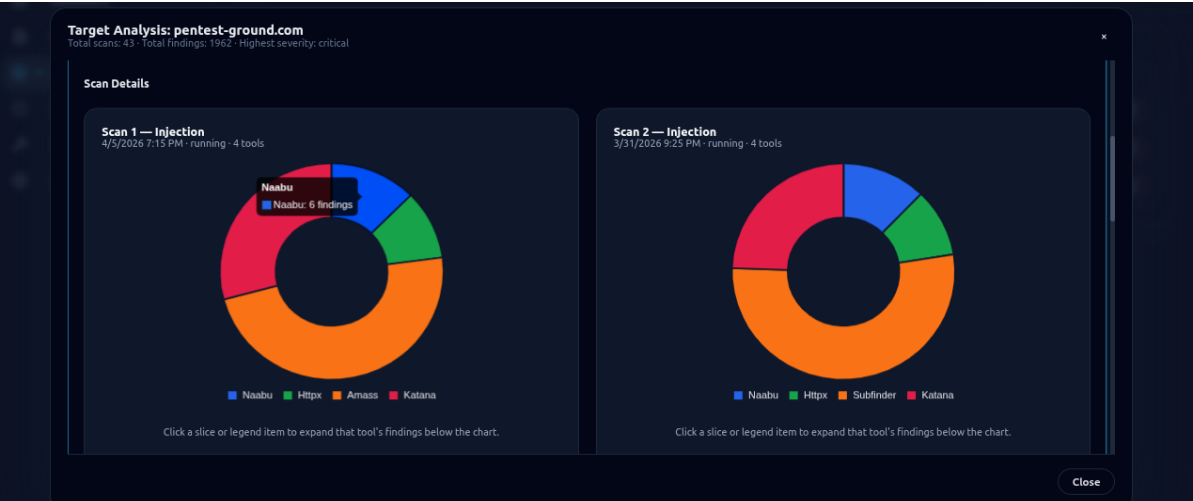


Figure 129 Clicked Naabu Slice on chart.

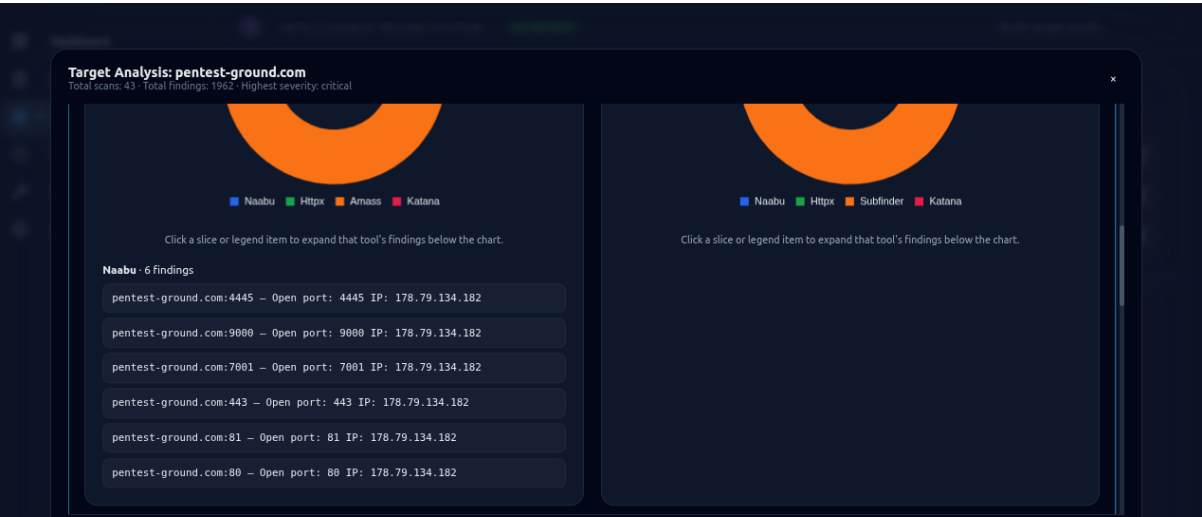


Figure 130 Findings details by Naabu.

4.3. SYSTEM TESTING

The system testing is performed when the entire system is assembled and ready. It tests the functionality of the entire system as per the requirements (Geeksforgeeks, 2025).

Test Case 1

System Testing	Objectives
Objective	To verify the end-to-end system process runs correctly without errors.
Action	1. Open Intelligent Recon 2. Enter target domain/URL/IP 3. Select OWASP category 4. Select one or more tools 5. Click Run Full Scan 6. Verify target has been added to Scanned Targets / summary 7. Wait until scan finishes 8. Save report / generate PDF
Expected Result	System completes full scan process and generates accurate results without errors and allows saved/generation of pdf.
Actual Result	As expected, system runs full scan and all other features.
Conclusion	The test was successful.

Table 18 System Test Case 1

Evidence

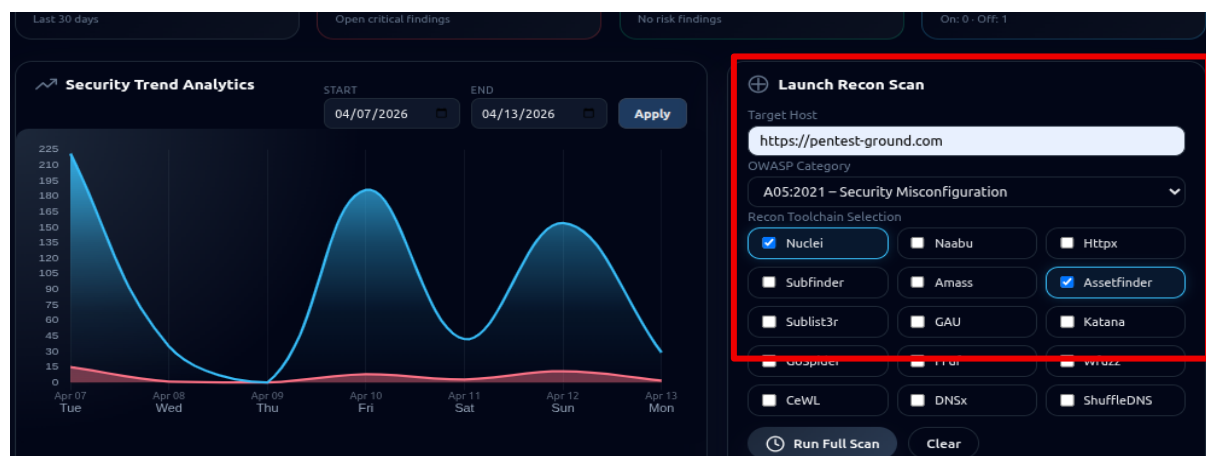


Figure 131 Start System and Filled Scan Requirement.

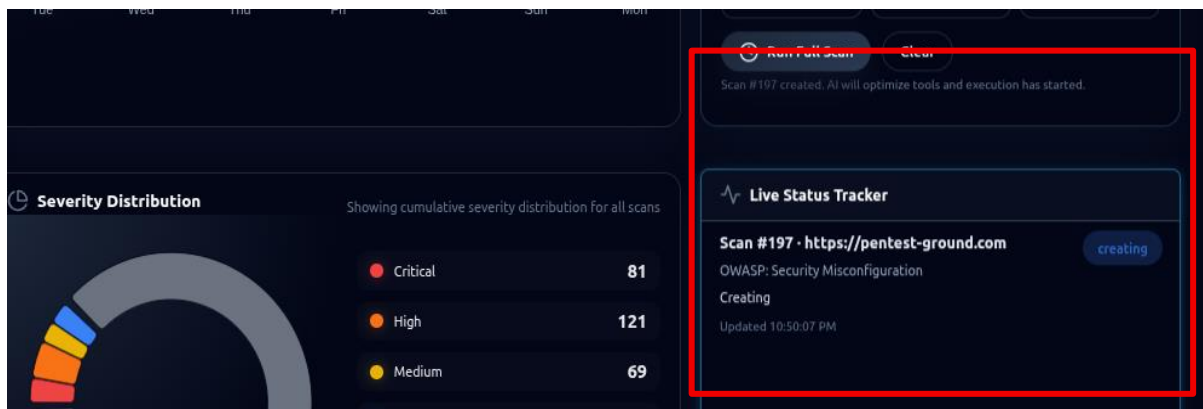


Figure 132 Scan Created Successfully

```
INFO: 127.0.0.1:49664 - "GET /api/scheduled-scans HTTP/1.1" 200 OK
INFO: 127.0.0.1:49658 - "GET /api/scans/stats/severity HTTP/1.1" 200 OK
INFO: 127.0.0.1:49664 - "GET /api/settings/alert-email HTTP/1.1" 200 OK
INFO: 127.0.0.1:49670 - "GET /api/scans/stats/severity HTTP/1.1" 200 OK
INFO: 127.0.0.1:49646 - "GET /api/scans/stats/trend?start_date=2026-04-07&end_date=2026-04-13 HTTP/1.1" 200 OK
INFO: 127.0.0.1:49658 - "GET /api/scheduled-scans/scan-results HTTP/1.1" 200 OK
INFO: 127.0.0.1:49670 - "GET /api/scans/stats/trend?start_date=2026-04-07&end_date=2026-04-13 HTTP/1.1" 200 OK
INFO: 127.0.0.1:49664 - "GET /api/scans?page_size=100&include_unsaved=true HTTP/1.1" 200 OK
INFO: 127.0.0.1:49664 - "GET /api/scheduled-scans HTTP/1.1" 200 OK
INFO: 127.0.0.1:49664 - "GET /api/scans?page_size=100&include_unsaved=false HTTP/1.1" 200 OK
INFO: 127.0.0.1:49664 - "GET /api/scans?include_unsaved=false HTTP/1.1" 200 OK
INFO: 127.0.0.1:35592 - "GET /api/scans/any-running HTTP/1.1" 200 OK
2026-04-13 22:50:07,156 - app.services.scan_service - INFO - Created scan 197 for target=pentest-ground.com, owasp=A05:2021
INFO: 127.0.0.1:35592 - "POST /api/scans HTTP/1.1" 200 OK
2026-04-13 22:50:07,203 - app.core.ws_updates - INFO - [Scan 197] Scan phase: clues gathering
2026-04-13 22:50:07,204 - app.services.scan_service - INFO - Scan 197: Running initial clues phase
2026-04-13 22:50:07,204 - app.core.ws_updates - INFO - [Scan 197] System: Starting initial reconnaissance with Naabu for port scanning...
2026-04-13 22:50:07,205 - app.core.ws_updates - INFO - [Scan 197] Scan phase: clues gathering
2026-04-13 22:50:07,205 - app.core.ws_updates - INFO - [Scan 197] System: Starting HTTP reconnaissance with Httpx...
2026-04-13 22:50:07,206 - app.core.ws_updates - INFO - [Scan 197] Scan phase: clues gathering
2026-04-13 22:50:07,207 - app.tools.executor - INFO - Executing Naabu for scan 197, target=pentest-ground.com, timeout=120s
2026-04-13 22:50:07,208 - app.core.ws_updates - INFO - [Scan 197] Tool started: Naabu
2026-04-13 22:50:07,210 - tool.naabu - INFO - Executing command: naabu -host pentest-ground.com -top-ports 1000 -ip-s -/home/prabesh/Dev
```

Figure 133 Scan created and run in backend.

```
INFO: 127.0.0.1:35592 - "GET /api/scans?page_size=100&include_unsaved=false HTTP/1.1" 200 OK
INFO: 127.0.0.1:35600 - "GET /api/scans/stats/severity HTTP/1.1" 200 OK
INFO: 127.0.0.1:35594 - "GET /api/scans/197/intelligence-summary HTTP/1.1" 200 OK
INFO: 127.0.0.1:35592 - "GET /api/scans?include_unsaved=false HTTP/1.1" 200 OK
INFO: 127.0.0.1:35594 - "GET /api/scans/stats/trend?start_date=2026-04-07&end_date=2026-04-13 HTTP/1.1" 200 OK
INFO: 127.0.0.1:35608 - "GET /api/scans?page_size=100&include_unsaved=true HTTP/1.1" 200 OK
2026-04-13 22:50:10,285 - tool.naabu - INFO - [Naabu] {"host": "pentest-ground.com", "ip": "178.79.134.182", "timestamp": "2026-04-13T17:05:10.282415
2022", "port": 80, "protocol": "tcp", "tls": false}
INFO: 127.0.0.1:35608 - "GET /api/scheduled-scans HTTP/1.1" 200 OK
2026-04-13 22:50:10,366 - tool.naabu - INFO - [Naabu] {"host": "pentest-ground.com", "ip": "178.79.134.182", "timestamp": "2026-04-13T17:05:10.361265
8Z", "port": 81, "protocol": "tcp", "tls": false}
2026-04-13 22:50:10,367 - tool.naabu - INFO - [Naabu] {"host": "pentest-ground.com", "ip": "178.79.134.182", "timestamp": "2026-04-13T17:05:10.365647
858Z", "port": 9000, "protocol": "tcp", "tls": false}
2026-04-13 22:50:10,373 - tool.naabu - INFO - [Naabu] {"host": "pentest-ground.com", "ip": "178.79.134.182", "timestamp": "2026-04-13T17:05:10.372650
138Z", "port": 7001, "protocol": "tcp", "tls": false}
INFO: 127.0.0.1:35608 - "GET /api/scans?page_size=100&include_unsaved=false HTTP/1.1" 200 OK
INFO: 127.0.0.1:35608 - "GET /api/scans?include_unsaved=false HTTP/1.1" 200 OK
INFO: 127.0.0.1:35608 - "GET /api/scans/197/intelligence-summary HTTP/1.1" 200 OK
2026-04-13 22:50:11,520 - tool.naabu - INFO - [Naabu] {"host": "pentest-ground.com", "ip": "178.79.134.182", "timestamp": "2026-04-13T17:05:11.519655
298Z", "port": 4445, "protocol": "tcp", "tls": false}
INFO: 127.0.0.1:35608 - "GET /api/scans/stats/severity HTTP/1.1" 200 OK
INFO: 127.0.0.1:35594 - "GET /api/scans/stats/trend?start_date=2026-04-07&end_date=2026-04-13 HTTP/1.1" 200 OK
INFO: 127.0.0.1:35594 - "GET /api/scans/197/status HTTP/1.1" 200 OK
```

Figure 134 Initial Clue Gathering Phase by naabu

```
(venv) prabesh@prabesh-Surface-Laptop-3:~/Desktop/Fyproject$ source /home/prabesh/Desktop/Fyproject/venv/bin/activate && uvicorn app.main:app --
host 0.0.0.0 --port 8080
2026-04-13 22:50:15,496 - app.tools.executor - INFO - Naabu completed: success=True, findings=5
2026-04-13 22:50:15,500 - app.tools.executor - INFO - Executing Httpx for scan 197, target=pentest-ground.com, timeout=120s
2026-04-13 22:50:15,501 - app.core.ws.updates - INFO - [Scan 197] Tool started: Httpx
2026-04-13 22:50:15,502 - tool.httpx - INFO - Executing command: /home/prabesh/go/bin/httpx -u http://pentest-ground.com:81 -u https://pentest-ground.com:81 -u http://pentest-ground.com:443 -u https://pentest-ground.com:443 -u http://pentest-ground.com:3000 -u https://pentest-ground.com:3000 -u http://pentest-ground.com:8888 -u https://pentest-ground.com:8888 -u http://pentest-ground.com:7001 -u https://pentest-ground.com:7001 -u http://pentest-ground.com:8443 -u https://pentest-ground.com:8443 -u http://pentest-ground.com:9000 -u https://pentest-ground.com:9000 -u http://pentest-ground.com:80 -u https://pentest-ground.com:80 -u http://pentest-ground.com:4445 -u https://pentest-ground.com:4445 -u http://pentest-ground.com:8080 -u https://pentest-ground.com:8080 -u http://pentest-ground.com:5000 -u https://pentest-ground.com:5000 -u http://pentest-ground.com:7070 -u https://pentest-ground.com:7070 -u http://pentest-ground.com:8000 -u https://pentest-ground.com:8000 -json -o /home/prabesh/Desktop/Fyproject/data/scans/197/httpx.out -silent -title -td -sc -web-server -ip -cname
2026-04-13 22:50:15,503 - app.core.ws.updates - INFO - [Scan 197] Httpx: Running command: /home/prabesh/go/bin/httpx -u http://pentest-ground.com:81 -u https://pentest-ground.com:81 -u http://pentest-ground.com:443 -u https://pentest-ground.com:443 -u http://pentest-ground.com:3000 -u https://pentest-ground.com:3000 -u http://pentest-ground.com:8888 -u https://pentest-ground.com:8888 -u http://pentest-ground.com:7001 -u https://pentest-ground.com:7001 -u http://pentest-ground.com:8443 -u https://pentest-ground.com:8443 -u http://pentest-ground.com:9000 -u https://pentest-ground.com:9000 -u http://pentest-ground.com:80 -u https://pentest-ground.com:80 -u http://pentest-ground.com:4445 -u https://pentest-ground.com:4445 -u http://pentest-ground.com:8080 -u https://pentest-ground.com:8080 -u http://pentest-ground.com:5000 -u https://pentest-ground.com:5000 -u http://pentest-ground.com:7070 -u https://pentest-ground.com:7070 -u http://pentest-ground.com:8000 -u https://pentest-ground.com:8000 -json -o /home/prabesh/Desktop/Fyproject/data/scans/197/httpx.out -silent -title -td -sc -web-server -ip -cname
INFO: 127.0.0.1:35594 - "GET /api/scans/197/intelligence-summary HTTP/1.1" 200 OK
INFO: 127.0.0.1:35594 - "GET /api/scans/stats/severity HTTP/1.1" 200 OK
INFO: 127.0.0.1:35594 - "GET /api/scans/197/status HTTP/1.1" 200 OK
INFO: 127.0.0.1:35594 - "GET /api/scans/197/intelligence-summary HTTP/1.1" 200 OK
```

Figure 135 Initial Clue gathering phase by httpx

```
(venv) prabesh@prabesh-Surface-Laptop-3:~/Desktop/Fyproject$ source /home/prabesh/Desktop/Fyproject/venv/bin/activate && uvicorn app.main:app --
host 0.0.0.0 --port 8080
INFO: 127.0.0.1:49620 - "GET /api/scans/197/status HTTP/1.1" 200 OK
2026-04-13 22:50:44,592 - app.ai.decision_node - INFO - Raw AI response: ``json
{
  "tools_to_run": [
    "Assetfinder",
    "Httpx",
    "Nuclei"
  ],
  "tools_skipped": [
    {
      "tool": "Assetfinder",
      "reason": "Initial clues indicate a limited set of open ports and HTTP services. While Assetfinder is useful for broad subdomain discovery, its output might be minimal given the current limited surface. However, it's a user-selected tool and can still provide some value."
    }
  ],
  "tools_added": [
    {
      "tool": "Naabu",
      "reason": "To comprehensively identify all open ports and services beyond the initial clues, which is crucial for discovering potential misconfigurations on non-standard ports."
    },
    {
      "tool": "Httpx",
      "reason": "To enrich the discovered HTTP/S services with detailed information like status codes, headers, and titles, which are vital for identifying security misconfigurations. This tool is essential for understanding the web surface."
    },
    {
      "tool": "Nuclei",
      "reason": "To actively scan for known security misconfigurations using its extensive template library, focusing on the OWASP A05:2021 category. It's a user-selected tool and a primary vulnerability scanner."
    }
  ],
  "execution_batches": [
```

Figure 136 AI decision for Tools

```
(venv) prabesh@prabesh-Surface-Laptop-3:~/Desktop/Fyproject$ source /home/prabesh/Desktop/Fyproject/venv/bin/activate && uvicorn app.main:app --host 0.0.0.0 --port 8080
...
INFO: 127.0.0.1:49620 - "GET /api/scans/197/intelligence-summary HTTP/1.1" 200 OK
2026-04-13 22:50:44,783 - app.services.scan_service - INFO - Scan 197: Creating tool run records for 1 tools (order: discovery -> other -> exploit)
2026-04-13 22:50:44,901 - app.services.scan_service - INFO - Scan 197: Starting tool execution phase
2026-04-13 22:50:44,904 - app.core.ws_updates - INFO - [Scan 197] System: Starting execution of 1 tools...
2026-04-13 22:50:44,907 - app.services.scan_service - INFO - Initial discovered_urls: 7 from clues
INFO: 127.0.0.1:49620 - "GET /api/scans/stats/severity HTTP/1.1" 200 OK
2026-04-13 22:50:45,025 - app.core.ws_updates - INFO - [Scan 197] System: Starting execution of Nuclei...
2026-04-13 22:50:45,029 - app.core.ws_updates - INFO - [Scan 197] Scan phase: tool_execution
2026-04-13 22:50:45,099 - app.services.scan_service - INFO - EXECUTE TOOL CALL: tool_name=Nuclei, is_exploit=True, discovered_urls_count=7, owasp_category=A05:2021
2026-04-13 22:50:45,105 - app.tools.executor - INFO - SMART TIMEOUT: Calculated Nuclei timeout=1800s (base=1800, urls=7, owasp=A05:2021)
2026-04-13 22:50:45,108 - app.tools.executor - INFO - NUCLEI FINAL TIMEOUT: timeout value=1800s, timeout param=1800, settings.TOOL_TIMEOUT=1800
2026-04-13 22:50:45,109 - app.tools.executor - INFO - Executing Nuclei for scan 197, target=pentest-ground.com, timeout=1800s
2026-04-13 22:50:45,109 - app.core.ws_updates - INFO - [Scan 197] Tool started: Nuclei
2026-04-13 22:50:45,121 - app.core.ws_updates - INFO - [Scan 197] Nuclei: Starting phase 1/3 (Critical/High) with critical,high severity templates.
2026-04-13 22:50:45,124 - tool.nuclei - INFO - Executing command: nuclei -l /home/prabesh/Desktop/Fyproject/data/scans/197/nuclei_urls.txt -jsonl -o /home/prabesh/Desktop/Fyproject/data/scans/197/nuclei_critical_high.out -silent -c 15 -bulk-size 15 -rl 75 -severity critical,high -tags misconfig,cors,headers,exposure -timeout 5 -retries 0 -no-color -disable-update-check -nmhe
2026-04-13 22:50:45,127 - app.core.ws_updates - INFO - [Scan 197] Nuclei: Running command: nuclei -l /home/prabesh/Desktop/Fyproject/data/scans/197/nuclei_urls.txt -jsonl -o /home/prabesh/Desktop/Fyproject/data/scans/197/nuclei_critical_high.out -silent -c 15 -bulk-size 15 -rl 75 -severity critical,high -tags misconfig,cors,headers,exposure -timeout 5 -retries 0 -no-color -disable-update-check -nmhe
INFO: 127.0.0.1:49620 - "GET /api/scans/197/intelligence-summary HTTP/1.1" 200 OK
INFO: 127.0.0.1:49620 - "GET /api/scans/stats/severity HTTP/1.1" 200 OK
INFO: 127.0.0.1:49620 - "GET /api/scans/197/status HTTP/1.1" 200 OK
INFO: 127.0.0.1:49620 - "GET /api/scans/197/intelligence-summary HTTP/1.1" 200 OK
INFO: 127.0.0.1:49620 - "GET /api/scans/stats/severity HTTP/1.1" 200 OK
INFO: 127.0.0.1:49620 - "GET /api/scans/197/status HTTP/1.1" 200 OK
INFO: 127.0.0.1:49600 - "GET /api/scans/197/intelligence-summary HTTP/1.1" 200 OK
INFO: 127.0.0.1:49600 - "GET /api/scans/stats/severity HTTP/1.1" 200 OK
INFO: 127.0.0.1:49600 - "GET /api/scans/197/intelligence-summary HTTP/1.1" 200 OK
```

Figure 137 Starting tool execution phase

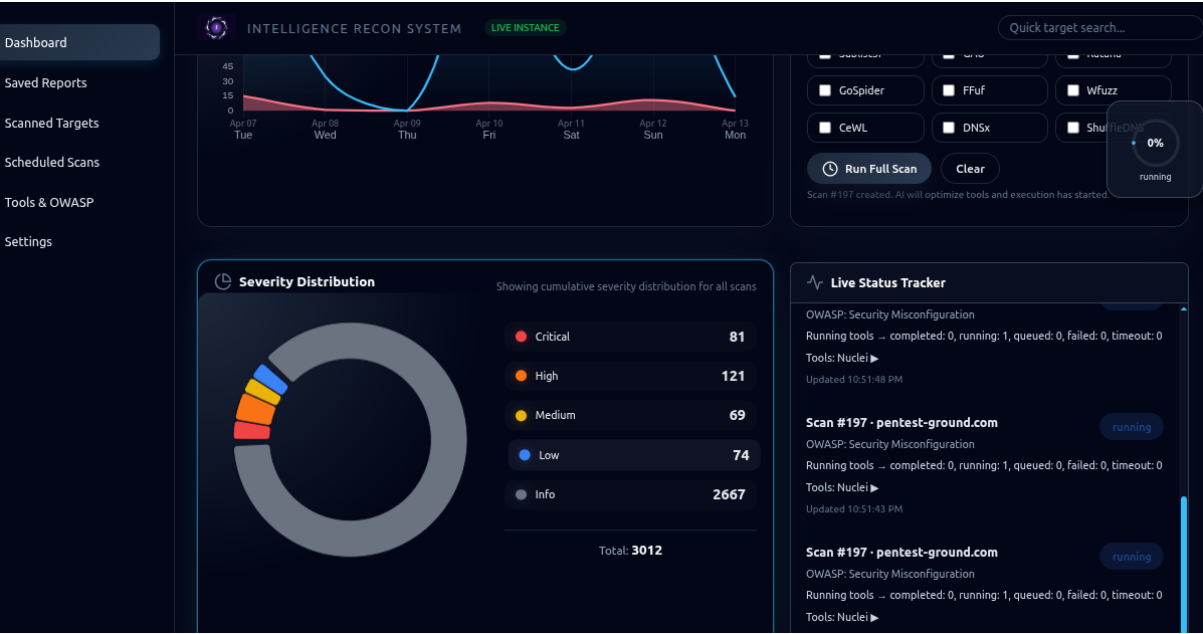


Figure 138 Live Status for Tool run


```
(venv) prabesh@prabesh-Surface-Laptop-3:~/Desktop/Fyproject$ source /home/prabesh/Desktop/Fyproject/venv/bin/activate && uvicorn app.main:app --
host 0.0.0.0 --port 8080
...
INFO: 127.0.0.1:49620 - "GET /api/scans/197/intelligence-summary HTTP/1.1" 200 OK
2026-04-13 22:50:44,783 - app.services.scan_service - INFO - Scan 197: Creating tool run records for 1 tools (order: discovery -> other -> explo
it)
2026-04-13 22:50:44,901 - app.services.scan_service - INFO - Scan 197: Starting tool execution phase
2026-04-13 22:50:44,904 - app.core.ws_updates - INFO - [Scan 197] System: Starting execution of 1 tools...
2026-04-13 22:50:44,907 - app.services.scan_service - INFO - Initial discovered_urls: 7 from clues
INFO: 127.0.0.1:49620 - "GET /api/scans/stats/severity HTTP/1.1" 200 OK
2026-04-13 22:50:45,025 - app.core.ws_updates - INFO - [Scan 197] System: Starting execution of Nuclei...
2026-04-13 22:50:45,029 - app.core.ws_updates - INFO - [Scan 197] Scan phase: tool_execution
2026-04-13 22:50:45,099 - app.services.scan_service - INFO - EXECUTE TOOL CALL: tool_name=Nuclei, is_exploit=True, discovered_urls_count=7, owasp
p_category=A05:2021
2026-04-13 22:50:45,105 - app.tools.executor - INFO - SMART TIMEOUT: Calculated Nuclei timeout=1800s (base=1800, urls=7, owasp=A05:2021)
2026-04-13 22:50:45,108 - app.tools.executor - INFO - NUCLEI FINAL TIMEOUT: timeout value=1800s, timeout param=1800, settings.TOOL_TIMEOUT=1800
2026-04-13 22:50:45,109 - app.tools.executor - INFO - Executing Nuclei for scan 197, target=pentest-ground.com, timeout=1800s
2026-04-13 22:50:45,109 - app.core.ws_updates - INFO - [Scan 197] Tool started: Nuclei
2026-04-13 22:50:45,121 - app.core.ws_updates - INFO - [Scan 197] Nuclei: Starting phase 1/3 (Critical/High) with critical,high severity templat
es.
2026-04-13 22:50:45,124 - tool.nuclei - INFO - Executing command: nuclei -l /home/prabesh/Desktop/Fyproject/data/scans/197/nuclei_urls.txt -json
l -o /home/prabesh/Desktop/Fyproject/data/scans/197/nuclei_critical_high.out -silent -c 15 -bulk-size 15 -rl 75 -severity critical,high -tags mi
sconfig,cors,headers,exposure -timeout 5 -retries 0 -no-color -disable-update-check -nmhe
2026-04-13 22:50:45,127 - app.core.ws_updates - INFO - [Scan 197] Nuclei: Running command: nuclei -l /home/prabesh/Desktop/Fyproject/data/scans/
197/nuclei_urls.txt -jsonl -o /home/prabesh/Desktop/Fyproject/data/scans/197/nuclei_critical_high.out -silent -c 15 -bulk-size 15 -rl 75 -severi
ty critical,high -tags misconfig,cors,headers,exposure -timeout 5 -retries 0 -no-color -disable-update-check -nmhe
INFO: 127.0.0.1:49620 - "GET /api/scans/197/intelligence-summary HTTP/1.1" 200 OK
INFO: 127.0.0.1:49620 - "GET /api/scans/stats/severity HTTP/1.1" 200 OK
INFO: 127.0.0.1:49620 - "GET /api/scans/197/status HTTP/1.1" 200 OK
INFO: 127.0.0.1:49620 - "GET /api/scans/197/intelligence-summary HTTP/1.1" 200 OK
INFO: 127.0.0.1:49620 - "GET /api/scans/stats/severity HTTP/1.1" 200 OK
INFO: 127.0.0.1:49620 - "GET /api/scans/197/status HTTP/1.1" 200 OK
INFO: 127.0.0.1:49600 - "GET /api/scans/197/intelligence-summary HTTP/1.1" 200 OK
INFO: 127.0.0.1:49600 - "GET /api/scans/stats/severity HTTP/1.1" 200 OK
INFO: 127.0.0.1:49600 - "GET /api/scans/197/intelligence-summary HTTP/1.1" 200 OK
```

Figure 139 Nuclei run as per AI decision Phase-1

```
(venv) prabesh@prabesh-Surface-Laptop-3:~/Desktop/Fyproject$ source /home/prabesh/Desktop/Fyproject/venv/bin/activate && uvicorn app.main:app --
host 0.0.0.0 --port 8080
...
INFO: 127.0.0.1:46572 - "GET /api/scans/stats/severity HTTP/1.1" 200 OK
INFO: 127.0.0.1:46572 - "GET /api/scans/197/intelligence-summary HTTP/1.1" 200 OK
INFO: 127.0.0.1:46572 - "GET /api/scans/stats/severity HTTP/1.1" 200 OK
INFO: 127.0.0.1:46572 - "GET /api/scans/197/status HTTP/1.1" 200 OK
INFO: 127.0.0.1:46572 - "GET /api/scans/197/intelligence-summary HTTP/1.1" 200 OK
INFO: 127.0.0.1:46572 - "GET /api/scans/stats/severity HTTP/1.1" 200 OK
INFO: 127.0.0.1:46572 - "GET /api/scans/197/status HTTP/1.1" 200 OK
INFO: 127.0.0.1:34994 - "GET /api/scans/197/intelligence-summary HTTP/1.1" 200 OK
INFO: 127.0.0.1:34994 - "GET /api/scans/stats/severity HTTP/1.1" 200 OK
2026-04-13 22:56:26,144 - tool.nuclei - INFO - Nuclei completed with return code: 0
2026-04-13 22:56:26,146 - app.core.ws_updates - INFO - [Scan 197] Nuclei: Completed phase 2/3 (Medium/Low): Nuclei did not identify any findings
.
2026-04-13 22:56:26,148 - app.core.ws_updates - INFO - [Scan 197] Nuclei: Starting phase 3/3 (Info) with info severity templates.
2026-04-13 22:56:26,149 - tool.nuclei - INFO - Executing command: nuclei -l /home/prabesh/Desktop/Fyproject/data/scans/197/nuclei_urls.txt -json
l -o /home/prabesh/Desktop/Fyproject/data/scans/197/nuclei_info.out -silent -c 20 -bulk-size 20 -rl 100 -severity info -tags misconfig,cors,head
ers,exposure -timeout 5 -retries 0 -no-color -disable-update-check -nmhe
2026-04-13 22:56:26,149 - app.core.ws_updates - INFO - [Scan 197] Nuclei: Running command: nuclei -l /home/prabesh/Desktop/Fyproject/data/scans/
197/nuclei_urls.txt -jsonl -o /home/prabesh/Desktop/Fyproject/data/scans/197/nuclei_info.out -silent -c 20 -bulk-size 20 -rl 100 -severity info
-tags misconfig,cors,headers,exposure -timeout 5 -retries 0 -no-color -disable-update-check -nmhe
INFO: 127.0.0.1:34994 - "GET /api/scans/197/intelligence-summary HTTP/1.1" 200 OK
INFO: 127.0.0.1:34994 - "GET /api/scans/stats/severity HTTP/1.1" 200 OK
INFO: 127.0.0.1:34994 - "GET /api/scans/197/status HTTP/1.1" 200 OK
INFO: 127.0.0.1:34994 - "GET /api/scans/197/intelligence-summary HTTP/1.1" 200 OK
INFO: 127.0.0.1:34994 - "GET /api/scans/stats/severity HTTP/1.1" 200 OK
INFO: 127.0.0.1:35006 - "GET /api/scans/stats/trend?start_date=2026-04-07&end_date=2026-04-13 HTTP/1.1" 200 OK
2026-04-13 22:56:29,727 - tool.nuclei - INFO - [Nuclei] [INFO] | Cookies without HttpOnly attribute - Detect | pentest-ground.com:81
2026-04-13 22:56:31,198 - tool.nuclei - INFO - [Nuclei] [INFO] | Cookies without Secure attribute - Detect | pentest-ground.com:81
INFO: 127.0.0.1:35006 - "GET /api/scans/197/intelligence-summary HTTP/1.1" 200 OK
INFO: 127.0.0.1:35006 - "GET /api/scans/stats/severity HTTP/1.1" 200 OK
INFO: 127.0.0.1:35006 - "GET /api/scans/197/status HTTP/1.1" 200 OK
2026-04-13 22:56:35,437 - tool.nuclei - INFO - [Nuclei] [INFO] | HTTP Missing Security Headers | matcher: strict-transport-security | https://pe
ntest-ground.com
2026-04-13 22:56:35,443 - tool.nuclei - INFO - [Nuclei] [INFO] | HTTP Missing Security Headers | matcher: permissions-policy | https://pentest-g
```

Figure 140 Nuclei run Phase 2 & 3

```
(venv) prabesh@prabesh-Surface-Laptop-3:~/Desktop/Fyproject$ source /home/prabesh/Desktop/Fyproject/venv/bin/activate && uvicorn app.main:app --h
host 0.0.0.0 --port 8080
INFO: 127.0.0.1:55508 - "GET /api/scans/stats/severity HTTP/1.1" 200 OK
INFO: 127.0.0.1:55508 - "GET /api/scans/197/status HTTP/1.1" 200 OK
INFO: 127.0.0.1:55508 - "GET /api/scans/197/intelligence-summary HTTP/1.1" 200 OK
INFO: 127.0.0.1:55508 - "GET /api/scans/stats/severity HTTP/1.1" 200 OK
2026-04-13 22:57:15,910 - tool.nuclei - INFO - Nuclei completed with return code: 0
2026-04-13 22:57:16,124 - app.core.ws_updates - INFO - [Scan 197] Nuclei: Completed phase 3/3 (Info): Nuclei findings by severity: Critical=0, H
igh=0, Medium=0, Low=0, Info=67.
2026-04-13 22:57:16,161 - app.core.ws_updates - INFO - [Scan 197] Tool completed: Nuclei (True)
2026-04-13 22:57:16,165 - app.tools.executor - INFO - Nuclei completed: success=True, findings=68
2026-04-13 22:57:16,167 - app.services.scan_service - INFO - Nuclei findings: 1 vulnerabilities, 67 information
2026-04-13 22:57:16,178 - app.services.scan_service - INFO - Tool Nuclei has 68 raw findings, 10 already exist in DB
2026-04-13 22:57:16,179 - app.services.scan_service - INFO - Added finding: Nuclei - vulnerability - high at pentest-ground.com:6379
2026-04-13 22:57:16,180 - app.services.scan_service - INFO - Added finding: Nuclei - information - info at pentest-ground.com:81
2026-04-13 22:57:16,181 - app.services.scan_service - INFO - Added finding: Nuclei - information - info at pentest-ground.com:81
2026-04-13 22:57:16,183 - app.services.scan_service - INFO - Added finding: Nuclei - information - info at https://pentest-ground.com
2026-04-13 22:57:16,190 - app.services.scan_service - INFO - Added finding: Nuclei - information - info at https://pentest-ground.com:9000
2026-04-13 22:57:16,193 - app.services.scan_service - INFO - Added finding: Nuclei - information - info at https://pentest-ground.com:7001
2026-04-13 22:57:16,198 - app.services.scan_service - INFO - Added finding: Nuclei - information - info at https://pentest-ground.com:81
2026-04-13 22:57:16,212 - app.services.scan_service - INFO - Added finding: Nuclei - information - info at https://pentest-ground.com:443/
2026-04-13 22:57:16,214 - app.services.scan_service - INFO - Added finding: Nuclei - information - info at https://pentest-ground.com:9000/
2026-04-13 22:57:16,216 - app.services.scan_service - INFO - Added finding: Nuclei - information - info at https://pentest-ground.com:81/
2026-04-13 22:57:16,218 - app.services.scan_service - INFO - Added finding: Nuclei - information - info at https://pentest-ground.com
2026-04-13 22:57:16,219 - app.services.scan_service - INFO - Added finding: Nuclei - information - info at https://pentest-ground.com:9000/opena
pi.json
2026-04-13 22:57:16,221 - app.services.scan_service - INFO - Added finding: Nuclei - information - info at https://pentest-ground.com:81
2026-04-13 22:57:16,267 - app.services.scan_service - INFO - Completed Nuclei: completed, added 13 new findings to DB
2026-04-13 22:57:16,293 - app.services.scan_service - INFO - Generated report for Nuclei: 13 findings
2026-04-13 22:57:16,296 - app.services.scan_service - INFO - Scan 197: Intelligence Layer - classifying endpoints
2026-04-13 22:57:16,296 - app.core.ws_updates - INFO - [Scan 197] System: Analyzing discovered endpoints and classifying by type/risk...
2026-04-13 22:57:16,296 - app.core.ws_updates - INFO - [Scan 197] Scan phase: intelligence layer
2026-04-13 22:57:16,340 - app.services.scan_service - INFO - Intelligence Layer: classified 5 endpoints for scan 197
2026-04-13 22:57:16,340 - app.services.scan_service - INFO - Scan 197: Active Testing Engine
2026-04-13 22:57:16,341 - app.core.ws_updates - INFO - [Scan 197] System: Running active testing for OWASP A05:2021...
```

Figure 141 Output by Nuclei from all 3 phases

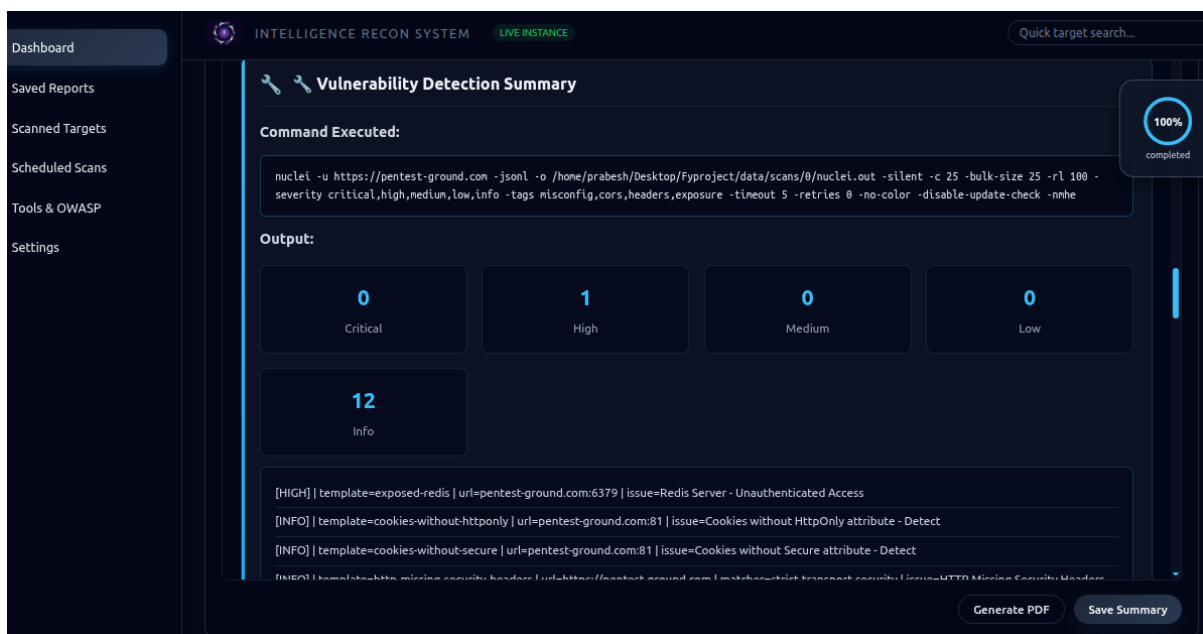


Figure 142 Scanned Target Summary, Nuclei Output


```
(venv) prabesh@prabesh-Surface-Laptop-3:~/Desktop/Fyproject$ source /home/prabesh/Desktop/Fyproject/venv/bin/activate && uvicorn app.main:app --host 0.0.0.0 --port 8080
2026-04-13 22:57:16,341 - app.core.ws_updates - INFO - [Scan 197] System: Running active testing for OWASP A05:2021...
2026-04-13 22:57:16,341 - app.core.ws_updates - INFO - [Scan 197] Scan phase: active testing
INFO: 127.0.0.1:55508 - "GET /api/scans/197/intelligence-summary HTTP/1.1" 200 OK
INFO: 127.0.0.1:55508 - "GET /api/scans/stats/severity HTTP/1.1" 200 OK
INFO: 127.0.0.1:55508 - "GET /api/scans/197/status HTTP/1.1" 200 OK
INFO: 127.0.0.1:55508 - "GET /api/scans/197/intelligence-summary HTTP/1.1" 200 OK
INFO: 127.0.0.1:55508 - "GET /api/scans/stats/severity HTTP/1.1" 200 OK
2026-04-13 22:57:21,815 - app.services.scan_service - INFO - Active Testing: completed 1 tests for scan 197
2026-04-13 22:57:21,815 - app.services.scan_service - INFO - Scan 197: Response Analysis
2026-04-13 22:57:21,816 - app.core.ws_updates - INFO - [Scan 197] System: Analyzing responses for vulnerability indicators...
2026-04-13 22:57:21,816 - app.core.ws_updates - INFO - [Scan 197] Scan phase: response analysis
2026-04-13 22:57:21,828 - app.services.scan_service - INFO - Response Analysis: found 1 vulnerabilities for scan 197
2026-04-13 22:57:21,828 - app.services.scan_service - INFO - Scan 197: Correlation
2026-04-13 22:57:21,829 - app.core.ws_updates - INFO - [Scan 197] System: Correlating findings across all tools...
2026-04-13 22:57:21,829 - app.core.ws_updates - INFO - [Scan 197] Scan phase: correlation
2026-04-13 22:57:21,839 - app.services.scan_service - INFO - Correlation: created 0 correlated findings for scan 197
2026-04-13 22:57:21,839 - app.services.scan_service - INFO - Scan 197: Smart Risk Scoring
2026-04-13 22:57:21,840 - app.core.ws_updates - INFO - [Scan 197] System: Applying smart risk scoring based on verified exploits...
2026-04-13 22:57:21,840 - app.core.ws_updates - INFO - [Scan 197] Scan phase: risk scoring
2026-04-13 22:57:21,851 - app.services.scan_service - INFO - Risk Scoring: https://pentest-ground.com:9000 severity low -> medium (tool: ActiveTester)
2026-04-13 22:57:21,865 - app.services.scan_service - INFO - Smart Risk Scoring: completed severity adjustments for scan 197
2026-04-13 22:57:21,867 - app.services.scan_service - INFO - Scan 197: Finalizing
2026-04-13 22:57:21,892 - app.services.scan_service - INFO - Scan 197 finalized with status: completed
INFO: 127.0.0.1:55508 - "GET /api/scans/197/status HTTP/1.1" 200 OK
INFO: 127.0.0.1:50998 - "GET /api/scans/197/intelligence-summary HTTP/1.1" 200 OK
INFO: 127.0.0.1:50998 - "GET /api/scans/stats/severity HTTP/1.1" 200 OK
INFO: 127.0.0.1:55508 - "GET /api/scans?page_size=100&include_unsaved=true HTTP/1.1" 200 OK
INFO: 127.0.0.1:50998 - "GET /api/scans/stats/severity HTTP/1.1" 200 OK
INFO: 127.0.0.1:50998 - "GET /api/scheduled-scans HTTP/1.1" 200 OK
INFO: 127.0.0.1:55508 - "GET /api/scans?page_size=100&include_unsaved=false HTTP/1.1" 200 OK
INFO: 127.0.0.1:50998 - "GET /api/scans/stats/trend?start_date=2026-04-07&end_date=2026-04-13 HTTP/1.1" 200 OK
INFO: 127.0.0.1:55508 - "GET /api/scans?include_unsaved=false HTTP/1.1" 200 OK
```

Figure 143 Active tester/Response Analyzer Engine run phase and completed full scan.

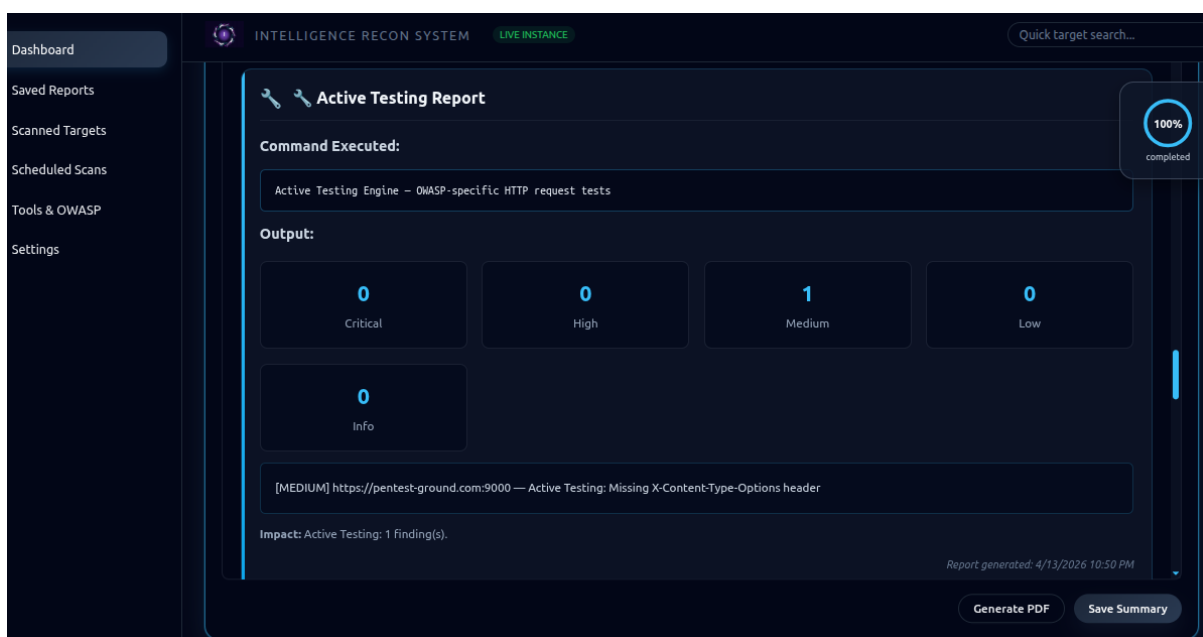


Figure 144 Active Testing Report

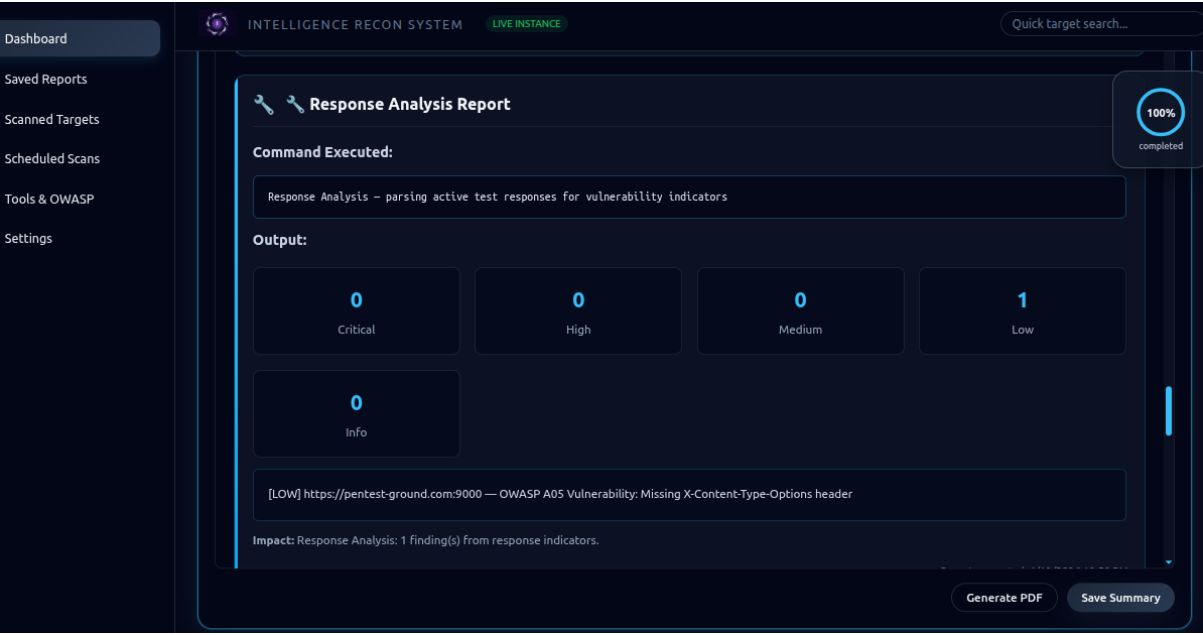


Figure 145 Response Analysis Report

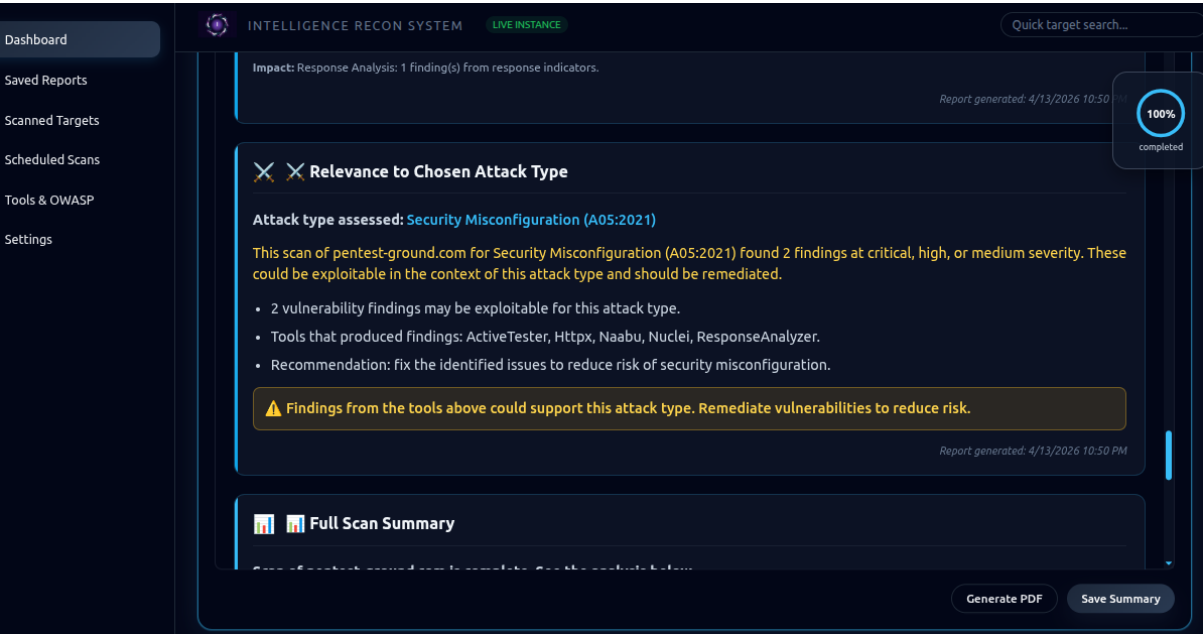


Figure 146 Relevance to Chosen Attack Type

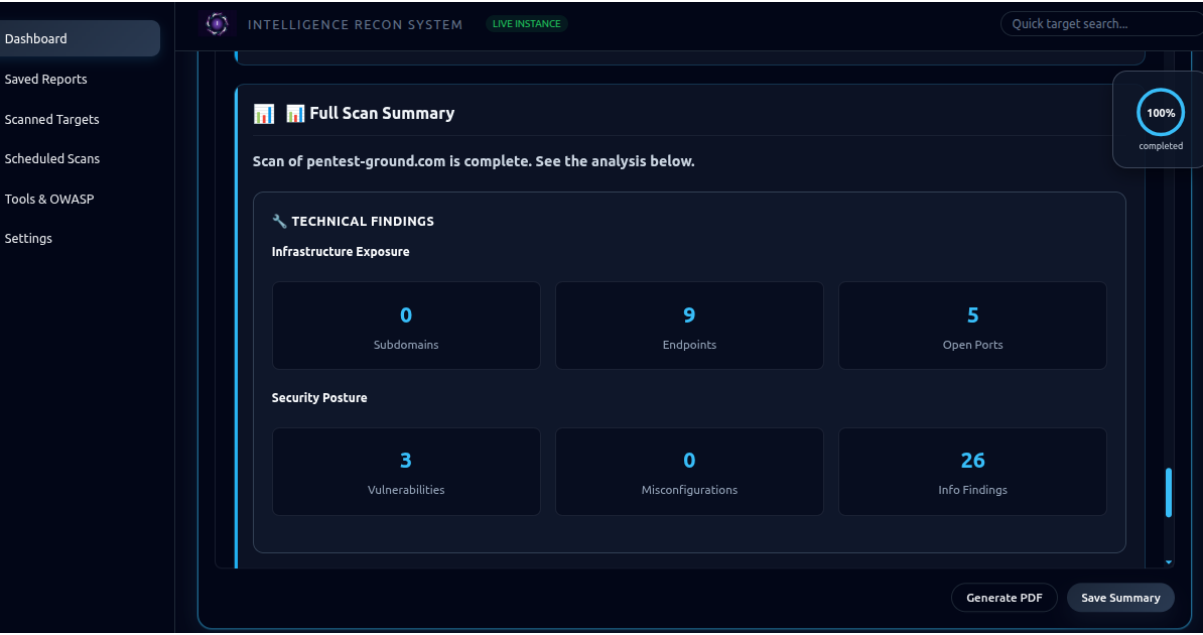


Figure 147 Full Scan Summary

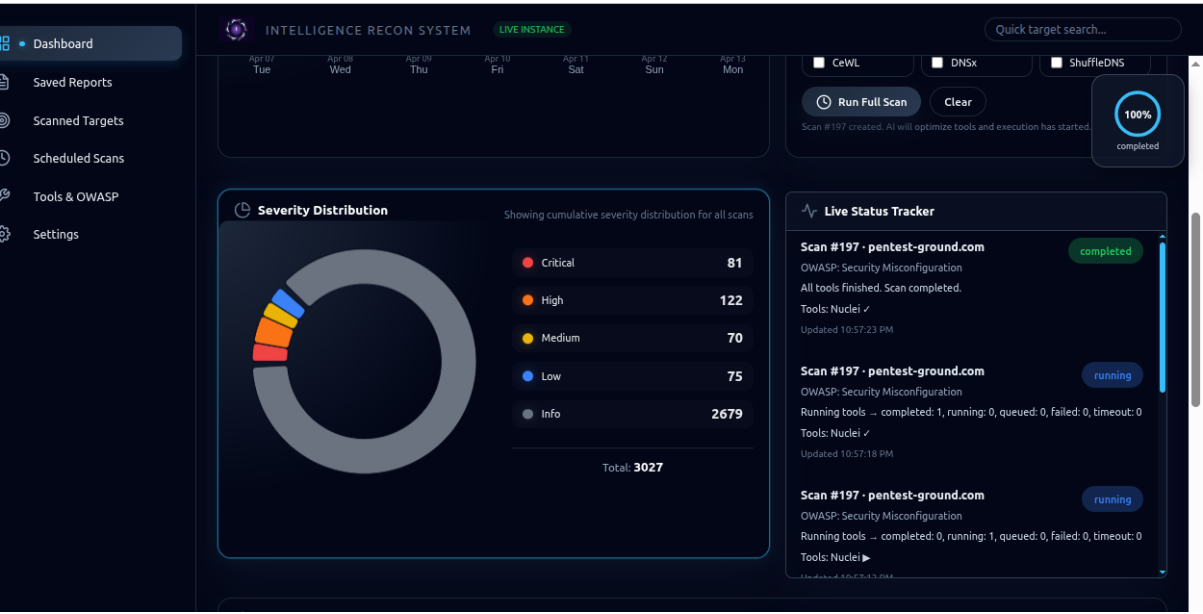


Figure 148 Completed Green Badge after scanning complete

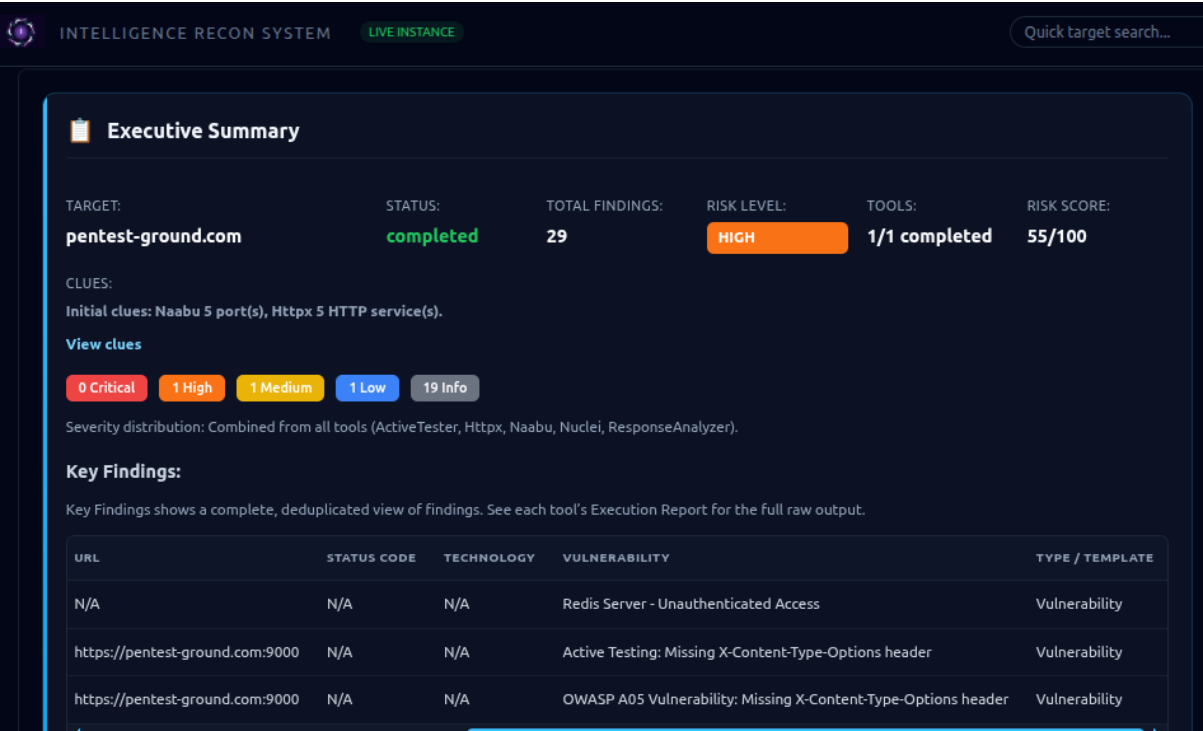


Figure 149 Executive Summary

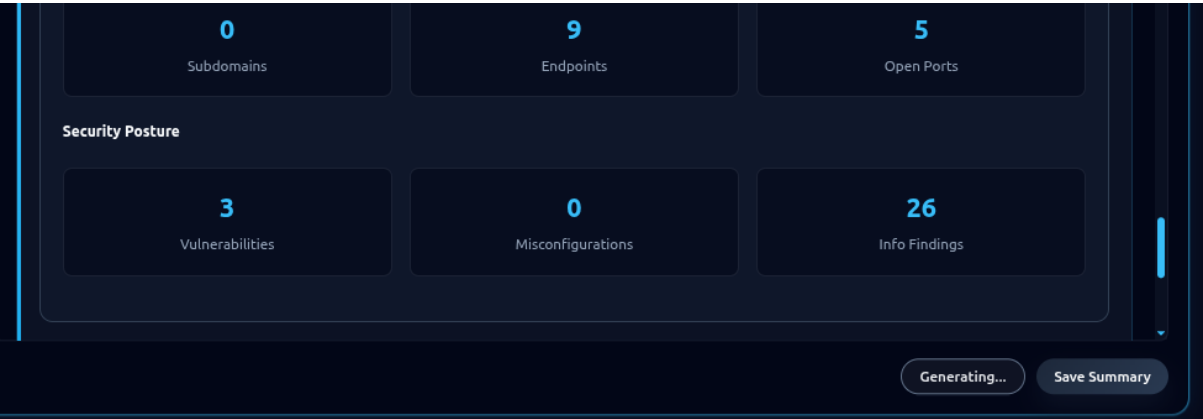


Figure 150 Generating PDF

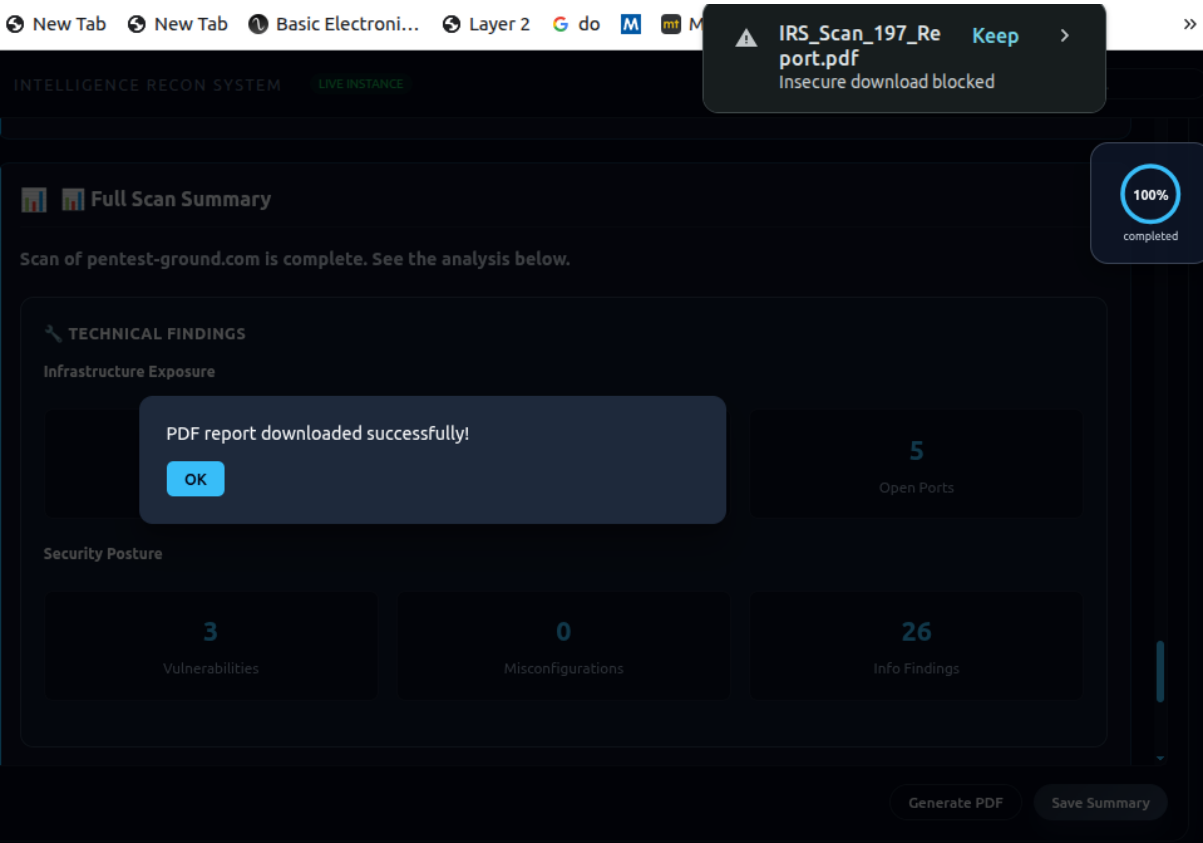


Figure 151 Successfully Downloaded Scan pdf

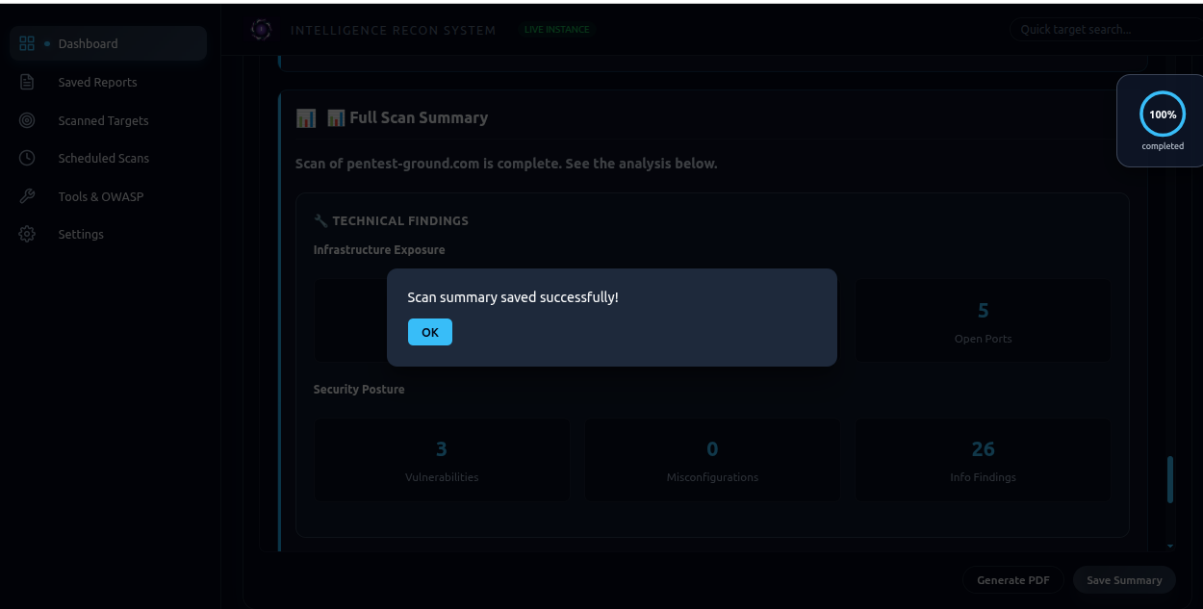


Figure 152 Scan Saved

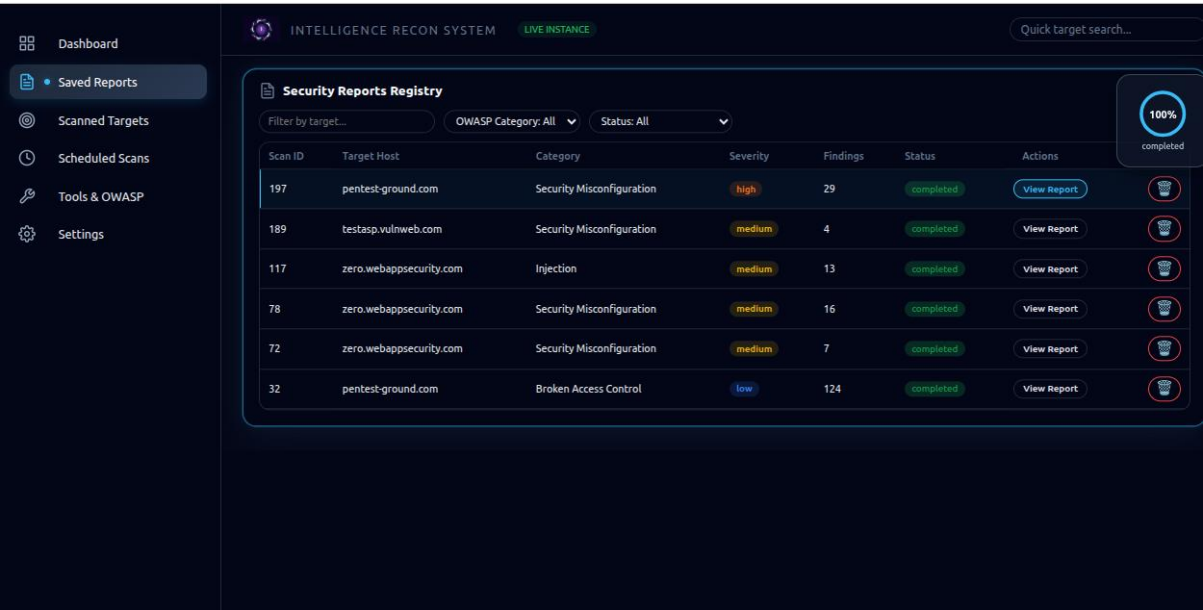


Figure 153 Saved scan added on Saved Reports

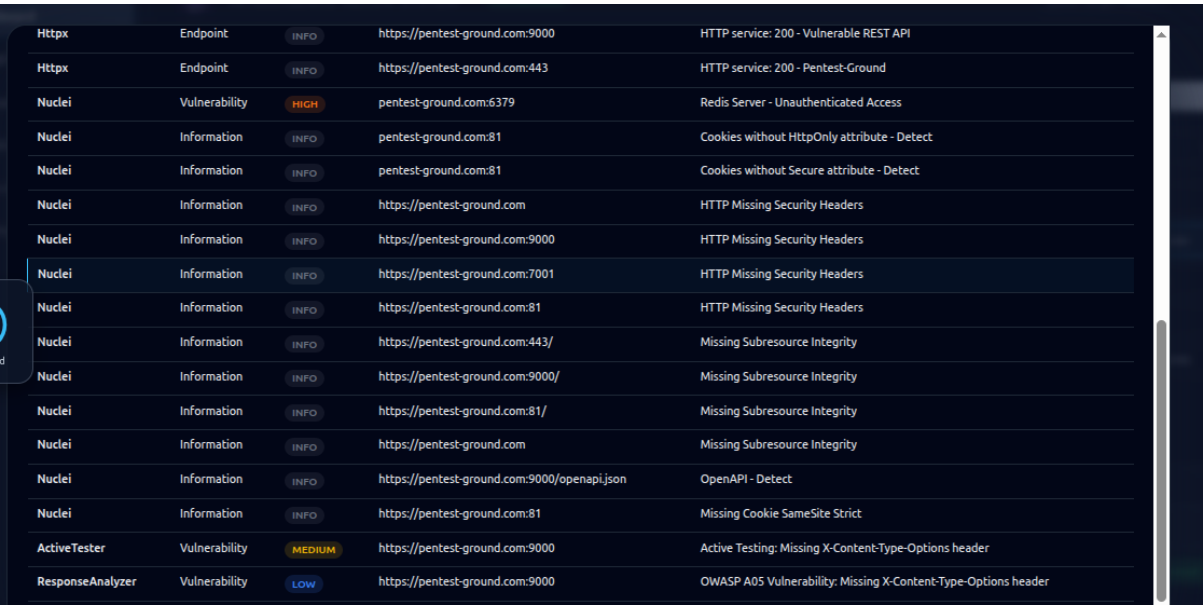


Figure 154 Findings list shown on Security Trend Analytics

Test Case 2

System Testing	Objectives
Objective	To verify end-to-end scheduled scan process without error.
Action	1. Open and filled needed form of target. 2. Scheduled Time 3. Clicked Add Scheduled
Expected Result	Scheduled scan should be added on Your Scheduled and run scan on scheduled time and provide report and save report features.
Actual Result	As expected, scan run on scheduled time and provide report after complete and can be able to view and save report.
Conclusion	The test was successful.

Table 19 System Test Case 2

Evidence

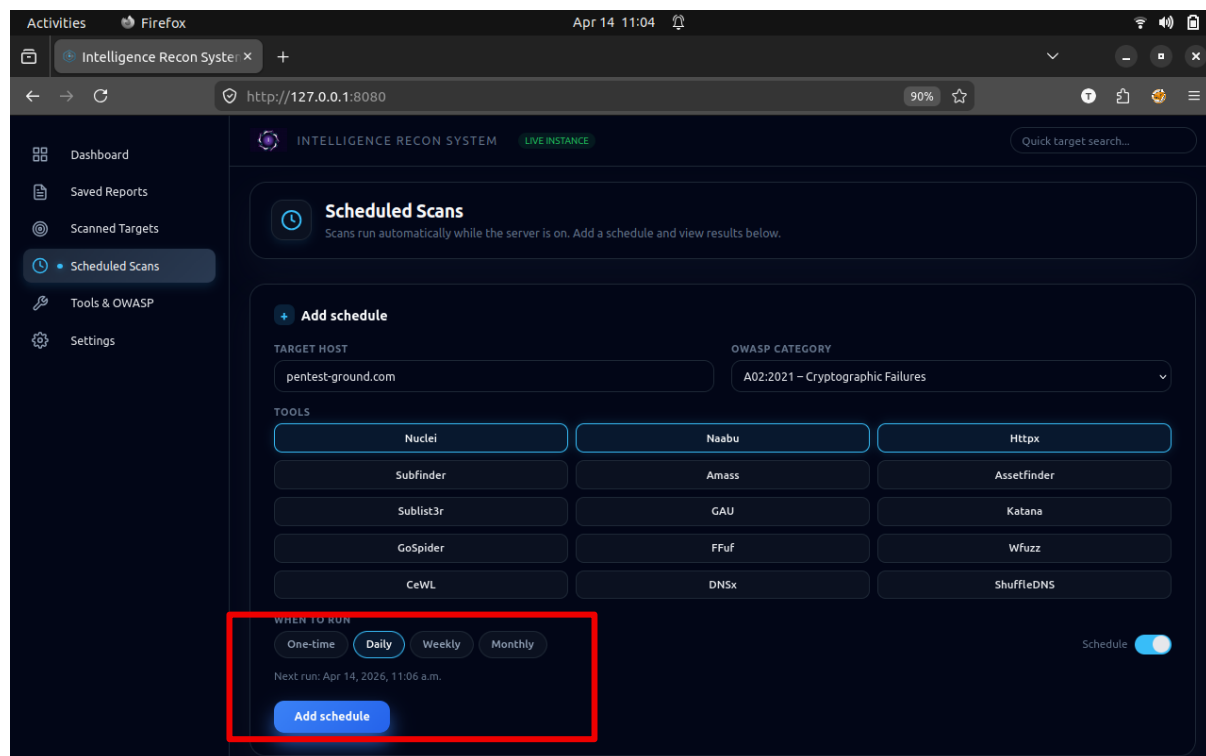


Figure 155 Filled requirement and Schedule Time for scanning

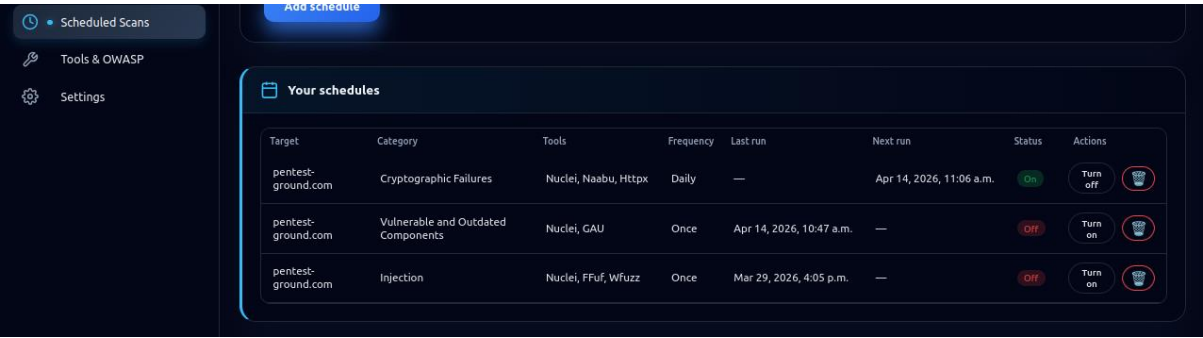


Figure 156 Schedule scan added on "Your Schedule"

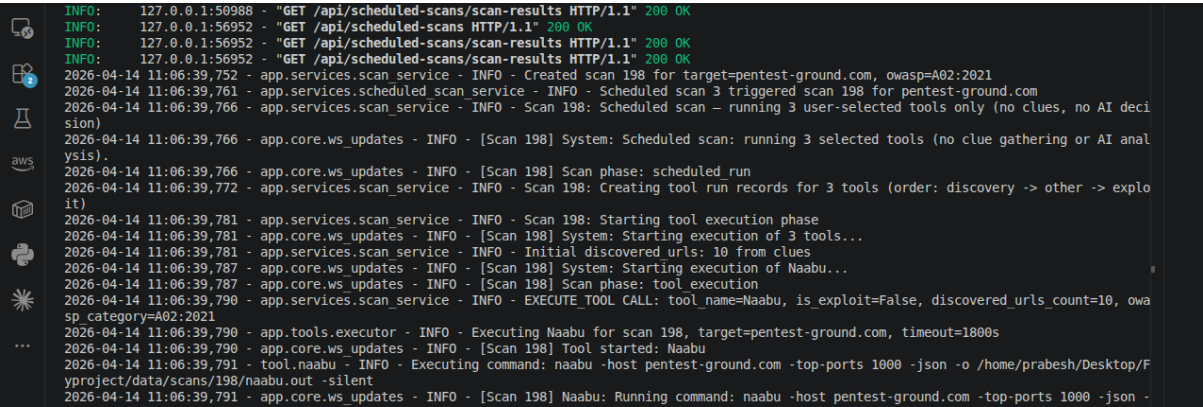


Figure 157 Scan had been triggered

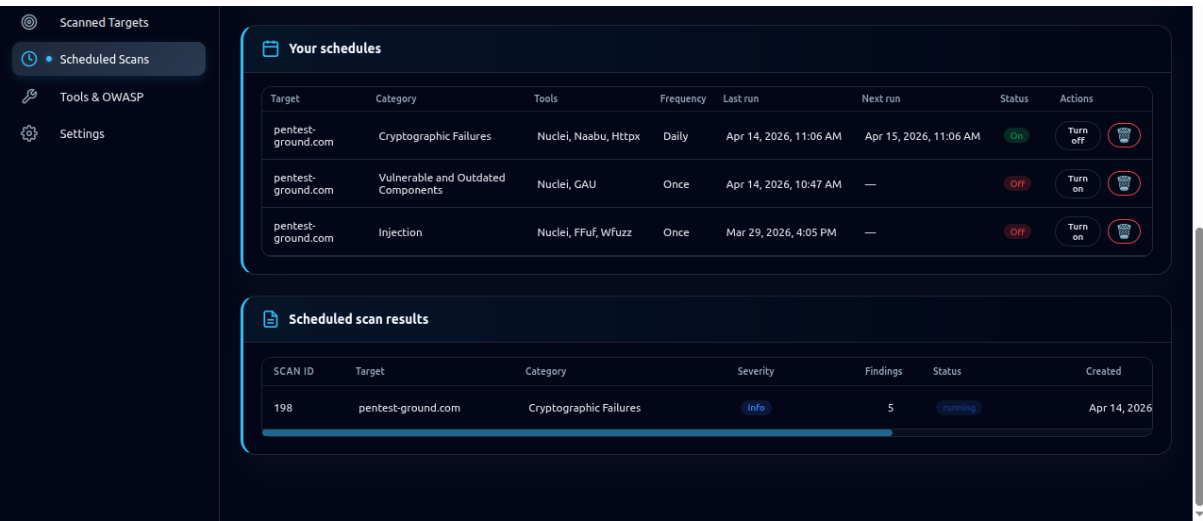


Figure 158 Scan appears in "Scheduled scan result"


```

host 0.0.0.0 --port 8080
2026-04-14 11:06:39,790 - app.core.ws updates - INFO - [Scan 198] Tool started: Naabu
2026-04-14 11:06:39,791 - tool.naabu - INFO - Executing command: naabu -host pentest-ground.com -top-ports 1000 -json -o /home/prabesh/Desktop/Fyproject/data/scans/198/naabu.out -silent
2026-04-14 11:06:39,791 - app.core.ws updates - INFO - [Scan 198] Naabu: Running command: naabu -host pentest-ground.com -top-ports 1000 -json -o /home/prabesh/Desktop/Fyproject/data/scans/198/naabu.out -silent
INFO: 127.0.0.1:50988 - "GET /api/scheduled-scans HTTP/1.1" 200 OK
INFO: 127.0.0.1:50988 - "GET /api/scheduled-scans/scan-results HTTP/1.1" 200 OK
INFO: 127.0.0.1:50988 - "GET /api/scheduled-scans/scan-results HTTP/1.1" 200 OK
INFO: 127.0.0.1:56952 - "GET /api/scheduled-scans HTTP/1.1" 200 OK
INFO: 127.0.0.1:56952 - "GET /api/scheduled-scans/scan-results HTTP/1.1" 200 OK
INFO: 127.0.0.1:56952 - "GET /api/scheduled-scans/scan-results HTTP/1.1" 200 OK
2026-04-14 11:06:42,297 - tool.naabu - INFO - [Naabu] {"host": "pentest-ground.com", "ip": "178.79.134.182", "timestamp": "2026-04-14T05:21:42.297124", "port": 80, "protocol": "tcp", "tls": false}
2026-04-14 11:06:42,297 - tool.naabu - INFO - [Naabu] {"host": "pentest-ground.com", "ip": "178.79.134.182", "timestamp": "2026-04-14T05:21:42.297321", "port": 81, "protocol": "tcp", "tls": false}
2026-04-14 11:06:42,332 - tool.naabu - INFO - [Naabu] {"host": "pentest-ground.com", "ip": "178.79.134.182", "timestamp": "2026-04-14T05:21:42.332227", "port": 9000, "protocol": "tcp", "tls": false}
2026-04-14 11:06:42,344 - tool.naabu - INFO - [Naabu] {"host": "pentest-ground.com", "ip": "178.79.134.182", "timestamp": "2026-04-14T05:21:42.344293", "port": 4445, "protocol": "tcp", "tls": false}
2026-04-14 11:06:42,352 - tool.naabu - INFO - [Naabu] {"host": "pentest-ground.com", "ip": "178.79.134.182", "timestamp": "2026-04-14T05:21:42.352600", "port": 443, "protocol": "tcp", "tls": false}
INFO: 127.0.0.1:50988 - "GET /api/scheduled-scans HTTP/1.1" 200 OK
INFO: 127.0.0.1:50988 - "GET /api/scheduled-scans/scan-results HTTP/1.1" 200 OK
INFO: 127.0.0.1:50988 - "GET /api/scheduled-scans/scan-results HTTP/1.1" 200 OK
INFO: 127.0.0.1:56952 - "GET /api/scheduled-scans HTTP/1.1" 200 OK
INFO: 127.0.0.1:56952 - "GET /api/scheduled-scans/scan-results HTTP/1.1" 200 OK
INFO: 127.0.0.1:56952 - "GET /api/scheduled-scans/scan-results HTTP/1.1" 200 OK
2026-04-14 11:06:47,161 - tool.naabu - INFO - [Naabu] {"host": "pentest-ground.com", "ip": "178.79.134.182", "timestamp": "2026-04-14T05:21:47.161530", "port": 80, "protocol": "tcp", "tls": false}
2026-04-14 11:06:47,162 - tool.naabu - INFO - [Naabu] {"host": "pentest-ground.com", "ip": "178.79.134.182", "timestamp": "2026-04-14T05:21:47.161561", "port": 81, "protocol": "tcp", "tls": false}
2026-04-14 11:06:47,162 - tool.naabu - INFO - [Naabu] {"host": "pentest-ground.com", "ip": "178.79.134.182", "timestamp": "2026-04-14T05:21:47.161566", "port": 9000, "protocol": "tcp", "tls": false}
2026-04-14 11:06:47,162 - tool.naabu - INFO - [Naabu] {"host": "pentest-ground.com", "ip": "178.79.134.182", "timestamp": "2026-04-14T05:21:47.161566", "port": 9000, "protocol": "tcp", "tls": false}

```

Figure 159 Tool Execution

```

host 0.0.0.0 --port 8080
2026-04-14 11:06:47,178 - app.services.scan_service - INFO - Added finding: Naabu - port - info at pentest-ground.com:4445
2026-04-14 11:06:47,178 - app.services.scan_service - INFO - Added finding: Naabu - port - info at pentest-ground.com:443
2026-04-14 11:06:47,186 - app.services.scan_service - INFO - Completed Naabu: completed, added 5 new findings to DB
2026-04-14 11:06:47,189 - app.services.scan_service - INFO - Generated report for Naabu: 5 findings
2026-04-14 11:06:47,193 - app.core.ws updates - INFO - [Scan 198] System: Starting execution of Httpx...
2026-04-14 11:06:47,193 - app.core.ws updates - INFO - [Scan 198] Scan phase: tool execution
2026-04-14 11:06:47,197 - app.services.scan_service - INFO - EXECUTE_TOOL CALL: tool_name=Httpx, is_exploit=False, discovered_urls_count=10, owa_sp_category=A02:2021
2026-04-14 11:06:47,197 - app.tools.executor - INFO - Executing Httpx for scan 198, target=pentest-ground.com, timeout=1800s
2026-04-14 11:06:47,197 - app.core.ws updates - INFO - [Scan 198] Tool started: Httpx
2026-04-14 11:06:47,197 - tool.httpx - INFO - Executing command: /home/prabesh/go/bin/httpx -u http://pentest-ground.com:80 -u https://pentest-ground.com:80 -u http://pentest-ground.com:443 -u https://pentest-ground.com:443 -u http://pentest-ground.com:8080 -u https://pentest-ground.com:8080 -u http://pentest-ground.com:8443 -u https://pentest-ground.com:8443 -u http://pentest-ground.com:3000 -u https://pentest-ground.com:3000 -u http://pentest-ground.com:5000 -u https://pentest-ground.com:5000 -u http://pentest-ground.com:7070 -u https://pentest-ground.com:7070 -u http://pentest-ground.com:8888 -u https://pentest-ground.com:8888 -u http://pentest-ground.com:9000 -u https://pentest-ground.com:9000 -json -o /home/prabesh/Desktop/Fyproject/data/scans/198/httpx.out -silent -title -td -sc -web-server -ip -cname
2026-04-14 11:06:47,197 - app.core.ws updates - INFO - [Scan 198] Httpx: Running command: /home/prabesh/go/bin/httpx -u http://pentest-ground.com:80 -u https://pentest-ground.com:80 -u http://pentest-ground.com:443 -u https://pentest-ground.com:443 -u http://pentest-ground.com:8080 -u https://pentest-ground.com:8080 -u http://pentest-ground.com:8443 -u https://pentest-ground.com:8443 -u http://pentest-ground.com:3000 -u https://pentest-ground.com:3000 -u http://pentest-ground.com:5000 -u https://pentest-ground.com:5000 -u http://pentest-ground.com:7070 -u https://pentest-ground.com:7070 -u http://pentest-ground.com:8888 -u https://pentest-ground.com:8888 -u http://pentest-ground.com:9000 -u https://pentest-ground.com:9000 -json -o /home/prabesh/Desktop/Fyproject/data/scans/198/httpx.out -silent -title -td -sc -web-server -ip -cname
INFO: 127.0.0.1:50988 - "GET /api/scheduled-scans HTTP/1.1" 200 OK
INFO: 127.0.0.1:50988 - "GET /api/scheduled-scans/scan-results HTTP/1.1" 200 OK
INFO: 127.0.0.1:50988 - "GET /api/scheduled-scans/scan-results HTTP/1.1" 200 OK
INFO: 127.0.0.1:56952 - "GET /api/scheduled-scans HTTP/1.1" 200 OK
INFO: 127.0.0.1:56952 - "GET /api/scheduled-scans/scan-results HTTP/1.1" 200 OK
INFO: 127.0.0.1:56952 - "GET /api/scheduled-scans/scan-results HTTP/1.1" 200 OK
INFO: 127.0.0.1:50988 - "GET /api/scheduled-scans HTTP/1.1" 200 OK
INFO: 127.0.0.1:50988 - "GET /api/scheduled-scans/scan-results HTTP/1.1" 200 OK
INFO: 127.0.0.1:50988 - "GET /api/scheduled-scans/scan-results HTTP/1.1" 200 OK

```

Figure 160 Tool Execution

```

host 0.0.0.0 --port 8080
INFO: 127.0.0.1:56952 - "GET /api/scheduled-scans HTTP/1.1" 200 OK
INFO: 127.0.0.1:56952 - "GET /api/scheduled-scans/scan-results HTTP/1.1" 200 OK
INFO: 127.0.0.1:56952 - "GET /api/scheduled-scans/scan-results HTTP/1.1" 200 OK
2026-04-14 11:07:08,785 - tool.httpx - INFO - Httpx completed with return code: 0
2026-04-14 11:07:08,786 - app.core.ws_updates - INFO - [Scan 198] Tool completed: Httpx (True)
2026-04-14 11:07:08,786 - app.tools.executor - INFO - Httpx completed: success=True, findings=0
2026-04-14 11:07:08,792 - app.services.scan_service - INFO - Completed Httpx: completed, added 0 new findings to DB
2026-04-14 11:07:08,794 - app.services.scan_service - INFO - Generated report for Httpx: 0 findings
2026-04-14 11:07:08,797 - app.core.ws_updates - INFO - [Scan 198] System: Starting execution of Nuclei...
2026-04-14 11:07:08,797 - app.core.ws_updates - INFO - [Scan 198] Scan phase: tool execution
2026-04-14 11:07:08,801 - app.services.scan_service - INFO - EXECUTE TOOL CALL: tool_name=Nuclei, is_exploit=True, discovered_urls_count=10, owa
sp_category=A02:2021
2026-04-14 11:07:08,801 - app.tools.executor - INFO - SMART TIMEOUT: Calculated Nuclei timeout=3150s (base=1800, urls=10, owasp=A02:2021)
2026-04-14 11:07:08,801 - app.tools.executor - INFO - NUCLEI FINAL TIMEOUT: timeout value=3150s, timeout param=3150, settings.TOOL_TIMEOUT=1800
2026-04-14 11:07:08,801 - app.tools.executor - INFO - Executing Nuclei for scan 198, target=pentest-ground.com, timeout=3150s
2026-04-14 11:07:08,801 - app.core.ws_updates - INFO - [Scan 198] Tool started: Nuclei
2026-04-14 11:07:08,802 - app.core.ws_updates - INFO - [Scan 198] Nuclei: Starting phase 1/3 (Critical/High) with critical,high severity templat
es.
2026-04-14 11:07:08,802 - tool.nuclei - INFO - Executing command: nuclei -l /home/prabesh/Desktop/Fyproject/data/scans/198/nuclei_urls.txt -json
l -o /home/prabesh/Desktop/Fyproject/data/scans/198/nuclei_critical_high.out -silent -c 15 -bulk-size 15 -rl 75 -severity critical,high -tags ss
l,tls,expired-ssl,certificate -timeout 5 -retries 0 -no-color -disable-update-check -nmhe
2026-04-14 11:07:08,802 - app.core.ws_updates - INFO - [Scan 198] Nuclei: Running command: nuclei -l /home/prabesh/Desktop/Fyproject/data/scans/
198/nuclei_urls.txt -jsonl -o /home/prabesh/Desktop/Fyproject/data/scans/198/nuclei_critical_high.out -silent -c 15 -bulk-size 15 -rl 75 -severi
ty critical,high -tags ssl,tls,expired-ssl,certificate -timeout 5 -retries 0 -no-color -disable-update-check -nmhe
2026-04-14 11:07:09,037 - tool.nuclei - INFO - [Nuclei] [stderr] [FTL] Could not run nuclei: no templates provided for scan
2026-04-14 11:07:09,043 - tool.nuclei - INFO - Nuclei completed with return code: 1
2026-04-14 11:07:09,044 - app.core.ws_updates - INFO - [Scan 198] Nuclei: Completed phase 1/3 (Critical/High): Nuclei did not identify any findi
ngs.
2026-04-14 11:07:09,044 - app.core.ws_updates - INFO - [Scan 198] Nuclei: Starting phase 2/3 (Medium/Low) with medium,low severity templates.
2026-04-14 11:07:09,045 - tool.nuclei - INFO - Executing command: nuclei -l /home/prabesh/Desktop/Fyproject/data/scans/198/nuclei_urls.txt -json
l -o /home/prabesh/Desktop/Fyproject/data/scans/198/nuclei_medium_low.out -silent -c 15 -bulk-size 15 -rl 75 -severity medium,low -tags ssl,tls,
expired-ssl,certificate -timeout 5 -retries 0 -no-color -disable-update-check -nmhe
2026-04-14 11:07:09,045 - app.core.ws_updates - INFO - [Scan 198] Nuclei: Running command: nuclei -l /home/prabesh/Desktop/Fyproject/data/scans/
198/nuclei_urls.txt -jsonl -o /home/prabesh/Desktop/Fyproject/data/scans/198/nuclei_medium_low.out -silent -c 15 -bulk-size 15 -rl 75 -severity
medium,low -tags ssl,tls,expired-ssl,certificate -timeout 5 -retries 0 -no-color -disable-update-check -nmhe

```

Figure 161 Tool Execution

```

host 0.0.0.0 --port 8080
2026-04-14 11:07:09,044 - app.core.ws_updates - INFO - [Scan 198] Nuclei: Completed phase 1/3 (Critical/High): Nuclei did not identify any findi
ngs.
2026-04-14 11:07:09,044 - app.core.ws_updates - INFO - [Scan 198] Nuclei: Starting phase 2/3 (Medium/Low) with medium,low severity templates.
2026-04-14 11:07:09,045 - tool.nuclei - INFO - Executing command: nuclei -l /home/prabesh/Desktop/Fyproject/data/scans/198/nuclei_urls.txt -json
l -o /home/prabesh/Desktop/Fyproject/data/scans/198/nuclei_medium_low.out -silent -c 15 -bulk-size 15 -rl 75 -severity medium,low -tags ssl,tls,
expired-ssl,certificate -timeout 5 -retries 0 -no-color -disable-update-check -nmhe
2026-04-14 11:07:09,045 - app.core.ws_updates - INFO - [Scan 198] Nuclei: Running command: nuclei -l /home/prabesh/Desktop/Fyproject/data/scans/
198/nuclei_urls.txt -jsonl -o /home/prabesh/Desktop/Fyproject/data/scans/198/nuclei_medium_low.out -silent -c 15 -bulk-size 15 -rl 75 -severity
medium,low -tags ssl,tls,expired-ssl,certificate -timeout 5 -retries 0 -no-color -disable-update-check -nmhe
INFO: 127.0.0.1:33082 - "GET /api/scheduled-scans HTTP/1.1" 200 OK
INFO: 127.0.0.1:33082 - "GET /api/scheduled-scans/scan-results HTTP/1.1" 200 OK
INFO: 127.0.0.1:33082 - "GET /api/scheduled-scans/scan-results HTTP/1.1" 200 OK
INFO: 127.0.0.1:56952 - "GET /api/scheduled-scans HTTP/1.1" 200 OK
INFO: 127.0.0.1:56952 - "GET /api/scheduled-scans/scan-results HTTP/1.1" 200 OK
INFO: 127.0.0.1:56952 - "GET /api/scheduled-scans/scan-results HTTP/1.1" 200 OK
INFO: 127.0.0.1:45532 - "GET /api/scheduled-scans HTTP/1.1" 200 OK
INFO: 127.0.0.1:45532 - "GET /api/scheduled-scans/scan-results HTTP/1.1" 200 OK
INFO: 127.0.0.1:45532 - "GET /api/scheduled-scans/scan-results HTTP/1.1" 200 OK
INFO: 127.0.0.1:56952 - "GET /api/scheduled-scans HTTP/1.1" 200 OK
INFO: 127.0.0.1:56952 - "GET /api/scheduled-scans/scan-results HTTP/1.1" 200 OK
INFO: 127.0.0.1:56952 - "GET /api/scheduled-scans/scan-results HTTP/1.1" 200 OK
2026-04-14 11:07:17,487 - tool.nuclei - INFO - Nuclei completed with return code: 0
2026-04-14 11:07:17,487 - app.core.ws_updates - INFO - [Scan 198] Nuclei: Completed phase 2/3 (Medium/Low): Nuclei did not identify any findings
.
2026-04-14 11:07:17,488 - app.core.ws_updates - INFO - [Scan 198] Nuclei: Starting phase 3/3 (Info) with info severity templates.
2026-04-14 11:07:17,488 - tool.nuclei - INFO - Executing command: nuclei -l /home/prabesh/Desktop/Fyproject/data/scans/198/nuclei_urls.txt -json
l -o /home/prabesh/Desktop/Fyproject/data/scans/198/nuclei_info.out -silent -c 20 -bulk-size 20 -rl 100 -severity info -tags ssl,tls,expired-ssl
,certificate -timeout 5 -retries 0 -no-color -disable-update-check -nmhe
2026-04-14 11:07:17,488 - app.core.ws_updates - INFO - [Scan 198] Nuclei: Running command: nuclei -l /home/prabesh/Desktop/Fyproject/data/scans/
198/nuclei_urls.txt -jsonl -o /home/prabesh/Desktop/Fyproject/data/scans/198/nuclei_info.out -silent -c 20 -bulk-size 20 -rl 100 -severity info
-tags ssl,tls,expired-ssl,certificate -timeout 5 -retries 0 -no-color -disable-update-check -nmhe
2026-04-14 11:07:18,425 - tool.nuclei - INFO - [Nuclei] [INFO] | Detect SSL Certificate Issuer | pentest-ground.com:443
2026-04-14 11:07:18,426 - tool.nuclei - INFO - [Nuclei] [INFO] | SSL DNS Names | pentest-ground.com:443

```

Figure 162 Tool Execution

```

host 0.0.0.0 --port 8080
2026-04-14 11:07:17,487 - app.core.ws_updates - INFO - [Scan 198] Nuclei: Completed phase 2/3 (Medium/Low): Nuclei did not identify any findings
.
2026-04-14 11:07:17,488 - app.core.ws_updates - INFO - [Scan 198] Nuclei: Starting phase 3/3 (Info) with info severity templates.
2026-04-14 11:07:17,488 - tool.nuclei - INFO - Executing command: nuclei -l /home/prabesh/Desktop/Fyproject/data/scans/198/nuclei_urls.txt -jsonl -o /home/prabesh/Desktop/Fyproject/data/scans/198/nuclei_info.out -silent -c 20 -bulk-size 20 -rl 100 -severity info -tags ssl,tls,expired-ssl,certificate -timeout 5 -retries 0 -no-color -disable-update-check -nmhe
2026-04-14 11:07:17,488 - app.core.ws_updates - INFO - [Scan 198] Nuclei: Running command: nuclei -l /home/prabesh/Desktop/Fyproject/data/scans/198/nuclei_urls.txt -jsonl -o /home/prabesh/Desktop/Fyproject/data/scans/198/nuclei_info.out -silent -c 20 -bulk-size 20 -rl 100 -severity info -tags ssl,tls,expired-ssl,certificate -timeout 5 -retries 0 -no-color -disable-update-check -nmhe
2026-04-14 11:07:18,425 - tool.nuclei - INFO - [Nuclei] [INFO] Detect SSL Certificate Issuer | pentest-ground.com:443
2026-04-14 11:07:18,426 - tool.nuclei - INFO - [Nuclei] [INFO] SSL DNS Names | pentest-ground.com:443
2026-04-14 11:07:18,455 - tool.nuclei - INFO - [Nuclei] [INFO] Detect SSL Certificate Issuer | pentest-ground.com:443
2026-04-14 11:07:18,455 - tool.nuclei - INFO - [Nuclei] [INFO] SSL DNS Names | pentest-ground.com:443
2026-04-14 11:07:18,455 - tool.nuclei - INFO - [Nuclei] [INFO] Detect SSL Certificate Issuer | pentest-ground.com:443
2026-04-14 11:07:18,456 - tool.nuclei - INFO - [Nuclei] [INFO] SSL DNS Names | pentest-ground.com:443
2026-04-14 11:07:18,456 - tool.nuclei - INFO - [Nuclei] [INFO] Detect SSL Certificate Issuer | pentest-ground.com:443
2026-04-14 11:07:18,457 - tool.nuclei - INFO - [Nuclei] [INFO] SSL DNS Names | pentest-ground.com:443
2026-04-14 11:07:18,469 - tool.nuclei - INFO - [Nuclei] [INFO] Detect SSL Certificate Issuer | pentest-ground.com:443
2026-04-14 11:07:18,469 - tool.nuclei - INFO - [Nuclei] [INFO] SSL DNS Names | pentest-ground.com:443
2026-04-14 11:07:21,155 - tool.nuclei - INFO - [Nuclei] [INFO] TLS Version - Detect | pentest-ground.com:443
2026-04-14 11:07:21,194 - tool.nuclei - INFO - [Nuclei] [INFO] TLS Version - Detect | pentest-ground.com:443
2026-04-14 11:07:21,245 - tool.nuclei - INFO - [Nuclei] [INFO] TLS Version - Detect | pentest-ground.com:443
2026-04-14 11:07:21,306 - tool.nuclei - INFO - [Nuclei] [INFO] TLS Version - Detect | pentest-ground.com:443
INFO: 127.0.0.1:56952 - "GET /api/scheduled-scans HTTP/1.1" 200 OK
INFO: 127.0.0.1:56952 - "GET /api/scheduled-scans/scan-results HTTP/1.1" 200 OK
INFO: 127.0.0.1:56952 - "GET /api/scheduled-scans/scan-results HTTP/1.1" 200 OK
2026-04-14 11:07:21,543 - tool.nuclei - INFO - [Nuclei] [INFO] TLS Version - Detect | pentest-ground.com:443
INFO: 127.0.0.1:56952 - "GET /api/scheduled-scans HTTP/1.1" 200 OK
INFO: 127.0.0.1:56952 - "GET /api/scheduled-scans/scan-results HTTP/1.1" 200 OK
INFO: 127.0.0.1:56952 - "GET /api/scheduled-scans/scan-results HTTP/1.1" 200 OK
2026-04-14 11:07:21,642 - tool.nuclei - INFO - [Nuclei] [INFO] TLS Version - Detect | pentest-ground.com:443
2026-04-14 11:07:21,695 - tool.nuclei - INFO - [Nuclei] [INFO] TLS Version - Detect | pentest-ground.com:443
2026-04-14 11:07:21,718 - tool.nuclei - INFO - [Nuclei] [INFO] TLS Version - Detect | pentest-ground.com:443

```

Figure 163 Tool Execution

```

host 0.0.0.0 --port 8080
INFO: 127.0.0.1:56952 - "GET /api/scheduled-scans/scan-results HTTP/1.1" 200 OK
INFO: 127.0.0.1:56952 - "GET /api/scheduled-scans/scan-results HTTP/1.1" 200 OK
2026-04-14 11:07:38,574 - tool.nuclei - INFO - Nuclei completed with return code: 0
2026-04-14 11:07:38,575 - app.core.ws_updates - INFO - [Scan 198] Nuclei: Completed phase 3/3 (Info): Nuclei findings by severity: Critical=0, High=0, Medium=0, Low=0, Info=3.
2026-04-14 11:07:38,575 - app.core.ws_updates - INFO - [Scan 198] Tool completed: Nuclei (True)
2026-04-14 11:07:38,575 - app.tools.executor - INFO - Nuclei completed: success=True, findings=3
2026-04-14 11:07:38,575 - app.services.scan_service - INFO - Nuclei findings: 0 vulnerabilities, 3 information
2026-04-14 11:07:38,576 - app.services.scan_service - INFO - Tool Nuclei has 3 raw findings, 5 already exist in DB
2026-04-14 11:07:38,577 - app.services.scan_service - INFO - Added finding: Nuclei - information - info at pentest-ground.com:443
2026-04-14 11:07:38,577 - app.services.scan_service - INFO - Added finding: Nuclei - information - info at pentest-ground.com:443
2026-04-14 11:07:38,577 - app.services.scan_service - INFO - Added finding: Nuclei - information - info at pentest-ground.com:443
2026-04-14 11:07:38,582 - app.services.scan_service - INFO - Completed Nuclei: completed, added 3 new findings to DB
2026-04-14 11:07:38,585 - app.services.scan_service - INFO - Generated report for Nuclei: 3 findings
2026-04-14 11:07:38,585 - app.services.scan_service - INFO - Scan 198: Intelligence Layer - classifying endpoints
2026-04-14 11:07:38,585 - app.core.ws_updates - INFO - [Scan 198] System: Analyzing discovered endpoints and classifying by type/risk...
2026-04-14 11:07:38,585 - app.core.ws_updates - INFO - [Scan 198] Scan phase: intelligence layer
2026-04-14 11:07:38,595 - app.services.scan_service - INFO - Intelligence Layer: classified 0 endpoints for scan 198
2026-04-14 11:07:38,595 - app.services.scan_service - INFO - Scan 198: Active Testing Engine
2026-04-14 11:07:38,595 - app.core.ws_updates - INFO - [Scan 198] System: Running active testing for OWASP A02:2021...
2026-04-14 11:07:38,595 - app.core.ws_updates - INFO - [Scan 198] Scan phase: active testing
2026-04-14 11:07:38,598 - app.services.scan_service - INFO - Active Testing: completed 0 tests for scan 198
2026-04-14 11:07:38,598 - app.services.scan_service - INFO - Scan 198: Response Analysis
2026-04-14 11:07:38,598 - app.core.ws_updates - INFO - [Scan 198] System: Analyzing responses for vulnerability indicators...
2026-04-14 11:07:38,598 - app.core.ws_updates - INFO - [Scan 198] Scan phase: response analysis
2026-04-14 11:07:38,598 - app.services.scan_service - INFO - Response Analysis: found 0 vulnerabilities for scan 198
2026-04-14 11:07:38,599 - app.services.scan_service - INFO - Scan 198: Correlation
2026-04-14 11:07:38,599 - app.core.ws_updates - INFO - [Scan 198] System: Correlating findings across all tools...
2026-04-14 11:07:38,599 - app.core.ws_updates - INFO - [Scan 198] Scan phase: correlation
2026-04-14 11:07:38,601 - app.services.scan_service - INFO - Correlation: created 0 correlated findings for scan 198
2026-04-14 11:07:38,601 - app.services.scan_service - INFO - Scan 198: Smart Risk Scoring
2026-04-14 11:07:38,601 - app.core.ws_updates - INFO - [Scan 198] System: Applying smart risk scoring based on verified exploits...
2026-04-14 11:07:38,601 - app.core.ws_updates - INFO - [Scan 198] Scan phase: risk scoring
2026-04-14 11:07:38,603 - app.services.scan_service - INFO - Smart Risk Scoring: completed severity adjustments for scan 198
2026-04-14 11:07:38,603 - app.services.scan_service - INFO - Scan 198: Finalizing
2026-04-14 11:07:38,608 - app.services.scan_service - INFO - Scan 198 finalized with status: completed
INFO: 127.0.0.1:54468 - "GET /api/scheduled-scans HTTP/1.1" 200 OK
INFO: 127.0.0.1:54468 - "GET /api/scheduled-scans/scan-results HTTP/1.1" 200 OK
INFO: 127.0.0.1:54468 - "GET /api/scheduled-scans/scan-results HTTP/1.1" 200 OK
INFO: 127.0.0.1:54468 - "GET /api/scans/stats/severity HTTP/1.1" 200 OK
INFO: 127.0.0.1:54468 - "GET /api/scans/stats/trend?start_date=2026-04-08&end_date=2026-04-14 HTTP/1.1" 200 OK
INFO: 127.0.0.1:56952 - "GET /api/scheduled-scans HTTP/1.1" 200 OK
INFO: 127.0.0.1:56952 - "GET /api/scheduled-scans/scan-results HTTP/1.1" 200 OK
INFO: 127.0.0.1:56952 - "GET /api/scheduled-scans/scan-results HTTP/1.1" 200 OK
INFO: 127.0.0.1:56952 - "GET /api/scheduled-scans/scan-results HTTP/1.1" 200 OK
INFO: 127.0.0.1:56952 - "GET /api/scans/stats/severity HTTP/1.1" 200 OK
INFO: 127.0.0.1:56952 - "GET /api/scans/stats/trend?start_date=2026-04-08&end_date=2026-04-14 HTTP/1.1" 200 OK
2026-04-14 11:07:42,815 - app.services.email_alert_service - INFO - Scan-finished alert email sent to ['taksari@gmail.com']
INFO: 127.0.0.1:54468 - "GET /api/scheduled-scans HTTP/1.1" 200 OK
INFO: 127.0.0.1:54468 - "GET /api/scheduled-scans/scan-results HTTP/1.1" 200 OK
INFO: 127.0.0.1:54468 - "GET /api/scheduled-scans/scan-results HTTP/1.1" 200 OK
INFO: 127.0.0.1:56952 - "GET /api/scheduled-scans HTTP/1.1" 200 OK
INFO: 127.0.0.1:56952 - "GET /api/scheduled-scans/scan-results HTTP/1.1" 200 OK
INFO: 127.0.0.1:56952 - "GET /api/scheduled-scans/scan-results HTTP/1.1" 200 OK
INFO: 127.0.0.1:33802 - "GET /api/scheduled-scans HTTP/1.1" 200 OK
INFO: 127.0.0.1:33802 - "GET /api/scheduled-scans/scan-results HTTP/1.1" 200 OK
INFO: 127.0.0.1:33802 - "GET /api/scheduled-scans/scan-results HTTP/1.1" 200 OK
INFO: 127.0.0.1:56952 - "GET /api/scheduled-scans HTTP/1.1" 200 OK
INFO: 127.0.0.1:56952 - "GET /api/scheduled-scans/scan-results HTTP/1.1" 200 OK
INFO: 127.0.0.1:56952 - "GET /api/scheduled-scans/scan-results HTTP/1.1" 200 OK
INFO: 127.0.0.1:56952 - "GET /api/scheduled-scans HTTP/1.1" 200 OK

```

Figure 164 Final phase of scan

```

host 0.0.0.0 --port 8080
2026-04-14 11:07:38,599 - app.core.ws_updates - INFO - [Scan 198] System: Correlating findings across all tools...
2026-04-14 11:07:38,599 - app.core.ws_updates - INFO - [Scan 198] Scan phase: correlation
2026-04-14 11:07:38,601 - app.services.scan_service - INFO - Correlation: created 0 correlated findings for scan 198
2026-04-14 11:07:38,601 - app.services.scan_service - INFO - Scan 198: Smart Risk Scoring
2026-04-14 11:07:38,601 - app.core.ws_updates - INFO - [Scan 198] System: Applying smart risk scoring based on verified exploits...
2026-04-14 11:07:38,601 - app.core.ws_updates - INFO - [Scan 198] Scan phase: risk scoring
2026-04-14 11:07:38,603 - app.services.scan_service - INFO - Smart Risk Scoring: completed severity adjustments for scan 198
2026-04-14 11:07:38,603 - app.services.scan_service - INFO - Scan 198: Finalizing
2026-04-14 11:07:38,608 - app.services.scan_service - INFO - Scan 198 finalized with status: completed
INFO: 127.0.0.1:54468 - "GET /api/scheduled-scans HTTP/1.1" 200 OK
INFO: 127.0.0.1:54468 - "GET /api/scheduled-scans/scan-results HTTP/1.1" 200 OK
INFO: 127.0.0.1:54468 - "GET /api/scheduled-scans/scan-results HTTP/1.1" 200 OK
INFO: 127.0.0.1:54468 - "GET /api/scans/stats/severity HTTP/1.1" 200 OK
INFO: 127.0.0.1:54468 - "GET /api/scans/stats/trend?start_date=2026-04-08&end_date=2026-04-14 HTTP/1.1" 200 OK
INFO: 127.0.0.1:56952 - "GET /api/scheduled-scans HTTP/1.1" 200 OK
INFO: 127.0.0.1:56952 - "GET /api/scheduled-scans/scan-results HTTP/1.1" 200 OK
INFO: 127.0.0.1:56952 - "GET /api/scheduled-scans/scan-results HTTP/1.1" 200 OK
INFO: 127.0.0.1:56952 - "GET /api/scheduled-scans/scan-results HTTP/1.1" 200 OK
INFO: 127.0.0.1:56952 - "GET /api/scans/stats/severity HTTP/1.1" 200 OK
INFO: 127.0.0.1:56952 - "GET /api/scans/stats/trend?start_date=2026-04-08&end_date=2026-04-14 HTTP/1.1" 200 OK
2026-04-14 11:07:42,815 - app.services.email_alert_service - INFO - Scan-finished alert email sent to ['taksari@gmail.com']
INFO: 127.0.0.1:54468 - "GET /api/scheduled-scans HTTP/1.1" 200 OK
INFO: 127.0.0.1:54468 - "GET /api/scheduled-scans/scan-results HTTP/1.1" 200 OK
INFO: 127.0.0.1:54468 - "GET /api/scheduled-scans/scan-results HTTP/1.1" 200 OK
INFO: 127.0.0.1:56952 - "GET /api/scheduled-scans HTTP/1.1" 200 OK
INFO: 127.0.0.1:56952 - "GET /api/scheduled-scans/scan-results HTTP/1.1" 200 OK
INFO: 127.0.0.1:56952 - "GET /api/scheduled-scans/scan-results HTTP/1.1" 200 OK
INFO: 127.0.0.1:33802 - "GET /api/scheduled-scans HTTP/1.1" 200 OK
INFO: 127.0.0.1:33802 - "GET /api/scheduled-scans/scan-results HTTP/1.1" 200 OK
INFO: 127.0.0.1:33802 - "GET /api/scheduled-scans/scan-results HTTP/1.1" 200 OK
INFO: 127.0.0.1:56952 - "GET /api/scheduled-scans HTTP/1.1" 200 OK
INFO: 127.0.0.1:56952 - "GET /api/scheduled-scans/scan-results HTTP/1.1" 200 OK
INFO: 127.0.0.1:56952 - "GET /api/scheduled-scans/scan-results HTTP/1.1" 200 OK
INFO: 127.0.0.1:56952 - "GET /api/scheduled-scans HTTP/1.1" 200 OK

```

Figure 165 Scanning Completed

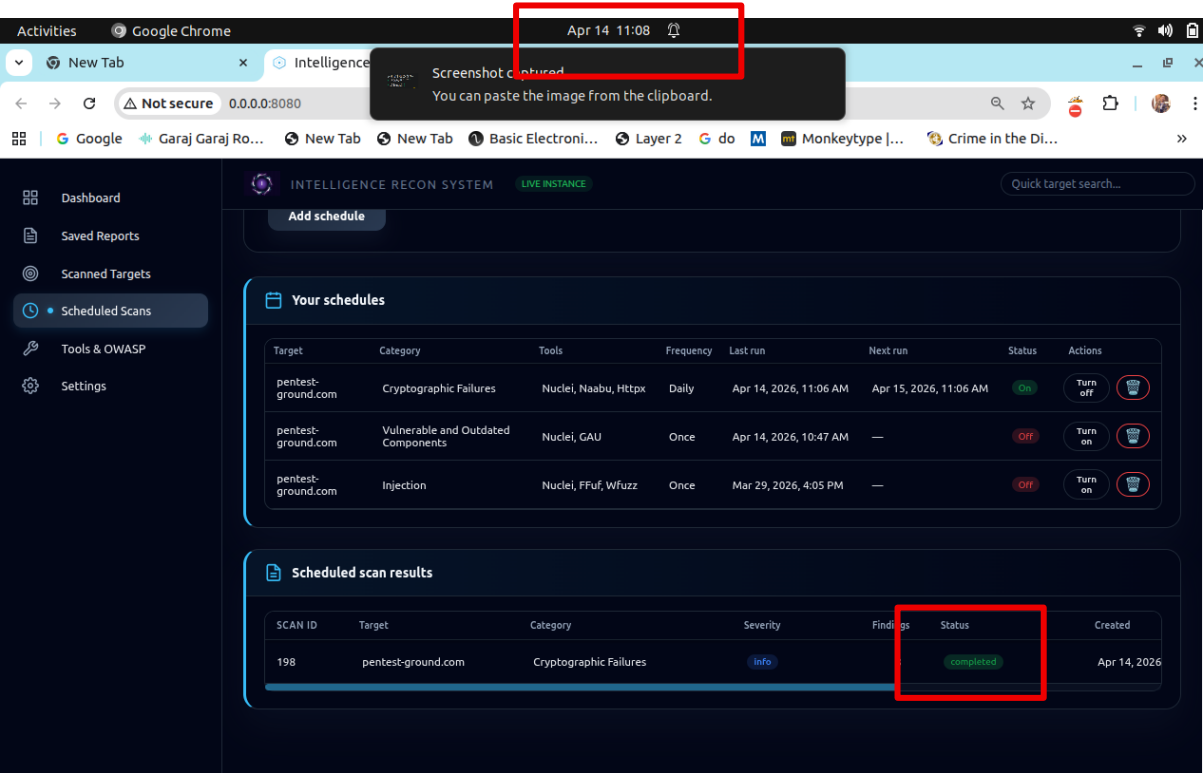


Figure 166 Scan Completed within 4 minutes

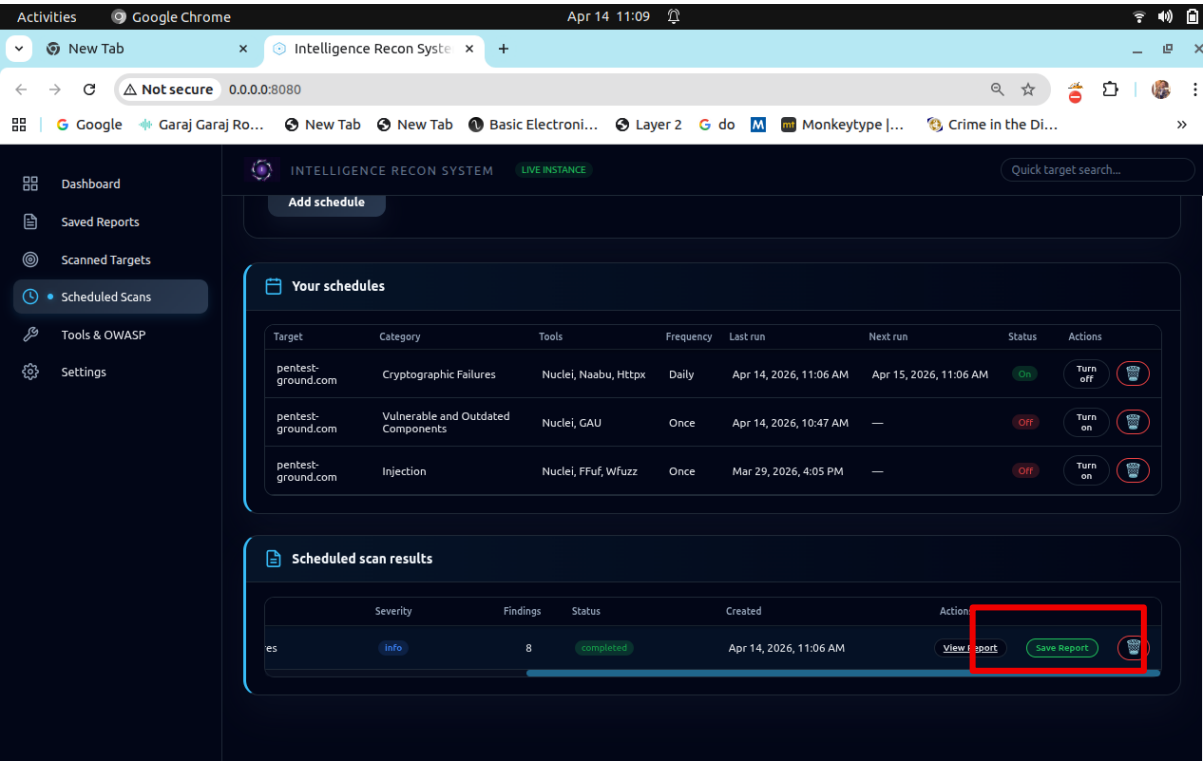


Figure 167 Saved Report

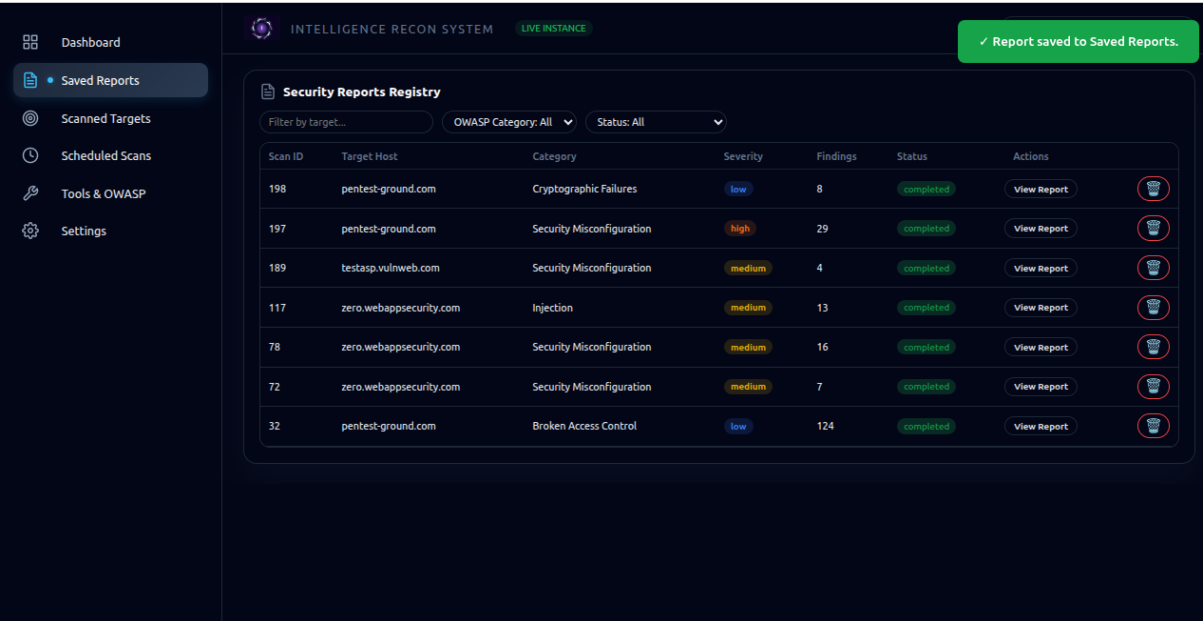


Figure 168 Scan Saved on "Saved Reports"

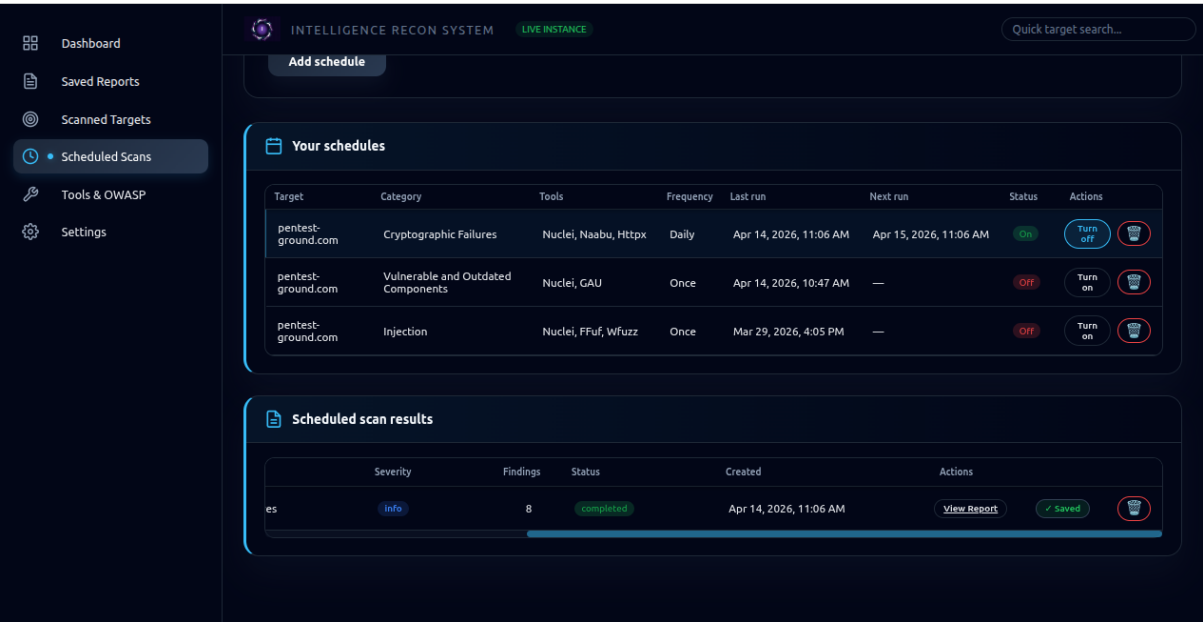


Figure 169 Turn off scan

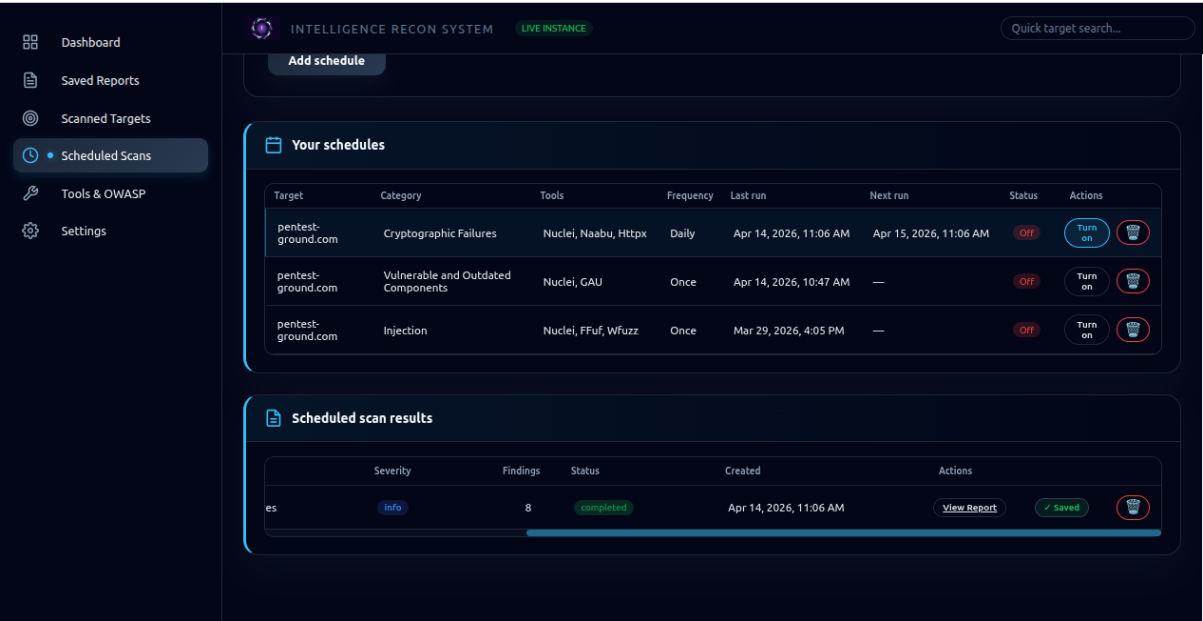


Figure 170 Turned off Successfully

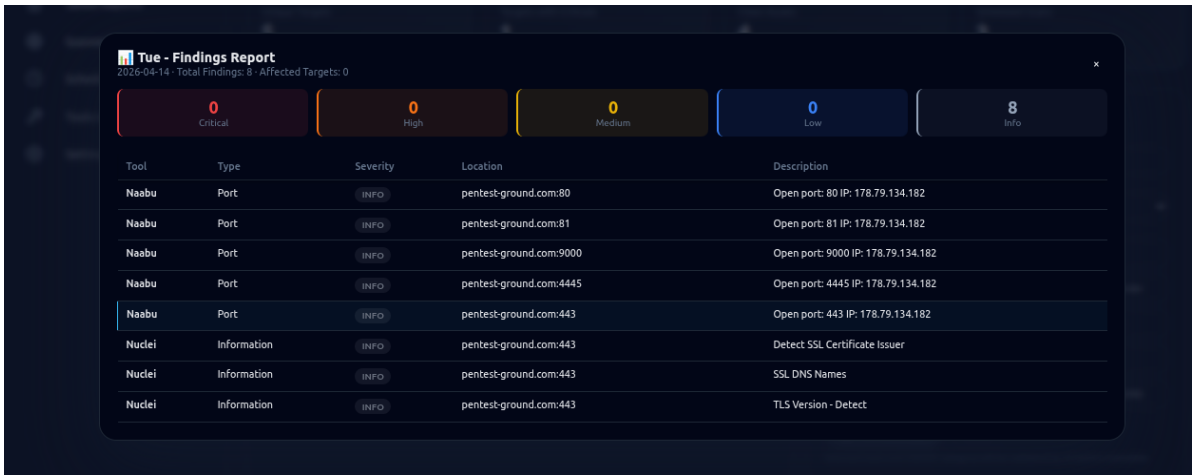


Figure 171 Findings shown on "Security Trend Analysis"

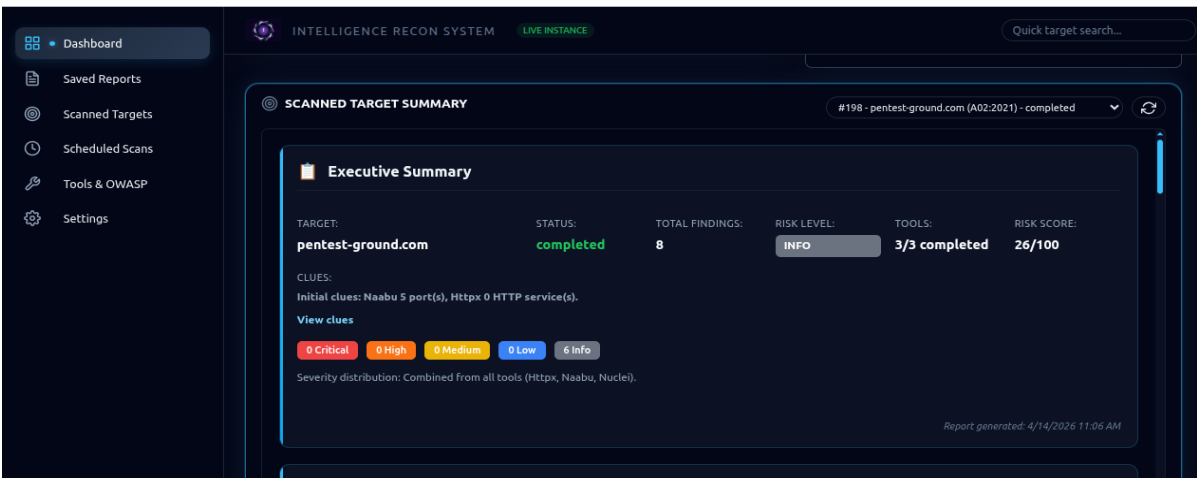


Figure 172 Scan Summary

4.4. CRITICAL ANALYSIS

The fifteen-unit test cases and the two system test cases were successfully completed, which proves that the IRS works as planned, within its core functional areas. The unit tests involved a wide variety of features such as backend startup, input checking, scan running, dashboard card response, email notification, report generation, scheduled scan setups, live status monitoring, and tool-level CLI output. Test Case 3 was especially useful as it confirmed three different input conditions without any tool selection, invalid target and valid target that the system effectively avoids incomplete or malformed scan requests reaching the execution layer. Test Case 5 also showed good data integrity by ensuring that the deletion of a scan correctly updates the Total Scans and Total Findings counters on the dashboard.

Some problems were detected and solved during testing. In Test Case 3, the validation message was not always displayed when no tools were selected, and this was determined and fixed until the final test run, verifying that the validation logic is reliable when correctly configured. Both system test cases ran without problems, and the end-to-end scan workflow and scheduled scan process generated the expected output such as the selection of tools with the help of AI, parallel tool execution, PDF report creation, and saving a scan within the expected time.

In general, the testing process indicated that the IRS platform is both stable and operational in the scope of the project as defined. The combination of unit and system testing provided reasonable coverage of the key workflows, and the issues encountered during testing were handled appropriately without affecting the final outcomes. The fact that all test cases have been successfully completed proves that the system has fulfilled its functional requirements and is now prepared to be developed and enhanced in the future in line with the project objectives.

CHAPTER 5: CONCLUSION

Security reconnaissance is traditionally inefficient because analysts must use multiple disconnected tools and manually combine the results, which delays the assessment process. IRS was constructed to transform that. At the time of development, the platform provided a complete and functional, AI-assisted workflow of reconnaissance that guides a user through target input, to a downloadable and structured security report in just one web session. Eight project milestones were achieved on time and six Agile Scrum sprints; all test cases were successfully completed and post-survey feedback revealed that the system was operating at or above professional expectation to its target audience. In addition to achieving its goals, IRS shows that AI-assisted tool orchestration is a potential and feasible solution to serious security assessment that can be accessible to teams and individuals that would otherwise not have the resources to conduct these assessments.

5.1. LEGAL, SOCIAL AND ETHICAL ISSUES

5.1.1. Legal Issues

The core legal requirements of IRS are fulfilled by the target validation, GDPR-compliant deletion of scans and OWASP scope enforcement. There is no Terms of Service or Acceptable Use Policy shown prior to scanning, and authorization has not been verified. The app is not secure because it uses HTTP and not HTTPS, which contravenes the GDPR and ISO/IEC 27001 requirements on transport security. The current system does not have any data-export workflow, breach-notification, or per-user data isolation. All these gaps are indicators of the difference between this prototype and a production release that is legally compliant. The next phase legal roadmap is comprised of HTTPS enforcement, an AUP gate, and a structured process of data-access.

5.1.2. Social Issues

IRS eliminates accessibility barrier to professional reconnaissance among students and small teams that do not have commercial tooling. In pre-survey findings, respondents expressed unanimous agreement on the value of automation; 61.6 percent of those who participated in the post-survey would adopt IRS professionally. The scheduling engine can provide 24/7 security coverage without the need to have a constant presence of the analyst. The leading social risk is dual-use misuse - there is no authorization declaration that can be made to prevent scanning of targets that the

user does not own. This is an omission in enforcing and can be addressed in a single development sprint. It is, however, the social responsibility that the following cycle must take care of first.

5.1.3. Ethical Issues

IRS gathers no information other than what is needed to scan operations and offers a permanent deletion option via the delete-scan feature. The live status tracker provides the user with full access to every execution step; the AI tool-selection step included. A fixed-tool fallback provides predictable behavior of the system if Gemini is unavailable. The AI Decision Node fails to make its reasoning visible to users, which lacks the explainable AI promise in the supporting report. The policies on data-use and non-monetization are only with intent and not on application, and neither is their disclosure nor privacy notice implemented. These gaps would be closed with an AUP step, HTTPS and an AI reasoning summary and prepare IRS to be professionally deployed.

5.2. ADVANTAGES

IRS is a viable and effective platform to conduct security assessments because it provides several significant benefits compared to the traditional manual reconnaissance workflows. The strengths of the built and tested system are as follows.

1. **Single Web Interface:** IRS integrates fifteen open-source CLI tools into one web-based interface, thus removing the necessity to operate multiple terminals, commands, and displays with multiple processes.
2. **AI-Assisted Tool Selection:** The AI Decision Node eliminates human bias by suggesting all contextually relevant tools, according to live reconnaissance clues and the chosen OWASP category, minimizing noise and unnecessary checks.
3. **Asynchronous and Isolated Execution:** Tools are executed in parallel and are isolated at the per-tool level, that is, failure in any one tool does not halt the larger scan and the total assessment time is considerably smaller.
4. **Persistent History:** All scan results, findings and reports are persistently stored, and teams can track trends in vulnerabilities and re-examine past assessments across engagements.
5. **Automated Scheduling:** The engine can be used to schedule daily, weekly and monthly scans which do not need analysis to always be present.

- 6. One-Click Reporting:** Professional HTML and PDF reports are created immediately, and the findings could be shared with the stakeholders immediately without any additional formatting.

5.3. LIMITATIONS

Although IRS has strengths, it has several limitations that are indicative of the fact that it is still a prototype, but not a production-ready system. These limitations reflect sincere limitations of the present-day implementation and lay the foundation of future development.

- 1. No User Authentication:** There is no user-authentication system or role-based access control, so any scanned data can be accessed by anyone who can access the system and there is no user-isolation.
- 2. None Transport Encryption:** By default, the application does not use HTTPS but uses HTTP, which means that all data during transit is not encrypted and, therefore, do not meet the requirements of GDPR and ISO/IEC 27001.
- 3. No Authorization Gate:** There is no Terms of Service or authorization statement that must be made prior to a scan being created and this presents a dual-use threat that the system cannot technically circumvent.
- 4. AI API Dependency:** The quality of tool selection will be a fixed fallback set when the Gemini API is not available or rate-limited, and the scan workflow is less intelligent.
- 5. Single-Instance Only:** Scalability of the system with multi-user concurrent workloads has not been tested and therefore the system is not suitable for use with a team or organization.

5.4. FUTURE WORK

There are still jobs that need to be improved and added. The following points are the plans of IRS for further work:

1. A secure Multi-User Authentication system can be adopted to provide privacy to data and its own scan control.
2. Switch to WebSocket or Server-Sent Events to support real-time, low-latency progress updates.

3. Multi-users are also available, allowing different user logins with independent data and role-based access control.
4. Auto remediation recommendation offers solutions to identify vulnerabilities to assist users to provide solutions at a shorter time.

For more Detail Understanding View [Appendix-10](#)

CHAPTER 6: REFERECES

Aguiar, A., 2024. *Security Boulevard*. [Online]

Available at: <https://securityboulevard.com/2024/01/forging-ahead-2023-recap-2024-cybersecurity-roadmap/>

[Accessed 13 April 2026].

Alexander Kolesnikov, V. M., 2024. *Secure List*. [Online]

Available at: <https://securelist.com/vulnerability-report-q1-2024/112554/>

[Accessed 29 December 2025].

Ariganello, J., 2022. *Anomali*. [Online]

Available at: <https://www.anomali.com/blog/more-is-less-the-challenge-of-utilizing-multiple-security-tools>

[Accessed 23 November 2025].

Cobrief, 2025. *Cobrief*. [Online]

Available at: <https://www.cobrief.app/resources/legal-glossary/methodology-overview-definition-and-example/>

[Accessed 30 November 2025].

Day, B., 2025. *Forbes*. [Online]

Available at: <https://www.forbes.com/advisor/business/what-is-waterfall-methodology/#what-is-the-waterfall-methodology->

[Accessed 27 November 2025].

Donetonic, 2025. *Donetonic*. [Online]

Available at: <https://donetonic.com/steps-to-configure-your-scrum-workflow/>

[Accessed 20 December 2025].

Doshi, H., 2016. *Scrum.org*. [Online]

Available at: <https://www.scrum.org/resources/blog/three-pillars-empiricism-scrum>

[Accessed 30 Noember 2025].

draw.io, 2023. *Draw.io*. [Online]

Available at: <https://www.drawio.com/blog/types-of-technical-diagrams>

[Accessed 3 December 2025].

Eastgate, 2025. *Eastgate*. [Online]

Available at: <https://eastgate-software.com/7-stages-of-software-development-life-cycle-sdlc-you-need-to-know/>

[Accessed 26 November 2025].

Fouz Barman, N. A. H. A. M. A. a. R. I., 2023. *A Methodical Framework for Conducting Reconnaissance and Enumeration in the Ethical Hacking Lifecycle*. Vol. 22, No. 1 ed. Athens, Greece : Academic Conferences International Limited, Reading, United Kingdom .

geeksforgeeks.org, 2025. *geeksforgeeks.org*. [Online]

Available at: <https://www.geeksforgeeks.org/system-design/use-case-diagram/>

[Accessed 29 December 2025].

Geeksforgeeks, 2025. *Geeksforgeeks*. [Online]

Available at: <https://www.geeksforgeeks.org/software-testing/system-testing/>

[Accessed 6 April 2026].

GeeksforGeeks, 2025. *GeeksforGeeks*. [Online]

Available at: <https://www.geeksforgeeks.org/software-testing/unit-testing-software-testing/>

[Accessed 26 03 2026].

GeeksForGeeks, 2025. *GeeksForGeeks*. [Online]

Available at: <https://www.geeksforgeeks.org/software-engineering/software-development-life-cycle-sdlc/>

[Accessed 26 November 2025].

GeeksForGeeks, 2025. *GeeksForGeeks*. [Online]

Available at: <https://www.geeksforgeeks.org/software-engineering/software-engineering-sdlc-v-model/>

[Accessed 27 November 2025].

Guilizzoni, P., 2025. *Balsamiq*. [Online]

Available at: <https://balsamiq.com/blog/what-are-wireframes/>

[Accessed 13 April 2026].

InterviewBit, 2023. *InterviewBit*. [Online]

Available at: <https://www.interviewbit.com/blog/system-architecture/>

[Accessed 24 December 2025].

Kamis, 2019. *materimateripti*. [Online]

Available at: <https://materimateripti.blogspot.com/2019/11/rekayasa-perangkat-lunak-rekayasa.html>

[Accessed 27 November 2025].

LaPrad, L., 2025. *TeamGantt*. [Online]

Available at: <https://www.teamgantt.com/what-is-a-gantt-chart>

[Accessed 25 December 2025].

Manager, P., 2025. *Project Manager*. [Online]

Available at: <https://www.projectmanager.com/guides/work-breakdown-structure>

[Accessed 29 November 2025].

Ojha, Y., 2025. *Github*. [Online]

Available at: <https://github.com/yogeshojha/engine>

[Accessed 17 December 2025].

Peek, S., 2024. *Business News Daily*. [Online]

Available at: <https://www.businessnewsdaily.com/4987-what-is-agile-scrum-methodology.html>

[Accessed 27 November 2025].

Sasovets, I., 2025. *Techmagic*. [Online]

Available at: <http://techmagic.co/blog/automated-security-testing>

[Accessed 26 November 2025].

Simpson, C., 2023. *ARMIS*. [Online]

Available at: <https://www.armis.com/blog/new-research-shows-global-organizations-fight-increasing-attack-surface-with-fragmented-tools-scattered-threat-intelligence-and-manual-spreadsheets/>

[Accessed 13 April 2026].

Simran, S., 2025. *bdtask*. [Online]

Available at: <https://www.bdtask.com/blog/software-development-life-cycle>

[Accessed 27 November 2025].

SOCRadar, 2025. *SOCRadar*. [Online]

Available at: <https://socradar.io/labs>

[Accessed 17 December 2025].

Staff, C., 2025. *Coursera*. [Online]

Available at: <https://www.coursera.org/articles/what-is-milestone?msockid=2b281f38e3ab640a32060985e2306557>

[Accessed 25 December 2025].

Sumanth, G., 2023. *Appsecengineer*. [Online]

Available at: <https://www.appsecengineer.com/blog/manual-vs-automated-reconnaissance>

[Accessed 13 April 2026].

Sumanth, G., 2023. *APPSECEngineer*. [Online]

Available at: <https://www.appsecengineer.com/blog/manual-vs-automated-reconnaissance>

[Accessed 23 November 2025].

Team, P., 2025. *Pentest Tools*. [Online]

Available at: [Team, Pentest](#)

[Accessed 17 December 2025].

Vasiliauskas, V., 2025. *teamhood*. [Online]

Available at: <https://teamhood.com/agile/scrum-advantages-disadvantages/>

[Accessed 27 December 2025].

Welch, J., 2025. *Profit.co*. [Online]

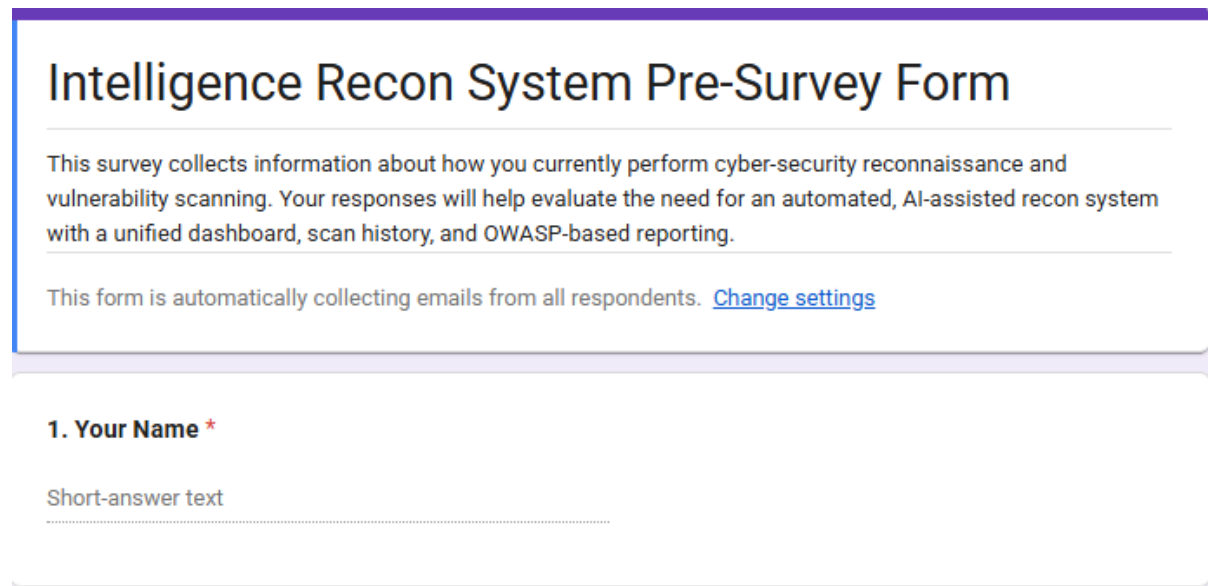
Available at: <https://www.profit.co/blog/task-management/the-5-phases-of-waterfall-project-management/>

[Accessed 27 November 2025].

CHAPTER 7: APPENDIX

7.1. APPENDIX 1: PRE-SURVEY

7.1.1. Pre-Survey Questions



Intelligence Recon System Pre-Survey Form

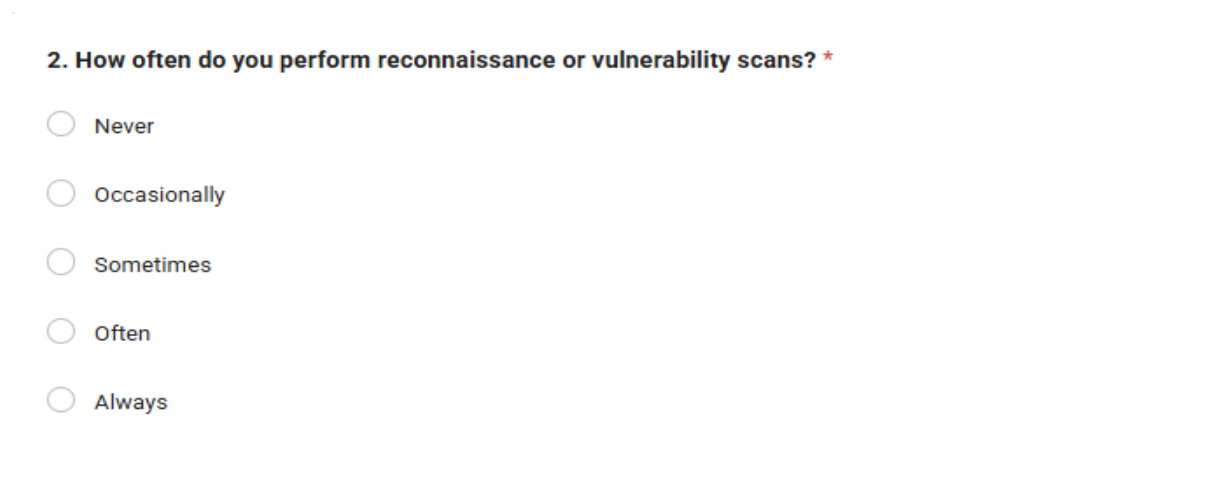
This survey collects information about how you currently perform cyber-security reconnaissance and vulnerability scanning. Your responses will help evaluate the need for an automated, AI-assisted recon system with a unified dashboard, scan history, and OWASP-based reporting.

This form is automatically collecting emails from all respondents. [Change settings](#)

1. Your Name *

Short-answer text

Figure 173 Pre-Survey Form: Personal Detail



2. How often do you perform reconnaissance or vulnerability scans? *

☐ Never

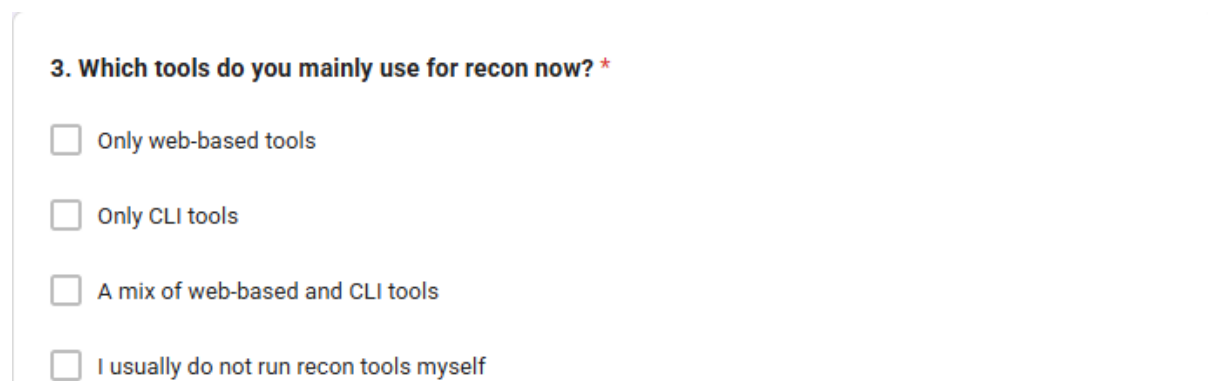
☐ Occasionally

☐ Sometimes

☐ Often

☐ Always

Figure 174 Pre-Survey Form: Question 1



3. Which tools do you mainly use for recon now? *

☐ Only web-based tools

☐ Only CLI tools

☐ A mix of web-based and CLI tools

☐ I usually do not run recon tools myself

Figure 175 Pre-Survey Form: Question 2

⋮

4. How do you usually run multiple tools for a single target? *

☐ I do not run multiple tools

☐ I run them one by one manually in terminal

☐ I use my own scripts to automate them

☐ I use an existing platform that chains tools

Figure 176 Pre-Survey Form: Question 3

5. What is the most difficult part of recon for you? *

☐ Remembering and using different CLI commands

☐ Understanding raw outputs from tools

☐ Combining results from many tools into one view

☐ Creating professional reports from scan results

Figure 177 Pre-Survey Form: Question 4

6. How useful would a single web dashboard that runs multiple recon tools for you be? *

☐ Not useful at all

☐ Slightly useful

☐ Moderately useful

☐ Very useful

☐ Extremely useful

Figure 178 Pre-Survey Form: Question 5

7. How interested are you in AI-based tool selection (AI decides which tools to run based on target and goal)? *

- ☐ Not interested
- ☐ Slightly interested
- ☐ Neutral
- ☐ Interested
- ☐ Very interested

Figure 179 Pre-Survey Form: Question 6

8. If AI sometimes selects wrong or unnecessary tools, what would you prefer? *

- ☐ Always let AI decide, even if sometimes wrong
- ☐ Let AI suggest but allow me to edit the list
- ☐ I want full manual control, no AI decisions
- ☐ Not sure

Figure 180 Pre-Survey Form: Question 7

9. How important is it to store all scan history and results in one centralized system? *

- ☐ Not important
- ☐ Slightly important
- ☐ Neutral
- ☐ Important
- ☐ Very important

Figure 181 Pre-Survey Form: Question 8

10. How useful would auto-generated OWASP-based vulnerability reports (HTML/PDF) be for you? *

- ☐ Not useful
- ☐ Slightly useful
- ☐ Neutral
- ☐ Useful
- ☐ Very useful

Figure 182 Pre-Survey Form: Question 9

11. For your own use, what format of report do you prefer? *

- ☐ Simple text summary
- ☐ Detailed technical report with raw findings
- ☐ Both summary and technical details
- ☐ I do not need reports

Figure 183 Pre-Survey Form: Question 10

12. If a system like IRS existed, how likely would you be to use it for your recon tasks? *

- ☐ Very unlikely
- ☐ Unlikely
- ☐ Not sure
- ☐ Likely
- ☐ Very likely

Figure 184 Pre-Survey Form: Question 11

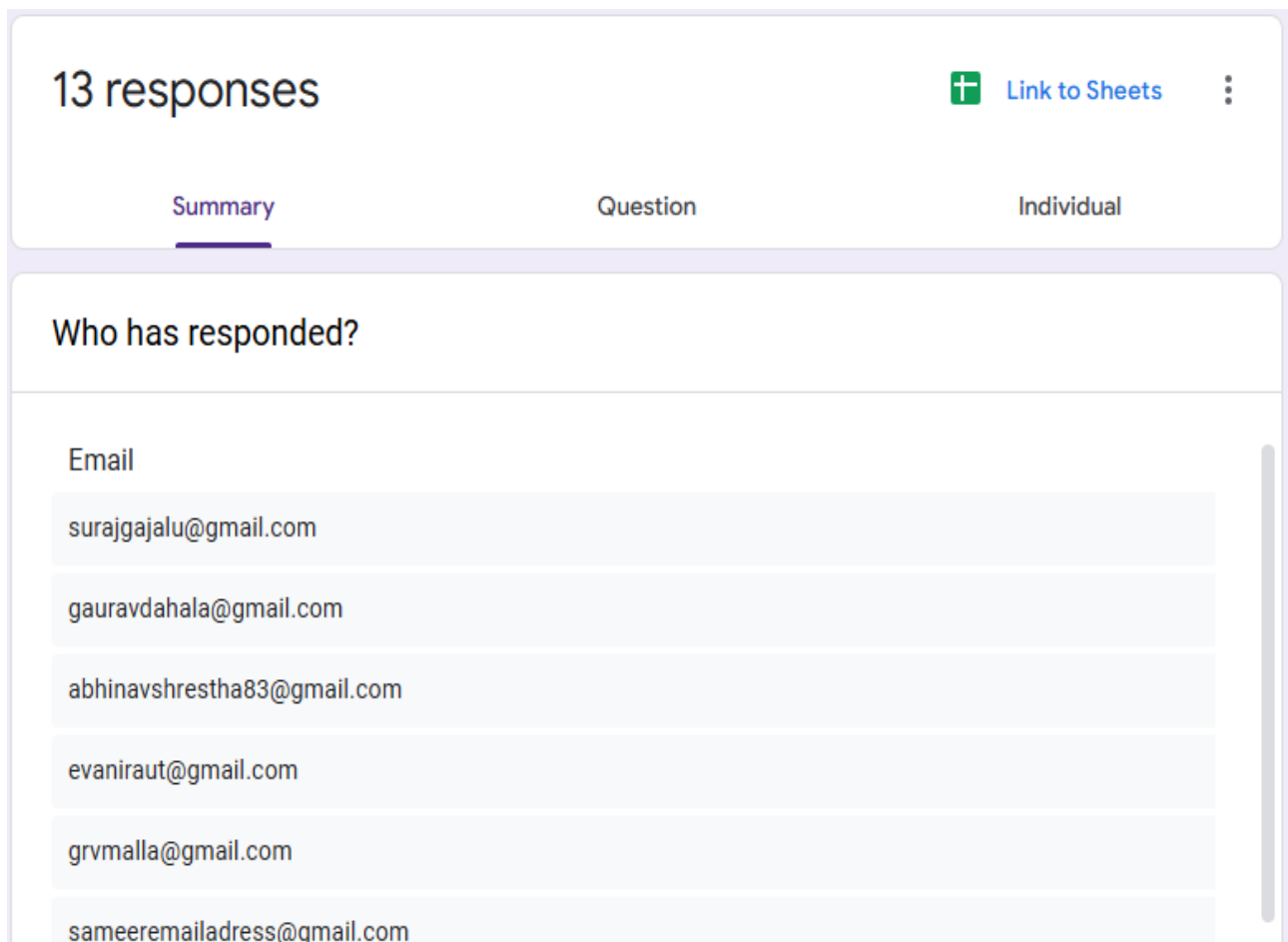
13. Any other features you would like in an automated recon system? *

Long-answer text

Figure 185 Pre-Survey Form: Question 12

7.1.2. Findings of the pre-survey

The IRS pre-survey involved 13 people who do actively work in the field of reconnaissance or vulnerability scanning, and they use both CLI and web-based tools. Most of the people interviewed run one or two tools at a time in terminal and said that the most challenging step about recon is getting the results of many tools in one display, then remembering various commands and interpreting raw data. Most of them replied that a single web dashboard, AI-assisted tool selection, centralized scan history, and auto-generated OWASP-based reports will be useful or especially useful, which points to high demand levels on the basic features, planned in IRS.



The screenshot displays a Google Forms interface for a survey titled "13 responses". At the top right, there is a green plus icon, a "Link to Sheets" button, and a vertical ellipsis menu. Below the title, there are three tabs: "Summary" (which is selected and underlined), "Question", and "Individual". The main content area is titled "Who has responded?". Below this title, there is a list of email addresses, each in its own light gray box. The email addresses are: surajgajalu@gmail.com, gauravdahala@gmail.com, abhinavshrestha83@gmail.com, evaniraut@gmail.com, grvmalla@gmail.com, and sameeremailaddress@gmail.com. A vertical scrollbar is visible on the right side of the email list.

Email
surajgajalu@gmail.com
gauravdahala@gmail.com
abhinavshrestha83@gmail.com
evaniraut@gmail.com
grvmalla@gmail.com
sameeremailaddress@gmail.com

Figure 186 Pre-Survey Form: Response Email ID

Who has responded?
grvmalla@gmail.com
sameeremailadress@gmail.com
shrevikask@gmail.com
timalsinabachan@gmail.com
prashna.malla77@gmail.com
satvishapanta2928@gmail.com
divyasitaula@gmail.com
sthprajina@gmail.com
ishanmakaju10@gmail.com

Figure 187 Pre-Survey Form: Response Emails ID

1. Your Name
13 responses
Prashant Rana
Evani Raut
Dibya Sitaula
Prashna malla
Prajina Taksari
Ishan Makaju

Figure 188 Pre-Survey Form: Response Identity

1. Your Name

13 responses

Prajina Taksari
Ishan Makaju
bachan
Sameer sah
Shrevika Khadka
Satvisha
Abhinav shrestha
Gaurav Dahal
Gaurav Malla

Figure 189 Pre-Survey Form: Response Identity

2. How often do you perform reconnaissance or vulnerability scans?

13 responses

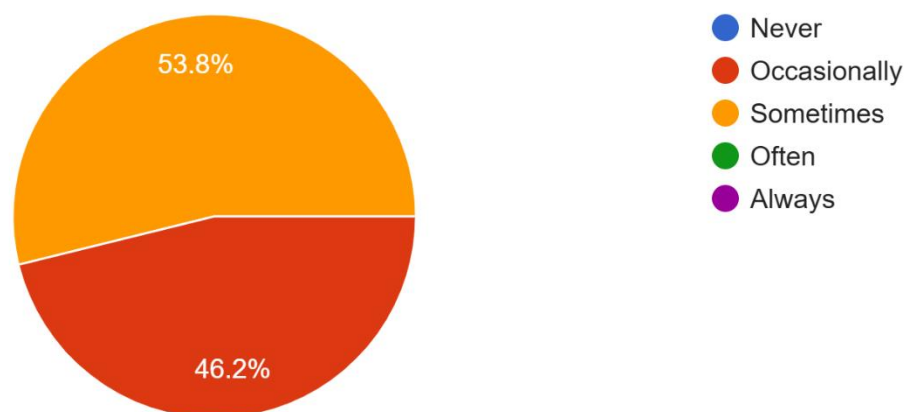


Figure 190 Pre-Survey Form Response: Question 1

3. Which tools do you mainly use for recon now?

13 responses

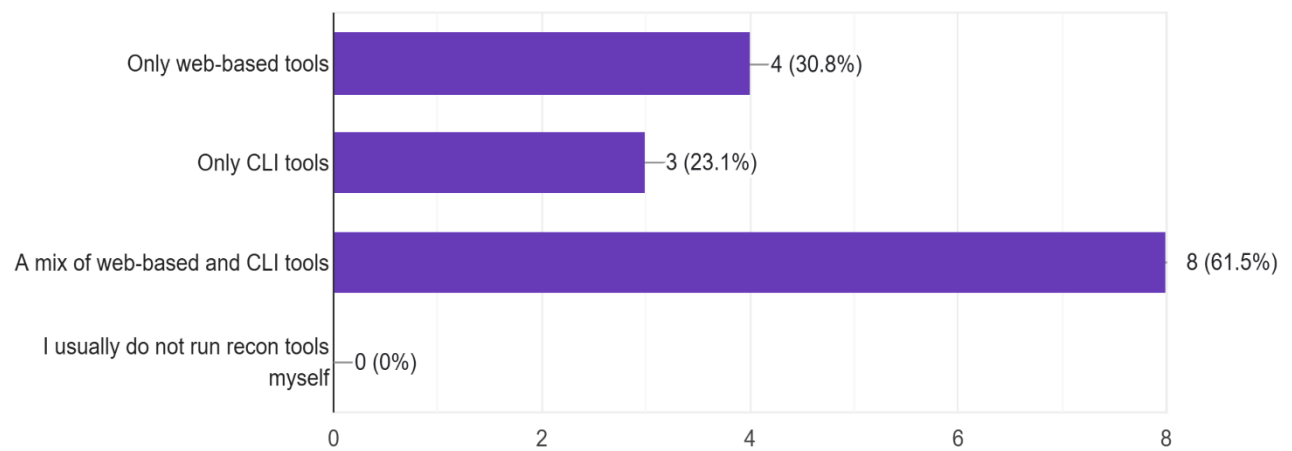


Figure 191 Pre-Survey Form Response: Question 2

4. How do you usually run multiple tools for a single target?

13 responses

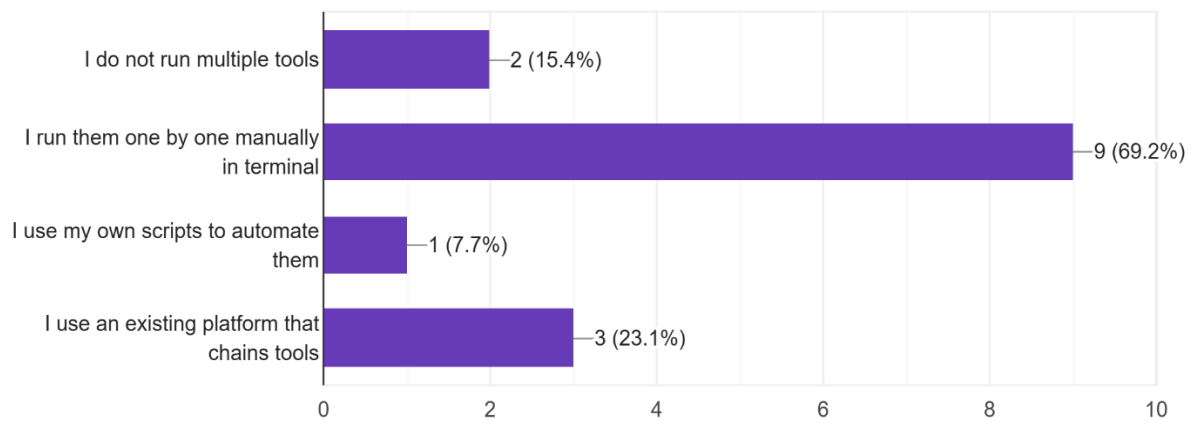


Figure 192 Pre-Survey Form Response: Question 3

5. What is the most difficult part of recon for you?

13 responses

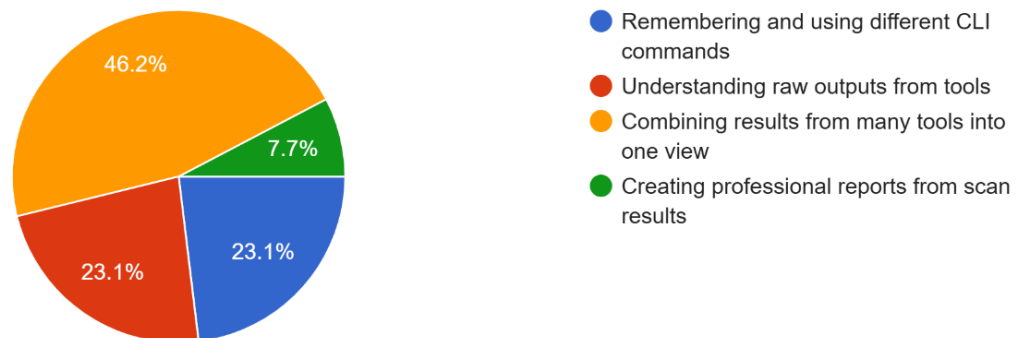


Figure 193 Pre-Survey Form Response: Question 4

6. How useful would a single web dashboard that runs multiple recon tools for you be?

13 responses

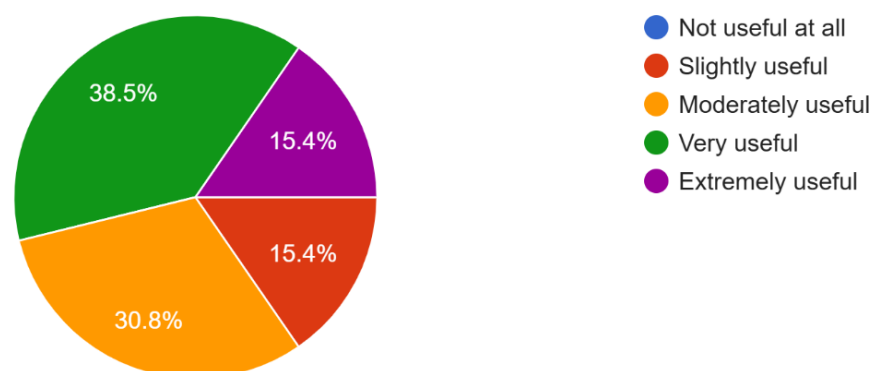


Figure 194 Pre-Survey Form Response: Question 5

7. How interested are you in AI-based tool selection (AI decides which tools to run based on target and goal)?

13 responses

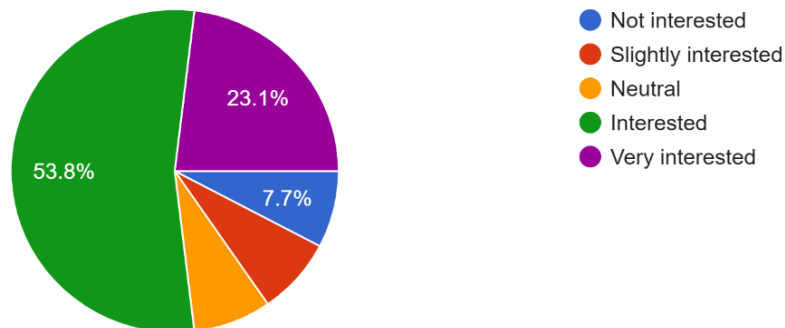


Figure 195 Pre-Survey Form Response: Question 6

8. If AI sometimes selects wrong or unnecessary tools, what would you prefer?

13 responses



Figure 196 Pre-Survey Form Response: Question 7

9. How important is it to store all scan history and results in one centralized system?

13 responses

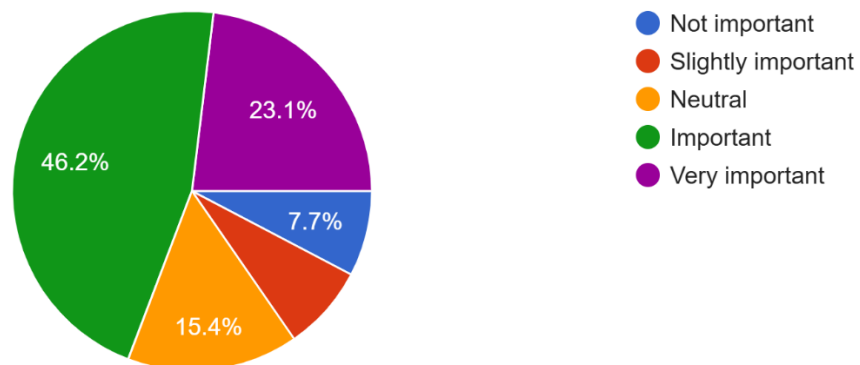


Figure 197 Pre-Survey Form Response: Question 8

10. How useful would auto-generated OWASP-based vulnerability reports (HTML/PDF) be for you?

13 responses

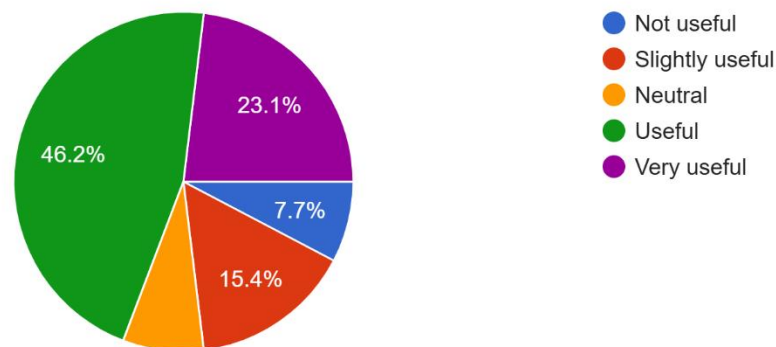


Figure 198 Pre-Survey Form Response: Question 9

11. For your own use, what format of report do you prefer?

13 responses

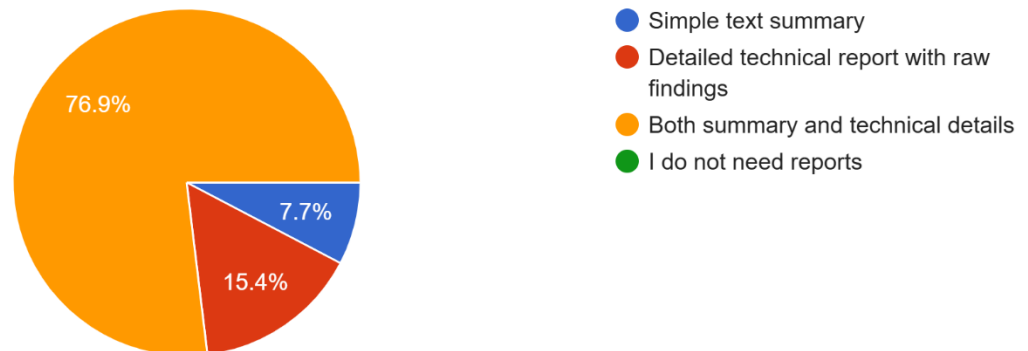


Figure 199 Pre-Survey Form Response: Question 10

12. If a system like IRS existed, how likely would you be to use it for your recon tasks?

13 responses

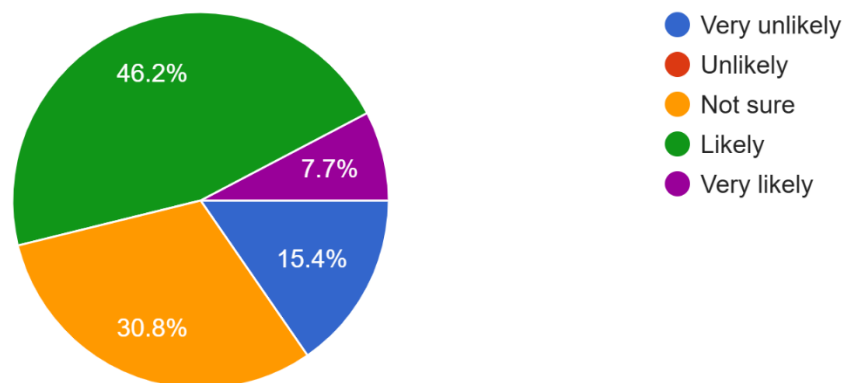


Figure 200 Pre-Survey Form Response: Question 11

13. Any other features you would like in an automated recon system?

11 responses

User friendly

No

Automatic mismatch detection

One click export

Vulnerability probing

NOT NECESSARY

Sub domain finder and OSINT

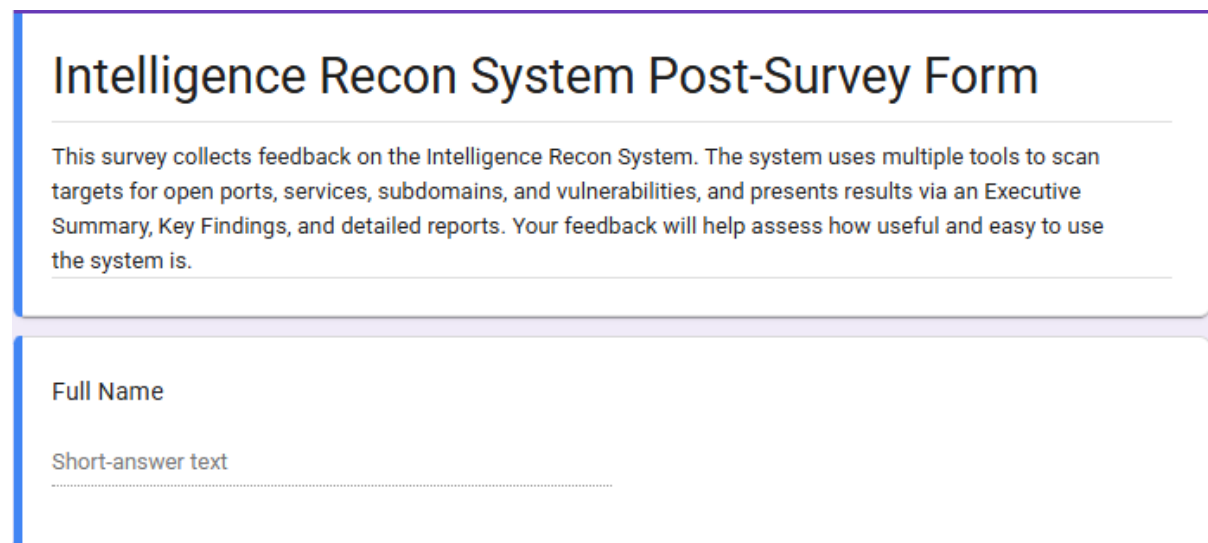
Smart Scan Recommendation Engine

Easy connection with common tools, live scan progress, clear risk levels, simple download options and easy to understand explanations.

Figure 201 Pre-Survey Form Response: Question 12[Click here](#) to go back.

7.2. APPENDIX: 2: POST SURVEY

7.2.1. Post-Survey Questions



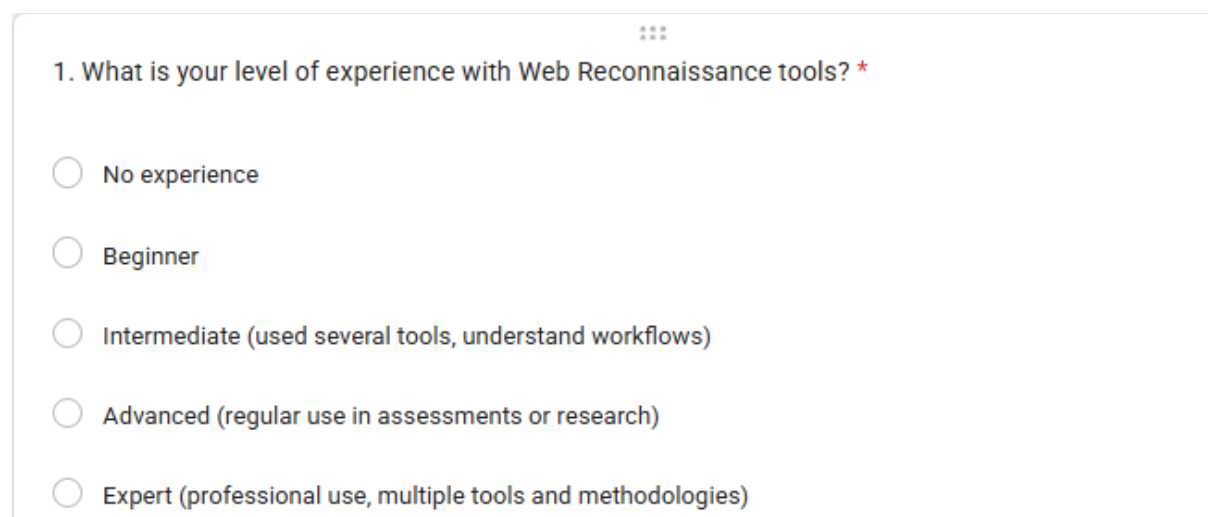
Intelligence Recon System Post-Survey Form

This survey collects feedback on the Intelligence Recon System. The system uses multiple tools to scan targets for open ports, services, subdomains, and vulnerabilities, and presents results via an Executive Summary, Key Findings, and detailed reports. Your feedback will help assess how useful and easy to use the system is.

Full Name

Short-answer text

Figure 202 Post-Survey Form: Personal Detail



1. What is your level of experience with Web Reconnaissance tools? *

- ☐ No experience
- ☐ Beginner
- ☐ Intermediate (used several tools, understand workflows)
- ☐ Advanced (regular use in assessments or research)
- ☐ Expert (professional use, multiple tools and methodologies)

Figure 203 Post-Survey Form: Question 1

2. How well does the system support real-world reconnaissance and security assessment workflows? *

- ☐ Very well
- ☐ Well
- ☐ Adequately
- ☐ Poorly
- ☐ Not at all

Figure 204 Post-Survey Form: Question 2

3. How clear and actionable are the findings for making security or remediation decisions? *

- ☐ Very clear and actionable
- ☐ Mostly clear
- ☐ Somewhat clear
- ☐ Rarely clear
- ☐ Not clear or actionable

Figure 205 Post-Survey Form: Question 3

4. How would you rate the quality of the reports (Executive Summary, Key Findings, Detailed Reconnaissance)? *

- ☐ Excellent
- ☐ Good with minor gaps
- ☐ Adequate
- ☐ Needs significant improvement
- ☐ Not suitable for professional use

Figure 206 Post-Survey Form: Question 4

5. How useful is the severity and risk presentation for prioritizing issues in a real engagement?

☒ Very useful for prioritization

☒ Useful

☐ Somewhat useful

☐ Of limited use

☐ Not useful

Figure 207 Post-Survey Form: Question 5

6. How likely would you be to use this system in a professional or organisational context (e.g. internal assessments, client work, research)?

☒ Very likely

☐ Likely

☐ Possibly

☐ Unlikely

☐ Very unlikely

Figure 208 Post-Survey Form: Question 6

7. How does the system compare to your expectations of a reconnaissance and reporting tool?

☒ Exceeds expectations

☐ Meets expectations

☐ Partly meets expectations

☐ Below expectations

☐ Far below expectations

Figure 209 Post-Survey Form: Question 7

8. How well does the tool integration (multiple scanners, single report) support efficiency in security assessments? *

- ☐ Very well
- ☐ Well
- ☐ Adequately
- ☐ Poorly
- ☐ Not at all

Figure 210 Post-Survey Form: Question 8

9. What would most improve the system for professional or organisational adoption? *

- ☐ Performance and scalability
- ☐ Report clarity and customisation
- ☐ Ease of use and onboarding
- ☐ Broader tool coverage and automation
- ☐ Compliance and audit-friendly outputs
- ☐ Other:

Figure 211 Post-Survey Form: Question 9

10. Who in your view would benefit most from this system? *

- ☐ Security teams and SOC's
- ☐ Penetration testers and red teams
- ☐ DevOps / platform teams
- ☐ Researchers and threat intelligence
- ☐ All of the above
- ☐ Other:

Figure 212 Post-Survey Form: Question 10

11. Where do you see the strongest use case for this system? *

- ☐ Internal security assessments
- ☐ External/client engagements
- ☐ Research and development
- ☐ Training and capability building
- ☐ Continuous monitoring / asset discovery
- ☐ Other:

Figure 213 Post-Survey Form: Question 11

12. When (in what context) would you be most likely to use this system? *

- ☐ As a primary reconnaissance and reporting platform
- ☐ Alongside existing tools for specific tasks
- ☐ For quick assessments or proof-of-concepts
- ☐ For training or lab environments
- ☐ Not sure / would need more evaluation

Figure 214 Post-Survey Form: Question 12

13. Any additional comments on gaps, strengths, or what would make it suitable for production or organisational use?

Long-answer text

Figure 215 Post-Survey Form: Question 13

7.2.2. Post-Survey Responses

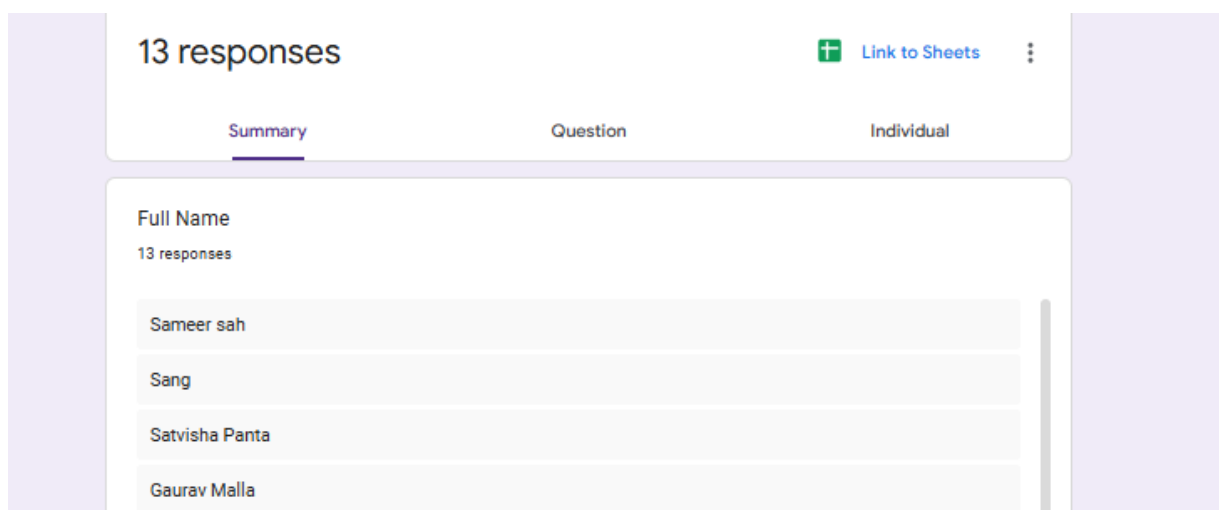


Figure 216 Post-Survey Form Response: Identity

13 responses		 Link to Sheets	⋮
Summary		Question	Individual
Full Name			
13 responses			
Gaurav Dahal			
Manish shrestha			
bachan			
Evani Raut			
Prashna Malla			
Piyush shrestha			
Prajina Taksari			
Ishan			
Prashant Rana			

Figure 217 Post-Survey Form Responses: Identity

1. What is your level of experience with Web Reconnaissance tools?

13 responses

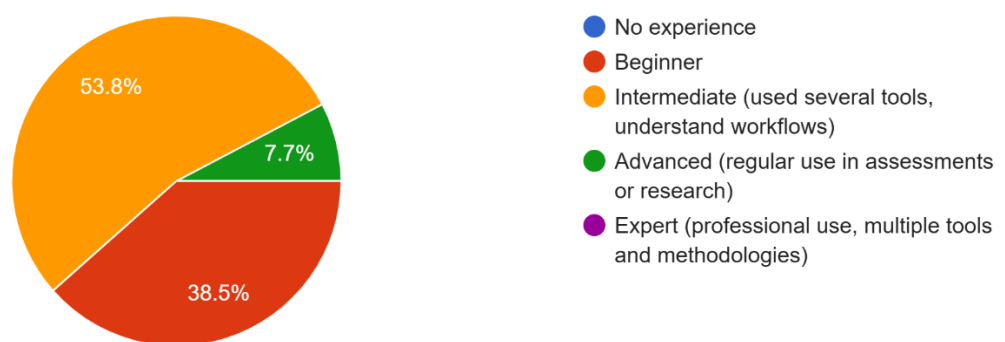


Figure 218 Post-Survey Form Responses: Question 1

2. How well does the system support real-world reconnaissance and security assessment workflows?

13 responses

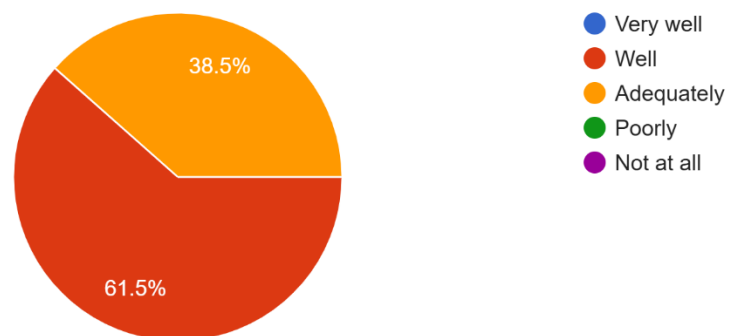


Figure 219 Post-Survey Form Responses: Question 2

3. How clear and actionable are the findings for making security or remediation decisions?

13 responses

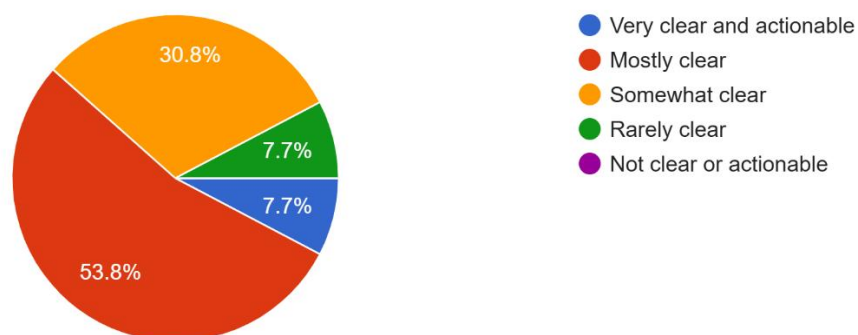


Figure 220 Post-Survey Form Responses: Question 3

4. How would you rate the quality of the reports (Executive Summary, Key Findings, Detailed Reconnaissance)?

13 responses

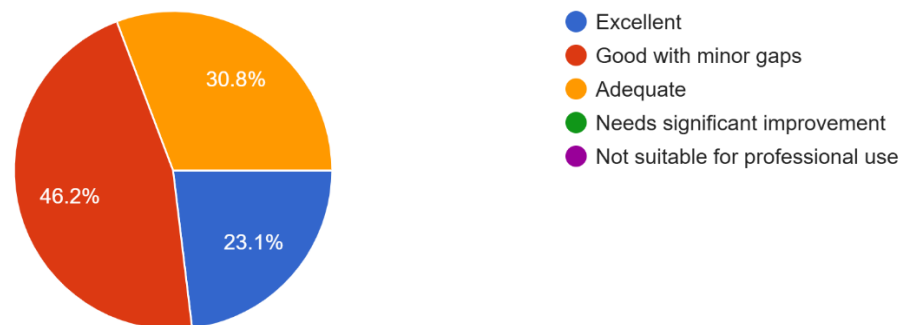


Figure 221 Post-Survey Form Responses: Question 4

5. How useful is the severity and risk presentation for prioritizing issues in a real engagement?

13 responses

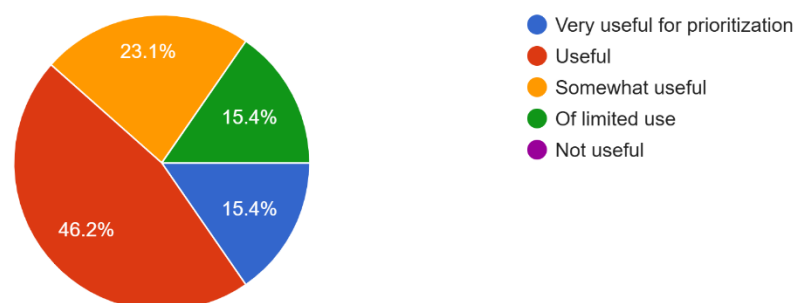


Figure 222 Post-Survey Form Responses: Question 5

6. How likely would you be to use this system in a professional or organisational context (e.g. internal assessments, client work, research)?

13 responses

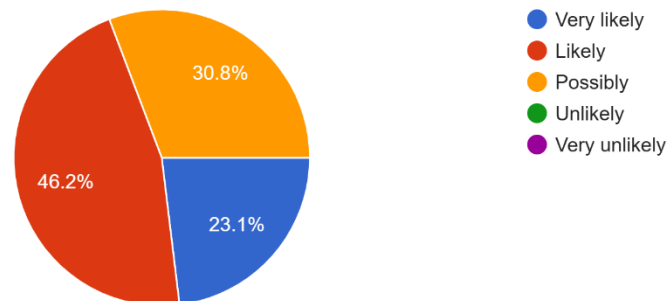


Figure 223 Post-Survey Form Responses: Question 6

7. How does the system compare to your expectations of a reconnaissance and reporting tool?

13 responses

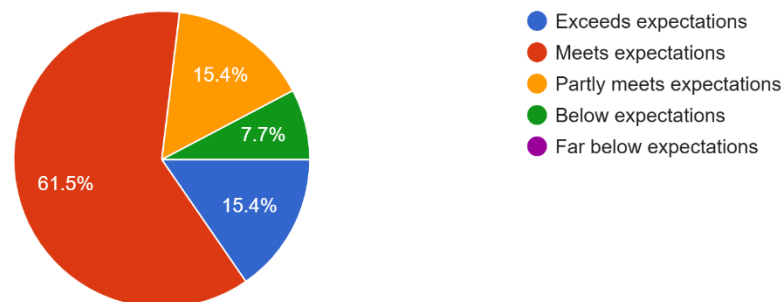


Figure 224 Post-Survey Form Responses: Question 7

8. How well does the tool integration (multiple scanners, single report) support efficiency in security assessments?

13 responses

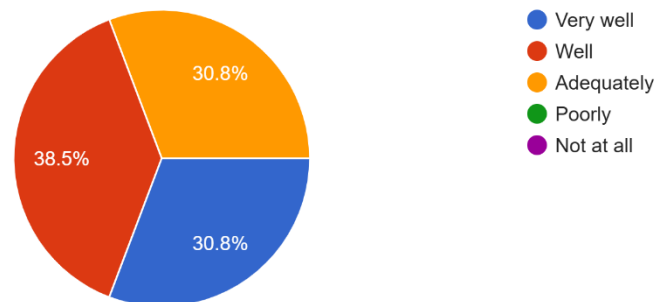


Figure 225 Post-Survey Form Responses: Question 8

9. What would most improve the system for professional or organisational adoption?

13 responses

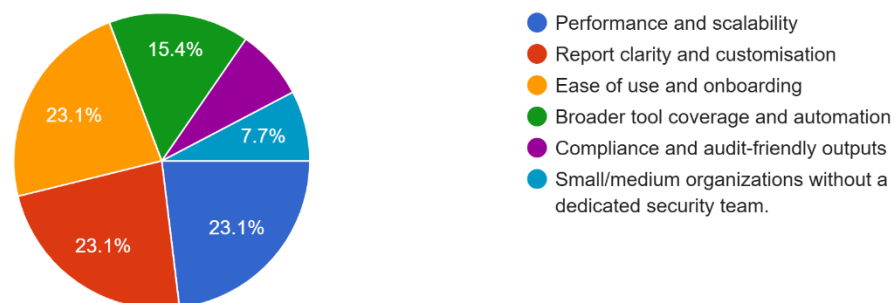


Figure 226 Post-Survey Form Responses: Question 9

10. Who in your view would benefit most from this system?

13 responses

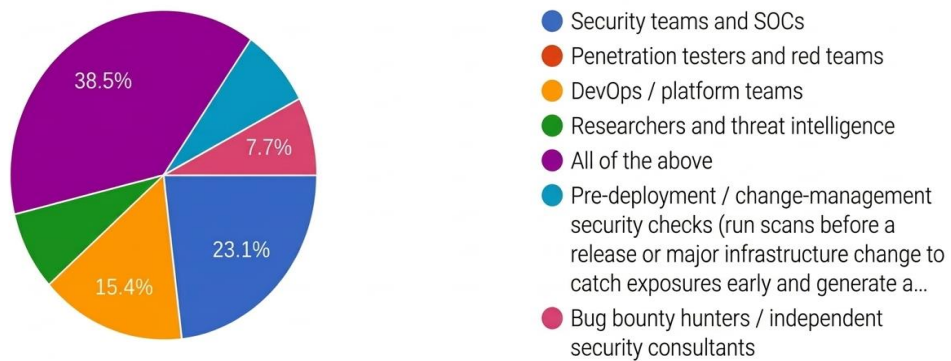


Figure 227 Post-Survey Form Responses: Question 10

11. Where do you see the strongest use case for this system?

13 responses

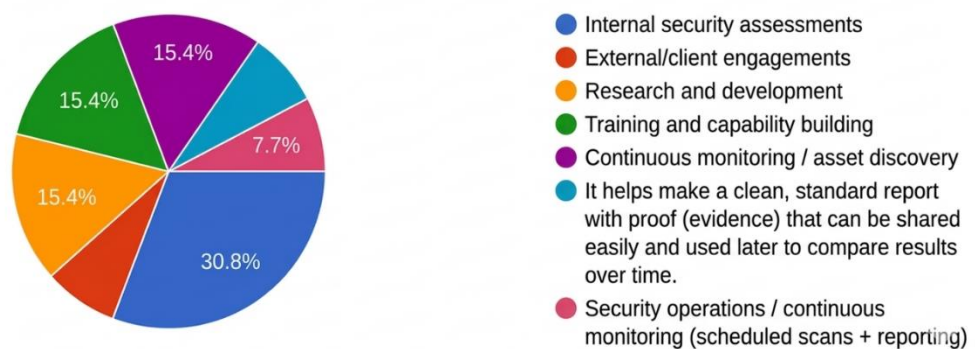


Figure 228 Post-Survey Form Responses: Question 11

12. When (in what context) would you be most likely to use this system?

13 responses

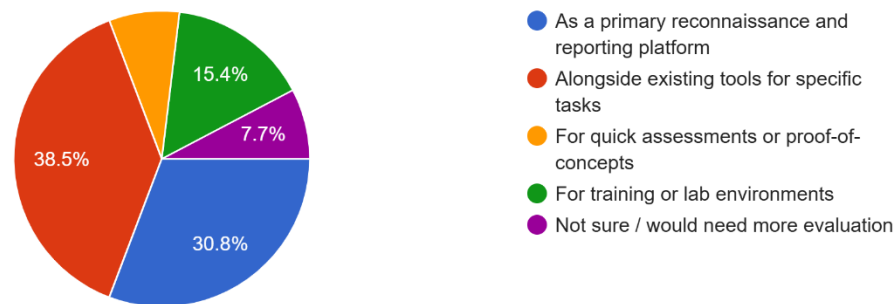


Figure 229 Post-Survey Form Responses: Question 12

13. Any additional comments on gaps, strengths, or what would make it suitable for production or organisational use?

2 responses

It demonstrates the core functionality well, but improvements in security, scalability, and documentation would be needed for production or organisational use.

Nothing

Figure 230 Post-Survey Form Responses: Question 13

[Click Here](#) to go back.

7.3. APPENDIX 3: CODES

7.3.1. Sprint 1: Initial Setup and Tools Installation

Sprint 1 aimed at establishing the base development environment of the Intelligence Recon System, Python, and Fast API, as well as project setup. It was also the installation and verification of all necessary reconnaissance and scanning tools on the target machine to guarantee that they execute appropriately via the command line. At the close of this sprint, the project had a working skeleton of the backend and verified external tools, and it was ready to be further integrated in subsequent sprints.

```
// ===== CONFIG =====
// API Base URL: use same-origin first so API works when app is served from the same server
let defaultApiBase = window.location.origin || `http://${window.location.hostname}:8080`;

const API_ROUTES = {
  config: () => "/api/config", // Fetch API config from backend
  createScan: () => "/api/scans",
  listScans: () => "/api/scans",
  scanDetail: (id) => `/api/scans/${id}`,
  scanStatus: (id) => `/api/scans/${id}/status`,
  scanIntelligence: (id) => `/api/scans/${id}/intelligence-summary`,
  health: () => "/api/health",
  reportHtml: (id) => `/api/scans/${id}/report`,
  reportPdf: (id) => `/api/scans/${id}/report/pdf`,
  markScanSaved: (id) => `/api/scans/${id}/mark-saved`,
  severityStatsCompleted: () => `/api/scans/stats/severity-completed`,
  purgeScans: () => "/api/scans/purge",
  toolCommand: (toolName, target) => `/api/tools/${encodeURIComponent(toolName)}/command${target ? `?target=${encodeURIComponent(target)}` : ""}`,
};
```

Figure 231 Project Path Configuration and Database URL Setup

```
// All 15 tools mapped to OWASP Top 10 2021 categories (each tool appears in at least one category)
const OWASP_MAP = [
  { id: "A01:2021", name: "Broken Access Control", tools: ["Nuclei", "FFuf", "Wfuzz", "Katana", "GoSpider", "GAU", "Subfinder", "Sublist3r"] },
  { id: "A02:2021", name: "Cryptographic Failures", tools: ["Nuclei", "Httpx"] },
  { id: "A03:2021", name: "Injection", tools: ["Nuclei", "FFuf", "Wfuzz", "Katana", "GoSpider", "GAU"] },
  { id: "A04:2021", name: "Insecure Design", tools: ["Nuclei", "Wfuzz", "Katana", "FFuf"] },
  { id: "A05:2021", name: "Security Misconfiguration", tools: ["Naabu", "Httpx", "Nuclei", "Subfinder", "DNSx", "Sublist3r"] },
  { id: "A06:2021", name: "Vulnerable and Outdated Components", tools: ["Nuclei", "Subfinder", "Amass", "Assetfinder", "Sublist3r", "DNSx", "ShuffleDNS"] },
  { id: "A07:2021", name: "Identification & Authentication Failures", tools: ["Nuclei", "FFuf", "Wfuzz", "CeWL"] },
  { id: "A08:2021", name: "Software & Data Integrity Failures", tools: ["Nuclei", "Httpx"] },
  { id: "A09:2021", name: "Security Logging & Monitoring Failures", tools: ["Nuclei", "Httpx"] },
  { id: "A10:2021", name: "Server-Side Request Forgery", tools: ["Httpx", "Nuclei", "GAU", "Subfinder", "Amass", "DNSx"] },
];
```

Figure 232 OWASP Top 10 2021 Categories Configuration

```
(venv) prabesh@prabesh-Surface-Laptop-3:~/Documents/FINAL YEAR PROJECT$ pip install -r requirements.txt
Defaulting to user installation because normal site-packages is not writeable
Collecting fastapi==0.104.1 (from -r requirements.txt (line 4))
  Using cached fastapi-0.104.1-py3-none-any.whl.metadata (24 kB)
Collecting uvicorn==0.24.0 (from uvicorn[standard]==0.24.0->-r requirements.txt (line 5))
  Using cached uvicorn-0.24.0-py3-none-any.whl.metadata (6.4 kB)
Collecting python-multipart==0.0.6 (from -r requirements.txt (line 6))
  Using cached python-multipart-0.0.6-py3-none-any.whl.metadata (2.5 kB)
Collecting sqlalchemy==2.0.23 (from sqlalchemy[asyncio]==2.0.23->-r requirements.txt (line 9))
  Using cached SQLAlchemy-2.0.23-cp310-cp310-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (9.6 kB)
Collecting aiosqlite==0.19.0 (from -r requirements.txt (line 10))
  Using cached aiosqlite-0.19.0-py3-none-any.whl.metadata (4.3 kB)
Collecting httpx==0.25.1 (from -r requirements.txt (line 13))
  Using cached httpx-0.25.1-py3-none-any.whl.metadata (7.1 kB)
Collecting pydantic==2.5.0 (from -r requirements.txt (line 16))
  Using cached pydantic-2.5.0-py3-none-any.whl.metadata (174 kB)
Collecting pydantic-settings==2.1.0 (from -r requirements.txt (line 17))
  Using cached pydantic-settings-2.1.0-py3-none-any.whl.metadata (2.9 kB)
Collecting python-dotenv==1.0.0 (from -r requirements.txt (line 20))
  Using cached python-dotenv-1.0.0-py3-none-any.whl.metadata (21 kB)
Collecting anyio==4.0.0 (from fastapi==0.104.1->-r requirements.txt (line 4))
  Using cached anyio-4.0.0-py3-none-any.whl.metadata (4.7 kB)
Collecting starlette==0.28.0 (from fastapi==0.104.1->-r requirements.txt (line 4))
  Using cached starlette-0.27.0-py3-none-any.whl.metadata (5.8 kB)
Requirement already satisfied: typing-extensions==4.8.0 in /home/prabesh/.local/lib/python3.10/site-packages (from fastapi==0.104.1->-r requirements.txt (line 4)) (4.8.0)
Requirement already satisfied: annotated-types==0.4.0 in /home/prabesh/.local/lib/python3.10/site-packages (from pydantic==2.5.0->-r requirements.txt (line 16)) (0.7.0)
Requirement already satisfied: click==7.0 in /usr/local/lib/python3.10/dist-packages (from uvicorn==0.24.0->-r requirements.txt (line 5)) (8.2.1)
Requirement already satisfied: h11==0.8 in /home/prabesh/.local/lib/python3.10/site-packages (from uvicorn==0.24.0->-r requirements.txt (line 5)) (0.16.0)
Requirement already satisfied: greenlet==0.4.17 in /home/prabesh/.local/lib/python3.10/site-packages (from sqlalchemy==2.0.23->-r requirements.txt (line 9)) (3.3.0)
Requirement already satisfied: certifi in /usr/lib/python3/dist-packages (from httpx==0.25.1->-r requirements.txt (line 13)) (2020.6.20)
Requirement already satisfied: httpcore in /home/prabesh/.local/lib/python3.10/site-packages (from httpx==0.25.1->-r requirements.txt (line 13)) (1.0.9)
Requirement already satisfied: idna in /usr/lib/python3/dist-packages (from httpx==0.25.1->-r requirements.txt (line 13)) (3.3)
Requirement already satisfied: sniffio in /home/prabesh/.local/lib/python3.10/site-packages (from httpx==0.25.1->-r requirements.txt (line 13)) (1.3.1)
Requirement already satisfied: httptools==0.5.0 in /home/prabesh/.local/lib/python3.10/site-packages (from uvicorn[standard]==0.24.0->-r requirements.txt (line 5)) (0.7.1)
Requirement already satisfied: pyyaml==5.1 in /home/prabesh/.local/lib/python3.10/site-packages (from uvicorn[standard]==0.24.0->-r requirements.txt (line 5)) (6.0.3)
Requirement already satisfied: uvloop==0.15.0 in /home/prabesh/.local/lib/python3.10/site-packages (from uvicorn[standard]==0.24.0->-r requirements.txt (line 5)) (0.22.1)
Requirement already satisfied: watchfiles==0.13 in /home/prabesh/.local/lib/python3.10/site-packages (from uvicorn[standard]==0.24.0->-r requirements.txt (line 5)) (1.1.1)
Requirement already satisfied: websockets==10.4 in /home/prabesh/.local/lib/python3.10/site-packages (from uvicorn[standard]==0.24.0->-r requirements.txt (line 5)) (15.0.1)
Requirement already satisfied: exceptiongroup in /home/prabesh/.local/lib/python3.10/site-packages (from anyio==4.0.0=>-r requirements.txt (line 4)) (1.3.1)
Downloading fastapi-0.104.1-py3-none-any.whl (92 kB)
Downloading pydantic-2.5.0-py3-none-any.whl (407 kB)
Downloading uvicorn-0.24.0-py3-none-any.whl (59 kB)
Downloading python-multipart-0.0.6-py3-none-any.whl (45 kB)
Downloading SQLAlchemy-2.0.23-cp310-cp310-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (3.0 MB)
3.0/3.0 MB 150.8 KB/s 0:00:20
Downloading aiosqlite-0.19.0-py3-none-any.whl (15 kB)
```

Figure 233 Installing Python dependencies

```

class BaseTool(ABC):
    """
    Abstract base class for all recon tools.

    Handles:
    - Command execution
    - Timeout handling
    - Output parsing
    - WebSocket notifications
    """

    Windsurf: Refactor | Explain | Generate Docstring
    def __init__(self, tool_name: str):
        self.tool_name = tool_name
        self.logger = get_logger(f"tool.{tool_name.lower()}")

    Windsurf: Refactor | Explain
    @abstractmethod
    def build_command(self, target: str, output_file: Path, **kwargs) -> List[str]:
        """
        Build the command line arguments for this tool.

        Returns:
        | List of strings representing the command.
        """
        pass

    Windsurf: Refactor | Explain
    @abstractmethod
    def parse_output(self, output: str, target: str) -> List[Dict[str, Any]]:
        """
        Parse raw tool output into a structured list of findings.

        Each finding is a dictionary with keys:
        - type
        - severity
        - location
        - description
        - evidence
        """
        pass

```

Figure 234 BaseTool Abstract Class Definition and Interface

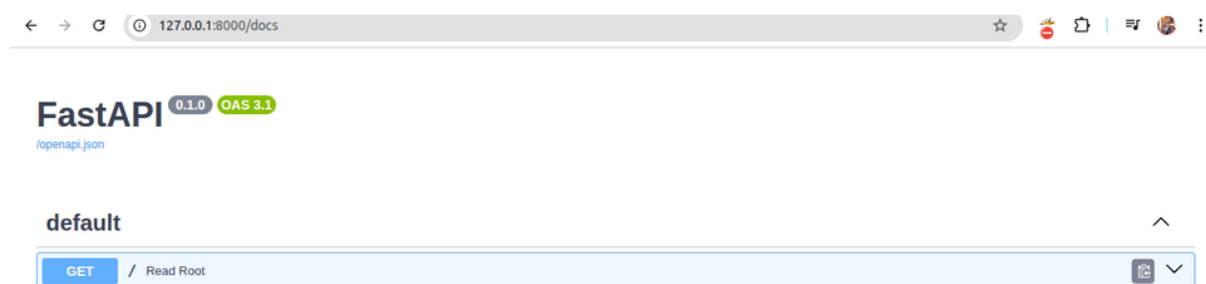


Figure 235 Fast API Swagger page showing the root endpoint

```
(venv) prabesh@prabesh-Surface-Laptop-3: ~/Documents/FINAL YEAR PROJECTS --retos
(venv) prabesh@prabesh-Surface-Laptop-3: ~/Documents/FINAL YEAR PROJECTS
(venv) prabesh@prabesh-Surface-Laptop-3: ~/Documents/FINAL YEAR PROJECTS go install -v github.com/projectdiscovery/naabu/v2/cmd/naabu@latest
go: github.com/projectdiscovery/naabu/v2@v2.3.7 requires go >= 1.24.0; switching to go1.24.11
github.com/projectdiscovery/gologger/levels
github.com/projectdiscovery/naabu/v2/pkg/result/confidence
github.com/Ullaakut/nmap/v3/pkg/osfamilies
github.com/projectdiscovery/utls/conversion
github.com/tidwall/match
golang.org/x/exp/constraints
github.com/projectdiscovery/utls/race
github.com/zmap/zcrypto/cryptobyte/asn1
github.com/projectdiscovery/utls/ptr
github.com/shirou/gopsutil/v3/common
github.com/projectdiscovery/naabu/v2/pkg/israce
github.com/projectdiscovery/utls/unit
github.com/projectdiscovery/utls/os
golang.org/x/exp/maps
github.com/projectdiscovery/utls/slice
github.com/projectdiscovery/utls/consts
github.com/projectdiscovery/gologger/formatter
github.com/projectdiscovery/naabu/v2/pkg/protocol
golang.org/x/sync/errgroup
github.com/projectdiscovery/utls/env
github.com/projectdiscovery/naabu/v2/pkg/port
github.com/projectdiscovery/utls/errkit
github.com/Ullaakut/nmap/v3
github.com/miekg/dns
github.com/projectdiscovery/naabu/v2/pkg/result
github.com/projectdiscovery/utls/strings
github.com/tidwall/gjson
github.com/projectdiscovery/gologger/writer
github.com/projectdiscovery/utls/ip
github.com/projectdiscovery/gologger
github.com/projectdiscovery/utls/errors
github.com/projectdiscovery/utls/file
github.com/weppos/publicsuffix-go/publicsuffix
github.com/projectdiscovery/clistats
github.com/cnf/structhash
github.com/projectdiscovery/utls/permission
github.com/projectdiscovery/utls/time
github.com/projectdiscovery/mapcidr
github.com/projectdiscovery/retryabledns/hostsfile
github.com/projectdiscovery/nmap/store/cache
```

Figure 236 Installing Nabbu tool with its go modules

```
(venv) prabesh@prabesh-Surface-Laptop-3:~/Documents/FINAL YEAR PROJECTS go install -v github.com/projectdiscovery/naabu/v2/cmd/naabu@latest
github.com/projectdiscovery/naabu/v2/internal/pdcp
(venv) prabesh@prabesh-Surface-Laptop-3:~/Documents/FINAL YEAR PROJECTS naabu -h
Naabu is a port scanning tool written in Go that allows you to enumerate open ports for hosts in a fast and reliable manner.

Usage:
  naabu [flags]

Flags:
INPUT:
  -host string[]      hosts to scan ports for (comma-separated)
  -list, -l string    list of hosts to scan ports (file)
  -exclude-hosts, -eh string  hosts to exclude from the scan (comma-separated)
  -exclude-file, -ef string  list of hosts to exclude from scan (file)

PORT:
  -port, -p string      ports to scan (80,443, 100-200)
  -top-ports, -tp string  top ports to scan (default 100) [full,100,1000]
  -exclude-ports, -ep string[]  ports to exclude from scan (file or comma-separated)
  -ports-file, -pf string[]  list of ports to scan (file or comma-separated)
  -port-threshold, -pts int  port threshold to skip port scan for the host
  -exclude-cdn, -ec         skip full port scans for CDN/WAF (only scan for port 80,443)
  -display-cdn, -cdn        display cdn in use

RATE-LIMIT:
  -c int    general internal worker threads (default 25)
  -rate int  packets to send per second (default 1000)

UPDATE:
  -up, -update          update naabu to latest version
  -duc, -disable-update-check  disable automatic naabu update check

OUTPUT:
  -o, -out string      file to write output to (optional)
  -lof, -list-output-fields  list of fields to output (comma separated)
  -eof, -exclude-output-fields string[]  exclude output fields output based on a condition
  -j, -json            write output in JSON lines format
  -csv                write output in csv format

CONFIGURATION:
  -config string  path to the naabu configuration file (default $HOME/.config/naabu/config.yaml)
  -scan-all-ips, -sa  scan all the IP's associated with DNS record
  -ip-version, -iv string[]  ip version to scan of hostname (4,6) - (default 4) (default ["4"])
  -scan-type, -s string  type of port scan (SYN/CONNECT) (default "c")
  -source-ip, -si string  source ip address to use for connection (default "0.0.0.0")
  -source-port, -sp int  source port to use for connection (default 8080)
```

Figure 237 Verify Successful Installation


```
(venv) prabesh@prabesh-Surface-Laptop-3:~/Documents/FINAL YEAR PROJECT$ go install -v github.com/projectdiscovery/nuclei/v3/cmd/nuclei@latest
go: downloading github.com/projectdiscovery/nuclei/v3 v3.6.1
go: github.com/projectdiscovery/nuclei/v3@v3.6.1 requires go >= 1.24.2; switching to go1.24.11
go: downloading github.com/projectdiscovery/gologger v1.1.64
go: downloading github.com/projectdiscovery/retryablehttp-go v1.1.1
go: downloading github.com/projectdiscovery/dsl v0.8.8
go: downloading github.com/projectdiscovery/fastdialer v0.4.20
go: downloading github.com/projectdiscovery/wappalizergo v0.2.59
go: downloading github.com/projectdiscovery/networkpolicy v0.1.32
go: downloading github.com/projectdiscovery/cdncheck v1.2.14
go: downloading github.com/projectdiscovery/retryabledns v1.0.111
go: downloading github.com/klauspost/compress v1.18.2
go: downloading github.com/minio/miniz v1.0.1
go: downloading github.com/pierrec/lz4/v4 v4.1.22
go: downloading github.com/spf13/afero v1.15.0
go: downloading golang.org/x/mod v0.29.0
go: downloading github.com/ericlagergren/decimal v0.0.0-20240411145413-00de7ca16731
go: downloading github.com/felixge/httpsnoop v1.0.4
go: downloading go.opentelemetry.io/otel/metric v1.38.0
go: downloading github.com/go-pg/zerochecker v0.2.0
go: downloading github.com/vmihailenco/msgpack/v5 v5.3.4
go: downloading github.com/vmihailenco/tagparser v0.1.2
go: downloading github.com/tmthrgd/go-hex v0.0.0-20190904060850-447a3041c3bc
go: downloading github.com/vmihailenco/bufpool v0.1.11
go: downloading github.com/gogo/protobuf v1.3.2
go: downloading github.com/go-logr/logr v1.4.3
go: downloading github.com/vmihailenco/tagparser/v2 v2.0.0
go: downloading github.com/go-logr/stdr v1.2.2
go: downloading go.opentelemetry.io/auto/sdk v1.1.0
go: downloading google.golang.org/protobuf v1.36.6
github.com/projectdiscovery/gologger/levels
github.com/klauspost/compress/internal/le
github.com/pierrec/lz4/v4/internal/lz4errors
github.com/klauspost/compress/internal/cpuinfo
github.com/klauspost/compress/internal/race
github.com/minio/miniz/internal/race
github.com/klauspost/compress
github.com/charmbracelet/x/ansi/parser
github.com/andybalholm/brotli/matchfinder
github.com/STARRY-S/zip
github.com/bodgit/secp256k1/internal/util
```

Figure 238 Installation of nuclei scanner and go

```
(venv) prabesh@prabesh-Surface-Laptop-3:~/Documents/FINAL YEAR PROJECT$ nuclei -h
-tid, -team-id string      upload scan results to given team id (optional) (default "none")
-cup, -cloud-upload        upload scan results to pdcp dashboard [DEPRECATED use -dashboard]
-sid, -scan-id string      upload scan results to existing scan id (optional)
-sname, -scan-name string  scan name to set (optional)
-pd, -dashboard            upload / view nuclei results in projectdiscovery cloud (pdc) UI dashboard
-pdu, -dashboard-upload string upload / view nuclei results file (jsonl) in projectdiscovery cloud (pdc) UI dashboard

AUTHENTICATION:
-sf, -secret-file string[] path to config file containing secrets for nuclei authenticated scan
-ps, -prefetch-secrets     prefetch secrets from the secrets file

EXAMPLES:
Run nuclei on single host:
$ nuclei -target example.com

Run nuclei with specific template directories:
$ nuclei -target example.com -t http/cves/ -t ssl

Run nuclei against a list of hosts:
$ nuclei -list hosts.txt

Run nuclei with a JSON output:
$ nuclei -target example.com -json-export output.json

Run nuclei with sorted Markdown outputs (with environment variables):
$ MARKDOWN_EXPORT_SORT_MODE=template nuclei -target example.com -markdown-export nuclei_report/

Additional documentation is available at: https://docs.nuclei.sh/getting-started/running
```

Figure 239 Verified installation of nuclei

```

* (venv) prabesh@prabesh-Surface-Laptop-3:~/Documents/FINAL YEAR PROJECT$ go install -v github.com/projectdiscovery/httpx/cmd/httpx@latest
go: downloading github.com/projectdiscovery/httpx/v1.7.4 requires go >= 1.24.1; switching to go1.24.11
* (venv) prabesh@prabesh-Surface-Laptop-3:~/Documents/FINAL YEAR PROJECT$ httpx --help
HTTPX 🐞

A next generation HTTP client.

Usage: httpx <URL> [OPTIONS]

-m, --method METHOD      Request method, such as GET, POST, PUT, PATCH, DELETE, OPTIONS, HEAD.
                        [Default: GET, or POST if a request body is included]

-p, --params <NAME VALUE> ... Query parameters to include in the request URL.

-c, --content TEXT      Byte content to include in the request body.

-d, --data <NAME VALUE> ... Form data to include in the request body.

-f, --files <NAME FILENAME> ... Form files to include in the request body.

-j, --json TEXT         JSON data to include in the request body.

-h, --headers <NAME VALUE> ... Include additional HTTP headers in the request.

--cookies <NAME VALUE> ... Cookies to include in the request.

--auth <USER PASS>      Username and password to include in the request. Specify '-' for the password to use a password prompt.
                        Note that using --verbose/-v will expose the Authorization header, including the password encoding in a
                        trivially reversible format.

--proxies URL            Send the request via a proxy. Should be the URL giving the proxy address.

--timeout FLOAT          Timeout value to use for network operations, such as establishing the connection, reading some data,
                        etc... [Default: 5.0]

--follow-redirects       Automatically follow redirects.

--no-verify              Disable SSL verification.

--http2                  Send the request using HTTP/2, if the remote server supports it.

--download FILE          Save the response content as a file, rather than displaying it.

```

Figure 240 Install httpx and verified

```

* (venv) prabesh@prabesh-Surface-Laptop-3:~/Documents/FINAL YEAR PROJECT$ go install -v github.com/projectdiscovery/subfinder/v2/cmd/subfinder@latest
go: downloading github.com/projectdiscovery/subfinder/v2 v2.11.0
go: downloading github.com/projectdiscovery/subfinder/v2 v2.11.0 requires go >= 1.24.0; switching to go1.24.11
github.com/projectdiscovery/gologger/levels
github.com/refraction-networking/utls/internal/fips140tls
github.com/refraction-networking/utls/internal/byteorder
github.com/projectdiscovery/utls/conversion
github.com/refraction-networking/utls/dicttls
github.com/gaissmai/bart/internal/art
github.com/zmap/zcrypto/cryptobyte/asn1
github.com/projectdiscovery/utls/ptr
github.com/projectdiscovery/utls/race
github.com/projectdiscovery/gologger/formatter
github.com/projectdiscovery/utls/errors
github.com/hako/durafmt
github.com/projectdiscovery/utls/os
github.com/refraction-networking/utls/internal/boring
github.com/refraction-networking/utls/internal/helper
github.com/refraction-networking/utls/internal/hkdf
github.com/projectdiscovery/gologger/writer
github.com/refraction-networking/utls/internal/quicvarint/protocol
github.com/refraction-networking/utls/internal/hpke
github.com/refraction-networking/utls/internal/tls12
github.com/refraction-networking/utls/internal/tls13
github.com/refraction-networking/utls/internal/quicvarint
github.com/projectdiscovery/ratelimit
github.com/projectdiscovery/hmap/store/cache
github.com/projectdiscovery/utls/slice
github.com/projectdiscovery/hmap/store/disk
github.com/projectdiscovery/utls/strings
github.com/projectdiscovery/utls/consts
github.com/refraction-networking/utls
github.com/projectdiscovery/utls/env
github.com/projectdiscovery/retryabledns/doh
github.com/gaissmai/bart/internal/bitset
github.com/projectdiscovery/gologger
github.com/projectdiscovery/utls/maps
github.com/projectdiscovery/utls/errkit
github.com/gaissmai/bart/internal/allot
github.com/projectdiscovery/utls/file
github.com/projectdiscovery/utls/ip
github.com/gaissmai/bart/internal/lpm
github.com/gaissmai/bart/internal/sparse

```

Figure 241 Install Subfinder


```
(venv) prabesh@prabesh-Surface-Laptop-3:~/Documents/FINAL YEAR PROJECTS$ go install -v github.com/projectdiscovery/subfinder/v2/cmd/subfinder@latest
(venv) prabesh@prabesh-Surface-Laptop-3:~/Documents/FINAL YEAR PROJECTS$ subfinder -h
Subfinder is a subdomain discovery tool that discovers subdomains for websites by using passive online sources.

Usage:
  subfinder [flags]

Flags:
INPUT:
  -d, -domain string[] domains to find subdomains for
  -dl, -list string      file containing list of domains for subdomain discovery

SOURCE:
  -s, -sources string[] specific sources to use for discovery (-s crtsh,github). Use -ls to display all available sources.
  -recursive            use only sources that can handle subdomains recursively rather than both recursive and non-recursive sources
  -all                 use all sources for enumeration (slow)
  -es, -exclude-sources string[] sources to exclude from enumeration (-es alienvault,zoomeyeapi)

FILTER:
  -m, -match string[] subdomain or list of subdomain to match (file or comma separated)
  -f, -filter string[] subdomain or list of subdomain to filter (file or comma separated)

RATE-LIMIT:
  -rl, -rate-limit int maximum number of http requests to send per second (global)
  -rls, -rate-limits value maximum number of http requests to send per second for providers in key-value format (-rls hackertarget=10/m) (default ["g
  =60/m", "pugreon=10/s", "robtex=18446744073709551615/m", "securitytrails=1/s", "shodan=1/s", "virustotal=4/m", "hackertarget=2/s", "waybackarchive=15
  ", "securitytrails=2/s", "sitedossier=8/m", "netlas=1/s", "github=83/m", "hudsonrock=3/s"])
  -t int               number of concurrent goroutines for resolving (-active only) (default 10)

UPDATE:
  -up, -update          update subfinder to latest version
  -duc, -disable-update-check disable automatic subfinder update check

OUTPUT:
  -o, -output string      file to write output to
  -oj, -json             write output in JSONL(lines) format
  -od, -output-dir string directory to write output (-dl only)
  -cs, -collect-sources include all sources in the output (-json only)
  -oi, -ip              include host IP in output (-active only)

CONFIGURATION:
  -config string          flag config file (default "/home/prabesh/.config/subfinder/config.yaml")
  -pc, -provider-config string provider config file (default "/home/prabesh/.config/subfinder/provider-config.yaml")
  -r string[]            comma separated list of resolvers to use
```

Figure 242 Verified installation of subfinder

```
(venv) prabesh@prabesh-Surface-Laptop-3:~/Documents/FINAL YEAR PROJECTS$ go install -v github.com/tomnomnom/assetfinder@latest
internal/coverage/rtcov
internal/godebugs
internal/unsafeheader
internal/goarch
internal/goexperiment
internal/byteorder
internal/gos
internal/profilerecord
runtime/internal/math
internal/abi
internal/cpu
internal/runtime/atomic
internal/runtime/syscall
runtime/internal/sys
internal/race
internal/chacha8rand
sync/atomic
unicode
cmp
unicode/utf8
encoding
math/bits
internal/bytealg
internal/itoa
internal/asan
internal/runtime/exithook
internal/msan
unicode/utf16
container/list
crypto/internal/alias
crypto/subtle
math
crypto/internal/boring/sig
vendor/golang.org/x/crypto/cryptobyte/asn1
vendor/golang.org/x/crypto/internal/alias
internal/nettrace
log/internal
internal/stringlite
runtime
iter
internal/weak
internal/reflectlite
```

Figure 243 Install Assetfinder with go and fetching dependencies

```

mimeType/multipart
crypto/tls
net/http/httptrace
net/http
● (venv) prabesh@prabesh-Surface-Laptop-3:~/Documents/FINAL YEAR PROJECTS$ assetfinder -h
Usage of assetfinder:
  -subs-only
    Only include subdomains of search domain
○ (venv) prabesh@prabesh-Surface-Laptop-3:~/Documents/FINAL YEAR PROJECTS$

```

Figure 244 Verified installation of assetfinder

[illegible]

Figure 245 Install Amass

```
(venv) prabesh@prabesh-Surface-Laptop-3:~/Documents/FINAL YEAR PROJECT$ git clone https://github.com/about131a/Sublist3r.git
remote: Total 383 (delta 0), reused 0 (delta 0), pack-reused 383 (from 1)
Resolving objects: 100% (33/33), 1.12 MiB | 568.00 KiB/s, done.
Resolving deltas: 100% (213/213), done.
* (venv) prabesh@prabesh-Surface-Laptop-3:~/Documents/FINAL YEAR PROJECT$ python3 sublist3r.py -h
python3: can't open file '/home/prabesh/Documents/FINAL YEAR PROJECT/sublist3r.py': [Errno 2] No such file or directory
* (venv) prabesh@prabesh-Surface-Laptop-3:~/Documents/FINAL YEAR PROJECT$ git clone https://github.com/about131a/Sublist3r.git
fatal: destination path 'Sublist3r' already exists and is not an empty directory.
* (venv) prabesh@prabesh-Surface-Laptop-3:~/Documents/FINAL YEAR PROJECT$ cd ./Sublist3r/
* (venv) prabesh@prabesh-Surface-Laptop-3:~/Documents/FINAL YEAR PROJECT/Sublist3r$ pip install -r requirements.txt
Defaulting to user installation because normal site-packages is not writable
Collecting argparse from -r requirements.txt (line 1)
  Downloading argparse-1.4.0-py2.py3-none-any.whl.metadata (2.8 kB)
Requirement already satisfied: dnspython in /usr/lib/python3/dist-packages (from -r requirements.txt (line 2)) (2.1.0)
Requirement already satisfied: requests in /home/prabesh/.local/lib/python3.10/site-packages (from -r requirements.txt (line 3)) (2.32.5)
Requirement already satisfied: charset-normalizer<4,>=2 in /home/prabesh/.local/lib/python3.10/site-packages (from requests->-r requirements.txt (line 3)) (3.4.4)
Requirement already satisfied: idna<4,>=2.5 in /usr/lib/python3/dist-packages (from requests->-r requirements.txt (line 3)) (3.3)
Requirement already satisfied: urllib3<3,>=1.21.1 in /home/prabesh/.local/lib/python3.10/site-packages (from requests->-r requirements.txt (line 3)) (2.5.0)
Requirement already satisfied: certifi<=2017.4.17 in /usr/lib/python3/dist-packages (from requests->-r requirements.txt (line 3)) (2020.6.20)
Downloading argparse-1.4.0-py2.py3-none-any.whl (23 kB)
Installing collected packages: argparse
Successfully installed argparse-1.4.0
* (venv) prabesh@prabesh-Surface-Laptop-3:~/Documents/FINAL YEAR PROJECT/Sublist3r$ python3 sublist3r.py -h
usage: sublist3r.py [-h] -d DOMAIN [-b [BRUTEFORCE]] [-p PORTS] [-v [VERBOSE]] [-e ENGINES] [-o OUTPUT] [-n]

OPTIONS:
  -h, --help            show this help message and exit
  -d DOMAIN, --domain DOMAIN
                        Domain name to enumerate it's subdomains
  -b [BRUTEFORCE], --bruteforce [BRUTEFORCE]
                        Enable the subbrute bruteforce module
  -p PORTS, --ports PORTS
                        Scan the found subdomains against specified tcp ports
  -v [VERBOSE], --verbose [VERBOSE]
                        Enable Verbosity and display results in realtime
  -t THREADS, --threads THREADS
                        Number of threads to use for subbrute bruteforce
  -e ENGINES, --engines ENGINES
                        Specify a comma-separated list of search engines
  -o OUTPUT, --output OUTPUT
                        Save the results to text file
  -n, --no-color         Output without color

Example: python3 sublist3r.py -d example.com
```

Figure 246 Cloning Sublist3r, installing requirements and verified installation

```

    config      ctt config tool
• (venv) prabesh@prabesh-Surface-Laptop-3:~/Documents/FINAL YEAR PROJECT$ go install -v github.com/lc/gau/v2/cmd/gau@latest
github.com/valyala/bytebufferpool
hash/adler32
github.com/modern-go/reflect2
html
golang.org/x/text/transform
github.com/deckarep/golang-set/v2
runtime/debug
golang.org/x/sys/unix
github.com/klauspost/compress/flate
github.com/valyala/fasthttp/fasthttputil
github.com/valyala/fasthttp/stackless
github.com/andybalholm/brotli
golang.org/x/text/unicode/bidi
github.com/modern-go/concurrent
encoding/csv
golang.org/x/text/unicode/norm
github.com/lynxsecurity/viper/internal/encoding
github.com/hashicorp/hcl/hcl/strconv
github.com/lynxsecurity/pflag
github.com/hashicorp/hcl/hcl/token
github.com/hashicorp/hcl/hcl/ast
golang.org/x/text/secure/bidirule
regexp/syntax
github.com/hashicorp/hcl/json/token
github.com/hashicorp/hcl/json/scanner
github.com/hashicorp/hcl/json/parser
golang.org/x/net/idna
text/tabwriter
github.com/klauspost/compress/gzip
github.com/klauspost/compress/zlib
github.com/lynxsecurity/viper/internal/encoding/json
github.com/pelletier/go-toml
github.com/mitchellh/mapstructure
github.com/json-iterator/go
github.com/bobesa/go-domain-util/domainutil
regexp

```

Figure 247 Install GAU

```

github.com/lc/gau/v2/runner
• (venv) prabesh@prabesh-Surface-Laptop-3:~/Documents/FINAL YEAR PROJECT$ gau -h
Usage of gau:
  --blacklist strings  list of extensions to skip
  --config string      location of config file (default $HOME/.gau.toml or %USERPROFILE%\gau.toml)
  --fc strings         list of status codes to filter
  --fp                remove different parameters of the same endpoint
  --from string        fetch urls from date (format: YYYYMM)
  --ft strings         list of mime-types to filter
  --json              output as json
  --mc strings         list of status codes to match
  --mt strings         list of mime-types to match
  --o string           filename to write results to
  --providers strings  list of providers to use (wayback,commoncrawl,otx,urlscan)
  --proxy string       http proxy to use
  --retries uint       retries for HTTP client
  --subs              include subdomains of target domain
  --threads uint       number of workers to spawn (default 1)
  --timeout uint       timeout (in seconds) for HTTP client (default 45)
  --to string          fetch urls to date (format: YYYYMM)
  --verbose            show verbose output
  --version            show gau version
• (venv) prabesh@prabesh-Surface-Laptop-3:~/Documents/FINAL YEAR PROJECT$

```

Figure 248 Verified installation of GAU

```

--version          show gau version
○ (venv) prabesh@prabesh-Surface-Laptop-3:~/Documents/FINAL YEAR PROJECTS$ go install -v github.com/projectdiscovery/katana/cmd/katana@latest
go: github.com/projectdiscovery/katana@v1.3.0 requires go >= 1.24.1; switching to go1.24.11
github.com/projectdiscovery/utls/conversion
golang.org/x/exp/constraints
github.com/projectdiscovery/utls/race
github.com/projectdiscovery/gologger/levels
github.com/projectdiscovery/katana/pkg/engine
github.com/go-rod/rod/lib/assets
github.com/go-rod/rod/lib/js
github.com/ysmood/leakless/pkg/shared
github.com/projectdiscovery/utls/ptr
golang.org/x/exp/maps
github.com/projectdiscovery/utls/env
github.com/projectdiscovery/utls/os
github.com/projectdiscovery/utls/slice
github.com/projectdiscovery/utls/strings
github.com/projectdiscovery/utls/permission
github.com/projectdiscovery/utls/time
github.com/projectdiscovery/utls/errkit
github.com/projectdiscovery/gologger/formatter
github.com/projectdiscovery/gologger/writer
github.com/andybalholm/cascadia
github.com/go-rod/rod/lib/launcher/flags
github.com/go-rod/rod/lib/utls
github.com/ysmood/leakless/pkg/utls
github.com/ysmood/got/lib/lcs
github.com/go-rod/rod/lib/proto
github.com/projectdiscovery/fastdialer/fastdialer/ja3/impersonate
github.com/ysmood/leakless
github.com/projectdiscovery/utls/file
github.com/projectdiscovery/utls/maps
github.com/projectdiscovery/gologger
github.com/go-rod/rod/lib/defaults
github.com/projectdiscovery/utls/consts
github.com/projectdiscovery/utls/ip
github.com/go-rod/rod/lib/cdp
github.com/projectdiscovery/utls/context
github.com/projectdiscovery/utls/url
github.com/projectdiscovery/fastdialer/fastdialer/utls
github.com/zmap/zcrypto/x509
github.com/PuerkitoBio/goquery
github.com/smacker/go-tree-sitter
golang.org/x/exp/slices

```

Figure 249 Install Katana

```

github.com/projectdiscovery/katana/internal/runner
○ (venv) prabesh@prabesh-Surface-Laptop-3:~/Documents/FINAL YEAR PROJECTS$ katana -h
Katana is a fast crawler focused on execution in automation
pipelines offering both headless and non-headless crawling.

Usage:
  katana [flags]

Flags:
INPUT:
  -u, -list string[]  target url / list to crawl
  -resume string      resume scan using resume.cfg
  -e, -exclude string[] exclude host matching specified filter ('cdn', 'private-ips', cidr, ip, regex)

CONFIGURATION:
  -r, -resolvers string[] list of custom resolver (file or comma separated)
  -d, -depth int          maximum depth to crawl (default 3)
  -jc, -js-crawl          enable endpoint parsing / crawling in javascript file
  -jsl, -jsluice           enable jsluice parsing in javascript file (memory intensive)
  -ct, -crawl-duration value maximum duration to crawl the target for (s, m, h, d) (default s)
  -kf, -known-files value  enable crawling of known files (all,robotstxt,sitemapxml), a minimum depth of 3 is required to ensu
  -mrs, -max-response-size int maximum response size to read (default 4194304)
  -timeout int            time to wait for request in seconds (default 10)
  -time-stable int        time to wait until the page is stable in seconds (default 1)
  -aff, -automatic-form-fill enable automatic form filling (experimental)
  -fx, -form-extraction    extract form, input, textarea & select elements in jsonl output
  -retry int              number of times to retry the request (default 1)
  -proxy string            http/socks5 proxy to use
  -td, -tech-detect        enable technology detection
  -H, -headers string[]    custom header/cookie to include in all http request in header:value format (file)
  -config string           path to the katana configuration file
  -fc, -form-config string path to custom form configuration file
  -flc, -field-config string path to custom field configuration file
  -s, -strategy string      Visit strategy (depth-first, breadth-first) (default "depth-first")
  -iqp, -ignore-query-params Ignore crawling same path with different query-param values
  -tlsi, -tls-impersonate  enable experimental client hello (ja3) tls randomization
  -dr, -disable-redirects  disable following redirects (default false)
  -pc, -path-climb          enable path climb (auto crawl parent paths)

DEBUG:
  -health-check, -hc      run diagnostic check up
  -elog, -error-log string file to write sent requests error log
  -pprof-server            enable pprof server

```

Figure 250 Verified installation of katana


```

version                                display project version
(venv) prabesh@prabesh-Surface-Laptop-3:~/Documents/FINAL YEAR PROJECT$ go install -v github.com/jaeles-project/gospider@latest
github.com/golang/groupcache/lru
golang.org/x/net/html/atom
golang.org/x/text/internal/utf8internal
golang.org/x/text/encoding/internal/identifier
github.com/gobwas/glob/util/runes
google.golang.org/protobuf/internal/flags
google.golang.org/protobuf/internal/set
golang.org/x/sys/internal/unsafeheader
internal/saferio
hash/fnv
golang.org/x/text/transform
golang.org/x/text/internal/tag
github.com/gobwas/glob/util/strings
github.com/saintfish/chardet
google.golang.org/protobuf/internal/pragma
encoding/gob
encoding/xml
golang.org/x/net/html
golang.org/x/text/internal/language
golang.org/x/text/encoding
golang.org/x/text/runes
github.com/antchfx/xpath
golang.org/x/text/encoding/internal
github.com/gobwas/glob/match
golang.org/x/text/encoding/charmap
golang.org/x/text/encoding/japanese
golang.org/x/text/encoding/korean
github.com/andybalholm/cascadia
golang.org/x/text/encoding/simplifiedchinese
golang.org/x/text/encoding/traditionalchinese
golang.org/x/text/encoding/unicode
golang.org/x/text/internal/language/compact
github.com/gobwas/glob/syntax/lexer
github.com/PuerkitoBio/goquery
github.com/gocolly/colly/v2/debug
github.com/gobwas/glob/syntax/ast
golang.org/x/text/language
github.com/gobwas/glob/compiler
github.com/gobwas/glob/syntax
net/http/cookiejar

```

Figure 251 Install Gospider

```

github.com/jaeles-project/gospider/core
(venv) prabesh@prabesh-Surface-Laptop-3:~/Documents/FINAL YEAR PROJECT$ gospider -h
Fast web spider written in Go - v1.1.6 by @thebl4ckturtle & @j3ssiej3j

Usage:
  gospider [flags]

Flags:
  -s, --site string           Site to crawl
  -S, --sites string          Site list to crawl
  -p, --proxy string          Proxy (Ex: http://127.0.0.1:8080)
  -o, --output string         Output folder
  -u, --user-agent string     User Agent to use
                               web: random web user-agent
                               mobi: random mobile user-agent
                               or you can set your special user-agent (default "web")
  --cookie string             Cookie to use (testA=a; testB=b)
  -H, --header stringArray    Header to use (Use multiple flag to set multiple header)
  --burp string               Load headers and cookie from burp raw http request
  --blacklist string           Blacklist URL Regex
  --whitelist string           Whitelist URL Regex
  --whitelist-domain string    Whitelist Domain
  -L, --filter-length string   Turn on length filter
  -t, --threads int            Number of threads (Run sites in parallel) (default 1)
  -c, --concurrent int         The number of the maximum allowed concurrent requests of the matching domains (default 5)
  -d, --depth int              MaxDepth limits the recursion depth of visited URLs. (Set it to 0 for infinite recursion) (default 1)
  -k, --delay int              Delay is the duration to wait before creating a new request to the matching domains (second)
  -K, --random-delay int       RandomDelay is the extra randomized duration to wait added to Delay before creating a new request
  -m, --timeout int            Request timeout (second) (default 10)
  -B, --base                   Disable all and only use HTML content
  --js                         Enable linkfinder in javascript file (default true)
  --sitemap                   Try to crawl sitemap.xml
  --robots                     Try to crawl robots.txt (default true)
  -a, --other-source           Find URLs from 3rd party (Archive.org, CommonCrawl.org, VirusTotal.com, AlienVault.com)
  -w, --include-subdomains     Include subdomains crawled from 3rd party. Default is main domain
  -r, --include-other-source    Also include other-source's urls (still crawl and request)
  --subs                       Include subdomains
  --debug                      Turn on debug mode
  --json                       Enable JSON output
  -v, --verbose                Turn on verbose
  -q, --quiet                  Suppress all the output and only show URL
  --no-redirect                Disable redirect
  --version                    Check version

```

Figure 252 Verified installation of gospider

```

• (venv) prabesh@prabesh-Surface-Laptop-3:~/Documents/FINAL YEAR PROJECT$ go install -v github.com/ffuf/ffuf/v2@latest
container/ring
golang.org/x/net/html/atom
os/signal
github.com/adrg/xdg/internal/pathutil
github.com/ffuf/pencode/pkg/pencode
github.com/pelletier/go-toml
golang.org/x/net/html
github.com/adrg/xdg
net/http/httputil
github.com/andybalholm/brotli
github.com/andybalholm/cascadia
github.com/ffuf/ffuf/v2/pkg/ffuf
github.com/PuerkitoBio/goquery
github.com/ffuf/ffuf/v2/pkg/filter
github.com/ffuf/ffuf/v2/pkg/input
github.com/ffuf/ffuf/v2/pkg/interactive
github.com/ffuf/ffuf/v2/pkg/output
github.com/ffuf/ffuf/v2/pkg/scraper
github.com/ffuf/ffuf/v2/pkg/runner
• (venv) prabesh@prabesh-Surface-Laptop-3:~/Documents/FINAL YEAR PROJECT$ ffuf -h
Fuzz Faster U Fool - v1.1.0

HTTP OPTIONS:
-H          Header "Name: Value", separated by colon. Multiple -H flags are accepted.
-X          HTTP method to use (default: GET)
-b          Cookie data "NAME1=VALUE1; NAME2=VALUE2" for copy as curl functionality.
-d          POST data
-ignore-body Do not fetch the response content. (default: false)
-r          Follow redirects (default: false)
-recursion  Scan recursively. Only FUZZ keyword is supported, and URL (-u) has to end in it. (default: false)
-recursion-depth Maximum recursion depth. (default: 0)
-replay-proxy Replay matched requests using this proxy.
-timeout    HTTP request timeout in seconds. (default: 10)
-u          Target URL
-x          HTTP Proxy URL

GENERAL OPTIONS:
-V          Show version information. (default: false)
-ac         Automatically calibrate filtering options (default: false)
-acc        Custom auto-calibration string. Can be used multiple times. Implies -ac
-c          Colorize output. (default: false)
-maxtime    Maximum running time in seconds for entire process. (default: 0)
-maxtime-job Maximum running time in seconds per job. (default: 0)

```

Figure 253 Install and Verified ffuf tool

```

• (venv) prabesh@prabesh-Surface-Laptop-3:~/Documents/FINAL YEAR PROJECT$ sudo apt install -y cewl
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
cewl is already the newest version (5.5.2-1).
The following packages were automatically installed and are no longer required:
  golang-1.18 golang-1.18-doc golang-1.18-go golang-1.18-src golang-src libxdo3
Use 'sudo apt autoremove' to remove them.
0 upgraded, 0 newly installed, 0 to remove and 22 not upgraded.
• (venv) prabesh@prabesh-Surface-Laptop-3:~/Documents/FINAL YEAR PROJECT$ cewl --help
CeWL 5.5.2 (Grouping) Robin Wood (robin@ninja) (https://ninja/)
Usage: cewl [OPTIONS] ... <url>

OPTIONS:
-h, --help: Show help.
-k, --keep: Keep the downloaded file.
-d <x>, --depth <x>: Depth to spider to, default 2.
-m, --min_word_length: Minimum word length, default 3.
-o, --outfile: Let the spider visit other sites.
--exclude: A file containing a list of paths to exclude
--allowed: A regex pattern that path must match to be followed
-w, --write: Write the output to the file.
-u, --ua <agent>: User agent to send.
-n, --no-words: Don't output the wordlist.
-g <x>, --groups <x>: Return groups of words as well
--lowercase: Lowercase all parsed words
--with-numbers: Accept words with numbers in as well as just letters
--convert-umlauts: Convert common ISO-8859-1 (Latin-1) umlauts (ä-ae, ö-oe, ü-ue, ß-ss)
-a, --meta: include meta data.
--meta_file file: Output file for meta data.
-e, --email: Include email addresses.
--email_file <file>: Output file for email addresses.
--meta-temp-dir <dir>: The temporary directory used by exiftool when parsing files, default /tmp.
-c, --count: Show the count for each word found.
-v, --verbose: Verbose.
--debug: Extra debug information.

Authentication
--auth_type: Digest or basic.
--auth_user: Authentication username.
--auth_pass: Authentication password.

Proxy Support

```

Figure 254 Install and verified cewl

```
(venv) prabesh@prabesh-Surface-Laptop-3:~/Documents/FINAL YEAR PROJECT$ go install -v github.com/projectdiscovery/dnsx/cmd/dnsx@latest
github.com/projectdiscovery/utls/file
github.com/projectdiscovery/utls/maps
github.com/projectdiscovery/utls/reader
github.com/projectdiscovery/utls/errors
github.com/projectdiscovery/gologger
github.com/projectdiscovery/ratelimit
github.com/projectdiscovery/utls/url
github.com/projectdiscovery/utls/folder
github.com/projectdiscovery/utls/process
github.com/projectdiscovery/retryabledns
github.com/projectdiscovery/hmap/store/hybrid
github.com/projectdiscovery/utls/update
github.com/projectdiscovery/goflags
github.com/projectdiscovery/networkpolicy
github.com/projectdiscovery/fastdialer/fastdialer/metafiles
github.com/projectdiscovery/cdncheck
github.com/projectdiscovery/fastdialer/fastdialer
github.com/projectdiscovery/dnsx/libs/dnsx
github.com/projectdiscovery/retryablehttp-go
github.com/projectdiscovery/utls/auth/pdcp
github.com/projectdiscovery/asnmap/libs
github.com/projectdiscovery/mapcidr/asn
github.com/projectdiscovery/dnsx/internal/runner
github.com/projectdiscovery/dnsx/cmd/dnsx
• (venv) prabesh@prabesh-Surface-Laptop-3:~/Documents/FINAL YEAR PROJECT$ dnsx -h
dnsx is a fast and multi-purpose DNS toolkit allow to run multiple probes using retryabledns library.

Usage:
  dnsx [flags]

Flags:
INPUT:
  -l, -list string      list of sub(domains)/hosts to resolve (file or stdin)
  -d, -domain string    list of domain to bruteforce (file or comma separated or stdin)
  -w, -wordlist string   list of words to bruteforce (file or comma separated or stdin)

QUERY:
  -a                  query A record (default)
  -aaaa              query AAAA record
  -cname              query CNAME record
  -ns                 query NS record
  -txt                query TXT record
  -srv                query SRV record
  -ptr                query PTR record
```

Figure 255 Install and Verified dnsx

```
• (venv) prabesh@prabesh-Surface-Laptop-3:~/Documents/FINAL YEAR PROJECT$ go install -v github.com/projectdiscovery/shuffledns/cmd/shuffledns@latest
go: downloading github.com/projectdiscovery/shuffledns v1.2.1
go: github.com/projectdiscovery/shuffledns@v1.2.1 requires go >= 1.24.0; switching to go1.24.11
github.com/projectdiscovery/utls/io
github.com/projectdiscovery/shuffledns/pkg/parser
github.com/projectdiscovery/utls/batcher
github.com/projectdiscovery/shuffledns/pkg/store
github.com/projectdiscovery/dnsx/libs/dnsx
github.com/projectdiscovery/utls/update
github.com/projectdiscovery/shuffledns/pkg/wildcards
github.com/projectdiscovery/shuffledns/pkg/massdns
github.com/projectdiscovery/shuffledns/pkg/runner
github.com/projectdiscovery/shuffledns/cmd/shuffledns
• (venv) prabesh@prabesh-Surface-Laptop-3:~/Documents/FINAL YEAR PROJECT$ shuffledns -h
shuffledns is a wrapper around massdns written in go that allows you to enumerate valid subdomains using active bruteforce as well as resolve subdomains with
easy input-output support.

Usage:
  shuffledns [flags]

Flags:
INPUT:
  -d, -domain string[]  Domain to find or resolve subdomains for
  -ad, -auto-domain      Automatically extract root domains
  -l, -list string       File containing list of subdomains to resolve
  -w, -wordlist string    File containing words to bruteforce for domain
  -r, -resolver string    File containing list of resolvers for enumeration
  -tr, -trusted-resolver string File containing list of trusted resolvers
  -ri, -raw-input string  Validate raw full massdns output
  -mode string           Execution mode (bruteforce, resolve, filter)

RATE-LIMIT:
  -t int  Number of concurrent massdns resolves (default 10000)

UPDATE:
  -up, -update          update shuffledns to latest version
  -duc, -disable-update-check disable automatic shuffledns update check

OUTPUT:
  -o, -output string      File to write output to (optional)
  -j, -json                Make output format as ndjson
  -wo, -wildcard-output string Dump wildcard ips to output file
```

Figure 256 Installed and Verified Shuffledns

```

@asynccontextmanager
async def lifespan(app: FastAPI):
    """Application lifespan events."""
    import asyncio
    from app.core.database import AsyncSessionLocal
    from app.services.scheduled_scan_service import run_due_schedules

    setup_logging()
    await init_db()

    Windsurf: Refactor | Explain
    async def scheduled_scan_loop():
        """Every minute, run due scheduled scans (by next_run_at)."""
        while True:
            await asyncio.sleep(60) # 1 minute
            try:
                async with AsyncSessionLocal() as db:
                    await run_due_schedules(db)
            except asyncio.CancelledError:
                break
            except Exception as e:
                from app.core.logging import get_logger
                get_logger(__name__).exception("Scheduled scan loop error: %s", e)

        task = asyncio.create_task(scheduled_scan_loop())

        yield

        task.cancel()
        try:
            await task
        except asyncio.CancelledError:
            pass

    app = FastAPI(
        title="Intelligence Recon System (IRS)",
        description="AI-assisted OWASP Top 10 reconnaissance platform",
        version="1.0.0",
        lifespan=lifespan,
    )

    app.add_middleware(NoCacheStaticMiddleware)
    app.add_middleware(
        CORSMiddleware,
        allow_origins=["*"],
        allow_credentials=True,
        allow_methods=["*"],
        allow_headers=["*"],
    )

```

Figure 257 FastAPI Application Lifespan, Middleware and Router Registration

```

.env
1  # Intelligence Recon System (IRS) - Environment Configuration
2
3  GEMINI_API_KEY=your_api_key_here
4
5  GEMINI_MODEL=gemini-2.5-flash
6
7
8
9  # Tool Execution Settings (optional)
10 TOOL_TIMEOUT=300
11 MAX_CONCURRENT_TOOLS=3
12

```

Figure 258 IRS environment configuration (.env) for Gemini and tool settings

Go back to Implementation [Sprint 1](#).

7.3.2. Sprint 2 (Backend API and Architecture Development) and Sprint 3 (Intelligent Workflow Development)

Sprint 2 involved development of the backend core, endpoints, database structures and services to store and normalize scan results of multiple reconnaissance tools. It also brought about asynchronous orchestration in case scans could be performed in the background and status, detail and reporting endpoints to be available later to integrate with the frontend.

Sprint 3 was aimed at creating the intelligent scan workflow, beginning with Naabu and Httpx to gather early evidence such as open ports and HTTP services to each target. All these clues are then transferred to an AI decision layer that is expected to select the most applicable reconnaissance tools in each target and OWASP category. The decision logic has been incorporated in the backend orchestration in a way that its output is able to self-control what tools to operate and what to store as the outcome. The integration of Gemini is not fully completed, and API settings and timely tuning are required to make the full utilization of AI-guided tool selection possible.

```
# Create FastAPI app
app = FastAPI(
    title="Intelligence Recon System (IRS)",
    description="AI-assisted OWASP Top 10 reconnaissance platform",
    version="1.0.0",
    lifespan=lifespan
)

# Add CORS middleware
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"], # In production, specify allowed origins
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

Figure 259 Fast API and CORS setup

```

***Prompt templates for AI decision node.***

TOOL_DESCRIPTIONS = {
    "Nuclei": "Fast vulnerability scanner with extensive template library for detecting security issues across web applications, APIs, and network services. Excellent for all OWASP categories.",
    "Nmap": "High-speed port scanner to discover open ports and services. Essential for initial reconnaissance and A05 (Security Misconfiguration).",
    "Httpx": "HTTP probe to analyze web servers, status codes, headers, titles, and technologies. Critical for initial clues and all web-based OWASP categories.",
    "Subfinder": "Passive subdomain discovery tool using multiple sources. Essential for asset discovery (A01, A05, A10).",
    "Amass": "Comprehensive subdomain enumeration and network mapping. Deep reconnaissance for all categories.",
    "Assetfinder": "Fast subdomain discovery tool. Good for quick asset enumeration (A01, A05).",
    "Sublist3r": "Subdomain enumeration using search engines. Complements other subdomain tools.",
    "GAU": "GetAllURLs - fetches known URLs from AlienVault's OTX, Wayback Machine, and Common Crawl. Excellent for A01, A03, A05, A10.",
    "Katana": "Next-generation web crawler for gathering URLs, endpoints, and parameters. Essential for A01, A03, A05.",
    "GoSpider": "Fast web spider for crawling and extracting endpoints. Good for A01, A03, A07.",
    "FFuf": "Fast web fuzzer for directory/file brute-forcing and parameter discovery. Critical for A01, A03, A05.",
    "Wfuzz": "Web application fuzzer for security testing and OWASP attack vector discovery. Excellent for A01, A03, A04, A07 - specializes in access control and injection testing.",
    "CwM": "Custom wordlist generator from web content for password attacks. Useful for A07 (Authentication Failures).",
    "DNSx": "Fast DNS toolkit for DNS enumeration and validation. Important for A05, A10.",
    "ShuffleDNS": "Subdomain bruteforcer with wildcard detection. Useful when passive methods are insufficient."
}

OWASP_NUCLEI_TAGS = {
    "A01:2021": "unauth, traversal, lfi, idor, redirect",
    "A02:2021": "ssl, tls, expired-ssl, certificate",
    "A03:2021": "sql, xss, command-injection, ssti, xxe",
    "A04:2021": "logic-bypass, auth-bypass, access-control, misconfig",
    "A05:2021": "misconfig, cors, headers, exposure",
    "A06:2021": "cve, known-vuln, version-detect, outdated",
    "A07:2021": "default-login, auth-bypass, jwt, login",
    "A08:2021": "deserialization, file-upload, xxe, path-traversal",
    "A09:2021": "log4j, logging, exposure, debug-page",
    "A10:2021": "ssrf, graphql"
}

```

Figure 260 Tool Descriptions and OWASP Nuclei Tag Mapping Configuration

```

)

return ScanListResponse(**result)

@router.delete("/{scan_id}")
async def delete_scan(
    scan_id: int,
    db: AsyncSession = Depends(get_db)
):
    """Delete a scan and its associated stored data."""
    deleted = await ScanService.delete_scan(db, scan_id)
    if not deleted:
        raise HTTPException(status_code=404, detail="Scan not found")
    return JSONResponse(status_code=200, content={"detail": "Scan deleted"})

@router.post("/{scan_id}/retry", response_model=ScanResponse)
async def retry_scan(
    scan_id: int,
    db: AsyncSession = Depends(get_db)
):
    scan = await ScanService.retry_scan(db, scan_id)

    return ScanResponse(
        scan_id=scan.id,
        status=scan.status.value,
        message="Scan retry initiated. AI decision and tool execution in progress."
    )

```

Figure 261 Scan delete and retry endpoint for managing scans in IRS API

```

@router.get("/{scan_id}/status", response_model=ScanStatus)
async def get_scan_status(
    scan_id: int,
    db: AsyncSession = Depends(get_db)
):
    status = await ScanService.get_scan_status(db, scan_id)

    if not status:
        raise HTTPException(status_code=404, detail="Scan not found")

    return ScanStatus(**status)

@router.get("/{scan_id}", response_model=ScanDetailResponse)
async def get_scan_detail(
    scan_id: int,
    db: AsyncSession = Depends(get_db)
):
    detail = await ScanService.get_scan_detail(db, scan_id)

    if not detail:
        raise HTTPException(status_code=404, detail="Scan not found")

    return ScanDetailResponse(**detail)

@router.get("", response_model=ScanListResponse)
async def list_scans(
    target: Optional[str] = Query(None, description="Filter by target"),
    status: Optional[str] = Query(None, description="Filter by status"),
    page: int = Query(1, ge=1, description="Page number"),
    page_size: int = Query(20, ge=1, le=100, description="Items per page"),
    db: AsyncSession = Depends(get_db)
):
    result = await ScanService.list_scans(
        db=db,
        target=target,
        status=status,
        page=page,
        page_size=page_size
    )

```

Figure 262 Scan status, detail, and list endpoints querying IRS scan results

```

"""Scan-related schemas."""
from typing import List, Optional, Dict, Any
from datetime import datetime
from urllib.parse import urlparse
from pydantic import BaseModel, Field, field_validator

Windsurf: Refactor | Explain
class ScanCreate(BaseModel):
    """Request schema for creating a new scan."""
    target: str = Field(..., min_length=1, max_length=255, description="Target domain or host")
    owasp_category: str = Field(..., description="OWASP Top 10 category ID (e.g., A01:2021)")
    selected_tools: List[str] = Field(default_factory=list, description="List of tool names to run")

Windsurf: Refactor | Explain
@field_validator("target")
@classmethod
def validate_target(cls, v: str) -> str:
    """Validate and sanitize target."""
    if not isinstance(v, str):
        raise ValueError("Target must be a string")

    v = v.strip()
    if not v:
        raise ValueError("Target cannot be empty")

    if v.startswith("//"):
        v = "http:" + v

    if v.startswith(("http://", "https://")):
        parsed = urlparse(v)
        host = parsed.hostname
        port = parsed.port
        if host:
            v = f"{host}:{port}" if port else host
        else:
            v = parsed.netloc or parsed.path

    # Remove any remaining path/query/fragment after URL normalization.
    v = v.split("/", 1)[0].split("?", 1)[0].split("#", 1)[0].strip()
    v = v.lower().rstrip("/")

    if not v:
        raise ValueError("Target cannot be empty")
    return v

Windsurf: Refactor | Explain
@field_validator("owasp_category")
@classmethod
def validate_owasp(cls, v: str) -> str:
    """Validate OWASP category."""
    valid_categories = [
        "A01:2021", "A02:2021", "A03:2021", "A04:2021", "A05:2021",
        "A06:2021", "A07:2021", "A08:2021", "A09:2021", "A10:2021"
    ]
    if v not in valid_categories:
        raise ValueError(f"Invalid OWASP category. Must be one of: {', '.join(valid_categories)}")
    return v

```

Figure 263 Scan Request Schema with Target and OWASP Category Validation

```

class ToolRunStatus(str, enum.Enum):
    """Tool run status enumeration."""
    QUEUED = "queued"
    RUNNING = "running"
    COMPLETED = "completed"
    FAILED = "failed"
    TIMEOUT = "timeout"

class ToolRun(BaseModel):
    """ToolRun model representing execution of a single tool within a scan."""
    __tablename__ = "tool_runs"

    # Scan relationship
    scan_id = Column(Integer, ForeignKey("scans.id", ondelete="CASCADE"), nullable=False, index=True)
    scan = relationship("Scan", back_populates="tool_runs")

    # Tool information
    tool_name = Column(String(100), nullable=False, index=True)
    status = Column(SQLEnum(ToolRunStatus), default=ToolRunStatus.QUEUED, nullable=False)

    # Execution timing
    started_at = Column(DateTime, nullable=True)
    finished_at = Column(DateTime, nullable=True)

    # Output and results
    raw_output_path = Column(String(500), nullable=True) # Path to raw output file
    summary = Column(Text, nullable=True) # Short summary of results
    error_message = Column(Text, nullable=True) # Error message if failed

```

Figure 264 Tool run status enum and Tool Run model

```

"""Database configuration and session management."""
from typing import AsyncGenerator
import json
import time
from sqlalchemy import text
from sqlalchemy.ext.asyncio import (
    AsyncEngine,
    AsyncSession,
    create_async_engine,
    async_sessionmaker
)
from sqlalchemy.orm import declarative_base

from app.core.config import settings

# Create async engine
engine: AsyncEngine = create_async_engine(
    settings.DATABASE_URL,
    echo=False,
    future=True,
    pool_pre_ping=True
)

# Create async session factory
AsyncSessionLocal = async_sessionmaker(
    engine,
    class_=AsyncSession,
    expire_on_commit=False,
    autocommit=False,
    autoflush=False
)

# Declarative base for models
Base = declarative_base()

```

Figure 265 Asynchronous Database Engine and Session Configuration

```

class ScanService:
    """Service for managing scan lifecycle and orchestration."""

    @staticmethod
    async def create_scan(
        db: AsyncSession,
        target: str,
        owasp_category: str,
        selected_tools: List[str]
    ) -> Scan:
        # Create scan record
        scan = Scan(
            target=target,
            owasp_category=owasp_category,
            status=ScanStatus.CREATING,
            user_selected_tools=json.dumps(selected_tools)
        )

        db.add(scan)
        await db.commit()
        await db.refresh(scan)

        logger.info(f"Created scan {scan.id} for target={target}, owasp={owasp_category}")

        # Start background workflow
        asyncio.create_task(
            ScanService._execute_scan_workflow(scan.id, target, owasp_category, selected_tools)
        )

        return scan

```

Figure 266 Scan Service create_scan

```

@staticmethod
async def _gather_initial_clues(scan_id: int, target: str) -> Dict[str, Any]:
    """Run initial clues tools and gather reconnaissance data.

    Args:
        scan_id: Scan ID
        target: Target domain

    Returns:
        Dictionary of clues
    """
    clues = {
        "open_ports": [],
        "http_services": [],
        "server_headers": [],
        "page_titles": [],
        "status_codes": [],
        "technologies": []
    }

    # Run Naabu (port scan)
    try:
        naabu_result = await execute_tool("Naabu", target, scan_id, timeout=120)
        if naabu_result.success:
            ports = [f["location"].split(":")[-1] for f in naabu_result.findings]
            clues["open_ports"] = ports[:20] # Limit
    except Exception as e:
        logger.error(f"Naabu clues failed: {e}")

    # Run Httpx (HTTP probe)
    try:
        httpx_result = await execute_tool("Httpx", target, scan_id, timeout=120)
        if httpx_result.success:
            for finding in httpx_result.findings:
                try:
                    evidence = json.loads(finding.get("evidence", "{}"))
                    clues["http_services"].append(evidence.get("url", ""))
                    if evidence.get("title"):
                        clues["page_titles"].append(evidence["title"])
                    if evidence.get("status_code"):
                        clues["status_codes"].append(str(evidence["status_code"]))
                    if evidence.get("technologies"):
                        clues["technologies"].extend(evidence["technologies"])
                except json.JSONDecodeError:
                    pass
    except Exception as e:
        logger.error(f"Httpx clues failed: {e}")

    logger.info(f"Clues gathered: {clues}")
    return clues

```

Figure 267 Gather initial clues with Naabu and Httpx


```

@staticmethod
async def _execute_tools(
    db: AsyncSession,
    scan_id: int,
    target: str,
    tool_names: List[str]
) -> None:

    async def execute_one_tool(tool_name: str):

        # Get tool run record
        result = await db.execute(
            select(ToolRun).where(
                ToolRun.scan_id == scan_id,
                ToolRun.tool_name == tool_name
            )
        )
        tool_run = result.scalar_one_or_none()

        if not tool_run:
            logger.error(f"ToolRun not found for {tool_name}")
            return

        tool_run.status = ToolRunStatus.RUNNING
        tool_run.started_at = datetime.utcnow()
        await db.commit()

        tool_result = await execute_tool(tool_name, target, scan_id)

        tool_run.finished_at = datetime.utcnow()

        if tool_result.success:
            tool_run.status = ToolRunStatus.COMPLETED
            tool_run.summary = tool_result.summary
            tool_run.raw_output_path = str(
                settings.SCANS_DIR / str(scan_id) / f"{tool_name.lower()}.out"
            )

            for finding_data in tool_result.findings:
                finding = Finding(
                    scan_id=scan_id,
                    tool_name=tool_name,
                    type=finding_data.get("type", "information"),
                    severity=finding_data.get("severity", "info"),
                    owasp_category=finding_data.get("owasp_category"),
                    location=finding_data.get("location", ""),
                    description=finding_data.get("description", ""),
                    evidence=finding_data.get("evidence")
                )
                db.add(finding)
            else:
                if tool_result.error_message and "timeout" in tool_result.error_message.lower():
                    tool_run.status = ToolRunStatus.TIMEOUT
                else:
                    tool_run.status = ToolRunStatus.FAILED
                    tool_run.error_message = tool_result.error_message

            await db.commit()
            logger.info(f"Completed {tool_name}: {tool_run.status}")

        for tool_name in tool_names:
            await execute_one_tool(tool_name)

```

Figure 268 Execute recon tools and save findings

```

12
13 @dataclass
14 class AIDecision:
15     """AI decision result."""
16     tools_to_run: List[str]
17     tools_skipped: List[List[str, str]]
18     execution_batches: Optional[List[List[str]]] = None
19     reasoning: Optional[str] = None
20     raw_response: Optional[str] = None
21     success: bool = True
22     error: Optional[str] = None
23
24
25 async def decide_tools(
26     target: str,
27     owasp_category: str,
28     owasp_name: str,
29     selected_tools: List[str],
30     clues: Dict[str, Any]
31 ) -> AIDecision:
32
33     logger.info(f"Requesting AI decision for target={target}, owasp={owasp_category}")
34
35     if not settings.GEMINI_API_KEY:
36         logger.error("GEMINI API KEY not configured")
37         return _fallback_decision(selected_tools, "Gemini API key not configured")
38
39     try:
40         # Build prompt
41         prompt = _build_decision_prompt(target, owasp_category, owasp_name, selected_tools, clues)
42
43         # Call Gemini API
44         url = f"https://generativelanguage.googleapis.com/v1beta/models/{settings.GEMINI_MODEL}:generateContent?key={settings.GEMINI_API_KEY}"
45
46         payload = {
47             "contents": [
48                 {
49                     "parts": [
50                         {
51                             "text": prompt
52                         }
53                     ]
54                 },
55             ],
56             "generationConfig": {
57                 "temperature": 0.3,
58                 "maxOutputTokens": 2048,
59             }
60         }
61
62         async with httpx.AsyncClient(timeout=30.0) as client:
63             response = await client.post(url, json=payload)
64             response.raise_for_status()
65
66         result = response.json()
67
68         # Extract response text
69         try:
70             response_text = result["candidates"][0]["content"]["parts"][0]["text"]
71         except (KeyError, IndexError) as e:
72             logger.error(f"Unexpected Gemini response structure: {e}")
73             return _fallback_decision(selected_tools, "Invalid API response structure")
74
75         logger.info(f"Raw AI response: {response_text}")
76

```

Figure 269 AI decision class and Gemini API call

```

12
13 async def execute_tool(
14     tool_name: str,
15     target: str,
16     scan_id: int,
17     timeout: Optional[int] = None
18 ) -> ToolResult:
19
20     # Get tool class
21     tool_class = TOOL_REGISTRY.get(tool_name)
22
23     if not tool_class:
24         logger.error(f"Unknown tool: {tool_name}")
25         return ToolResult(
26             tool_name=tool_name,
27             success=False,
28             summary="Unknown tool",
29             error_message=f"Tool '{tool_name}' not found in registry"
30         )
31
32     # Create output file path
33     output_dir = settings.SCANS_DIR / str(scan_id)
34     output_dir.mkdir(parents=True, exist_ok=True)
35     output_file = output_dir / f"{tool_name.lower()}.out"
36
37     # Execute tool
38     tool: BaseTool = tool_class()
39     timeout_value = timeout or settings.TOOL_TIMEOUT
40
41     logger.info(f"Executing {tool_name} for scan {scan_id}, target={target}, timeout={timeout_value}s")
42
43     result = await tool.execute(
44         target=target,
45         output_file=output_file,
46         timeout=timeout_value
47     )
48

```

Figure 270 Execute a single tool with timeout and output file

[Click here](#) to go back Sprint 2, [Click Here](#) to go back Sprint 3

7.3.3. Sprint 4: Frontend Development

Sprint 4 included the entire frontend programming of the Intelligence Recon System in HTML, CSS, and JavaScript. Implementation aimed at creating the single page dashboard interface comprising of scan launch form, real time status tracking, API route configuration and the OWASP-to-tool mapping logic. The frontend interacts with the backend by making calls to a REST API as specified in app.js, and the ws_updates.py quietly maintains live status updates by polling.

```
const API_ROUTES = {
  config: () => "/api/config", // Fetch API config from backend
  createScan: () => "/api/scans",
  listScans: () => "/api/scans",
  scanDetail: (id) => `/api/scans/${id}`,
  scanStatus: (id) => `/api/scans/${id}/status`,
  scanIntelligence: (id) => `/api/scans/${id}/intelligence-summary`,
  health: () => "/api/health",
  reportHtml: (id) => `/api/scans/${id}/report`,
  reportPdf: (id) => `/api/scans/${id}/report/pdf`,
  markScanSaved: (id) => `/api/scans/${id}/mark-saved`,
  severityStatsCompleted: () => "/api/scans/stats/severity-completed",
  purgeScans: () => "/api/scans/purge",
  toolCommand: (toolName, target) => `/api/tools/${encodeURIComponent(toolName)}/commands${target ? `?target=${encodeURIComponent(target)}` : ""}`,
};
```

Figure 271 API Routes Configuration

```
Windsurf: Refactor | Explain
async function initializeApiConfig() {
  try {
    // Use relative fetch so it always stays same-origin (works in internal/external browsers).
    const response = await fetch("/api/config");
    if (response.ok) {
      const config = await response.json();
      // Only update apiBase if backend returns something compatible with the current origin.
      // Otherwise, keep the safe same-origin default (prevents 127.0.0.1/localhost from breaking external browsers).
      try {
        const curOrigin = (window.location.origin || "").replace(/\/$/, "");
        const cfgOrigin = (config.api_base_url || "").replace(/\/$/, "");
        if (curOrigin && cfgOrigin && curOrigin === cfgOrigin) {
          defaultApiBase = config.api_base_url;
          apiBase = config.api_base_url;
          console.log("✓ API Config loaded from backend:", apiBase);
        } else {
          apiBase = defaultApiBase;
        }
      } catch (_) {
        apiBase = defaultApiBase;
      }
    }
  } catch (error) {
    console.warn("⚠ Could not fetch API config from backend, using same-origin:", defaultApiBase);
    apiBase = defaultApiBase;
  }
}
```

Figure 272 API Config Initialization and Same-Origin Handling

```

let liveStatus = []; // Rolling snapshot list (max 6 updates)
let statusPollingIntervals = {}; // Per-scan polling intervals
let commandTracking = {}; // Per-tool command/output tracking
let fallbackNotificationsShown = {}; // Track fallback alerts by scanId
const LIVE_STATUS_UPDATE_INTERVAL_MS = 5000; // Throttle: only show updates every 5 seconds
const lastLiveStatusUpdateByScan = {}; // scanId -> timestamp
const lastLiveStatusDisplayTimeByScan = {}; // scanId -> displayed timestamp

// Initialize liveStatus with OWASP category field
liveStatus.forEach(item => {
  if (item.owaspCategory === undefined) {
    item.owaspCategory = null;
  }
});

// EventSource placeholder retained for compatibility with older code paths.
let eventSource = null;
let currentScanId = null;

// Real-time command tracking data
let toolStatus = {};
const ALLOWED_THEMES = ["dark-soc", "light-minimal"];
let activeTheme = () => {
  const stored = localStorage.getItem("irs_theme") || "dark-soc";
  return ALLOWED_THEMES.includes(stored) ? stored : "dark-soc";
}();
let lastScanId = null;
let scanSummaryInterval = null;

```

Figure 273 Live Status Tracker State and Polling Variable

```

Windsurf: Refactor | Explain | Generate JSDoc
function createOrUpdateProgressWidget(scanId, progressPercent, statusText) {
  let widget = document.getElementById('scanProgressWidget');
  if (!widget) {
    widget = document.createElement('div');
    widget.id = 'scanProgressWidget';
    widget.innerHTML = `
      <div class="scan-progress-ring" data-percent="${progressPercent}">
        <svg viewBox="0 0 36 36" class="circular-chart" aria-hidden="true">
          <path class="circle-bg" d="M18 2.0845
            a 15.9155 15.9155 0 0 1 0 31.831
            a 15.9155 15.9155 0 0 1 0 -31.831" />
          <path class="circle" stroke-dasharray="${progressPercent},100" d="M18 2.0845
            a 15.9155 15.9155 0 0 1 0 31.831
            a 15.9155 15.9155 0 0 1 0 -31.831" />
          </svg>
          <span class="scan-progress-label">${progressPercent}%</span>
        </div>
        <div class="scan-progress-text">${statusText} || 'Running'</div>
      `;
    document.body.appendChild(widget);
    initProgressWidgetDrag(widget);
  }
  const ring = widget.querySelector('.scan-progress-ring');
  if (ring) {
    ring.dataset.percent = progressPercent;
    const path = ring.querySelector('.circle');
    if (path) {
      path.setAttribute('stroke-dasharray', `${progressPercent},100`);
    }
  }
  const label = widget.querySelector('.scan-progress-label');
  if (label) label.textContent = `${progressPercent}%`;
  const text = widget.querySelector('.scan-progress-text');
  if (text) text.textContent = statusText || 'Running';
}

Windsurf: Refactor | Explain | Generate JSDoc
function removeProgressWidget() {
  const widget = document.getElementById('scanProgressWidget');
  if (widget) {
    widget.remove();
  }
}

```

Figure 274 Scan Progress Widget and Tracking

```

function getApiUrl(path) {
  // Always use same-origin relative API URLs to prevent host mismatch
  // (e.g., internal works on localhost/127.0.0.1 but external uses 0.0.0.0).
  if (!path) return path;
  if (String(path).startsWith("http://") || String(path).startsWith("https://")) return path;
  if (String(path).startsWith("/")) return path;
  return (apiBase || window.location.origin) + "/" + path;
}

Windsurf: Refactor | Explain | Generate JSDoc
async function apiRequest(path, options = {}) {
  const url = getApiUrl(path);
  const resp = await fetch(url, options);
  if (!resp.ok) {
    const text = await resp.text();
    // Try to parse JSON error response to extract detail field
    try {
      const errorData = JSON.parse(text);
      // Extract only the detail/message value, nothing else
      const detail = errorData.detail || errorData.message;
      if (detail) {
        throw new Error(detail);
      } else {
        throw new Error(text);
      }
    } catch (e) {
      // If not JSON or no detail field, use the raw text
      if (e instanceof Error && e.message !== text) {
        throw e;
      }
      throw new Error(text);
    }
  }
  const contentType = resp.headers.get("content-type") || "";
  if (contentType.includes("application/json")) {
    return await resp.json();
  }
  return await resp.text();
}

```

Figure 275 API Request Handler and URL Builder

```

Windsurf: Refactor | Explain
function connectSSE(scanId) {
  if (eventSource) {
    eventSource.close();
    eventSource = null;
  }
  currentScanId = scanId;
  console.debug(`SSE is disabled; using polling for scan ${scanId}`);
}

Windsurf: Refactor | Explain | Generate JSDoc
function disconnectSSE() {
  if (eventSource) {
    eventSource.close();
    eventSource = null;
    currentScanId = null;
  }
}

```

Figure 276 SSE Connection and Disconnect Handler

```

155 <!-- Row 1: Launch Recon Scan -->
156 <div class="card">
157   <div class="card-header">
158     <span class="card-title"><svg class="card-icon" aria-hidden="true"><use href="#icon-launch"/></svg> Launch Recon Scan</span>
159   </div>
160   <div class="card-body">
161     <form id="launchForm" class="launch-form">
162       <label class="field">
163         <span class="field-label">Target Host</span>
164         <input type="text" id="targetInput" placeholder="domain.com or IP address" required>
165       </label>
166       <label class="field">
167         <span class="field-label">OWASP Category</span>
168         <select id="owaspSelect" required>
169           <option value="">Select category...</option>
170           <option value="A01:2021">A01:2021 Broken Access Control</option>
171           <option value="A02:2021">A02:2021 Cryptographic Failures</option>
172           <option value="A03:2021">A03:2021 Injection</option>
173           <option value="A04:2021">A04:2021 Insecure Design</option>
174           <option value="A05:2021">A05:2021 Security Misconfiguration</option>
175           <option value="A06:2021">A06:2021 Vulnerable and Outdated Components</option>
176           <option value="A07:2021">A07:2021 Identification and Authentication Failures</option>
177           <option value="A08:2021">A08:2021 Software and Data Integrity Failures</option>
178           <option value="A09:2021">A09:2021 Security Logging and Monitoring Failures</option>
179           <option value="A10:2021">A10:2021 Server-Side Request Forgery</option>
180         </select>
181       </label>
182       <div class="field">
183         <span class="field-label">Recon Toolchain Selection</span>
184         <div id="toolCheckboxGrid" class="tool-grid"></div>
185       </div>
186       <div class="launch-actions">
187         <button type="submit" id="runScanBtn" class="btn-primary"><svg class="btn-icon" aria-hidden="true"><use href="#icon-scan"/></svg> Run Full Scan</button>
188         <button type="button" id="clearLaunchBtn" class="btn-ghost">Clear</button>
189       </div>
190       <p class="field-help" id="launchHelpText">
191         Selected tools and OWASP category will be validated by AI before execution.
192       </p>
193     </form>
194   </div>
195 </div>

```

Figure 277 Scan Launch Form HTML Structure

[Go back](#)

7.3.4. Sprint 5: Reporting Module Development

Sprint 5 used the entire reporting pipeline including the integration of Gemini AI to produce professional HTML and PDF reports based on completed scan data. Report service is used to perform in-memory caching to avoid repeated AI call, whereas report generator is used to perform multi-backend retry logic and deduplication finding. api/reports.py report endpoints provide support of scan persistence and HTML and PDF generation.

```

@staticmethod
async def generate_html_report(
    db: AsyncSession, scan_id: int, for_pdf: bool = False
):
    """Generate HTML report for a scan.

    - If report was already generated (stored in scan.report_html or in-memory cache),
      returns it without calling AI. Re-clicking View Report never calls AI after first generation.
    - First time: runs AI, builds HTML, persists to scan.report_html, then returns.
    - When for_pdf=True, returns (full_html, front_html, main_html) for PDF merge so
      main content page numbers start at 1. Otherwise returns full_html (str).
    """
    if scan_id in _REPORT_HTML_CACHE and not for_pdf:
        return _REPORT_HTML_CACHE[scan_id]

    # Get scan data
    result = await db.execute(select(Scan).where(Scan.id == scan_id))
    scan = result.scalar_one_or_none()

    if not scan:
        return None

    _CURRENT_REPORT_MARKER = "irs-summary-page"
    _REPORT_VERSION_MARKER = "irs-report-pdf-v4"
    report_is_current = False
    if (
        scan.report_html
        and scan.report_html.strip()
        and _CURRENT_REPORT_MARKER in scan.report_html
        and _REPORT_VERSION_MARKER in scan.report_html
    ):

```

Figure 278 HTML Report Generation and Cache Check

```

    if scan.updated_at <= scan.created_at:
        report_is_current = True
    if report_is_current and not for_pdf:
        _REPORT_HTML_CACHE[scan_id] = scan.report_html
        return scan.report_html
    if scan.report_html and (
        _CURRENT_REPORT_MARKER not in scan.report_html
        or _REPORT_VERSION_MARKER not in scan.report_html
    ):
        scan.report_html = None
        await db.commit()

    # Get tool runs
    result = await db.execute(
        select(ToolRun)
        .where(ToolRun.scan_id == scan_id)
        .order_by(ToolRun.tool_name)
    )
    tool_runs = result.scalars().all()

    result = await db.execute(
        select(Finding)
        .where(Finding.scan_id == scan_id)
        .order_by(Finding.severity, Finding.type)
    )
    findings = result.scalars().all()

    deduped_findings = {}
    for f in findings:
        key = (
            (f.location or "").strip().lower(),
            (f.description or "").strip().lower(),
        )
        current = deduped_findings.get(key)
        if not current:
            deduped_findings[key] = f
            continue
        if ReportService.severity_rank(f.severity.value if hasattr(f.severity, "value") else str(f.severity)) > ReportService.severity_rank(
            current.severity.value if hasattr(current.severity, "value") else str(current.severity)
        ):
            deduped_findings[key] = f

    deduped_findings_list = list(deduped_findings.values())

```

Figure 279 Report Validation, Tool Runs and Findings Deduplication

```

deduped_findings_list = list(deduped_findings.values())

tools_to_run = json.loads(scan.ai_tools_to_run) if scan.ai_tools_to_run else []
user_tools = json.loads(scan.user_selected_tools) if scan.user_selected_tools else []
tools_used = tools_to_run or user_tools or [tr.tool_name for tr in tool_runs]
severity_counts = {"critical": 0, "high": 0, "medium": 0, "low": 0, "info": 0}
for f in deduped_findings_list:
    sev = f.severity.value if hasattr(f.severity, "value") else str(f.severity)
    if sev in severity_counts:
        severity_counts[sev] += 1
# Top findings for the AI executive summary: deduplicated by (severity, description)
seen_top = set()
top_findings = []
for f in deduped_findings_list:
    sev = f.severity.value if hasattr(f.severity, "value") else str(f.severity)
    desc = (f.description or "")[:150]
    key = (sev, desc)
    if key in seen_top:
        continue
    seen_top.add(key)
    top_findings.append({"severity": sev, "description": desc})
    if len(top_findings) >= 10:
        break
owasp_name = next(
    (c["name"] for c in settings.OWASP_CATEGORIES if c["id"] == scan.owasp_category),
    scan.owasp_category,
)

```

Figure 280 Severity Counts and Top Findings Builder

```

Windsurf: Refactor | Explain
async def generate_report_executive_summary(
    target: str,
    owasp_category: str,
    owasp_name: str,
    severity_counts: Dict[str, int],
    tools_used: List[str],
    top_findings: List[Dict[str, Any]],
) -> str:
    """Generate AI executive summary for the scan report. AI only; no fallback."""
    _require_gemini_api_key()

    prompt = f"""You are a professional security analyst. Write a concise 2-4 sentence executive summary for a security scan report.

Scan context:
- Target: {target}
- Attack Type: {owasp_category} - {owasp_name}
- Tools used: {'; '.join(tools_used)} if tools used else 'N/A'
- Severity breakdown: Critical={severity_counts.get('critical',0)},
  High={severity_counts.get('high',0)}, Medium={severity_counts.get('medium',0)}, Low={severity_counts.get('low',0)}, Info={severity_counts.get('info',0)}

Top findings (if any):
{chr(10).join(f"- {f.get('severity','')}: {f.get('description','')[:100]}" for f in top_findings[:5]) if top_findings else "None"}

Write a professional executive summary. Example style: "The target domain X was analyzed
for OWASP Top 10 vulnerabilities. Multiple medium and high severity issues were identified including [types].
Immediate remediation is recommended."
If no serious issues: "The target was analyzed. No critical or high severity vulnerabilities were detected. [X] informational
findings were documented."
Output ONLY the summary paragraph, no headings or labels."""

    try:
        payload = {
            "contents": [{"parts": [{"text": prompt}]}],
            "generationConfig": {"temperature": 0.3, "maxOutputTokens": 512},
        }
        resp = await gemini_post_with_retry(payload, timeout=15.0)
        text = resp.json()["candidates"][0]["content"][0]["text"]
        return text.strip()
    except Exception as e:
        logger.warning("AI executive summary failed: %s", e)
        raise

```

Figure 281 AI Executive Summary Generation with Gemini


```

Windsurf: Refactor | Explain
async def generate_report_conclusion(
    target: str,
    severity_counts: Dict[str, int],
    findings_count: int,
    recommendations: List[str],
) -> str:
    """Generate AI conclusion and remediation summary for the report. AI only; no fallback."""
    _require_gemini_api_key()

    prompt = f"""You are a security consultant. Write a 3-5 sentence conclusion and remediation summary for a security scan report.

Context:
- Target: {target}
- Total findings: {findings_count}
- Severity: Critical={severity_counts.get('critical',0)}, High={severity_counts.get('high',0)},
Medium={severity_counts.get('medium',0)}, Low={severity_counts.get('low',0)}, Info={severity_counts.get('info',0)}

Existing recommendations to incorporate:
{chr(10).join('- ' + r for r in recommendations[:8]) if recommendations else 'None'}

Write a professional conclusion that summarizes the security posture and provides actionable remediation guidance. I
nclude technical implementation options where relevant (e.g., parameterized queries for SQLi, security headers, input validation).
Output ONLY the conclusion paragraph(s), no headings."""

    try:
        payload = {
            "contents": [{"parts": [{"text": prompt}]}],
            "generationConfig": {"temperature": 0.3, "maxOutputTokens": 1024},
        }
        resp = await _gemini_post_with_retry(payload, timeout=20.0)
        text = resp.json()["candidates"][0]["content"]["parts"][0]["text"]
        return text.strip()
    except Exception as e:
        logger.warning("AI conclusion failed: %s", e)
        raise

```

Figure 282 AI Conclusion and Remediation Summary Generation

[Go back.](#)

7.3.5. Sprint 6: Integration and Improve

Sprint 6 completed the system by adding the scheduled scan service, AI triage validation and finding normalization logic. The scan scheduling service computes the scan frequency and automatically initiates due scan. The AI validation module checks the accuracy of tool execution and the finding normalization logic checks that the appropriate OWASP category is scoped in all scan results.

```

"""Run due scheduled scans - used by API and by the in-process scheduler."""
import json
from datetime import datetime, timedelta

from sqlalchemy import select
from sqlalchemy.ext.asyncio import AsyncSession

from app.core.logging import get_logger
from app.models.scheduled_scan import ScheduledScan
from app.services.scan_service import ScanService

logger = get_logger(__name__)

Windsurf: Refactor | Explain
def _next_run_after(last_run: datetime, frequency: str) -> datetime | None:
    """Compute next run time after last_run. Returns None for 'once'."""
    if frequency == "once":
        return None
    if frequency == "daily":
        return last_run + timedelta(days=1)
    if frequency == "weekly":
        return last_run + timedelta(days=7)
    if frequency == "monthly":
        # Run again after 30 days
        return last_run + timedelta(days=30)
    return None

```

Figure 283 Scheduled Scan Imports and Frequency Calculator

```

async def run_due_schedules(db: AsyncSession) -> int:
    """
    Find all enabled scheduled scans where next_run_at <= now and start a scan for each.

    Behaviour:
    - If the server was online at the scheduled time (within a small grace window),
      the scan is created as normal.
    - If the scheduled time is in the past beyond the grace window (server was down),
      the run is considered "missed": no scan is created, and next_run_at is advanced
      to the next occurrence (or the schedule is disabled for 'once').

    Returns the number of scans started.
    """
    now = datetime.utcnow()
    # How far past the scheduled time we still consider a run "on time".
    # Anything older than this is treated as a missed run and skipped.
    grace_window = timedelta(minutes=2)
    result = await db.execute(
        select(ScheduledScan).where(
            ScheduledScan.enabled.is_(True),
            ScheduledScan.next_run_at.isnot(None),
            ScheduledScan.next_run_at <= now,
        )
    )
    schedules = result.scalars().all()
    started = 0
    for s in schedules:
        # If this schedule's next_run_at is far in the past, treat it as missed.
        if s.next_run_at and s.next_run_at < now - grace_window:
            next_run = _next_run_after(now, s.frequency)
            if next_run is None:
                # 'once' schedule missed while server was down: disable without running
                s.enabled = False
                s.next_run_at = None
            else:
                s.next_run_at = next_run
            s.last_run_at = None
            await db.commit()
            logger.info(
                f"Scheduled scan {s.id} missed its window at {s.next_run_at} and was skipped; "
                f"next run is scheduled at {s.next_run_at} or 'disabled'"
            )
            continue
        tools_str = s.selected_tools
        if isinstance(tools_str, str):
            try:
                tools_list = json.loads(tools_str) if tools_str else []
            except Exception:
                tools_list = []
        else:
            tools_list = list(tools_str) if tools_str else []

        scan = await ScanService.create_scan(
            db=db,
            target=s.target,
            owasp_category=s.owasp_category,
            selected_tools=tools_list,
            scheduled_scan_id=s.id,
        )
        s.last_run_at = now
        next_run = _next_run_after(now, s.frequency)
        if next_run is None:
            # once: disable after run
            s.enabled = False
            s.next_run_at = None
        else:
            s.next_run_at = next_run
        await db.commit()
        started += 1
        logger.info(f"Scheduled scan {s.id} triggered scan {scan.id} for {s.target}")
    except ValueError as e:
        logger.warning(f"Scheduled scan {s.id} skipped: {e}")
        continue
    except Exception as e:
        logger.exception(f"Scheduled scan {s.id} failed: {e}")
        continue
    return started

```

Figure 284 Due Schedule Runner and Missed Scan Handler

```

else:
    tools_list = list(tools_str) if tools_str else []
if not tools_list:
    continue
try:
    # scheduled_scan_id set - scan runs as scheduled flow (no clues, no AI; run chosen tools only)
    scan = await ScanService.create_scan(
        db=db,
        target=s.target,
        owasp_category=s.owasp_category,
        selected_tools=tools_list,
        scheduled_scan_id=s.id,
    )
    s.last_run_at = now
    next_run = _next_run_after(now, s.frequency)
    if next_run is None:
        # once: disable after run
        s.enabled = False
        s.next_run_at = None
    else:
        s.next_run_at = next_run
    await db.commit()
    started += 1
    logger.info(f"Scheduled scan {s.id} triggered scan {scan.id} for {s.target}")
except ValueError as e:
    logger.warning(f"Scheduled scan {s.id} skipped: {e}")
    continue
except Exception as e:
    logger.exception(f"Scheduled scan {s.id} failed: {e}")
    continue
return started

```

Figure 285 Scan Trigger, Schedule Update and Error Handling

[Go back.](#)

7.4. APPENDIX 4: DESIGN

7.4.1. Gantt Chart

7.4.1.1. Proposal Gantt Chart

The following screenshots are the images of Gantt chart which was created on proposal phase:

1. Project Planning

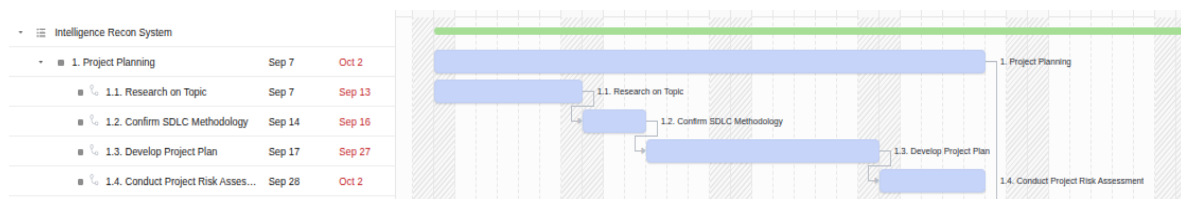


Figure 286 Project Planning

2. Requirement Analysis

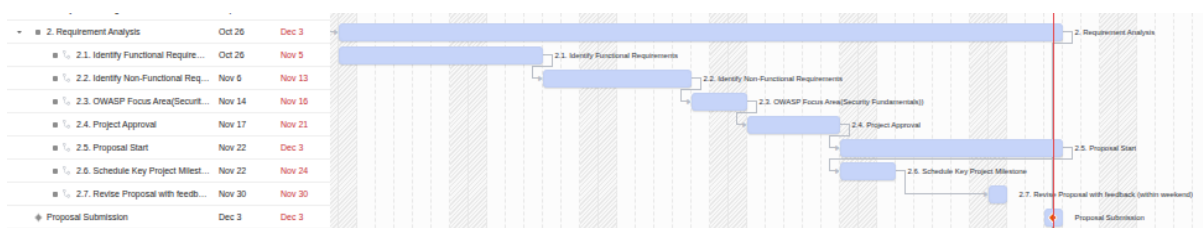


Figure 287 Requirement Analysis

3. Sprint 1

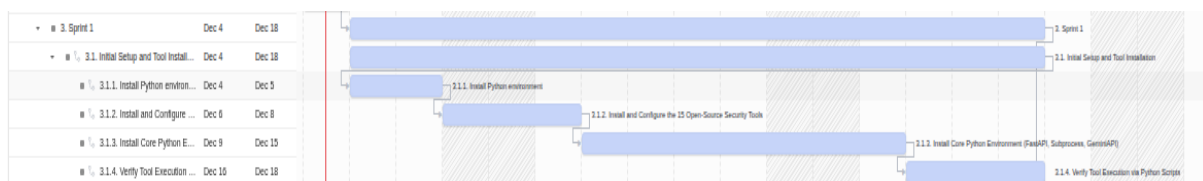


Figure 288 Sprint 1

4. Sprint 2

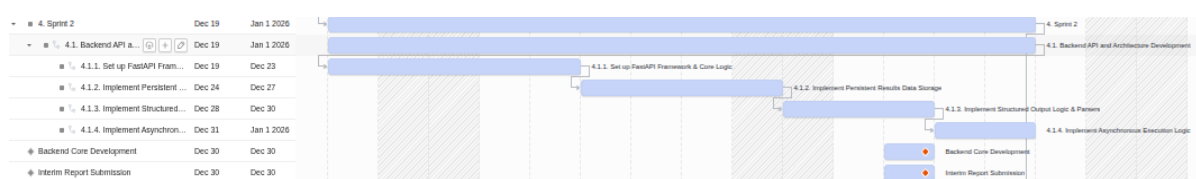


Figure 289 Sprint 2

5. Sprint 3

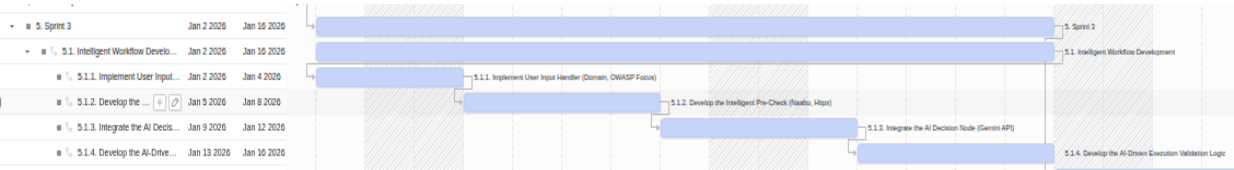


Figure 290 Sprint 3

6. Sprint 4

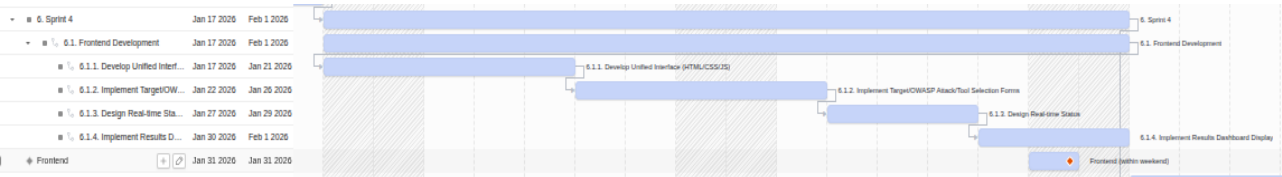


Figure 291 Sprint 4

7. Sprint 5

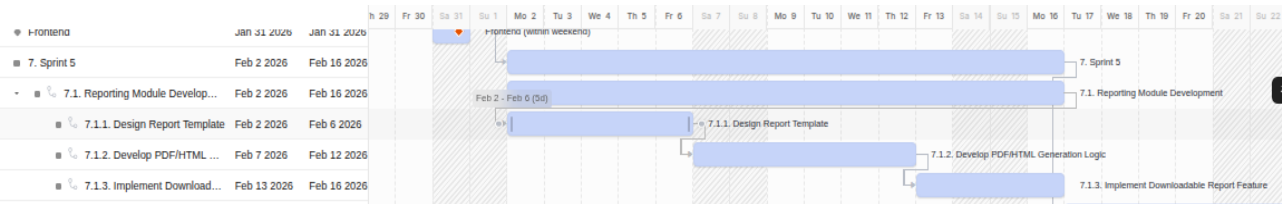


Figure 292 Sprint 5

8. Sprint 6

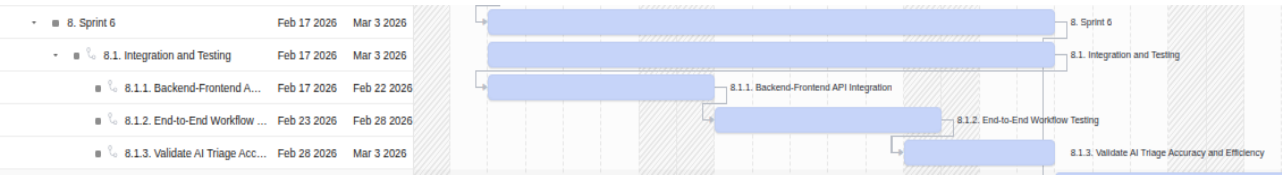


Figure 293 Sprint 6

9. Control

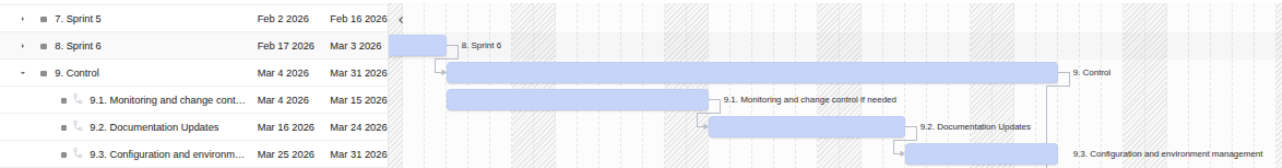


Figure 294 Control.

10. Closure

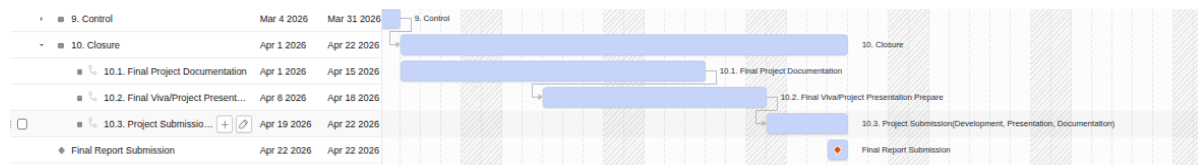


Figure 295 Closure!

7.4.1.2. Interim Report Gantt Chart

There is a deviation in the updated Gantt because Sprint 2 was done earlier than expected. This led to the early initiation of some Sprint 3 activities. These changes show speed up progress and ensure that the remaining sprints and final deliveries are well organized in the new schedule.

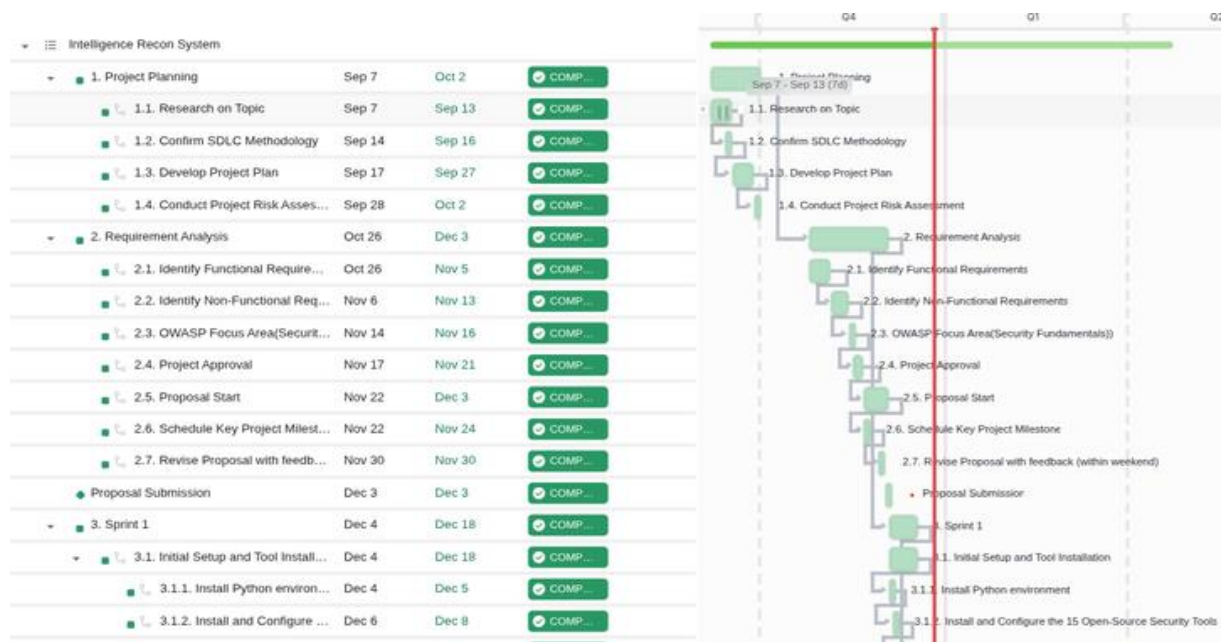


Figure 296 Interim Report Gantt Chart

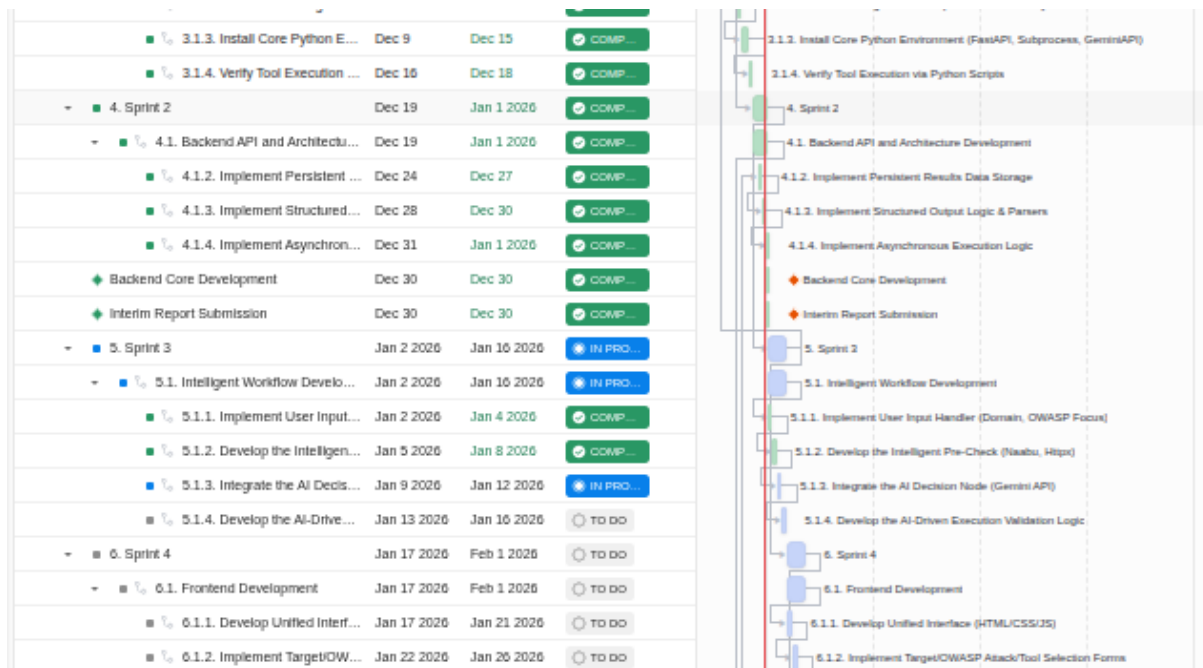


Figure 297 Interim Report Gantt Chart

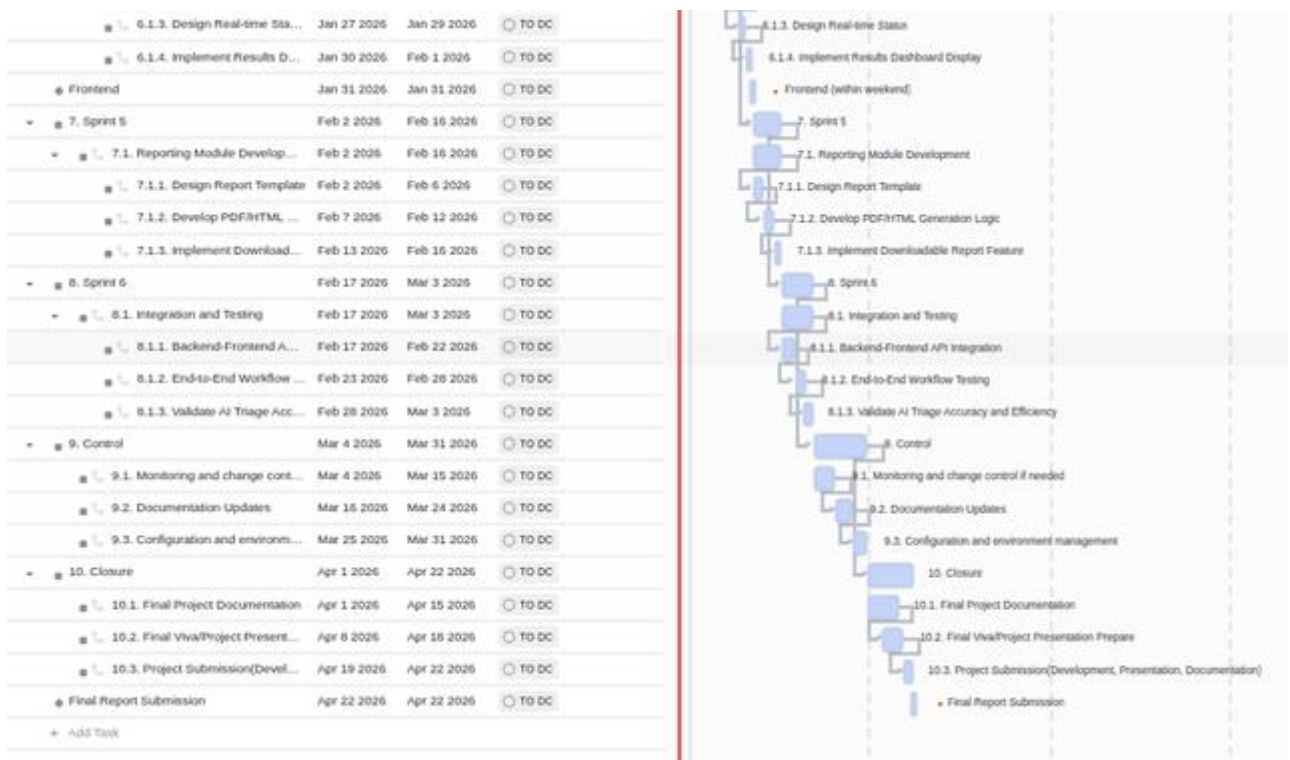


Figure 298 Interim Report Gantt Chart

Tasks	Start Date	End Date	Status
Project Planning	9/7/2025	10/2/2025	Completed
Research on Topic	9/7/2025	9/13/2025	Completed
Confirm SDLC Methodology	9/14/2025	9/26/2025	Completed
Develop Project Plan	9/17/2025	9/27/2025	Completed
Conduct Project Risk Assessment	9/28/2025	10/2/2025	Completed
Requirement Analysis	10/26/2025	12/3/2025	Completed
Identify Functional Requirements	10/26/2025	11/5/2025	Completed
Identify Non-Functional Requirements	11/6/2025	11/13/2025	Completed
OWASP Focus Area (Security Fundamentals)	11/14/2025	11/16/2025	Completed
Project Approval	11/17/2025	11/21/2025	Completed
Proposal Start	11/22/2025	12/3/2025	Completed
Schedule Key Project Milestone	11/22/2025	11/24/2025	Completed
Revise Proposal with feedback	11/30/2025	11/30/2025	Completed
Proposal Submission	12/3/2025	12/3/2025	Completed
Sprint 1	12/4/2025	12/18/2025	Completed
Initial Setup and Tool Installation	12/4/2025	12/4/2025	Completed
Install Python environment	12/4/2025	12/5/2025	Completed
Install and configure the 15 Open-Source Security Tools	12/6/2025	12/8/2025	Completed
Install Core Python Environment (Fast API, Subprocess, Gemini API)	12/9/2025	12/15/2025	Completed
Verify Tool Execution via Python Scripts	12/16/2025	12/18/2025	Completed
Sprint 2	12/19/2025	1/1/2026	Completed
Backend API and Architecture Development	12/19/2025	1/1/2026	Completed
Set up Fast API Framework & Core Logic	12/19/2025	12/23/2025	Completed
Implement Persistent Results Data Storage	12/24/2025	12/27/2025	Completed
Implement Structured Output Logic & Parsers	12/28/2025	12/30/2025	Completed
Implement Asynchronous Execution Logic	12/31/2025	1/1/2026	Completed
Backend Core Development	12/30/2025	12/30/2025	Completed
Interim Report Submission	12/30/2025	12/30/2025	Completed
Sprint 3	1/2/2026	1/16/2026	In progress
Intelligent Workflow Development	1/2/2026	1/16/2026	In progress
Implement User Input Handler (Domain, OWASP Focus)	1/2/2026	1/4/2026	Completed
Develop the Intelligent Pre-Check (Naabu, Httpx)	1/5/2026	1/8/2026	Completed
Integrate the AI Decision Node (Gemini API)	1/9/2026	1/12/2026	In progress
Develop the AI-Driven Execution Validation Logic	1/13/2026	1/16/2026	To Do
Sprint 4	1/17/2026	2/1/2026	To Do
Frontend Development	1/17/2026	2/1/2026	To Do
Develop Unified Interface (HTML/CSS/JS)	1/17/2026	1/21/2026	To Do
Implement Target/OWASP Attack/Tool Selection Forms	1/22/2026	1/26/2026	To Do
Design Real-time Status	1/27/2026	1/29/2026	To Do
Implement Results Dashboard Display	1/30/2026	2/1/2026	To Do
Frontend	1/31/2026	1/31/2026	To Do

Figure 299 Interim Report Gantt Chart in Detail

Sprint 5	2/2/2026	2/16/2026	To Do
Reporting Module Development	2/2/2026	2/16/2026	To Do
Design Report Template	2/2/2026	2/6/2026	To Do
Develop PDF/HTML Generation Logic	2/7/2026	2/12/2026	To Do
Implement Downloadable Report Feature	2/13/2026	2/16/2026	To Do
Sprint 6	2/17/2026	3/3/2026	To Do
Integration and Testing	2/17/2026	3/3/2026	To Do
Backend-Frontend API Integration	2/17/2026	2/22/2026	To Do
End-to-End Workflow Testing	2/23/2026	2/28/2026	To Do
Validate AI Triage Accuracy and Efficiency	2/28/2026	3/3/2026	To Do
Control	3/4/2026	3/31/2026	To Do
Monitoring and change control if needed	3/4/2026	3/15/2026	To Do
Documentation Updates	3/16/2026	3/24/2026	To Do
Configuration and environment management	3/25/2026	3/31/2026	To Do
Closure	4/1/2026	4/22/2026	To Do
Final Project Documentation	4/1/2026	4/15/2026	To Do
Final Viva/Project Presentation Prepare	4/8/2026	4/18/2026	To Do
Project Submission (Development, Presentation, Documentation)	4/19/2026	4/22/2026	To Do
Final Report Submission	4/22/2026	4/22/2026	To Do

Figure 300 Updated Gantt Chart in detail

To go back [Click here](#).

7.4.1.3. Reasons for Switch Gantt Chart Tool

The Gantt chart was originally created with ClickUp, but a large amount of its functionality, such as updating tasks and timelines, was limited by its premium plan. Due to this limitation, it was not possible to properly track and update project progress. Considering the importance of keeping the project under constant changes and the inability to upgrade the project because of financial issues, the decision was made to switch to TeamGantt. TeamGantt was easier to access, update on a regular basis, and offered a more transparent and easier to use means of controlling the project schedule.

[Click Here](#) to go back.

7.4.2. WORK BREAKDOWN STRUCTURE

7.4.2.1. PREVIOUS WORK BREAKDOWN STRUCTURE

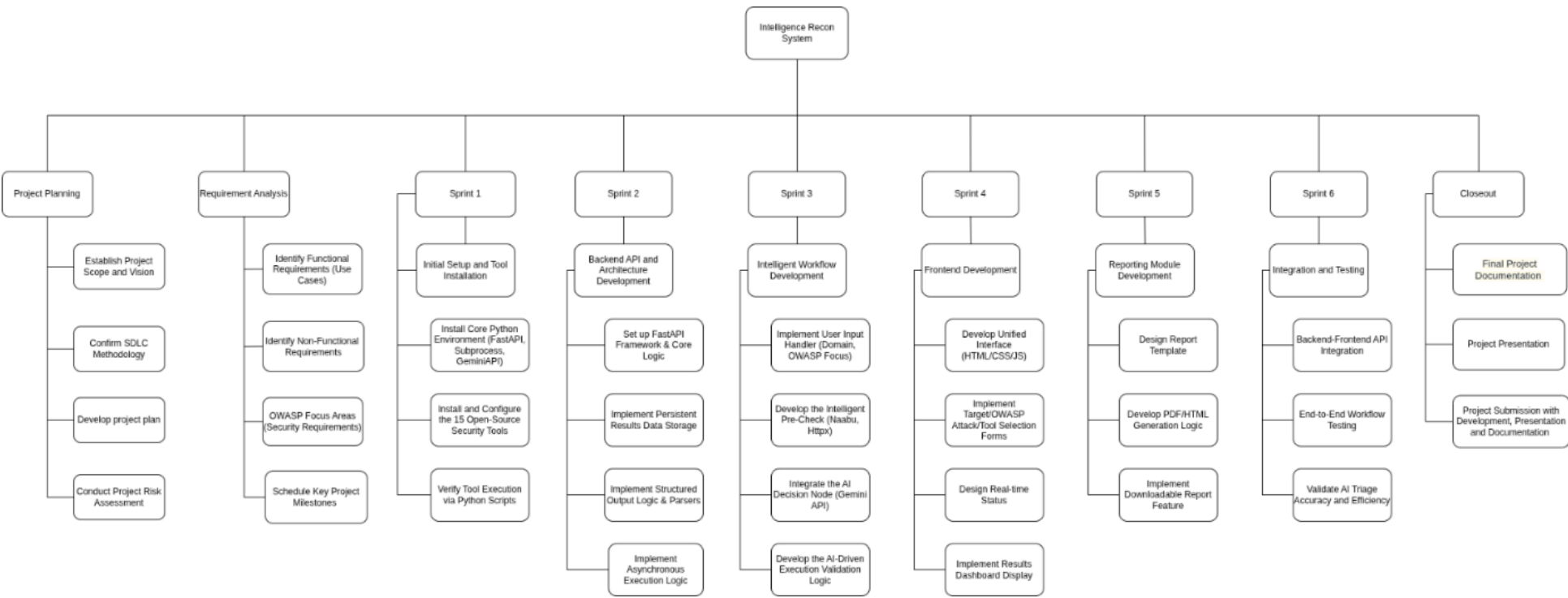


Figure 301 Previous WBS

7.4.2.2. INTERIM REPORT WORK BREAKDOWN STRUCTURE

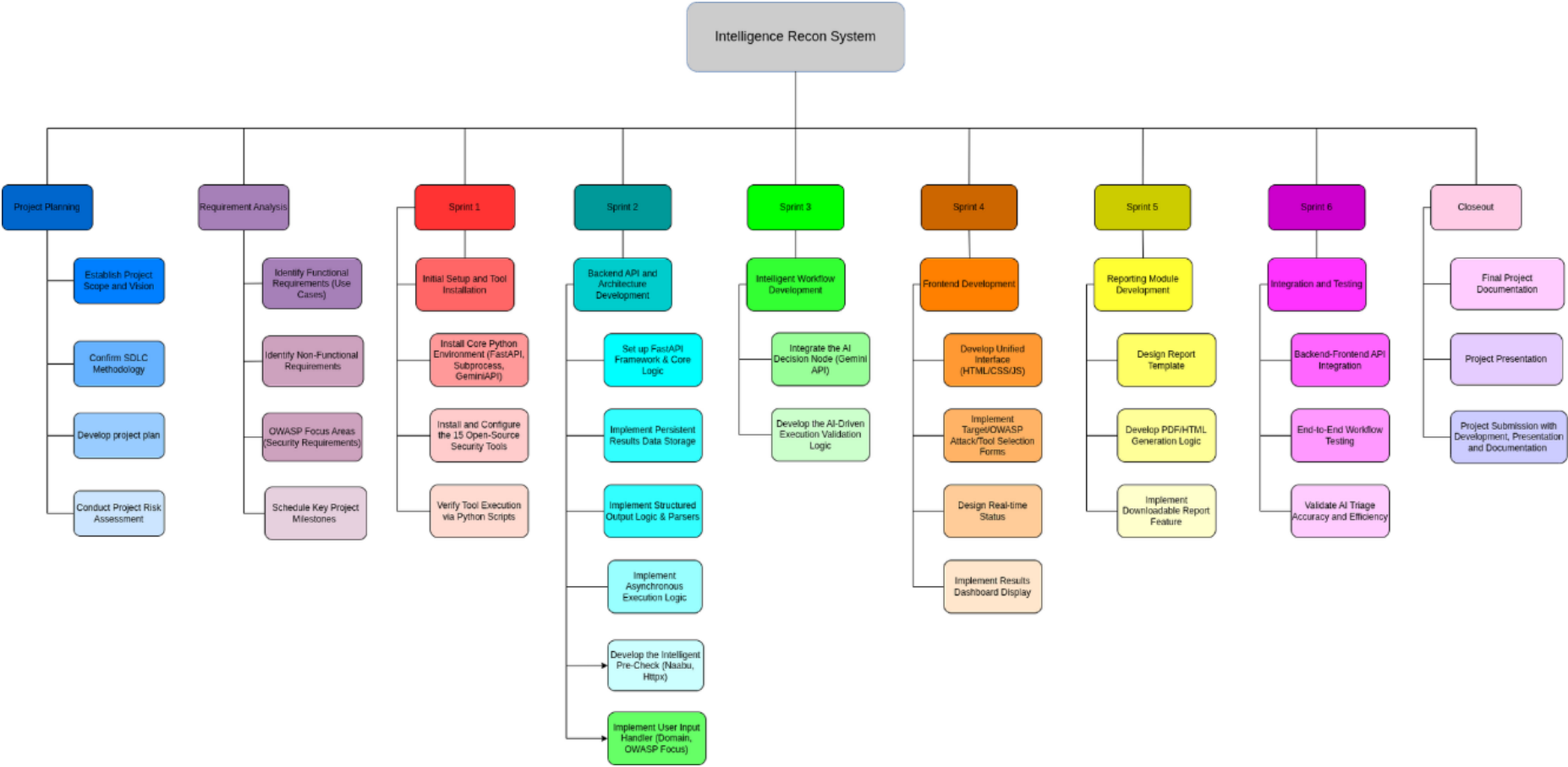


Figure 302 Updated WBS

7.4.2.3. MAIN REPORT UPDATED WORK BREAKDOWN STRUCTURE

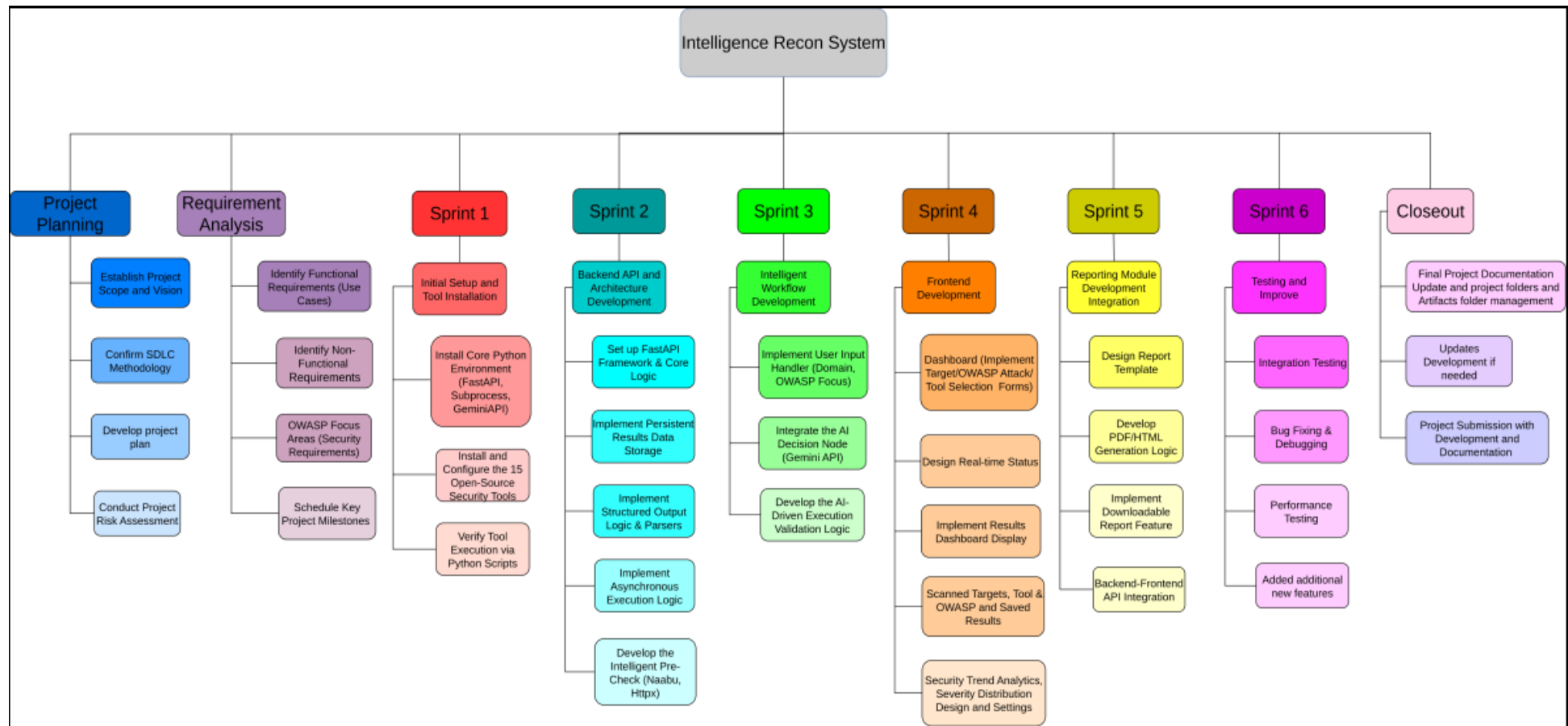


Figure 303 Updated Work Breakdown Structure

[Click Here](#) to go back.

7.4.3. MILESTONE

The project milestone is a reference or benchmark to showcase a significant goal, event, or task in a project lifecycle (Staff, 2025).

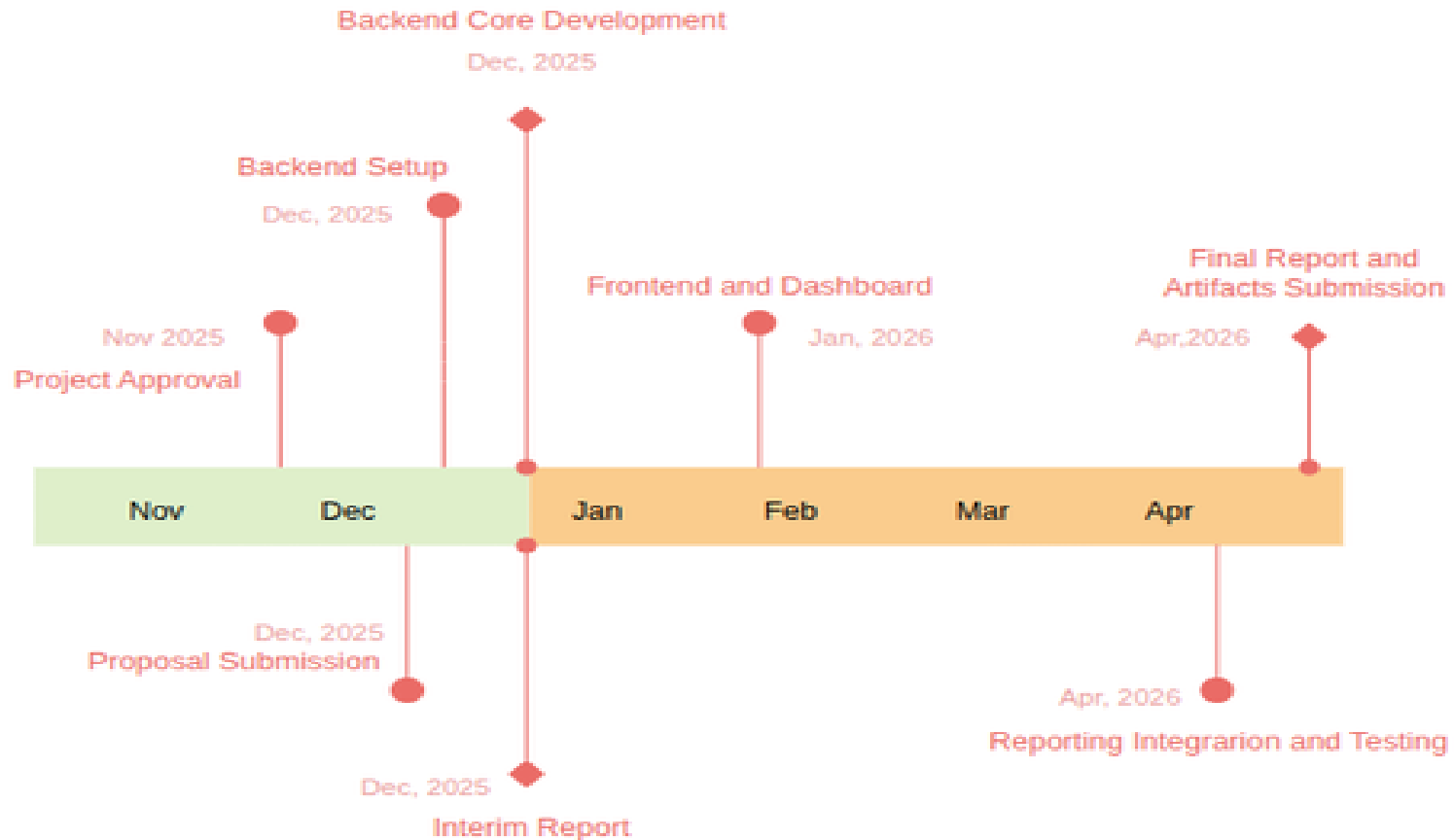


Figure 304 Milestone

[Click Here](#) to go back

Milestone 1: Project Approval

The milestone includes finding the topic of the project, clarifying the project goals, and getting the approval of the supervisor and department. It involves picking the Intelligence Recon System as the project name, preparing the original concept, and definition of the problem, scope and planned results of the research and development part.

Milestone 2: Proposal Submission

This milestone is dedicated to the preparation and submission of the official project proposal. It contains a record of background research, problem analysis, literature review, and system objectives. The proposal identifies the intended methodology, system architecture, tools and technologies to be employed and an approximate timeframe of the development with assessment criteria.

Milestone 3: Backend Setup

This milestone is concerned with defining the initial framework of system development. It involves the installation of the back-end framework with Fast API, the initial database setup, setting up the environment variables, and developing the bare bones of the application in which functionalities and testing will be performed.

Milestone 4: Backend Core Development

The milestone is dedicated to the creation and development of the main logic of the backend, which is the logic behind the functioning of the system. It involves the application of data models, API routing and scan management logic, integration of AI modules, connection to database and coordination of real time processes. The milestone will make sure that the backend will be able to support the process of the tools, data storage, and secure transactions.

Milestone 5: Interim Report Submission

This milestone includes the process of preparing the interim report to show the general progress, analysis, and current development status. It also involves brief reporting of system design, methodology, initial implementation findings, issues facing and modification of the following plan of work as per interim evaluation findings.

Milestone 6: Frontend and Dashboard

This milestone is aimed at creating the interface of the Intelligence Recon System. It involves the design of the dashboard with the help of HTML, CSS, and JavaScript, and the development of the sections to control targets, the choice of the tools, and the input of the OWASP categories. It also includes interface responsiveness, accessibility design, and support to backend APIs to be responsive in real-time.

Milestone 7: Reporting Integration and Testing

This milestone focuses on incorporation of user reporting capability and system level testing. It involves writing structured HTML-based reports, connecting the frontend and backend modules to generate outputs and testing important processes, including scanning, AI-based decision-making, and errors, as well as overall system performance.

Milestone 8: Final Report and Artifacts Submission

This includes final report creation and submission of all final project documentation and deliverables. It comprises the creation of the full system report, the presentation of materials, and technical documentation such as code, configuration files, or user manual. And the Final Report and all project folders will be submitted early 2 days of deadline due to the completion of all development and report.

[Click here](#) to go back.

7.4.4. FLOWCHART

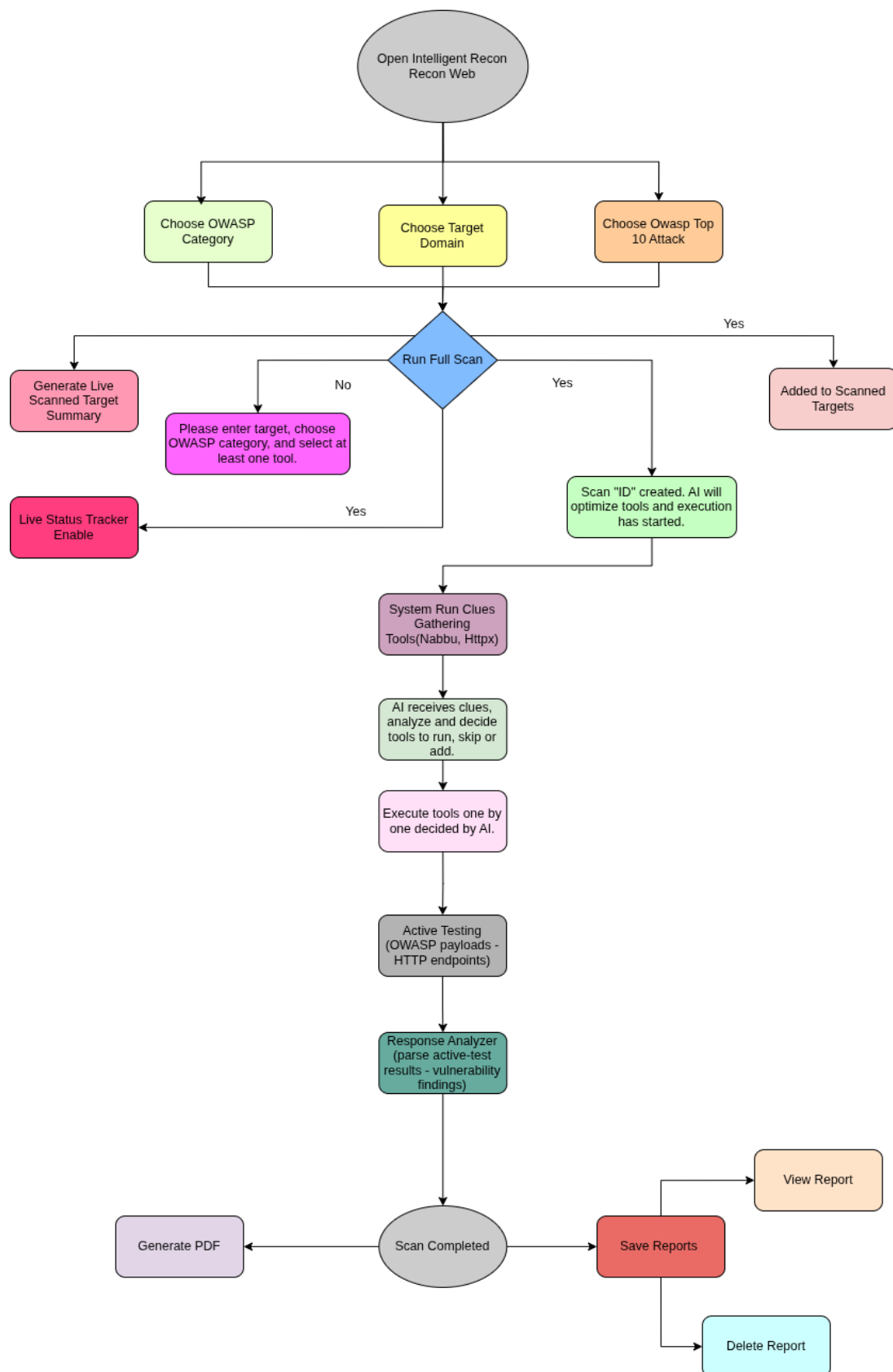


Figure 305 Clear System Flow Chart

[Go back](#)

7.4.5. SYSTEM ARCHITECTURE DIAGRAM

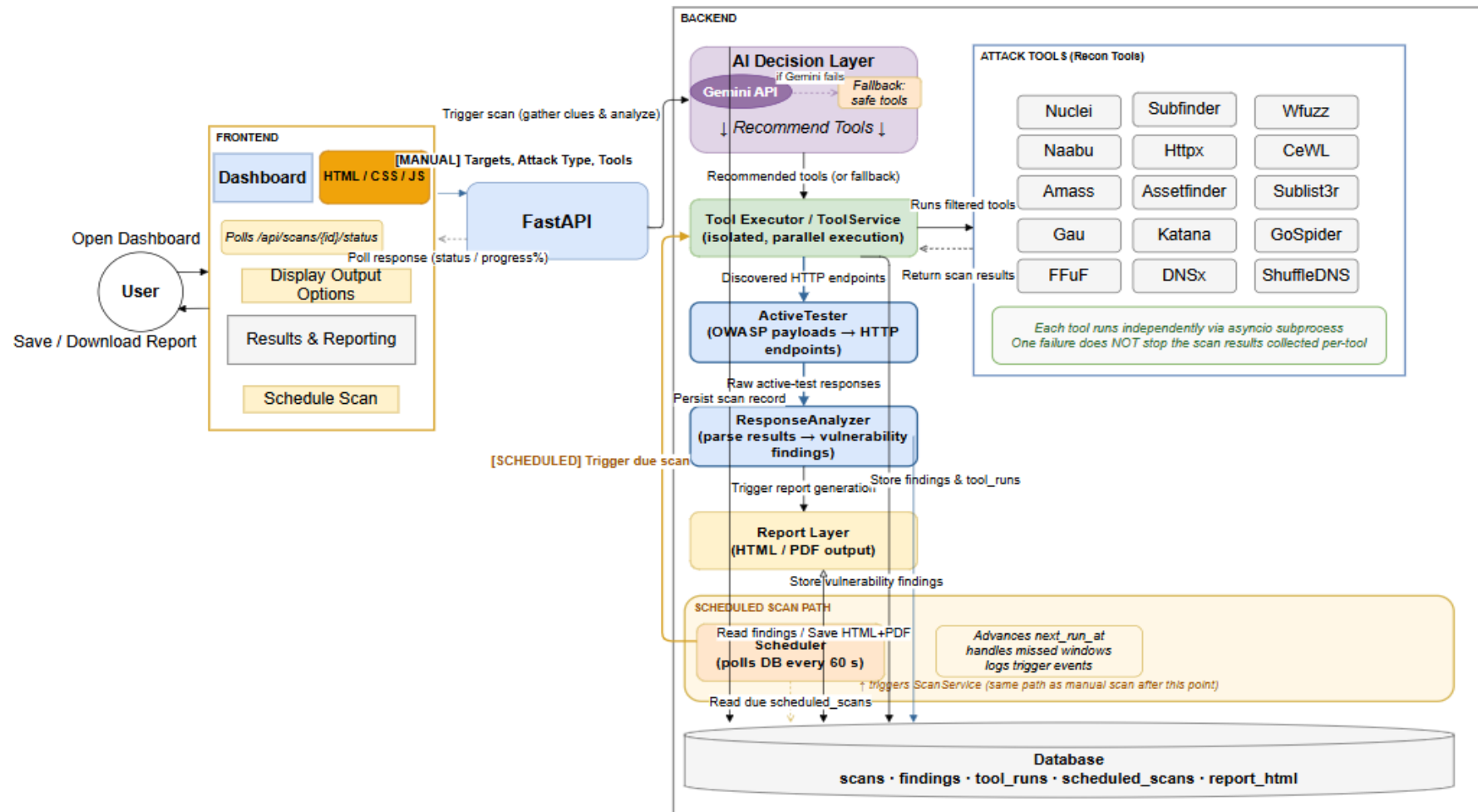


Figure 306 Clear System Architecture Diagram

[Go back](#)

7.5. APPENDIX 5: SCREENSHOTS OF THE SYSTEM

The screenshots below give the full user interface of the Intelligence Recon System (IRS). These images provide the representation of the most important pages and elements of the created system such as the main dashboard, scan execution, live status tracking, saved reports, scheduled scans, settings, and report generation.

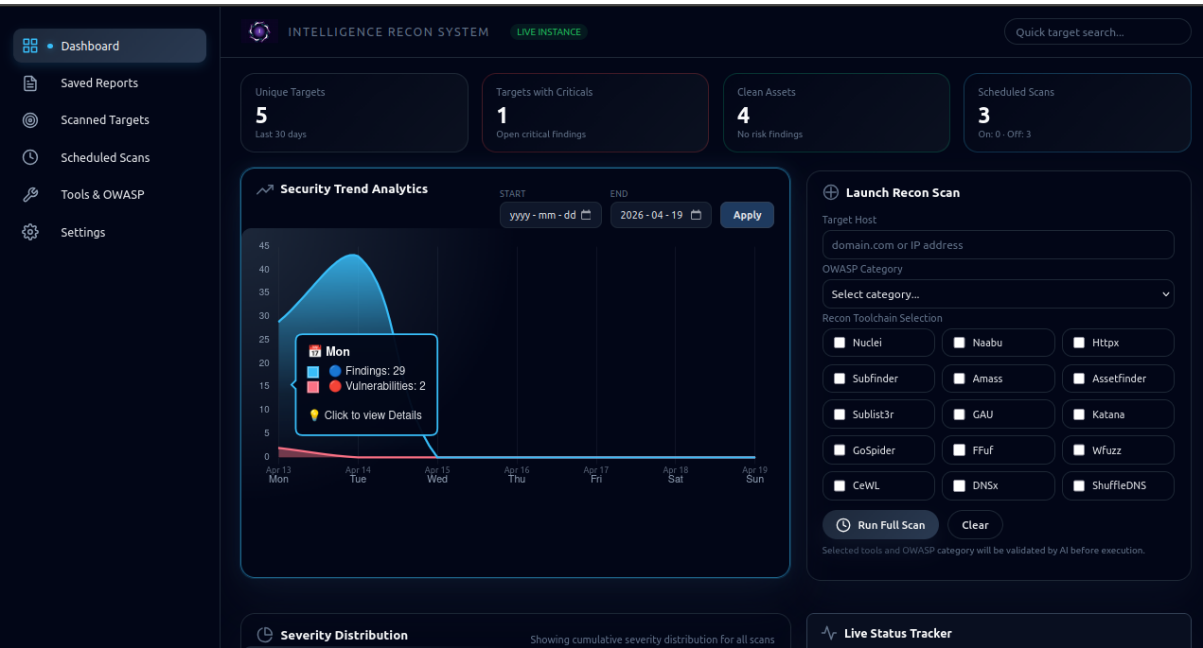


Figure 307 Dashboard of IRS

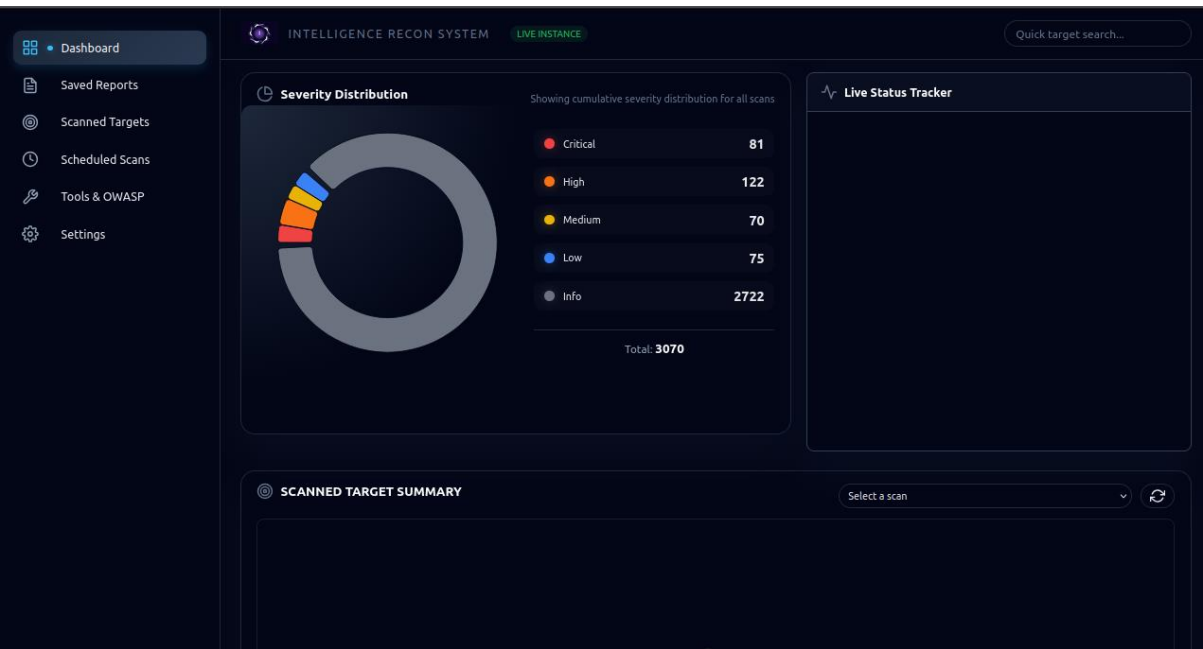


Figure 308 Dashboard of IRS

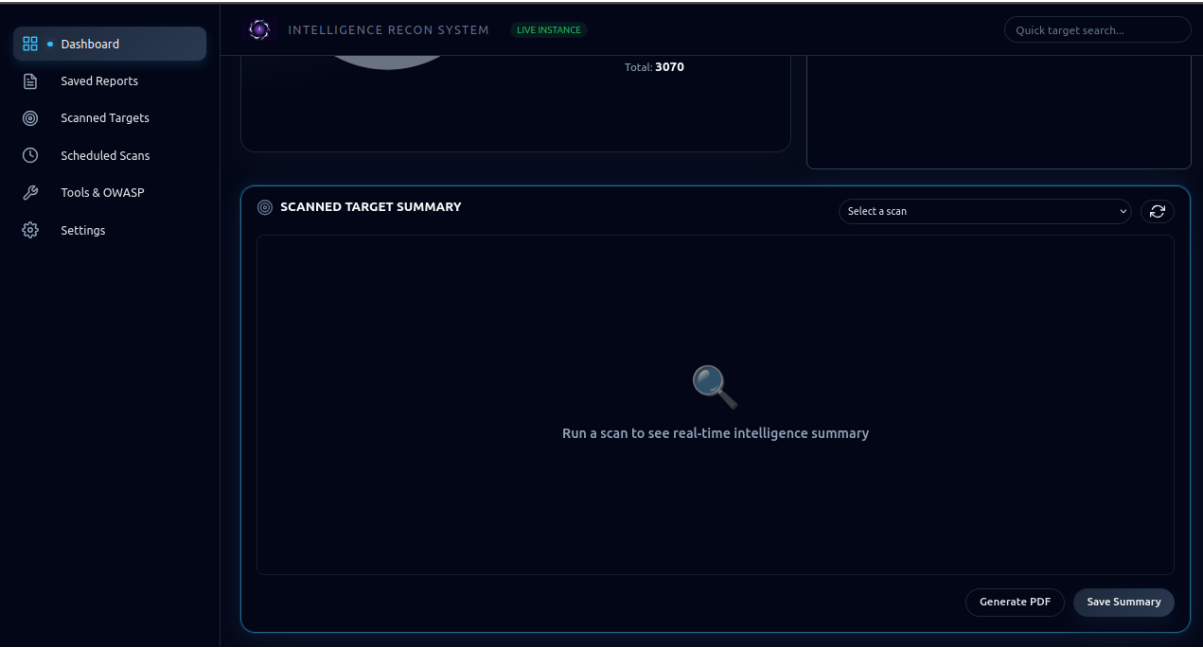


Figure 309 Dashboard Output Section of IRS

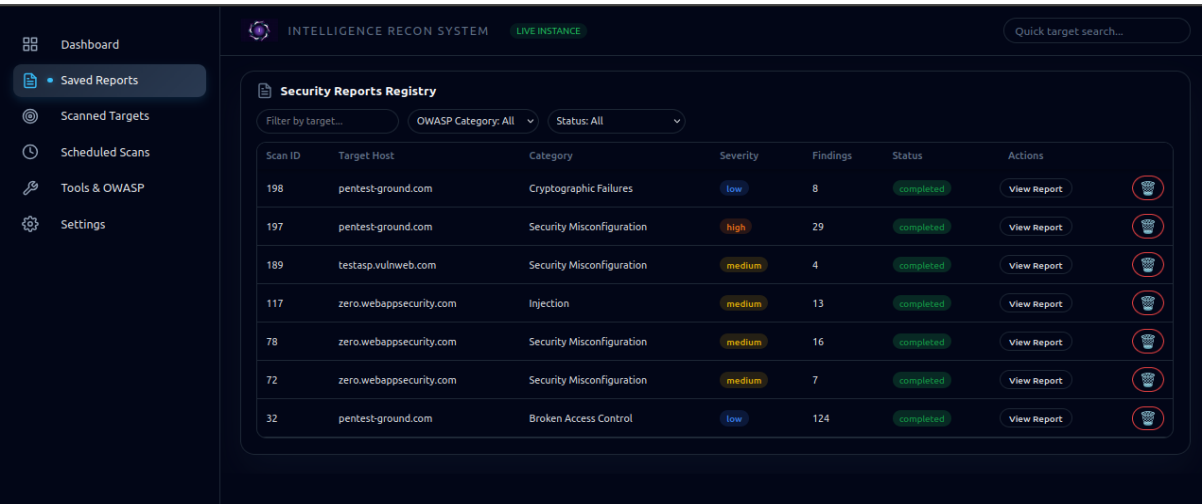


Figure 310 Saved Reports of IRS

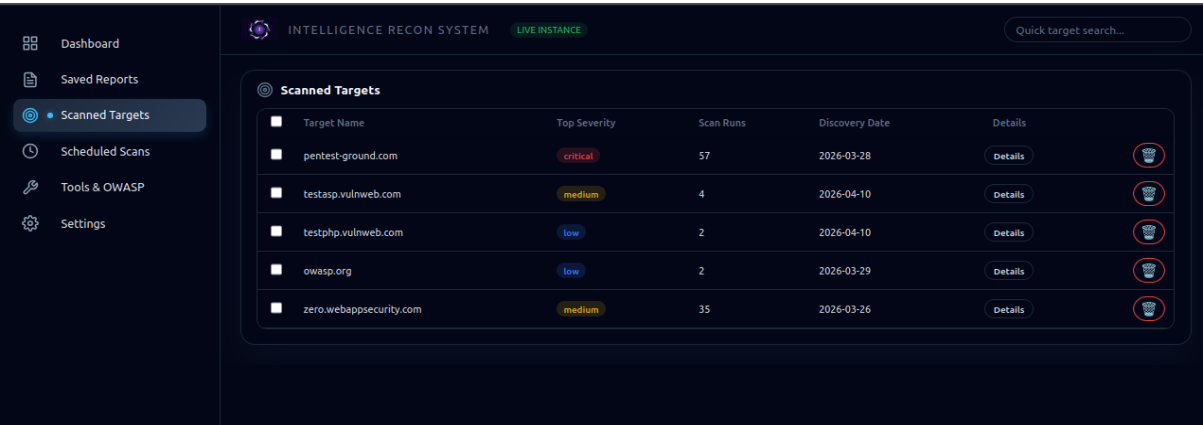


Figure 311 Scanned Targets of IRS

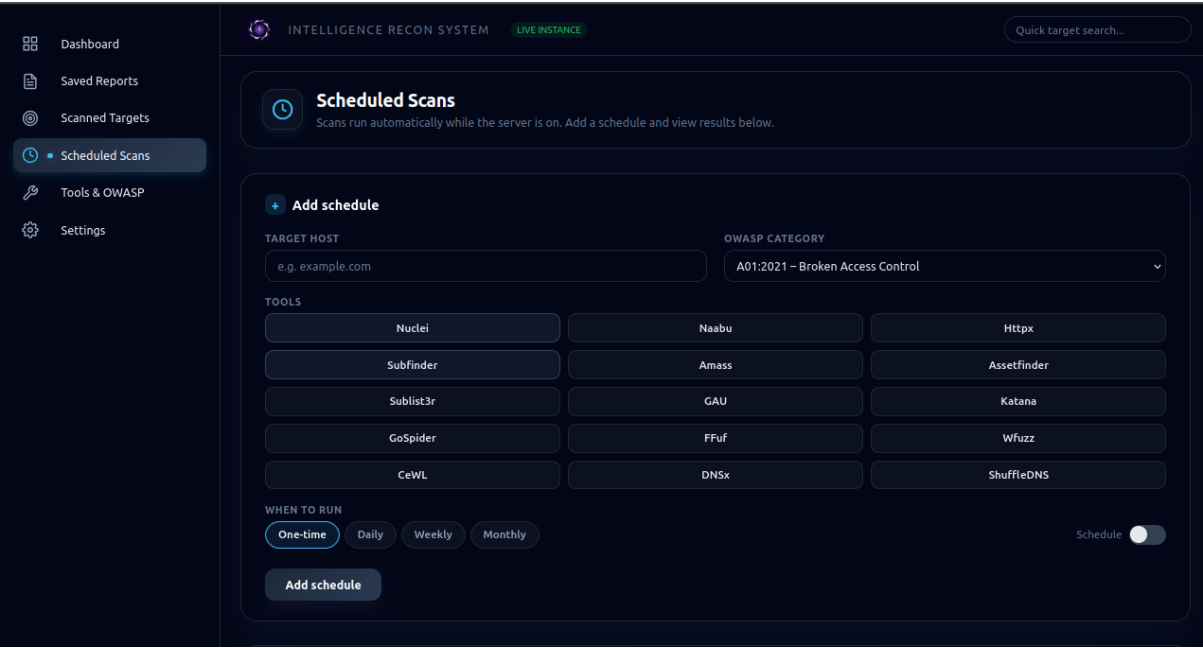


Figure 312 Scheduled Scans of IRS

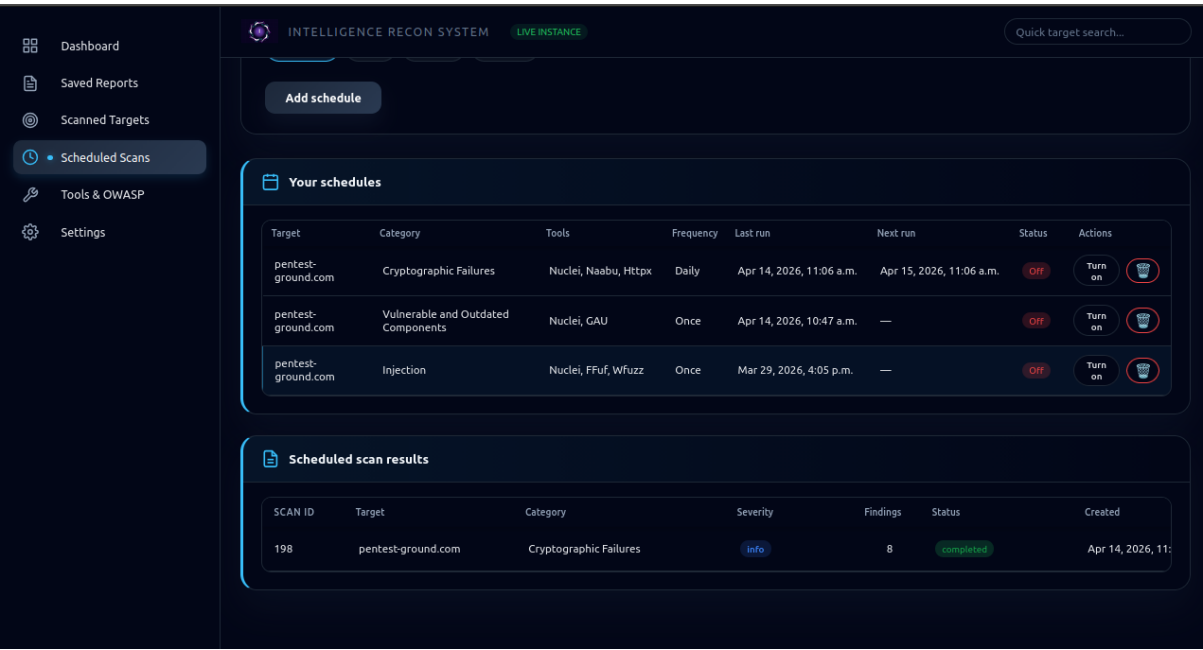


Figure 313 Scheduled Scans of IRS

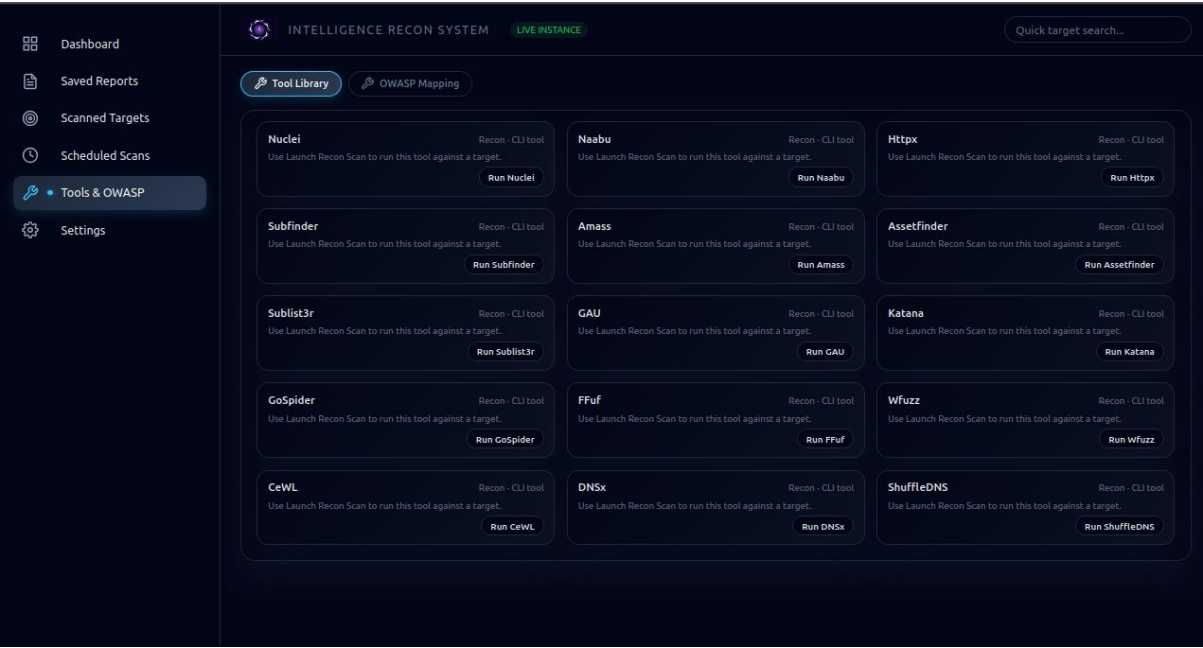


Figure 314 Tools & OWASP Mapping

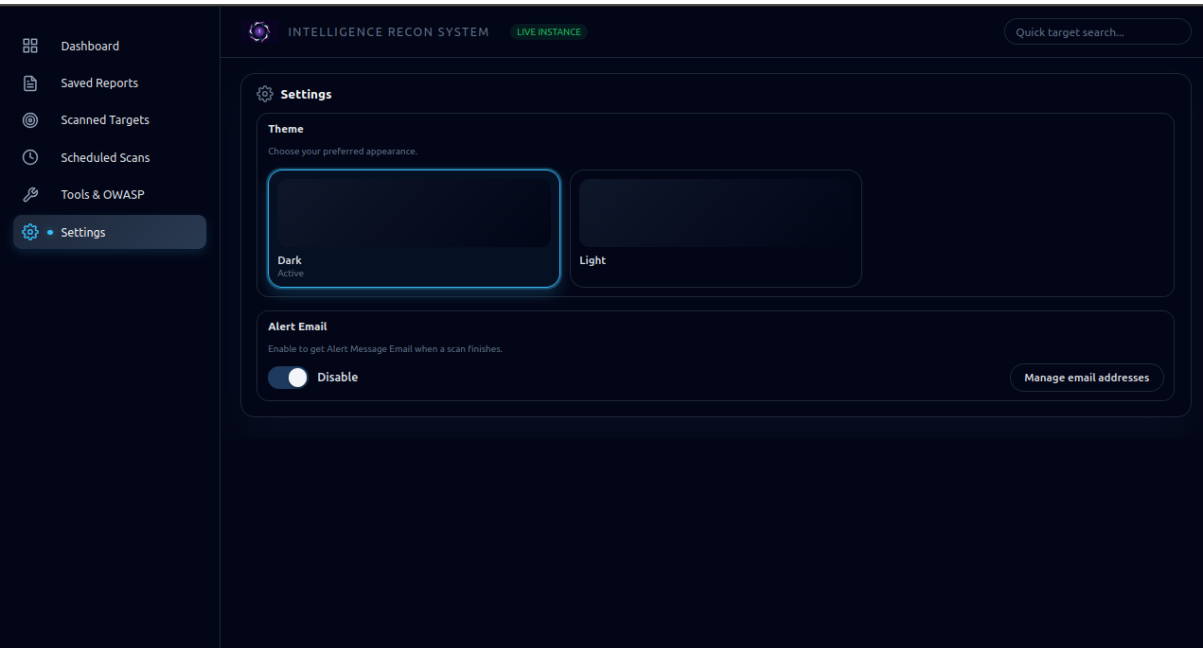


Figure 315 Settings of IRS

[Click Here](#) to go back.

7.6. APPENDIX 6: WATERFALL METHODOLOGY

Waterfall methodology refers to the linear project approach that is executed in phases where each phase (requirements, design, implementation, testing, deployment, and maintenance) is accomplished to completion which is followed by the subsequent one. It is based on high level upfront planning and documentation and is less flexible in terms of scope modification after development has begun. This is practical in stable and predictable projects but not practical in dynamic or research driven work (Day, 2025).



Figure 316 Waterfall Methodology (Welch, 2025).

Why Waterfall was considered.

- Simple and easy to follow, having a definite line of stages.
- Good records in every stage, which is suitable for academic reporting and assessment.
- Works with the requirements that may appear clear at the start of the project.

Why was Waterfall not chosen.

- All your project needs (AI behavior, tool chaining, UI) will change probably as you play around.
- Revising the previous stages is difficult, and this is dangerous to an innovative security system.
- You require feedback, testing, and improvement regularly which can be best facilitated by the iterative Agile/Scrum. [Click here](#) to go back.

7.7. APPENDIX 7: V-MODEL METHODOLOGY

V-Model is a verification/ validation SDLC model in which each development phase on the left (requirements, system design, detailed design, coding) has an equivalent testing phase on the right (unit, integration, system, acceptance testing). The rationale is to design tests in advance and make sure that each requirement is met by a test activity (GeeksForGeeks, 2025).

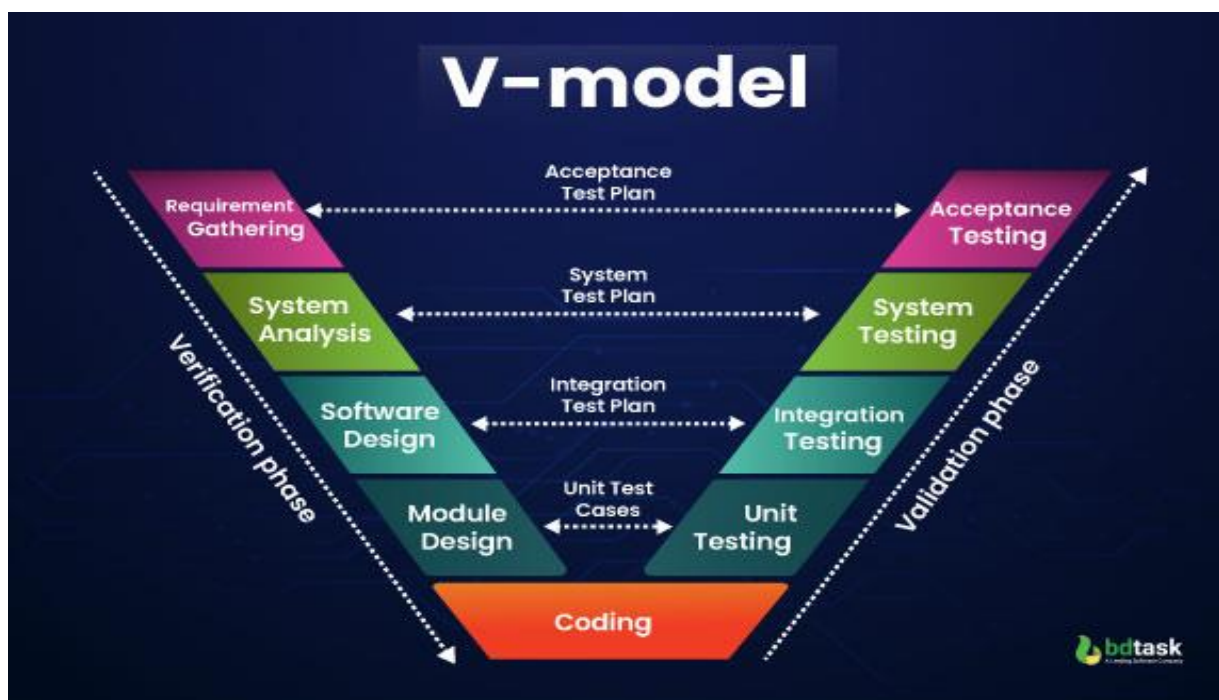


Figure 317 V-Model Methodology (Simran, 2025).

Why was it considered.

V-Model is desirable due to the following reasons:

- The specifications are quite precise: single internet-based platform, OWASP based recon, AI triage, 15 tools, reporting, etc., thus, it is possible to map each requirement to tests.
- It is a small to medium sized project and that is where V-Model is best suited and its well-written documentation and test planning can make a student project look like an academic endeavor.
- Security domain the security domain enjoys good verification: unit tests of parsers, integration tests of tool execution, and system tests of end-to-end scans are all compatible with the systematic testing focus of the V-Model.

Why was it not chosen.

Nevertheless, V-Model is not perfectly appropriate in your case, and Agile/Scrum (sprint-based plan that you are applying) is more suitable:

- The requirements will tend to change, requirements will vary as you experiment, and V-model assumes that requirements will be fixed and stable, whereas the V-model is inflexible when you need to make mid-project changes.
- In pure V-Model, working software is delivered late, whereas this project requires early prototypes (backend first, then intelligent workflow, then frontend) to demonstrate progress at interim milestones and make revisions.
- V-Model is paperwork-intensive and not as flexible as can be an overhead to a one-developer student project where rapid turnaround and quick completion is more significant than phase gates.

[Click here](#) to go back.

7.8. APPENDIX 8: AGILE METHODOLOGY

Agile methodology is an adaptable approach to project management through small, iterative units of work and releasing them at an appropriate frequency, followed by constant feedback and interaction amongst the team members to enable quick learning to adapt to the change (Peek, 2024). Agile project management is chosen to be used in the Intelligence Recon System as it allows iterative development, constant feedback and allowing flexibility in case of change in requirements or technical details.



Figure 318 Agile Methodology (Kamis, 2019).

Agile project management has four key values which can be directly applied to the IRS project:

- **People back processes and tools:** Cooperation between the developer and supervisors is encouraged to streamline recon workflows, OWASP-concerned features, and AI prompts, rather than observe a pre-determined process.
- **Coding more than documentation:** With each iteration, a working portion of IRS (e.g., executing a subset of the tools for creating an initial report) must be created, so that operational software is the main gauge of success.

- **Academic cooperation within the limits that are fixed:** Continued cooperation with academic supervisors and the FYP department is made more than keeping the original project scope unaltered.
- **Reactive to change after a plan:** As the behavior of tools, AI models, or constraints in a project evolve, Agile enables a backlog and design decision modification to make the system realistic and maintainable.

[Click here](#) to go back.

7.9. APPENDIX 9: SCRUM METHODOLOGY

Scrum is a special agile model that puts this concept into practice by short sprints that are definitely fixed and in which the team takes high priority tasks out of a backlog and delivers a usable increment at the end of each sprint (Peek, 2024). It encourages teamwork, transparency, and constant enhancement, which is in line with the requirements of IRS as a multifaceted system with various tools and AI logic.

- The project implies research, AI integration, various tools, and thus the requirements will be changing during development.
- The supervisors and other stakeholders should provide frequent feedback to improve features such as the dashboard, AI triage, and reporting.
- The team should demonstrate ongoing improvements (prototypes, partial features) rather than till the very end.

7.9.1. Role of scrum

Product Owner (role equivalent): The project supervisor, or stakeholder, who communicates the project vision, assists in prioritization of features.

Scrum Master (role equivalent): The person who oversees the process, by making sure that the Scrum practices (sprint planning, Scrum reviews, Scrum retrospectives) are adhered to as well as barriers prevented and overcome.

Scrum Team: The developer(s) who implements the development parts of the project.

Advantages

- Scrum is very dynamic and flexible, and it is suited to dynamic environments where the requirements are not clear.
- It encourages innovation and creativity by means of close working of ideas.
- It allows acceleration of product delivery or feature, leading to time to market.
- It can reduce the expenses by having less paperwork and formal control as compared to the conventional approach.
- It is more transparent and the ownership is shared, which enhances trust, productivity, and work quality.
- Constant feedback and regular changes tend to make the customer more satisfied.
- In Scrum, individual accountability and ownership enhance motivation and satisfaction of the teams.

Disadvantages

- Scrum needs experienced members of the team that can work hard and provide quality feedback.
- Teams require a long training course which may take months without experience.
- It is complicated and hard to control the process of scaling Scrum to large projects or numerous teams.
- Concentration on little steps does not ensure the delivery of the total project time.
- Scrum may be difficult to blend with more conventional and plan-based models such as waterfall.

These are the key points which show the advantages and disadvantages of scrum methodology (Vasiliauskas, 2025).

[Click here](#) to go back.

7.9.2. Justification for choosing Methodology.

The best methodological choice to the project of the Intelligence Recon System is Agile Scrum due to its ability to support requirements changes, feedback, and continuous delivery of the working part.

- **The changing demands and experimentation.**

The project will be associated with the integration of numerous security tools, tuning AI prompt and changing dashboards and reports based on the testing of the needs, and thus, the requirements cannot be entirely determined initially. The benefits of Scrum in changing scope and priorities between iterations include short sprints and backlog refinement, which do not disrupt the entire plan.

- **Early and incremental working software required.**

The project roadmap is sprint-based (backend setup, intelligent workflow, frontend, reporting, integration) with well-defined interim and final milestones, thus there is an advantage of delivering a working increment at the end of each sprint (for e.g. backend API on Sprint 2, intelligent workflow on Sprint 3, UI on the Sprint 4). Scrum is specifically intended to provide a potentially shippable increment at every sprint, which is in line with the idea of showing progress during reviews and to supervisors.

- **Close cooperation, feedback and quality**

The project demands unending feedback from supervisors and testers regarding security workflow, AI behavior and usability, which Scrum sustains through sprint reviews and retrospectives. Agile principles also adopt the importance of continuous testing and enhancement of each sprint, which also plays an important role in a security-oriented system that reliability and correctness are paramount.

[Click here](#) to go back.

7.10. APPENDIX 10: FUTURE PLAN

1. User Authentication & Access Control

The idea is to transform the tool into a professional site where the users will have to sign in to view their data. The system secures sensitive information using Login and Logout features. Once the user logs in, the system assigns him a special kind of digital key (stated as JWT) which is used to identify the user. Upon their logout that key is destroyed so that no other person can jump into his/her session and view his/her confidential security information.

2. Scan Tracking in Real-Time

At this point the site must continue to query the server. Currently system is using polling to show Live Status of scan time system will utilize WebSocket (which is an open phone line between the server and system screen). This enables the system to push the information to you in real-time. When a tool identifies vulnerability, it will be shown on your screen right away, and the entire process will feel a lot faster and easier.

3. Multi-user support

Multi-user support will allow multiple users to use the system at the same time with individual accounts, so every user can only access his or her scan data and reports. Role-based access control will be used in which the administrators are able to control the users and the system settings, and the normal users have restricted access. This will enhance security and scalability and enable the system to be used by a team.

4. Automatic Remediation

Automatic remediation can be provided and implemented on the system to identify the vulnerabilities and even provide the appropriate remedy suggestions. In this case, it can provide recommendations to upgrade software, alter settings, switch off the services that are not secure, or create more powerful passwords. This feature is a time-saving feature, and it enables users to find solutions to security issues easily without additional research.

[Click here](#) to go back.

7.11. APPENDIX 11: SOFTWARE REQUIREMENTS SPECIFICATIONS

7.11.1. Software Requirements Specifications

An elaborate Software Requirements Specification (SRS) document is critical to the development of software that satisfies business requirements as well as user expectations. Concise, clear, and executable requirements unify project teams, provide the clarity of functionality, and serve as one source of truth during the development. In the case of the Intelligence Recon System (IRS), the SRS is structured in IEEE-like fashion and includes functional and non-functional requirements of the AI-supported reconnaissance platform.

7.11.1.1. Introduction

7.11.1.1.1. Purpose

This SRS outlines the specifications of the Intelligence Recon System (IRS), which is an AI-assisted web platform that coordinates various open-source security tools to execute a specific reconnaissance and vulnerability scan, depending on OWASP categories. The system enables a user to enter a target, retrieve initial clues, have an AI decision node decide the right tools to use, perform scans asynchronously and view results of normalized findings and reports on a dashboard.

The primary intended user is:

Security Analysts: Analysts perform authorized scans, monitor status, analyze results based on severity and OWASP, and report on the results to proceed with the risk analysis or penetration.

Scope: IRS combines and coordinates existing open-source tools (e.g., Naabu) with Httpx, Nuclei, Subfinder, Amass) through a Fast API backend, AI decision node and web frontend. The scope includes the creation of scans, making intelligent choices of tools, running in parallel, normalizing the results, their storage and reporting; exploitation or constant monitoring is not in the scope of the present FYP.

7.11.1.1.2. Definitions, Acronyms, and Abbreviations.

- **IRS:** Intelligence Recon System - AI based reconnaissance and vulnerability scan system.

- **OWASP:** Open Worldwide Application Security Project; offers the OWASP Top 10 types of scans to be focused.
- **AI Decision Node / LLM:** Large language model based on Gemini that decides on which tools to execute given the first clues and OWASP category.
- **Scan:** One test case under a target using a target set of tools and OWASP focus, which is recorded in the database.
- **Tool Run:** Profile of a particular security tool in a scan, status, start and end times and summary.
- **Result:** The normalized vulnerability or reconnaissance outcome of type, severity, location, summary and evidence.

7.11.1.2. Overall Description

7.11.1.2.1. Product Perspective

IRS is an independent web application that integrates reconnaissance processes in such a way that would otherwise be conducted manually through a variety of independent command line tools. It is a browser-based dashboard, Fast API business-logic services on a backend with a business-logic node, AI decision node, and a tool-execution layer, running 15 open-source tools, storing the results in a relational database. REST APIs are made available in such a way that it allows the integration of other systems with IRS in a programmatic way.

7.11.1.2.2. Product Functions

Key functions of IRS include:

- **Target scan creation:** Takes a target (e.g., domain) and OWASP category (optionally permitting the user to select tools) and constructs a new scan record and begins processing.
- **First clue collection:** The automatic selection of Naabu and Httpx to detect open ports, HTTP services, and basic technologies is conducted, and then other tools are not chosen.
- **AI-aided tool selection:** Takes the first indications and OWASP focus and suggests tools to operate or not, with a harmless default option if the AI is not available.

- **Parallel execution:** The tool execution is provided as the selected tools execute concurrently and limited by concurrency and timeouts, and the status of the tools and errors are documented.
- Result normalization and storage of Parses heterogeneous tool results into structured results by severity and location and stores all raw and processed data to be reviewed later.
- Reporting and history Reporting and history Gives a dashboard view of scans and findings and creates HTML (and PDF-ready) reports with severity and tool summaries.

7.11.1.2.3. Classes and Characteristics of User

- **Security Analysts**
 - Make and follow scans, find and report results.
 - Need OWASP-views, severity grouping and little need to touch CLI tools.

7.11.1.2.4. Operating Environment

IRS prototype is written on Ubuntu/Linux, and Python 3, Fast Api and SQLite to persist data in the database while developing and testing, although it is not mentioned that the database schema and ORM are designed to be compatible with SQLite when the project becomes production-scale. The frontend is served as static HTML/CSS/JavaScript by Fast API and all interaction between the browser and the server is based on HTTP/JSON APIs.

7.11.1.2.5. Constraints of Design and Implementation

- Only open and stable reconnaissance tools are employed; they must all allow safe invocation based on argument and not shell injection.
- The names of all target input and tools should be verified and sterilized to avoid command injection or abuse.
- The implementation of AI should not go beyond the boundaries of the provider; there should be timeouts and fallbacks when the AI is not fast enough or is not available.
- There should be blocking constraints to prevent the depletion of resources when there are numerous tools or scans running.

7.11.1.3. Specific Requirements

7.11.1.3.1. Functional Requirements (FR)

FR1: Creation and Management of Scans.

IRS will enable security analysts to generate scans by giving them a target and category of OWASP and follow-up the status of each scan through creation to completion.

FR2: Preliminary Evidence Collection (Naabu, Httpx)

IRS will default on Naabu and Httpx when each scan has a new scan to gather open ports and HTTP service data prior to one of the other tools being chosen.

FR3: AI-Driven Tool Selection

IRS will provide the first hints, target specifications and OWASP category to the AI decision node and be provided with a list of tools to execute or abandon, default safe set in case of failure of the AI.

FR4: Asynchronous Execution of Tools.

IRS will run the imposed tools asynchronously with several tools per scan and per-tool timeouts being set.

FR5: Findings Normalization and storage.

IRS will extract tool results in form of structured findings (type, severity, location, summary, evidence) and store them in the database together with raw output references.

FR6: Status and Dashboard Views.

IRS will have dashboard pages to create scans, scan status, past scan list and inspect detailed results and tool summary.

FR7: Reporting

IRS will produce an HTML summary of the findings of a chosen scan, by severity and by the tool used, and allow export to PDF, with an external renderer being used when configured.

7.11.1.3.2. Non-Functional Requirements (NFR)**7.11.1.4. Performance**

The system will be able to run up to 3 tools in parallel per scan and at the same time maintain the UI responsive.

7.11.1.5. Security

The execution of any command should be done with argument lists and without shell=True, user inputs should be strictly checked, and the keys of AI API and other secrets should be saved in environment variables and should never be logged or copied in any response.

7.11.1.6. Reliability

Breakages of a specific tool or the decision node of the AI should not terminate the entire scan, IRS should indicate the failed Tool Runs suitably but still should keep the successful results and enable generating the report.

7.11.1.7. Usability

The dashboard will employ easy to read three column design with live status update, severity badges and brief messages to make sure that an analyst can get a quick view of the scan progress, and scan results.

7.11.1.8. Scalability and Extensibility Scalability and Extensibility.

It will allow the addition of new tools and replace AI models with minor updates and will allow migrating to SQLite in case of bigger deployments.

[Click Here](#) to go back.

7.12. APPENDIX 12: PRODUCT BACKLOG

8. User Story	Justification	Points
As a security analyst, I would like to provide a target domain and choose a particular OWASP Top 10 category to ensure that the scan is limited to the relevant vulnerabilities.	Maintains clear input-related requirements as well as being in line with OWASP-based assessment objectives.	3
Being a security analyst, I would like to browse and pick among a list of 15 integrated recon and vulnerability tools so that I can manage the execution of these tools.	Enables the configuration of scans to be transparent and controllable and yet still able to validate AI.	5
As a security analyst, I would like the AI decision node to test and optimize my choice of the tool using initial hints (ports, technologies) to ensure that superfluous tools are not executed.	Elaborates scans of pertinent tools, which minimize noise and execution time.	8
As a system administrator, I would like scans to run in the background and asynchronously with obvious status reporting to ensure that the UI is responsive when scans run.	Scales become better and become more user-friendly when performing a multitude of scans or long scans.	5
As a security analyst, I would like all the tool results to be normalized, parsed, and kept in a structured format to allow me to review and correlate results easily.	Facilitates the ability to compare the output of heterogeneous tools as well as facilitate subsequent reporting.	8
As a security analyst, I would like to have a consolidated view of the findings based on the severity and OWASP category to enable me to remediate the findings easily.	Convert raw scan data into prioritized, action to be taken insight.	5

As a user, I would like to create and save an HTML/ PDF report to any completed scan to be able to share assessment results with the stakeholders.	Gives formal support of audits, presentations, and project evaluation.	5
As a system administrator, I would like to have simple monitoring, logging and error handling around scan execution to diagnose and correct failures in an efficient way.	Improves maintainability and helps with stable real time usage.	5
As a technical writer, I wish to write user and developer documentation with architecture and API usage diagrams and step-by-step scan processes to allow others to deploy and extend the system.	Make the system readable by the system assessors, operators and future developers.	5

Table 20 Product Backlog

[Click here](#) to go back

8.13. APPENDIX 13: PROJECT RISKS, THREATS & CONTINGENCY PLAN

Threats and risks can be encountered during the project development process since every single system can be exposed to risks. The following risks and threats are mitigated along with contingency plans to minimize systemic defects.

- **AI Triage Dependency Risk**

Risk: AI Triage (Gemini API) provides faulty filtering, which reduces the effectiveness of scans.

Threats: Gemini API update software failure, AI misunderstands target or proposes a tool that is not relevant.

Contingency plan

- Introduce Manual Override and Fallback to operate 5 most important tools without AI.
- Have a strict version locked prompt template.

- **Creation of tool Risk and Data Inconsistency risk.**

Risk: Security tools malfunction, crash system or invalid output of raw results.

Threats: Critical tool will not run with update, change of format output or tool requires unconfigured environmental variables.

Contingency plan

- Install Version Locking and Isolation on all the 15 security tools.
- Crash of subprocesses is isolated using the try/except in Python.
- In case of parsing failures, do away with inconsistent data and record the error.

- **Performance and Scalability risk.**

Risk: Asynchronous Execution Core is inefficient in managing concurrent tools, thus leading to system lag.

Threats: Subprocess module will be a bottleneck, server will run out of resources or database will write slowly.

Contingency plan

- A Fixed Parallel Limit will be implemented to control simultaneous tools.
- Make use of the inbuilt asyncio of Python to have a responsive UI.

[Click here](#) to go back.

8.14. APPENDIX 14: RESOURCES REQUIREMENTS

8.14.1. Hardware Requirement

Personal devices (Laptop) will be used as development machines.

8.14.2. Software Requirement

Category	Software Requirements	Specifications
OS	Ubuntu	Open-source and stable Linux distribution is the development environment.
Development Tools	VS Code	I used IDE to code, debug, and version control.
	HTML, CSS, JavaScript	Technologies of User Interface development.

	Python/Fast API	Python backend language: fast Api, high performance, asynchronous API.
Core AI Model	Gemini2.5 Flash	Intelligent triage and data validation of the AI Decision Node
Data Storage	SQLite (aiosqlite)	Storage of all scan results and history record keeping.
Gantt Chart	Click up and TeamGantt	Project timeline and progress tracking visual tools.
Documentation	MS Word	Keep records of the project.
Diagram	Eraser, Draw.io, Visual Paradigm	Shows ideas and processes.
Version control	Git	Tracks the changes to the source file
Project Management Tool	Trello	Used to keep track of milestones

Table 21 Table of Software Requirements

[Click Here](#) to go back